saturnCFD

0.1.0

Vinaykanth

Generated by Doxygen 1.12.0

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Class Index 3
2.1 Class List . . . 3

3 File Index 5
3.1 File List . . . 5

4 Namespace Documentation 7
4.1 saturn Namespace Reference . . . 7
4.1.1 Typedef Documentation . . . 9
4.1.1.1 Int3 . . . 9
4.1.1.2 Int4 . . . 9
4.1.1.3 MDRange3 . . . 9
4.1.1.4 MDRange4 . . . 9
4.1.2 Enumeration Type Documentation . . . 9
4.1.2.1 BoundaryType . . . 9
4.1.2.2 Reconstruction . . . 10
4.1.2.3 RiemannSolver . . . 10
4.1.3 Function Documentation . . . 10
4.1.3.1 advanceRk4() . . . 10
4.1.3.2 advanceSsprk3() . . . 11
4.1.3.3 applyBoundaryConditions() . . . 11
4.1.3.4 dispatchExecutionSpace() . . . 11
4.1.3.5 ensureParent() . . . 11
4.1.3.6 fieldAddScaled() . . . 11
4.1.3.7 fieldCombine() . . . 11
4.1.3.8 fieldCopy() . . . 12
4.1.3.9 makeMdrange3() . . . 12
4.1.3.10 makeMdrange4() . . . 12
4.1.3.11 mpiAll() . . . 12
4.1.3.12 mpiMax() . . . 12
4.1.3.13 mpiRank() . . . 12
4.1.3.14 mpiSize() . . . 12
4.1.3.15 mpiSum() . . . 12
4.1.3.16 parseBoundaryType() . . . 13
4.1.3.17 rankSuffix() . . . 13
4.1.3.18 readCheckpoint() . . . 13
4.1.3.19 reconstructionStencilWidth() . . . 13
4.1.3.20 scalarAdvectionDiffusionRhs() [1/2] . . . 13
4.1.3.21 scalarAdvectionDiffusionRhs() [2/2] . . . 13
4.1.3.22 scalarAdvectionDiffusionRhsShell() . . . 14

4.1.3.23 scalarRhsValue() . . . 14
4.1.3.24 unavailableBackend() . . . 14
4.1.3.25 writeCheckpoint() . . . 14
4.1.3.26 writeVti() . . . 14
4.1.3.27 writeVtiFields() . . . 15
4.1.4 Variable Documentation . . . 15
4.1.4.1 is_host_execution_space . . . 15
4.1.4.2 kAuthor . . . 15
4.1.4.3 kInnerTile . . . 15
4.1.4.4 kProjectName . . . 15
4.1.4.5 kVersion . . . 15
4.2 saturn::cavity Namespace Reference . . . 15
4.2.1 Function Documentation . . . 16
4.2.1.1 advanceSsprk3() . . . 16
4.2.1.2 divergence() . . . 17
4.2.1.3 filterVelocity() . . . 17
4.2.1.4 limitedUpwind() . . . 17
4.2.1.5 minmod() . . . 18
4.2.1.6 momentumRhs() . . . 18
4.2.1.7 project() . . . 18
4.2.1.8 projectClustered() . . . 19
4.2.1.9 weno5Derivative() . . . 19
4.3 saturn::channel Namespace Reference . . . 19
4.3.1 Function Documentation . . . 20
4.3.1.1 advanceSsprk3() . . . 20
4.3.1.2 divergence() . . . 20
4.3.1.3 filter4Channel() . . . 21
4.3.1.4 filterDimension() . . . 21
4.3.1.5 momentumRhs() . . . 21
4.3.1.6 project() . . . 21
4.4 saturn::compressible Namespace Reference . . . 22
4.4.1 Function Documentation . . . 23
4.4.1.1 assembleRhs() . . . 23
4.4.1.2 central4Derivative() . . . 23
4.4.1.3 compressibleRhs() . . . 23
4.4.1.4 computeFaceFluxesX() . . . 23
4.4.1.5 computeFaceFluxesY() . . . 24
4.4.1.6 computeFaceFluxesZ() . . . 24
4.4.1.7 computeGradients() . . . 24
4.4.1.8 computeStress() . . . 24
4.4.1.9 fieldDerivative() . . . 24
4.4.1.10 fromNormal() . . . 25

4.4.1.11 gradientLine() . . . 25
4.4.1.12 inviscidFaceFlux() . . . 25
4.4.1.13 loadPrimitive() . . . 25
4.4.1.14 maxKinematicDiffusivity() . . . 25
4.4.1.15 maxWaveSpeed() . . . 26
4.4.1.16 reconstructFace() . . . 26
4.4.1.17 soundSpeed() . . . 26
4.4.1.18 temperature() . . . 26
4.4.1.19 toNormal() . . . 26
4.5 saturn::edac Namespace Reference . . . 26
4.5.1 Function Documentation . . . 27
4.5.1.1 advanceSsprk3() . . . 27
4.5.1.2 laplacian() . . . 28
4.5.1.3 pressureRhs() . . . 28
4.5.1.4 subtract() . . . 28
4.5.1.5 velocityRhs() . . . 28
4.6 saturn::incompressible Namespace Reference . . . 29
4.6.1 Function Documentation . . . 29
4.6.1.1 advanceSsprk3() . . . 29
4.6.1.2 combine() . . . 30
4.6.1.3 filter4() . . . 30
4.6.1.4 filterDimension() . . . 30
4.6.1.5 momentumRhs() . . . 30
4.7 saturn::poisson Namespace Reference . . . 30
4.7.1 Enumeration Type Documentation . . . 31
4.7.1.1 Boundary . . . 31
4.8 saturn::poisson::detail Namespace Reference . . . 31
4.8.1 Function Documentation . . . 31
4.8.1.1 extendedIndex() . . . 31
4.8.1.2 minusIndex() . . . 32
4.8.1.3 plusIndex() . . . 32
4.9 saturn::projection Namespace Reference . . . 32
4.9.1 Typedef Documentation . . . 32
4.9.1.1 ScalarView . . . 32
4.9.2 Function Documentation . . . 33
4.9.2.1 divergence() . . . 33
4.9.2.2 gradientX() . . . 33
4.9.2.3 gradientY() . . . 33
4.9.2.4 gradientZ() . . . 33
4.9.2.5 project() . . . 33
4.10 saturn::recon Namespace Reference . . . 34
4.10.1 Function Documentation . . . 34

4.10.1.1 central2() . . . 34
4.10.1.2 central4() . . . 35
4.10.1.3 central6() . . . 35
4.10.1.4 reconstructLeft() [1/2] . . . 35
4.10.1.5 reconstructLeft() [2/2] . . . 35
4.10.1.6 reconstructRight() [1/2] . . . 35
4.10.1.7 reconstructRight() [2/2] . . . 36
4.10.1.8 stencil6Smoothness() . . . 36
4.10.1.9 teno6() . . . 36
4.10.1.10 tenoInverseSmoothness() . . . 36
4.10.1.11 weno5Smoothness() . . . 36
4.10.1.12 weno5z() . . . 37
4.10.2 Variable Documentation . . . 37
4.10.2.1 kEpsilon . . . 37
4.10.2.2 kTenoCutoff . . . 37
4.10.2.3 kTenoExponent . . . 37
4.10.2.4 kWenoWeight0 . . . 37
4.10.2.5 kWenoWeight1 . . . 37
4.10.2.6 kWenoWeight2 . . . 37
4.11 saturn::riemann Namespace Reference . . . 38
4.11.1 Function Documentation . . . 38
4.11.1.1 ausmPlusUp() . . . 38
4.11.1.2 averageFlux() . . . 38
4.11.1.3 flux() . . . 38
4.11.1.4 hllc() . . . 39
4.11.1.5 hllcImpl() . . . 39
4.11.1.6 hllcLowMach() . . . 39
4.11.1.7 llf() . . . 39
4.11.1.8 machMinus1() . . . 39
4.11.1.9 machMinus2() . . . 39
4.11.1.10 machPlus1() . . . 39
4.11.1.11 machPlus2() . . . 40
4.11.1.12 maxSignalSpeed() . . . 40
4.11.1.13 physicalFlux() . . . 40
4.11.1.14 pressureMinus5() . . . 40
4.11.1.15 pressurePlus5() . . . 40
4.11.1.16 toConservative() . . . 40
4.11.1.17 toPrimitive() . . . 40
4.11.1.18 valid() . . . 40
4.12 saturn::vof Namespace Reference . . . 41
4.12.1 Typedef Documentation . . . 41
4.12.1.1 Face . . . 41

4.12.1.2 Polyhedron . . . 42
4.12.2 Function Documentation . . . 42
4.12.2.1 clipAxis() . . . 42
4.12.2.2 clipHalfPlane() [1/2] . . . 42
4.12.2.3 clipHalfPlane() [2/2] . . . 42
4.12.2.4 columnHeight() [1/2] . . . 42
4.12.2.5 columnHeight() [2/2] . . . 42
4.12.2.6 cross() . . . 42
4.12.2.7 curvatureLevelset() . . . 43
4.12.2.8 curvatureX() . . . 43
4.12.2.9 curvatureY() . . . 43
4.12.2.10 curvatureZ() . . . 43
4.12.2.11 dot() . . . 43
4.12.2.12 fluidPolygon() . . . 43
4.12.2.13 fluidPolyhedron() . . . 43
4.12.2.14 operator∗() . . . 44
4.12.2.15 operator+() . . . 44
4.12.2.16 operator-() . . . 44
4.12.2.17 polygonArea() . . . 44
4.12.2.18 polyhedronVolume() . . . 44
4.12.2.19 reconstruct() [1/2] . . . 44
4.12.2.20 reconstruct() [2/2] . . . 44
4.12.2.21 redistribute() [1/2] . . . 45
4.12.2.22 redistribute() [2/2] . . . 45
4.12.2.23 rowHeight() . . . 45
4.12.2.24 smoothBinomial() . . . 45
4.12.2.25 unitCube() . . . 45
4.12.2.26 unitSquare() . . . 45

5 Class Documentation **47**

5.1 saturn::Box Struct Reference . . . 47
5.1.1 Member Function Documentation . . . 47
5.1.1.1 contains() . . . 47
5.1.1.2 empty() . . . 48
5.1.1.3 fromExtents() . . . 48
5.1.1.4 grow() . . . 48
5.1.1.5 intersect() . . . 48
5.1.1.6 length() . . . 48
5.1.1.7 numCells() . . . 48
5.1.1.8 operator"!=() . . . 48
5.1.1.9 operator==() . . . 48
5.1.1.10 shift() . . . 48

5.1.1.11 shrink() . . . 49
5.1.2 Member Data Documentation . . . 49
5.1.2.1 hi . . . 49
5.1.2.2 lo . . . 49
5.2 saturn::BoxExchange< ExecSpace > Class Template Reference . . . 49
5.2.1 Constructor & Destructor Documentation . . . 49
5.2.1.1 BoxExchange() . . . 49
5.2.2 Member Function Documentation . . . 49
5.2.2.1 exchange() . . . 49
5.3 saturn::CheckpointHeader Struct Reference . . . 50
5.3.1 Member Data Documentation . . . 50
5.3.1.1 global_hi . . . 50
5.3.1.2 global_lo . . . 50
5.3.1.3 local_hi . . . 50
5.3.1.4 local_lo . . . 50
5.3.1.5 magic . . . 50
5.3.1.6 ng . . . 50
5.3.1.7 nvar . . . 51
5.3.1.8 rank . . . 51
5.3.1.9 size . . . 51
5.3.1.10 step . . . 51
5.3.1.11 time . . . 51
5.3.1.12 version . . . 51
5.4 saturn::cavity::ClusteredPoisson2D Class Reference . . . 51
5.4.1 Detailed Description . . . 52
5.4.2 Constructor & Destructor Documentation . . . 52
5.4.2.1 ClusteredPoisson2D() . . . 52
5.4.3 Member Function Documentation . . . 52
5.4.3.1 apply() . . . 52
5.4.3.2 solve() . . . 52
5.5 saturn::compressible::CompressibleWorkspace< ExecSpace > Class Template Reference . . . 52
5.5.1 Member Typedef Documentation . . . 53
5.5.1.1 face_view . . . 53
5.5.1.2 memory_space . . . 53
5.5.2 Constructor & Destructor Documentation . . . 53
5.5.2.1 CompressibleWorkspace() . . . 53
5.5.3 Member Data Documentation . . . 53
5.5.3.1 fx . . . 53
5.5.3.2 fy . . . 53
5.5.3.3 fz . . . 54
5.5.3.4 grad . . . 54
5.5.3.5 stress . . . 54

5.6 saturn::ConservativeState Struct Reference . . . 54
5.6.1 Member Data Documentation . . . 54
5.6.1.1 energy . . . 54
5.6.1.2 mom_n . . . 54
5.6.1.3 mom_t1 . . . 54
5.6.1.4 mom_t2 . . . 55
5.6.1.5 rho . . . 55
5.7 saturn::vof::Curvature2D Struct Reference . . . 55
5.7.1 Member Data Documentation . . . 55
5.7.1.1 kappa . . . 55
5.7.1.2 slope . . . 55
5.7.1.3 valid . . . 55
5.8 saturn::vof::Curvature3D Struct Reference . . . 55
5.8.1 Member Data Documentation . . . 56
5.8.1.1 hx . . . 56
5.8.1.2 hy . . . 56
5.8.1.3 kappa . . . 56
5.8.1.4 valid . . . 56
5.9 saturn::Decomposition Struct Reference . . . 56
5.9.1 Member Function Documentation . . . 57
5.9.1.1 boxForCoords() . . . 57
5.9.1.2 create() . . . 57
5.9.1.3 partner() . . . 57
5.9.2 Member Data Documentation . . . 57
5.9.2.1 coords . . . 57
5.9.2.2 dims . . . 57
5.9.2.3 global . . . 57
5.9.2.4 local . . . 58
5.9.2.5 periodic . . . 58
5.9.2.6 rank . . . 58
5.9.2.7 size . . . 58
5.10 saturn::FaceExchange< ExecSpace > Class Template Reference . . . 58
5.10.1 Member Typedef Documentation . . . 58
5.10.1.1 buffer_type . . . 58
5.10.1.2 host_buffer_type . . . 59
5.10.1.3 memory_space . . . 59
5.10.2 Constructor & Destructor Documentation . . . 59
5.10.2.1 FaceExchange() . . . 59
5.10.3 Member Function Documentation . . . 59
5.10.3.1 begin() . . . 59
5.10.3.2 end() . . . 59
5.10.3.3 exchange() . . . 59

5.10.3.4 pack() . . . 59
5.10.3.5 unpack() . . . 60
5.11 saturn::Flux Struct Reference . . . 60
5.11.1 Member Data Documentation . . . 60
5.11.1.1 energy . . . 60
5.11.1.2 mom_n . . . 60
5.11.1.3 mom_t1 . . . 60
5.11.1.4 mom_t2 . . . 60
5.11.1.5 rho . . . 61
5.12 saturn::compressible::GasProperties Struct Reference . . . 61
5.12.1 Member Data Documentation . . . 61
5.12.1.1 gamma . . . 61
5.12.1.2 gas_constant . . . 61
5.12.1.3 mu . . . 61
5.12.1.4 prandtl . . . 61
5.13 saturn::poisson::GeometricMultigrid< ExecSpace > Class Template Reference . . . 62
5.13.1 Detailed Description . . . 62
5.13.2 Member Typedef Documentation . . . 63
5.13.2.1 memory_space . . . 63
5.13.2.2 view_type . . . 63
5.13.3 Constructor & Destructor Documentation . . . 63
5.13.3.1 GeometricMultigrid() [1/2] . . . 63
5.13.3.2 GeometricMultigrid() [2/2] . . . 63
5.13.4 Member Function Documentation . . . 63
5.13.4.1 computeResidual() . . . 63
5.13.4.2 levels() . . . 63
5.13.4.3 norm() . . . 64
5.13.4.4 projectMeanImpl() . . . 64
5.13.4.5 prolongAdd() . . . 64
5.13.4.6 residualNorm() . . . 64
5.13.4.7 restrict() . . . 64
5.13.4.8 smooth() . . . 64
5.13.4.9 solve() . . . 64
5.13.4.10 vcycle() . . . 65
5.14 saturn::cavity::Geometry Struct Reference . . . 65
5.14.1 Detailed Description . . . 65
5.14.2 Member Function Documentation . . . 66
5.14.2.1 dmapX() . . . 66
5.14.2.2 mapX() . . . 66
5.14.2.3 xc() . . . 66
5.14.2.4 xf() . . . 66
5.14.2.5 yc() . . . 66

5.14.2.6 yf() . . . 66
5.14.3 Member Data Documentation . . . 66
5.14.3.1 adv . . . 66
5.14.3.2 adv_blend . . . 66
5.14.3.3 beta . . . 67
5.14.3.4 hx . . . 67
5.14.3.5 hy . . . 67
5.14.3.6 hz . . . 67
5.14.3.7 n0 . . . 67
5.14.3.8 n1 . . . 67
5.14.3.9 n2 . . . 67
5.14.3.10 u_lid . . . 67
5.14.3.11 z_wall . . . 67
5.15 saturn::channel::Geometry Struct Reference . . . 68
5.15.1 Member Data Documentation . . . 68
5.15.1.1 gx . . . 68
5.15.1.2 hx . . . 68
5.15.1.3 hy . . . 68
5.15.1.4 hz . . . 68
5.15.1.5 ly . . . 68
5.15.1.6 n0 . . . 68
5.15.1.7 n1 . . . 69
5.15.1.8 n2 . . . 69
5.15.1.9 nu . . . 69
5.15.1.10 u_top . . . 69
5.16 saturn::GlobalExchange< ExecSpace > Class Template Reference . . . 69
5.16.1 Constructor & Destructor Documentation . . . 69
5.16.1.1 GlobalExchange() . . . 69
5.16.2 Member Function Documentation . . . 70
5.16.2.1 exchange() . . . 70
5.17 saturn::vof::Grid2D Class Reference . . . 70
5.17.1 Constructor & Destructor Documentation . . . 70
5.17.1.1 Grid2D() . . . 70
5.17.2 Member Function Documentation . . . 71
5.17.2.1 advance() . . . 71
5.17.2.2 data() [1/2] . . . 71
5.17.2.3 data() [2/2] . . . 71
5.17.2.4 hx() . . . 71
5.17.2.5 hy() . . . 71
5.17.2.6 mass() . . . 71
5.17.2.7 nx() . . . 71
5.17.2.8 ny() . . . 71

5.17.2.9 operator()() [1/2] . . . 71
5.17.2.10 operator()() [2/2] . . . 72
5.17.2.11 reconstructions() . . . 72
5.17.2.12 sweepX() . . . 72
5.17.2.13 sweepY() . . . 72
5.18 saturn::vof::Grid3D Class Reference . . . 72
5.18.1 Constructor & Destructor Documentation . . . 73
5.18.1.1 Grid3D() . . . 73
5.18.2 Member Function Documentation . . . 73
5.18.2.1 advance() . . . 73
5.18.2.2 cellVolume() . . . 73
5.18.2.3 data() [1/2] . . . 73
5.18.2.4 data() [2/2] . . . 73
5.18.2.5 hx() . . . 73
5.18.2.6 hy() . . . 73
5.18.2.7 hz() . . . 73
5.18.2.8 mass() . . . 74
5.18.2.9 nx() . . . 74
5.18.2.10 ny() . . . 74
5.18.2.11 nz() . . . 74
5.18.2.12 operator()() [1/2] . . . 74
5.18.2.13 operator()() [2/2] . . . 74
5.18.2.14 reconstructions() . . . 74
5.18.2.15 sweepX() . . . 74
5.18.2.16 sweepY() . . . 74
5.18.2.17 sweepZ() . . . 75
5.19 saturn::poisson::GeometricMultigrid< ExecSpace >::Level Struct Reference . . . 75
5.19.1 Member Data Documentation . . . 75
5.19.1.1 b . . . 75
5.19.1.2 h . . . 75
5.19.1.3 n . . . 75
5.19.1.4 r . . . 75
5.19.1.5 x . . . 76
5.20 saturn::vof::LevelSetCurvature Struct Reference . . . 76
5.20.1 Member Data Documentation . . . 76
5.20.1.1 kappa . . . 76
5.20.1.2 valid . . . 76
5.21 saturn::poisson::Options Struct Reference . . . 76
5.21.1 Detailed Description . . . 77
5.21.2 Member Data Documentation . . . 77
5.21.2.1 coarse_smooth . . . 77
5.21.2.2 max_cycles . . . 77

5.21.2.3 post_smooth . . . 77
5.21.2.4 pre_smooth . . . 77
5.21.2.5 tolerance . . . 77
5.21.2.6 verbose . . . 77
5.22 saturn::edac::Parameters Struct Reference . . . 77
5.22.1 Member Data Documentation . . . 78
5.22.1.1 c . . . 78
5.22.1.2 nu . . . 78
5.22.1.3 nu_p . . . 78
5.23 saturn::PatchField< ExecSpace > Class Template Reference . . . 78
5.23.1 Member Typedef Documentation . . . 78
5.23.1.1 execution_space . . . 78
5.23.1.2 memory_space . . . 79
5.23.1.3 view_type . . . 79
5.23.2 Constructor & Destructor Documentation . . . 79
5.23.2.1 PatchField() [1/2] . . . 79
5.23.2.2 PatchField() [2/2] . . . 79
5.23.3 Member Function Documentation . . . 79
5.23.3.1 box() . . . 79
5.23.3.2 data() [1/2] . . . 79
5.23.3.3 data() [2/2] . . . 79
5.23.3.4 fillPeriodic() . . . 80
5.23.3.5 ng() . . . 80
5.23.3.6 nvar() . . . 80
5.23.3.7 operator()() . . . 80
5.24 saturn::vof::Point Struct Reference . . . 80
5.24.1 Member Data Documentation . . . 80
5.24.1.1 x . . . 80
5.24.1.2 y . . . 81
5.25 saturn::PrimitiveState Struct Reference . . . 81
5.25.1 Member Data Documentation . . . 81
5.25.1.1 p . . . 81
5.25.1.2 rho . . . 81
5.25.1.3 u . . . 81
5.25.1.4 v . . . 81
5.25.1.5 w . . . 81
5.26 saturn::vof::Reconstruction Struct Reference . . . 82
5.26.1 Member Data Documentation . . . 82
5.26.1.1 alpha . . . 82
5.26.1.2 nx . . . 82
5.26.1.3 ny . . . 82
5.27 saturn::vof::Reconstruction3D Struct Reference . . . 82

5.27.1 Member Data Documentation . . . 82
5.27.1.1 alpha . . . 82
5.27.1.2 nx . . . 83
5.27.1.3 ny . . . 83
5.27.1.4 nz . . . 83
5.28 saturn::poisson::Result Struct Reference . . . 83
5.28.1 Member Data Documentation . . . 83
5.28.1.1 converged . . . 83
5.28.1.2 cycles . . . 83
5.28.1.3 final_residual . . . 83
5.28.1.4 initial_residual . . . 84
5.29 saturn::UniformGrid Struct Reference . . . 84
5.29.1 Member Function Documentation . . . 84
5.29.1.1 fromCells() . . . 84
5.29.1.2 fromCellsLocal() . . . 85
5.29.1.3 localBox() . . . 85
5.29.1.4 numCells() . . . 85
5.29.2 Member Data Documentation . . . 85
5.29.2.1 global . . . 85
5.29.2.2 h . . . 85
5.29.2.3 hi . . . 85
5.29.2.4 lo . . . 85
5.29.2.5 local . . . 85
5.29.2.6 ng . . . 86
5.30 saturn::vof::Vec3 Struct Reference . . . 86
5.30.1 Member Data Documentation . . . 86
5.30.1.1 x . . . 86
5.30.1.2 y . . . 86
5.30.1.3 z . . . 86

6 File Documentation **87**

6.1 apps/cavity_sim.cpp File Reference . . . 87
6.1.1 Detailed Description . . . 87
6.1.2 Macro Definition Documentation . . . 88
6.1.2.1 SATURN_DEFAULT_SPACE . . . 88
6.1.3 Function Documentation . . . 88
6.1.3.1 main() . . . 88
6.2 apps/channel_sim.cpp File Reference . . . 88
6.2.1 Macro Definition Documentation . . . 89
6.2.1.1 SATURN_DEFAULT_SPACE . . . 89
6.2.2 Function Documentation . . . 89
6.2.2.1 main() . . . 89

6.3 apps/compressible_mms.cpp File Reference . . . 89
6.3.1 Macro Definition Documentation . . . 90
6.3.1.1 SATURN_DEFAULT_SPACE . . . 90
6.3.2 Function Documentation . . . 90
6.3.2.1 main() . . . 90
6.4 apps/compressible_sim.cpp File Reference . . . 90
6.4.1 Macro Definition Documentation . . . 91
6.4.1.1 SATURN_DEFAULT_SPACE . . . 91
6.4.2 Function Documentation . . . 91
6.4.2.1 main() . . . 91
6.5 apps/edac_sim.cpp File Reference . . . 91
6.5.1 Macro Definition Documentation . . . 92
6.5.1.1 SATURN_DEFAULT_SPACE . . . 92
6.5.2 Function Documentation . . . 92
6.5.2.1 main() . . . 92
6.6 apps/euler1d.cpp File Reference . . . 92
6.6.1 Function Documentation . . . 92
6.6.1.1 main() . . . 92
6.7 apps/halo_bench.cpp File Reference . . . 93
6.7.1 Macro Definition Documentation . . . 93
6.7.1.1 SATURN_DEFAULT_SPACE . . . 93
6.7.2 Function Documentation . . . 93
6.7.2.1 main() . . . 93
6.8 apps/incompressible_sim.cpp File Reference . . . 94
6.8.1 Macro Definition Documentation . . . 94
6.8.1.1 SATURN_DEFAULT_SPACE . . . 94
6.8.2 Function Documentation . . . 94
6.8.2.1 main() . . . 94
6.9 apps/scalar_mms.cpp File Reference . . . 95
6.9.1 Macro Definition Documentation . . . 95
6.9.1.1 SATURN_DEFAULT_SPACE . . . 95
6.9.2 Function Documentation . . . 95
6.9.2.1 main() . . . 95
6.10 apps/stencil_bench.cpp File Reference . . . 96
6.10.1 Macro Definition Documentation . . . 96
6.10.1.1 SATURN_DEFAULT_SPACE . . . 96
6.10.2 Function Documentation . . . 96
6.10.2.1 main() . . . 96
6.11 apps/stream_bench.cpp File Reference . . . 96
6.11.1 Macro Definition Documentation . . . 97
6.11.1.1 SATURN_DEFAULT_SPACE . . . 97
6.11.2 Function Documentation . . . 97

6.11.2.1 main() . . . 97
6.12 src/core/execution_space.hpp File Reference . . . 97
6.13 execution_space.hpp . . . 98
6.14 src/core/mpi.hpp File Reference . . . 99
6.15 mpi.hpp . . . 99
6.16 src/core/parallel.hpp File Reference . . . 100
6.17 parallel.hpp . . . 100
6.18 src/core/version.hpp File Reference . . . 101
6.19 version.hpp . . . 101
6.20 src/fields/boundary_conditions.hpp File Reference . . . 101
6.21 boundary_conditions.hpp . . . 102
6.22 src/fields/patch_field.hpp File Reference . . . 103
6.23 patch_field.hpp . . . 104
6.24 src/halo/box_exchange.hpp File Reference . . . 105
6.25 box_exchange.hpp . . . 105
6.26 src/halo/face_exchange.hpp File Reference . . . 108
6.27 face_exchange.hpp . . . 108
6.28 src/halo/global_exchange.hpp File Reference . . . 111
6.29 global_exchange.hpp . . . 111
6.30 src/io/checkpoint.hpp File Reference . . . 113
6.31 checkpoint.hpp . . . 114
6.32 src/io/vtk_writer.hpp File Reference . . . 115
6.33 vtk_writer.hpp . . . 116
6.34 src/mesh/box.hpp File Reference . . . 118
6.35 box.hpp . . . 119
6.36 src/mesh/decomposition.hpp File Reference . . . 120
6.37 decomposition.hpp . . . 120
6.38 src/mesh/uniform_grid.hpp File Reference . . . 122
6.39 uniform_grid.hpp . . . 122
6.40 src/numerics/compressible/compressible_ns.hpp File Reference . . . 123
6.40.1 Detailed Description . . . 124
6.41 compressible_ns.hpp . . . 124
6.42 src/numerics/compressible/reconstruction.hpp File Reference . . . 131
6.42.1 Detailed Description . . . 132
6.43 reconstruction.hpp . . . 132
6.44 src/numerics/compressible/riemann.hpp File Reference . . . 135
6.45 riemann.hpp . . . 137
6.46 src/numerics/compressible/scalar_advection.hpp File Reference . . . 142
6.47 scalar_advection.hpp . . . 142
6.48 src/numerics/compressible/time_integration.hpp File Reference . . . 144
6.49 time_integration.hpp . . . 144
6.50 src/numerics/incompressible/cavity.hpp File Reference . . . 146

6.50.1 Detailed Description . . . 147
6.51 cavity.hpp . . . 148
6.52 src/numerics/incompressible/channel.hpp File Reference . . . 156
6.53 channel.hpp . . . 157
6.54 src/numerics/incompressible/channel_poisson.hpp File Reference . . . 161
6.55 channel_poisson.hpp . . . 161
6.56 src/numerics/incompressible/edac.hpp File Reference . . . 163
6.57 edac.hpp . . . 164
6.58 src/numerics/incompressible/fft_poisson.hpp File Reference . . . 166
6.59 fft_poisson.hpp . . . 166
6.60 src/numerics/incompressible/incompressible.hpp File Reference . . . 168
6.60.1 Detailed Description . . . 169
6.61 incompressible.hpp . . . 169
6.62 src/numerics/incompressible/poisson.hpp File Reference . . . 172
6.62.1 Detailed Description . . . 173
6.63 poisson.hpp . . . 173
6.64 src/numerics/incompressible/projection.hpp File Reference . . . 179
6.64.1 Detailed Description . . . 180
6.65 projection.hpp . . . 180
6.66 src/numerics/multiphase/dambreak2d.hpp File Reference . . . 181
6.66.1 Detailed Description . . . 181
6.67 dambreak2d.hpp . . . 182
6.68 src/numerics/multiphase/twophase2d.hpp File Reference . . . 189
6.69 twophase2d.hpp . . . 189
6.70 src/numerics/vof/vof2d.hpp File Reference . . . 195
6.70.1 Detailed Description . . . 196
6.71 vof2d.hpp . . . 196
6.72 src/numerics/vof/vof3d.hpp File Reference . . . 199
6.73 vof3d.hpp . . . 200
6.74 src/numerics/vof/vof_curvature2d.hpp File Reference . . . 204
6.75 vof_curvature2d.hpp . . . 205
6.76 src/numerics/vof/vof_curvature3d.hpp File Reference . . . 206
6.77 vof_curvature3d.hpp . . . 207
6.78 src/numerics/vof/vof_curvature_levelset2d.hpp File Reference . . . 208
6.79 vof_curvature_levelset2d.hpp . . . 208
6.80 src/numerics/vof/vof_redistribute.hpp File Reference . . . 209
6.81 vof_redistribute.hpp . . . 210

Index **211**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

saturn 7
saturn::cavity 15
saturn::channel 19
saturn::compressible 22
saturn::edac 26
saturn::incompressible 29
saturn::poisson 30
saturn::poisson::detail 31
saturn::projection 32
saturn::recon 34
saturn::riemann 38
saturn::vof 41

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

saturn::Box . . . 47
saturn::BoxExchange< ExecSpace > . . . 49
saturn::CheckpointHeader . . . 50
saturn::cavity::ClusteredPoisson2D
    Host variable-coefficient Poisson solver for the clustered 2D grid . . . 51
saturn::compressible::CompressibleWorkspace< ExecSpace > . . . 52
saturn::ConservativeState . . . 54
saturn::vof::Curvature2D . . . 55
saturn::vof::Curvature3D . . . 55
saturn::Decomposition . . . 56
saturn::FaceExchange< ExecSpace > . . . 58
saturn::Flux . . . 60
saturn::compressible::GasProperties . . . 61
saturn::poisson::GeometricMultigrid< ExecSpace >
    Geometric multigrid for the uniform-grid Poisson equation . . . 62
saturn::cavity::Geometry
    Grid, lid speed, advection scheme, and optional wall-clustering map . . . 65
saturn::channel::Geometry . . . 68
saturn::GlobalExchange< ExecSpace > . . . 69
saturn::vof::Grid2D . . . 70
saturn::vof::Grid3D . . . 72
saturn::poisson::GeometricMultigrid< ExecSpace >::Level . . . 75
saturn::vof::LevelSetCurvature . . . 76
saturn::poisson::Options
    Multigrid cycle and tolerance controls . . . 76
saturn::edac::Parameters . . . 77
saturn::PatchField< ExecSpace > . . . 78
saturn::vof::Point . . . 80
saturn::PrimitiveState . . . 81
saturn::vof::Reconstruction . . . 82
saturn::vof::Reconstruction3D . . . 82
saturn::poisson::Result . . . 83
saturn::UniformGrid . . . 84
saturn::vof::Vec3 . . . 86

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

apps/cavity_sim.cpp
    Lid-driven cavity driver: Ghia validation, GPU runs, clustering . . . 87
apps/channel_sim.cpp . . . 88
apps/compressible_mms.cpp . . . 89
apps/compressible_sim.cpp . . . 90
apps/edac_sim.cpp . . . 91
apps/euler1d.cpp . . . 92
apps/halo_bench.cpp . . . 93
apps/incompressible_sim.cpp . . . 94
apps/scalar_mms.cpp . . . 95
apps/stencil_bench.cpp . . . 96
apps/stream_bench.cpp . . . 96
src/core/execution_space.hpp . . . 97
src/core/mpi.hpp . . . 99
src/core/parallel.hpp . . . 100
src/core/version.hpp . . . 101
src/fields/boundary_conditions.hpp . . . 101
src/fields/patch_field.hpp . . . 103
src/halo/box_exchange.hpp . . . 105
src/halo/face_exchange.hpp . . . 108
src/halo/global_exchange.hpp . . . 111
src/io/checkpoint.hpp . . . 113
src/io/vtk_writer.hpp . . . 115
src/mesh/box.hpp . . . 118
src/mesh/decomposition.hpp . . . 120
src/mesh/uniform_grid.hpp . . . 122
src/numerics/compressible/compressible_ns.hpp
    Compressible Navier-Stokes fluxes and source terms . . . 123
src/numerics/compressible/reconstruction.hpp
    Face reconstruction: central, WENO5-Z, and TENO6 . . . 131
src/numerics/compressible/riemann.hpp . . . 135
src/numerics/compressible/scalar_advection.hpp . . . 142
src/numerics/compressible/time_integration.hpp . . . 144
src/numerics/incompressible/cavity.hpp
    Lid-driven cavity on a staggered MAC grid, uniform or wall-clustered . . . 146

src/numerics/incompressible/channel.hpp . . . 156
src/numerics/incompressible/channel_poisson.hpp . . . 161
src/numerics/incompressible/edac.hpp . . . 163
src/numerics/incompressible/fft_poisson.hpp . . . 166
src/numerics/incompressible/incompressible.hpp
Incompressible MAC momentum, filtering, and SSP-RK3 helpers . . . 168
src/numerics/incompressible/poisson.hpp
Cell-centred geometric-multigrid Poisson solver . . . 172
src/numerics/incompressible/projection.hpp
MAC velocity layout and discrete divergence/gradient operators . . . 179
src/numerics/multiphase/dambreak2d.hpp
Wall-bounded two-phase dam-break solver . . . 181
src/numerics/multiphase/twophase2d.hpp . . . 189
src/numerics/vof/vof2d.hpp
Two-dimensional PLIC volume-of-fluid reconstruction and advection . . . 195
src/numerics/vof/vof3d.hpp . . . 199
src/numerics/vof/vof_curvature2d.hpp . . . 204
src/numerics/vof/vof_curvature3d.hpp . . . 206
src/numerics/vof/vof_curvature_levelset2d.hpp . . . 208
src/numerics/vof/vof_redistribute.hpp . . . 209

Chapter 4

Namespace Documentation

4.1 saturn Namespace Reference

Namespaces

- namespace cavity
- namespace channel
- namespace compressible
- namespace edac
- namespace incompressible
- namespace poisson
- namespace projection
- namespace recon
- namespace riemann
- namespace vof

Classes

- struct Box
- class BoxExchange
- struct CheckpointHeader
- struct ConservativeState
- struct Decomposition
- class FaceExchange
- struct Flux
- class GlobalExchange
- class PatchField
- struct PrimitiveState
- struct UniformGrid

Typedefs

- template<class ExecSpace >
  using MDRange3 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<3>>
- template<class ExecSpace >
  using MDRange4 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<4>>
- using Int3 = Kokkos::Array<int, 3>
- using Int4 = Kokkos::Array<int, 4>

Enumerations

- enum class BoundaryType { Periodic , Transmissive , SlipWall }
- enum class Reconstruction {
  Central2 , Central4 , Central6 , Weno5Z ,
  Teno6 }
- enum class RiemannSolver { Llf , Hllc , HllcLowMach , AusmPlusUp }

Functions

- int unavailableBackend (const std::string &name)
- template<class F >
  int dispatchExecutionSpace (const std::string &requested, const char *default_space, F &&f)
- int mpiRank ()
- int mpiSize ()
- double mpiSum (double value)
- double mpiMax (double value)
- bool mpiAll (bool value)
- template<class ExecSpace >
  MDRange3< ExecSpace > makeMdrange3 (Int3 lower, Int3 upper)
- template<class ExecSpace >
  MDRange4< ExecSpace > makeMdrange4 (Int4 lower, Int4 upper)
- BoundaryType parseBoundaryType (const std::string &name)
- template<class ExecSpace >
  void applyBoundaryConditions (PatchField< ExecSpace > &q, const BoundaryType bc[3][2])
- template<class ExecSpace >
  bool writeCheckpoint (const std::string &base, const PatchField< ExecSpace > &q, int step, double time, const Decomposition &decomposition)
- template<class ExecSpace >
  bool readCheckpoint (const std::string &base, PatchField< ExecSpace > &q, int &step, double &time, const Decomposition &decomposition)
- std::string rankSuffix (int rank)
- void ensureParent (const std::string &path)
- template<class ExecSpace >
  bool writeVtiFields (const std::string &base, const PatchField< ExecSpace > &q, const std::vector< std↩
  ::string > &names, const std::array< double, 3 > &origin, const std::array< double, 3 > &spacing, const Box &global, double time)
- template<class ExecSpace >
  bool writeVti (const std::string &base, const PatchField< ExecSpace > &q, const std::array< double, 3 > &origin, const std::array< double, 3 > &spacing, const Box &global, double time)
- constexpr int reconstructionStencilWidth (Reconstruction scheme) noexcept
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION double scalarRhsValue (const PatchType &c, int i, int j, int k, double ux, double uy, double uz, double hx, double hy, double hz, double nu)
- template<class ExecSpace >
  void scalarAdvectionDiffusionRhs (const PatchField< ExecSpace > &c, PatchField< ExecSpace > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h, const Box &region)
- template<class ExecSpace >
  void scalarAdvectionDiffusionRhs (const PatchField< ExecSpace > &c, PatchField< ExecSpace > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void scalarAdvectionDiffusionRhsShell (const PatchField< ExecSpace > &c, PatchField< ExecSpace > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h, const Box &box)
- template<class Field >
  void fieldCombine (Field &destination, double a, const Field &x, double b, const Field &y)

- template<class Field >
  void fieldAddScaled (Field &destination, double scale, const Field &x)
- template<class Field >
  void fieldCopy (const Field &source, Field &destination)
- template<class Field , class ComputeRhs >
  void advanceSsprk3 (Field &q, Field &stage, Field &next, Field &rhs, double dt, double time, ComputeRhs &computeRhs)
- template<class Field , class ComputeRhs >
  void advanceRk4 (Field &q, Field &stage, Field &rhs, Field &k1, Field &k2, Field &k3, double dt, double time, ComputeRhs &computeRhs)

Variables

- constexpr int kInnerTile = 32
- template<class ExecSpace >
  constexpr bool is_host_execution_space
- constexpr const char * kProjectName = "saturnCFD"
- constexpr const char * kAuthor = "Vinaykanth"
- constexpr const char * kVersion = "0.1.0"

4.1.1 Typedef Documentation

4.1.1.1 Int3

```
using saturn::Int3 = Kokkos::Array<int, 3>
```

4.1.1.2 Int4

```
using saturn::Int4 = Kokkos::Array<int, 4>
```

4.1.1.3 MDRange3

```
template<class ExecSpace >
using saturn::MDRange3 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<3>>
```

4.1.1.4 MDRange4

```
template<class ExecSpace >
using saturn::MDRange4 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<4>>
```

4.1.2 Enumeration Type Documentation

4.1.2.1 BoundaryType

```
enum class saturn::BoundaryType  [strong]
```

Enumerator

Periodic	
Transmissive	
SlipWall	

4.1.2.2 Reconstruction

```
enum class saturn::Reconstruction  [strong]
```

Enumerator

Central2	
Central4	
Central6	
Weno5Z	
Teno6	

4.1.2.3 RiemannSolver

```
enum class saturn::RiemannSolver  [strong]
```

Enumerator

Llf	
Hllc	
HllcLowMach	
AusmPlusUp	

4.1.3 Function Documentation

4.1.3.1 advanceRk4()

```
template<class Field , class ComputeRhs >
void saturn::advanceRk4 (
            Field & q,
            Field & stage,
            Field & rhs,
            Field & k1,
            Field & k2,
            Field & k3,
            double dt,
            double time,
            ComputeRhs & computeRhs)
```

4.1.3.2 advanceSsprk3()

```
template<class Field , class ComputeRhs >
void saturn::advanceSsprk3 (
            Field & q,
            Field & stage,
            Field & next,
            Field & rhs,
            double dt,
            double time,
            ComputeRhs & computeRhs)
```

4.1.3.3 applyBoundaryConditions()

```
template<class ExecSpace >
void saturn::applyBoundaryConditions (
            PatchField< ExecSpace > & q,
            const BoundaryType bc[3][2])
```

4.1.3.4 dispatchExecutionSpace()

```
template<class F >
int saturn::dispatchExecutionSpace (
            const std::string & requested,
            const char * default_space,
            F && f)
```

4.1.3.5 ensureParent()

```
void saturn::ensureParent (
            const std::string & path)  [inline]
```

4.1.3.6 fieldAddScaled()

```
template<class Field >
void saturn::fieldAddScaled (
            Field & destination,
            double scale,
            const Field & x)
```

4.1.3.7 fieldCombine()

```
template<class Field >
void saturn::fieldCombine (
            Field & destination,
            double a,
            const Field & x,
            double b,
            const Field & y)
```

4.1.3.8 fieldCopy()

```
template<class Field >
void saturn::fieldCopy (
            const Field & source,
            Field & destination)
```

4.1.3.9 makeMdrange3()

```
template<class ExecSpace >
MDRange3< ExecSpace > saturn::makeMdrange3 (
            Int3 lower,
            Int3 upper)
```

4.1.3.10 makeMdrange4()

```
template<class ExecSpace >
MDRange4< ExecSpace > saturn::makeMdrange4 (
            Int4 lower,
            Int4 upper)
```

4.1.3.11 mpiAll()

```
bool saturn::mpiAll (
            bool value)  [inline]
```

4.1.3.12 mpiMax()

```
double saturn::mpiMax (
            double value)  [inline]
```

4.1.3.13 mpiRank()

```
int saturn::mpiRank ()  [inline]
```

4.1.3.14 mpiSize()

```
int saturn::mpiSize ()  [inline]
```

4.1.3.15 mpiSum()

```
double saturn::mpiSum (
            double value)  [inline]
```

4.1.3.16 parseBoundaryType()

```
BoundaryType saturn::parseBoundaryType (
            const std::string & name)  [inline]
```

4.1.3.17 rankSuffix()

```
std::string saturn::rankSuffix (
            int rank)  [inline]
```

4.1.3.18 readCheckpoint()

```
template<class ExecSpace >
bool saturn::readCheckpoint (
            const std::string & base,
            PatchField< ExecSpace > & q,
            int & step,
            double & time,
            const Decomposition & decomposition)
```

4.1.3.19 reconstructionStencilWidth()

```
int saturn::reconstructionStencilWidth (
            Reconstruction scheme)  [constexpr], [noexcept]
```

4.1.3.20 scalarAdvectionDiffusionRhs() [1/2]

```
template<class ExecSpace >
void saturn::scalarAdvectionDiffusionRhs (
            const PatchField< ExecSpace > & c,
            PatchField< ExecSpace > & rhs,
            const std::array< double, 3 > & velocity,
            double nu,
            const std::array< double, 3 > & h)
```

4.1.3.21 scalarAdvectionDiffusionRhs() [2/2]

```
template<class ExecSpace >
void saturn::scalarAdvectionDiffusionRhs (
            const PatchField< ExecSpace > & c,
            PatchField< ExecSpace > & rhs,
            const std::array< double, 3 > & velocity,
            double nu,
            const std::array< double, 3 > & h,
            const Box & region)
```

4.1.3.22 scalarAdvectionDiffusionRhsShell()

```
template<class ExecSpace >
void saturn::scalarAdvectionDiffusionRhsShell (
            const PatchField< ExecSpace > & c,
            PatchField< ExecSpace > & rhs,
            const std::array< double, 3 > & velocity,
            double nu,
            const std::array< double, 3 > & h,
            const Box & box)
```

4.1.3.23 scalarRhsValue()

```
template<class PatchType >
KOKKOS_INLINE_FUNCTION double saturn::scalarRhsValue (
            const PatchType & c,
            int i,
            int j,
            int k,
            double ux,
            double uy,
            double uz,
            double hx,
            double hy,
            double hz,
            double nu)
```

4.1.3.24 unavailableBackend()

```
int saturn::unavailableBackend (
            const std::string & name)  [inline]
```

4.1.3.25 writeCheckpoint()

```
template<class ExecSpace >
bool saturn::writeCheckpoint (
            const std::string & base,
            const PatchField< ExecSpace > & q,
            int step,
            double time,
            const Decomposition & decomposition)
```

4.1.3.26 writeVti()

```
template<class ExecSpace >
bool saturn::writeVti (
            const std::string & base,
            const PatchField< ExecSpace > & q,
            const std::array< double, 3 > & origin,
            const std::array< double, 3 > & spacing,
            const Box & global,
            double time)
```

4.1.3.27 writeVtiFields()

```
template<class ExecSpace >
bool saturn::writeVtiFields (
            const std::string & base,
            const PatchField< ExecSpace > & q,
            const std::vector< std::string > & names,
            const std::array< double, 3 > & origin,
            const std::array< double, 3 > & spacing,
            const Box & global,
            double time)
```

4.1.4 Variable Documentation

4.1.4.1 is_host_execution_space

```
template<class ExecSpace >
bool saturn::is_host_execution_space  [inline], [constexpr]
```

Initial value:

```
=
    Kokkos::SpaceAccessibility<Kokkos::HostSpace,
                               typename ExecSpace::memory_space>::accessible
```

4.1.4.2 kAuthor

```
const char* saturn::kAuthor = "Vinaykanth"  [inline], [constexpr]
```

4.1.4.3 kInnerTile

```
int saturn::kInnerTile = 32  [inline], [constexpr]
```

4.1.4.4 kProjectName

```
const char* saturn::kProjectName = "saturnCFD"  [inline], [constexpr]
```

4.1.4.5 kVersion

```
const char* saturn::kVersion = "0.1.0"  [inline], [constexpr]
```

4.2 saturn::cavity Namespace Reference

Classes

- class ClusteredPoisson2D

  Host variable-coefficient Poisson solver for the clustered 2D grid.
- struct Geometry

  Grid, lid speed, advection scheme, and optional wall-clustering map.

Functions

- KOKKOS_INLINE_FUNCTION double minmod (double a, double b)
- KOKKOS_INLINE_FUNCTION double weno5Derivative (double qm3, double qm2, double qm1, double q0, double qp1, double qp2, double qp3, double vel, double h)
- KOKKOS_INLINE_FUNCTION double limitedUpwind (double q0, double qm1, double qm2, double qp1, double qp2, double vel, double h)
- template<class ExecSpace >
  void divergence (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &div, const Geometry &g)
- template<class ExecSpace >
  void momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const Geometry &g, double nu)

  Momentum right-hand side on the MAC faces.
- template<class ExecSpace >
  void filterVelocity (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u_out, projection::ScalarView< ExecSpace > &v_out, projection::ScalarView< ExecSpace > &w_out, const Geometry &g, double alpha)

  Weak low-pass/dealiasing filter applied to the velocity field.
- template<class ExecSpace >
  void projectClustered (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g)

  Pressure projection on the wall-clustered 2D grid.
- template<class ExecSpace , class PoissonSolver >
  poisson::Result project (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g, PoissonSolver &solver, const poisson::Options &options)

  MAC pressure projection on the uniform grid.
- template<class ExecSpace , class Project >
  void advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const Geometry &g, double nu, double dt, Project &project)

  SSP-RK3 step with a pressure projection after every stage.

4.2.1 Function Documentation

4.2.1.1 advanceSsprk3()

```
template<class ExecSpace , class Project >
void saturn::cavity::advanceSsprk3 (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & u1,
            projection::ScalarView< ExecSpace > & v1,
```

```
            projection::ScalarView< ExecSpace > & w1,
            projection::ScalarView< ExecSpace > & u2,
            projection::ScalarView< ExecSpace > & v2,
            projection::ScalarView< ExecSpace > & w2,
            projection::ScalarView< ExecSpace > & ku,
            projection::ScalarView< ExecSpace > & kv,
            projection::ScalarView< ExecSpace > & kw,
            const Geometry & g,
            double nu,
            double dt,
            Project & project)
```

SSP-RK3 step with a pressure projection after every stage.

4.2.1.2 divergence()

```
template<class ExecSpace >
void saturn::cavity::divergence (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & div,
            const Geometry & g)
```

4.2.1.3 filterVelocity()

```
template<class ExecSpace >
void saturn::cavity::filterVelocity (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & u_out,
            projection::ScalarView< ExecSpace > & v_out,
            projection::ScalarView< ExecSpace > & w_out,
            const Geometry & g,
            double alpha)
```

Weak low-pass/dealiasing filter applied to the velocity field.

4.2.1.4 limitedUpwind()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::limitedUpwind (
            double q0,
            double qm1,
            double qm2,
            double qp1,
            double qp2,
            double vel,
            double h)
```

4.2.1.5 minmod()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::minmod (
            double a,
            double b)
```

4.2.1.6 momentumRhs()

```
template<class ExecSpace >
void saturn::cavity::momentumRhs (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & du,
            projection::ScalarView< ExecSpace > & dv,
            projection::ScalarView< ExecSpace > & dw,
            const Geometry & g,
            double nu)
```

Momentum right-hand side on the MAC faces.

Parameters

u,v,w	Face-centred velocities.
du,dv,dw	Output momentum tendencies.
g	Geometry, including the optional clustering map.
nu	Kinematic viscosity.

4.2.1.7 project()

```
template<class ExecSpace , class PoissonSolver >
poisson::Result saturn::cavity::project (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & phi,
            projection::ScalarView< ExecSpace > & scratch,
            const Geometry & g,
            PoissonSolver & solver,
            const poisson::Options & options)
```

MAC pressure projection on the uniform grid.

Parameters

solver	Geometric multigrid used for the pressure Poisson equation.

4.2.1.8 projectClustered()

```
template<class ExecSpace >
void saturn::cavity::projectClustered (
             projection::ScalarView< ExecSpace > & u,
             projection::ScalarView< ExecSpace > & v,
             projection::ScalarView< ExecSpace > & w,
             projection::ScalarView< ExecSpace > & phi,
             projection::ScalarView< ExecSpace > & scratch,
             const Geometry & g)
```

Pressure projection on the wall-clustered 2D grid.

The pressure solve is host-side; the velocity update is on the device.

4.2.1.9 weno5Derivative()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::weno5Derivative (
             double qm3,
             double qm2,
             double qm1,
             double q0,
             double qp1,
             double qp2,
             double qp3,
             double vel,
             double h)
```

4.3 saturn::channel Namespace Reference

Classes

- struct Geometry

Functions

- template<class ExecSpace >
  void divergence (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &div, const Geometry &g)
- template<class ExecSpace >
  void momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const Geometry &g, double nu)
- template<class ExecSpace , class PoissonSolver >
  poisson::Result project (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g, PoissonSolver &solver, const poisson::Options &options)

- template<class ExecSpace , class Project >
  void advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< Exec↩ Space > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const Geometry &g, double nu, double dt, Project &project)
- template<class ExecSpace >
  void filterDimension (const projection::ScalarView< ExecSpace > &in, projection::ScalarView< ExecSpace > &out, double alpha, int dimension, int component, const Geometry &g)
- template<class ExecSpace >
  void filter4Channel (projection::ScalarView< ExecSpace > &u, projection::ScalarView< Exec↩ Space > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &tu, projection::ScalarView< ExecSpace > &tv, projection::ScalarView< ExecSpace > &tw, const Geometry &g, double alpha)

4.3.1 Function Documentation

4.3.1.1 advanceSsprk3()

```
template<class ExecSpace , class Project >
void saturn::channel::advanceSsprk3 (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & u1,
            projection::ScalarView< ExecSpace > & v1,
            projection::ScalarView< ExecSpace > & w1,
            projection::ScalarView< ExecSpace > & u2,
            projection::ScalarView< ExecSpace > & v2,
            projection::ScalarView< ExecSpace > & w2,
            projection::ScalarView< ExecSpace > & ku,
            projection::ScalarView< ExecSpace > & kv,
            projection::ScalarView< ExecSpace > & kw,
            const Geometry & g,
            double nu,
            double dt,
            Project & project)
```

4.3.1.2 divergence()

```
template<class ExecSpace >
void saturn::channel::divergence (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & div,
            const Geometry & g)
```

4.3.1.3 filter4Channel()

```
template<class ExecSpace >
void saturn::channel::filter4Channel (
             projection::ScalarView< ExecSpace > & u,
             projection::ScalarView< ExecSpace > & v,
             projection::ScalarView< ExecSpace > & w,
             projection::ScalarView< ExecSpace > & tu,
             projection::ScalarView< ExecSpace > & tv,
             projection::ScalarView< ExecSpace > & tw,
             const Geometry & g,
             double alpha)
```

4.3.1.4 filterDimension()

```
template<class ExecSpace >
void saturn::channel::filterDimension (
             const projection::ScalarView< ExecSpace > & in,
             projection::ScalarView< ExecSpace > & out,
             double alpha,
             int dimension,
             int component,
             const Geometry & g)
```

4.3.1.5 momentumRhs()

```
template<class ExecSpace >
void saturn::channel::momentumRhs (
             const projection::ScalarView< ExecSpace > & u,
             const projection::ScalarView< ExecSpace > & v,
             const projection::ScalarView< ExecSpace > & w,
             projection::ScalarView< ExecSpace > & du,
             projection::ScalarView< ExecSpace > & dv,
             projection::ScalarView< ExecSpace > & dw,
             const Geometry & g,
             double nu)
```

4.3.1.6 project()

```
template<class ExecSpace , class PoissonSolver >
poisson::Result saturn::channel::project (
             projection::ScalarView< ExecSpace > & u,
             projection::ScalarView< ExecSpace > & v,
             projection::ScalarView< ExecSpace > & w,
             projection::ScalarView< ExecSpace > & phi,
             projection::ScalarView< ExecSpace > & scratch,
             const Geometry & g,
             PoissonSolver & solver,
             const poisson::Options & options)
```

4.4 saturn::compressible Namespace Reference

Classes

- class CompressibleWorkspace
- struct GasProperties

Functions

- KOKKOS_INLINE_FUNCTION double soundSpeed (const PrimitiveState &state, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION double temperature (const PrimitiveState &state, const GasProperties &gas)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION PrimitiveState loadPrimitive (const PatchType &q, int i, int j, int k, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION PrimitiveState toNormal (const PrimitiveState &state, int direction)
- KOKKOS_INLINE_FUNCTION Flux fromNormal (const Flux &flux, int direction)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION void reconstructFace (const PatchType &q, int i, int j, int k, int direction, Reconstruction scheme, const GasProperties &gas, PrimitiveState &left, PrimitiveState &right)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION Flux inviscidFaceFlux (const PatchType &q, int i, int j, int k, int direction, Reconstruction scheme, RiemannSolver solver, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION double central4Derivative (double minus2, double minus1, double plus1, double plus2, double h)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION void gradientLine (const PatchType &q, int i, int j, int k, int direction, double h, const GasProperties &gas, double derivative[4])
- template<class ExecSpace >
  void computeGradients (const PatchField< ExecSpace > &q, PatchField< ExecSpace > &grad, const UniformGrid &grid, const GasProperties &gas, const Box &region)
- template<class ExecSpace >
  void computeStress (const PatchField< ExecSpace > &grad, PatchField< ExecSpace > &stress, const GasProperties &gas, const Box &region)
- template<class ExecSpace >
  void computeFaceFluxesX (const PatchField< ExecSpace > &q, const CompressibleWorkspace< Exec↩ Space > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class ExecSpace >
  void computeFaceFluxesY (const PatchField< ExecSpace > &q, const CompressibleWorkspace< Exec↩ Space > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class ExecSpace >
  void computeFaceFluxesZ (const PatchField< ExecSpace > &q, const CompressibleWorkspace< Exec↩ Space > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class FieldType >
  KOKKOS_INLINE_FUNCTION double fieldDerivative (const FieldType &field, int component, int i, int j, int k, int di, int dj, int dk, double h)
- template<class ExecSpace >
  void assembleRhs (const PatchField< ExecSpace > &q, const CompressibleWorkspace< ExecSpace > &workspace, PatchField< ExecSpace > &rhs, const UniformGrid &grid, const GasProperties &gas, bool dissipation_sink)
- template<class ExecSpace >
  void compressibleRhs (const PatchField< ExecSpace > &q, CompressibleWorkspace< ExecSpace > &workspace, PatchField< ExecSpace > &rhs, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver, bool dissipation_sink=false)

- template<class ExecSpace >
  double maxWaveSpeed (const PatchField< ExecSpace > &q, const GasProperties &gas)
- template<class ExecSpace >
  double maxKinematicDiffusivity (const PatchField< ExecSpace > &q, const GasProperties &gas)

4.4.1 Function Documentation

4.4.1.1 assembleRhs()

```
template<class ExecSpace >
void saturn::compressible::assembleRhs (
            const PatchField< ExecSpace > & q,
            const CompressibleWorkspace< ExecSpace > & workspace,
            PatchField< ExecSpace > & rhs,
            const UniformGrid & grid,
            const GasProperties & gas,
            bool dissipation_sink)
```

4.4.1.2 central4Derivative()

```
KOKKOS_INLINE_FUNCTION double saturn::compressible::central4Derivative (
            double minus2,
            double minus1,
            double plus1,
            double plus2,
            double h)
```

4.4.1.3 compressibleRhs()

```
template<class ExecSpace >
void saturn::compressible::compressibleRhs (
            const PatchField< ExecSpace > & q,
            CompressibleWorkspace< ExecSpace > & workspace,
            PatchField< ExecSpace > & rhs,
            const UniformGrid & grid,
            const GasProperties & gas,
            Reconstruction scheme,
            RiemannSolver solver,
            bool dissipation_sink = false)
```

4.4.1.4 computeFaceFluxesX()

```
template<class ExecSpace >
void saturn::compressible::computeFaceFluxesX (
            const PatchField< ExecSpace > & q,
            const CompressibleWorkspace< ExecSpace > & workspace,
            const UniformGrid & grid,
            const GasProperties & gas,
            Reconstruction scheme,
            RiemannSolver solver)
```

4.4.1.5 computeFaceFluxesY()

```
template<class ExecSpace >
void saturn::compressible::computeFaceFluxesY (
             const PatchField< ExecSpace > & q,
             const CompressibleWorkspace< ExecSpace > & workspace,
             const UniformGrid & grid,
             const GasProperties & gas,
             Reconstruction scheme,
             RiemannSolver solver)
```

4.4.1.6 computeFaceFluxesZ()

```
template<class ExecSpace >
void saturn::compressible::computeFaceFluxesZ (
             const PatchField< ExecSpace > & q,
             const CompressibleWorkspace< ExecSpace > & workspace,
             const UniformGrid & grid,
             const GasProperties & gas,
             Reconstruction scheme,
             RiemannSolver solver)
```

4.4.1.7 computeGradients()

```
template<class ExecSpace >
void saturn::compressible::computeGradients (
             const PatchField< ExecSpace > & q,
             PatchField< ExecSpace > & grad,
             const UniformGrid & grid,
             const GasProperties & gas,
             const Box & region)
```

4.4.1.8 computeStress()

```
template<class ExecSpace >
void saturn::compressible::computeStress (
             const PatchField< ExecSpace > & grad,
             PatchField< ExecSpace > & stress,
             const GasProperties & gas,
             const Box & region)
```

4.4.1.9 fieldDerivative()

```
template<class FieldType >
KOKKOS_INLINE_FUNCTION double saturn::compressible::fieldDerivative (
             const FieldType & field,
             int component,
             int i,
             int j,
             int k,
             int di,
             int dj,
             int dk,
             double h)
```

4.4.1.10 fromNormal()

```
KOKKOS_INLINE_FUNCTION Flux saturn::compressible::fromNormal (
            const Flux & flux,
            int direction)
```

4.4.1.11 gradientLine()

```
template<class PatchType >
KOKKOS_INLINE_FUNCTION void saturn::compressible::gradientLine (
            const PatchType & q,
            int i,
            int j,
            int k,
            int direction,
            double h,
            const GasProperties & gas,
            double derivative[4])
```

4.4.1.12 inviscidFaceFlux()

```
template<class PatchType >
KOKKOS_INLINE_FUNCTION Flux saturn::compressible::inviscidFaceFlux (
            const PatchType & q,
            int i,
            int j,
            int k,
            int direction,
            Reconstruction scheme,
            RiemannSolver solver,
            const GasProperties & gas)
```

4.4.1.13 loadPrimitive()

```
template<class PatchType >
KOKKOS_INLINE_FUNCTION PrimitiveState saturn::compressible::loadPrimitive (
            const PatchType & q,
            int i,
            int j,
            int k,
            const GasProperties & gas)
```

4.4.1.14 maxKinematicDiffusivity()

```
template<class ExecSpace >
double saturn::compressible::maxKinematicDiffusivity (
            const PatchField< ExecSpace > & q,
            const GasProperties & gas)
```

4.4.1.15 maxWaveSpeed()

```
template<class ExecSpace >
double saturn::compressible::maxWaveSpeed (
            const PatchField< ExecSpace > & q,
            const GasProperties & gas)
```

4.4.1.16 reconstructFace()

```
template<class PatchType >
KOKKOS_INLINE_FUNCTION void saturn::compressible::reconstructFace (
            const PatchType & q,
            int i,
            int j,
            int k,
            int direction,
            Reconstruction scheme,
            const GasProperties & gas,
            PrimitiveState & left,
            PrimitiveState & right)
```

4.4.1.17 soundSpeed()

```
KOKKOS_INLINE_FUNCTION double saturn::compressible::soundSpeed (
            const PrimitiveState & state,
            const GasProperties & gas)
```

4.4.1.18 temperature()

```
KOKKOS_INLINE_FUNCTION double saturn::compressible::temperature (
            const PrimitiveState & state,
            const GasProperties & gas)
```

4.4.1.19 toNormal()

```
KOKKOS_INLINE_FUNCTION PrimitiveState saturn::compressible::toNormal (
            const PrimitiveState & state,
            int direction)
```

4.5 saturn::edac Namespace Reference

Classes

- struct Parameters

Functions

- template<class ExecSpace >
  void laplacian (const projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void subtract (projection::ScalarView< ExecSpace > &a, const projection::ScalarView< ExecSpace > &b)
- template<class ExecSpace >
  void velocityRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, const projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, projection::ScalarView< ExecSpace > &gx, projection::ScalarView< ExecSpace > &gy, projection::ScalarView< ExecSpace > &gz, const std::array< double, 3 > &h, double nu)
- template<class ExecSpace >
  void pressureRhs (const projection::ScalarView< ExecSpace > &p, const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &dp, projection::ScalarView< ExecSpace > &div, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h, const Parameters &params)
- template<class ExecSpace >
  void advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &p1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &p2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, projection::ScalarView< ExecSpace > &kp, projection::ScalarView< ExecSpace > &gx, projection::ScalarView< ExecSpace > &gy, projection::ScalarView< ExecSpace > &gz, projection::ScalarView< ExecSpace > &div, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h, const Parameters &params, double dt)

4.5.1 Function Documentation

4.5.1.1 advanceSsprk3()

```
template<class ExecSpace >
void saturn::edac::advanceSsprk3 (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & p,
            projection::ScalarView< ExecSpace > & u1,
            projection::ScalarView< ExecSpace > & v1,
            projection::ScalarView< ExecSpace > & w1,
            projection::ScalarView< ExecSpace > & p1,
            projection::ScalarView< ExecSpace > & u2,
            projection::ScalarView< ExecSpace > & v2,
            projection::ScalarView< ExecSpace > & w2,
            projection::ScalarView< ExecSpace > & p2,
            projection::ScalarView< ExecSpace > & ku,
            projection::ScalarView< ExecSpace > & kv,
            projection::ScalarView< ExecSpace > & kw,
            projection::ScalarView< ExecSpace > & kp,
            projection::ScalarView< ExecSpace > & gx,
            projection::ScalarView< ExecSpace > & gy,
```

```
            projection::ScalarView< ExecSpace > & gz,
            projection::ScalarView< ExecSpace > & div,
            projection::ScalarView< ExecSpace > & lap,
            const std::array< double, 3 > & h,
            const Parameters & params,
            double dt)
```

4.5.1.2 laplacian()

```
template<class ExecSpace >
void saturn::edac::laplacian (
            const projection::ScalarView< ExecSpace > & p,
            projection::ScalarView< ExecSpace > & lap,
            const std::array< double, 3 > & h)
```

4.5.1.3 pressureRhs()

```
template<class ExecSpace >
void saturn::edac::pressureRhs (
            const projection::ScalarView< ExecSpace > & p,
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & dp,
            projection::ScalarView< ExecSpace > & div,
            projection::ScalarView< ExecSpace > & lap,
            const std::array< double, 3 > & h,
            const Parameters & params)
```

4.5.1.4 subtract()

```
template<class ExecSpace >
void saturn::edac::subtract (
            projection::ScalarView< ExecSpace > & a,
            const projection::ScalarView< ExecSpace > & b)
```

4.5.1.5 velocityRhs()

```
template<class ExecSpace >
void saturn::edac::velocityRhs (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            const projection::ScalarView< ExecSpace > & p,
            projection::ScalarView< ExecSpace > & du,
            projection::ScalarView< ExecSpace > & dv,
            projection::ScalarView< ExecSpace > & dw,
            projection::ScalarView< ExecSpace > & gx,
            projection::ScalarView< ExecSpace > & gy,
            projection::ScalarView< ExecSpace > & gz,
            const std::array< double, 3 > & h,
            double nu)
```

4.6 saturn::incompressible Namespace Reference

Functions

- template<class ExecSpace >
  void momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< Exec↩ Space > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const std::array< double, 3 > &h, double nu)
- template<class ExecSpace >
  void combine (const projection::ScalarView< ExecSpace > &a, double alpha, const projection::ScalarView< ExecSpace > &b, double beta, const projection::ScalarView< ExecSpace > &c, double gamma, projection::ScalarView< ExecSpace > &out)
- template<class ExecSpace >
  void filterDimension (const projection::ScalarView< ExecSpace > &in, projection::ScalarView< ExecSpace > &out, double alpha, int dimension)
- template<class ExecSpace >
  void filter4 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &tu, projection::ScalarView< ExecSpace > &tv, projection::ScalarView< ExecSpace > &tw, double alpha)
- template<class ExecSpace , class Project >
  void advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< Exec↩ Space > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const std::array< double, 3 > &h, double nu, double dt, Project &project)

4.6.1 Function Documentation

4.6.1.1 advanceSsprk3()

```
template<class ExecSpace , class Project >
void saturn::incompressible::advanceSsprk3 (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & u1,
            projection::ScalarView< ExecSpace > & v1,
            projection::ScalarView< ExecSpace > & w1,
            projection::ScalarView< ExecSpace > & u2,
            projection::ScalarView< ExecSpace > & v2,
            projection::ScalarView< ExecSpace > & w2,
            projection::ScalarView< ExecSpace > & ku,
            projection::ScalarView< ExecSpace > & kv,
            projection::ScalarView< ExecSpace > & kw,
            const std::array< double, 3 > & h,
            double nu,
            double dt,
            Project & project)
```

4.6.1.2 combine()

```
template<class ExecSpace >
void saturn::incompressible::combine (
            const projection::ScalarView< ExecSpace > & a,
            double alpha,
            const projection::ScalarView< ExecSpace > & b,
            double beta,
            const projection::ScalarView< ExecSpace > & c,
            double gamma,
            projection::ScalarView< ExecSpace > & out)
```

4.6.1.3 filter4()

```
template<class ExecSpace >
void saturn::incompressible::filter4 (
            projection::ScalarView< ExecSpace > & u,
            projection::ScalarView< ExecSpace > & v,
            projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & tu,
            projection::ScalarView< ExecSpace > & tv,
            projection::ScalarView< ExecSpace > & tw,
            double alpha)
```

4.6.1.4 filterDimension()

```
template<class ExecSpace >
void saturn::incompressible::filterDimension (
            const projection::ScalarView< ExecSpace > & in,
            projection::ScalarView< ExecSpace > & out,
            double alpha,
            int dimension)
```

4.6.1.5 momentumRhs()

```
template<class ExecSpace >
void saturn::incompressible::momentumRhs (
            const projection::ScalarView< ExecSpace > & u,
            const projection::ScalarView< ExecSpace > & v,
            const projection::ScalarView< ExecSpace > & w,
            projection::ScalarView< ExecSpace > & du,
            projection::ScalarView< ExecSpace > & dv,
            projection::ScalarView< ExecSpace > & dw,
            const std::array< double, 3 > & h,
            double nu)
```

4.7 saturn::poisson Namespace Reference

Namespaces

- namespace detail

Classes

- class GeometricMultigrid

  Geometric multigrid for the uniform-grid Poisson equation.
- struct Options

  Multigrid cycle and tolerance controls.
- struct Result

Enumerations

- enum class Boundary { Periodic , Neumann , Dirichlet }

4.7.1 Enumeration Type Documentation

4.7.1.1 Boundary

```
enum class saturn::poisson::Boundary  [strong]
```

Enumerator

Periodic	
Neumann	
Dirichlet	

4.8 saturn::poisson::detail Namespace Reference

Functions

- KOKKOS_INLINE_FUNCTION int plusIndex (int i, int n, Boundary bc, int &sign)
- KOKKOS_INLINE_FUNCTION int minusIndex (int i, int n, Boundary bc, int &sign)
- KOKKOS_INLINE_FUNCTION int extendedIndex (int index, int n, Boundary low, Boundary high, int &sign)

4.8.1 Function Documentation

4.8.1.1 extendedIndex()

```
KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::extendedIndex (
            int index,
            int n,
            Boundary low,
            Boundary high,
            int & sign)
```

4.8.1.2 minusIndex()

```
KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::minusIndex (
            int i,
            int n,
            Boundary bc,
            int & sign)
```

4.8.1.3 plusIndex()

```
KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::plusIndex (
            int i,
            int n,
            Boundary bc,
            int & sign)
```

4.9 saturn::projection Namespace Reference

Typedefs

- template<class ExecSpace >
  using ScalarView

Functions

- template<class ExecSpace >
  void divergence (const ScalarView< ExecSpace > &u, const ScalarView< ExecSpace > &v, const ScalarView< ExecSpace > &w, ScalarView< ExecSpace > &div, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void gradientX (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gx, double hx)
- template<class ExecSpace >
  void gradientY (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gy, double hy)
- template<class ExecSpace >
  void gradientZ (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gz, double hz)
- template<class ExecSpace , class PoissonSolver >
  poisson::Result project (ScalarView< ExecSpace > &u, ScalarView< ExecSpace > &v, ScalarView< ExecSpace > &w, ScalarView< ExecSpace > &phi, ScalarView< ExecSpace > &scratch, const std::array< double, 3 > &h, double dt, PoissonSolver &solver, const poisson::Options &options)

4.9.1 Typedef Documentation

4.9.1.1 ScalarView

```
template<class ExecSpace >
using saturn::projection::ScalarView
```

Initial value:

```
 Kokkos::View<double***, Kokkos::LayoutRight,
                            typename ExecSpace::memory_space>
```

4.9.2 Function Documentation

4.9.2.1 divergence()

```
template<class ExecSpace >
void saturn::projection::divergence (
            const ScalarView< ExecSpace > & u,
            const ScalarView< ExecSpace > & v,
            const ScalarView< ExecSpace > & w,
            ScalarView< ExecSpace > & div,
            const std::array< double, 3 > & h)
```

4.9.2.2 gradientX()

```
template<class ExecSpace >
void saturn::projection::gradientX (
            const ScalarView< ExecSpace > & p,
            ScalarView< ExecSpace > & gx,
            double hx)
```

4.9.2.3 gradientY()

```
template<class ExecSpace >
void saturn::projection::gradientY (
            const ScalarView< ExecSpace > & p,
            ScalarView< ExecSpace > & gy,
            double hy)
```

4.9.2.4 gradientZ()

```
template<class ExecSpace >
void saturn::projection::gradientZ (
            const ScalarView< ExecSpace > & p,
            ScalarView< ExecSpace > & gz,
            double hz)
```

4.9.2.5 project()

```
template<class ExecSpace , class PoissonSolver >
poisson::Result saturn::projection::project (
            ScalarView< ExecSpace > & u,
            ScalarView< ExecSpace > & v,
            ScalarView< ExecSpace > & w,
            ScalarView< ExecSpace > & phi,
            ScalarView< ExecSpace > & scratch,
            const std::array< double, 3 > & h,
            double dt,
            PoissonSolver & solver,
            const poisson::Options & options)
```

4.10 saturn::recon Namespace Reference

Functions

- KOKKOS_INLINE_FUNCTION double central2 (double, double, double v0, double vp1, double, double)
- KOKKOS_INLINE_FUNCTION double central4 (double, double vm1, double v0, double vp1, double vp2, double)
- KOKKOS_INLINE_FUNCTION double central6 (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION void weno5Smoothness (double vm2, double vm1, double v0, double vp1, double vp2, double &beta0, double &beta1, double &beta2)
- KOKKOS_INLINE_FUNCTION double weno5z (double vm2, double vm1, double v0, double vp1, double vp2, double)
- KOKKOS_INLINE_FUNCTION double stencil6Smoothness (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double tenoInverseSmoothness (double normalized_beta)
- KOKKOS_INLINE_FUNCTION double teno6 (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double reconstructLeft (Reconstruction scheme, double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double reconstructRight (Reconstruction scheme, double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- template<class Accessor >
  KOKKOS_INLINE_FUNCTION double reconstructLeft (Reconstruction scheme, const Accessor &accessor)
- template<class Accessor >
  KOKKOS_INLINE_FUNCTION double reconstructRight (Reconstruction scheme, const Accessor &accessor)

Variables

- constexpr double kEpsilon = 1.0e-40
- constexpr double kTenoCutoff = 1.0e-5
- constexpr double kTenoExponent = 7.0
- constexpr double kWenoWeight0 = 1.0 / 10.0
- constexpr double kWenoWeight1 = 3.0 / 5.0
- constexpr double kWenoWeight2 = 3.0 / 10.0

4.10.1 Function Documentation

4.10.1.1 central2()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::central2 (
            double ,
            double ,
            double v0,
            double vp1,
            double ,
            double )
```

4.10.1.2 central4()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::central4 (
            double ,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double )
```

4.10.1.3 central6()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::central6 (
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double vp3)
```

4.10.1.4 reconstructLeft() [1/2]

```
template<class Accessor >
KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructLeft (
            Reconstruction scheme,
            const Accessor & accessor)
```

4.10.1.5 reconstructLeft() [2/2]

```
KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructLeft (
            Reconstruction scheme,
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double vp3)
```

4.10.1.6 reconstructRight() [1/2]

```
template<class Accessor >
KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructRight (
            Reconstruction scheme,
            const Accessor & accessor)
```

4.10.1.7 reconstructRight() [2/2]

```
KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructRight (
            Reconstruction scheme,
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double vp3)
```

4.10.1.8 stencil6Smoothness()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::stencil6Smoothness (
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double vp3)
```

4.10.1.9 teno6()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::teno6 (
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double vp3)
```

4.10.1.10 tenoInverseSmoothness()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::tenoInverseSmoothness (
            double normalized_beta)
```

4.10.1.11 weno5Smoothness()

```
KOKKOS_INLINE_FUNCTION void saturn::recon::weno5Smoothness (
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double & beta0,
            double & beta1,
            double & beta2)
```

4.10.1.12 weno5z()

```
KOKKOS_INLINE_FUNCTION double saturn::recon::weno5z (
            double vm2,
            double vm1,
            double v0,
            double vp1,
            double vp2,
            double )
```

4.10.2 Variable Documentation

4.10.2.1 kEpsilon

```
double saturn::recon::kEpsilon = 1.0e-40  [inline], [constexpr]
```

4.10.2.2 kTenoCutoff

```
double saturn::recon::kTenoCutoff = 1.0e-5  [inline], [constexpr]
```

4.10.2.3 kTenoExponent

```
double saturn::recon::kTenoExponent = 7.0  [inline], [constexpr]
```

4.10.2.4 kWenoWeight0

```
double saturn::recon::kWenoWeight0 = 1.0 / 10.0  [inline], [constexpr]
```

4.10.2.5 kWenoWeight1

```
double saturn::recon::kWenoWeight1 = 3.0 / 5.0  [inline], [constexpr]
```

4.10.2.6 kWenoWeight2

```
double saturn::recon::kWenoWeight2 = 3.0 / 10.0  [inline], [constexpr]
```

4.11 saturn::riemann Namespace Reference

Functions

- KOKKOS_INLINE_FUNCTION bool valid (const PrimitiveState &state)
- KOKKOS_INLINE_FUNCTION double maxSignalSpeed (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION ConservativeState toConservative (const PrimitiveState &state, double gamma)
- KOKKOS_INLINE_FUNCTION PrimitiveState toPrimitive (const ConservativeState &state, double gamma)
- KOKKOS_INLINE_FUNCTION Flux physicalFlux (const PrimitiveState &state, double gamma)
- KOKKOS_INLINE_FUNCTION Flux averageFlux (const Flux &left, const Flux &right)
- KOKKOS_INLINE_FUNCTION Flux llf (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux hllcImpl (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux hllc (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux hllcLowMach (const PrimitiveState &left, const PrimitiveState &right, double gamma, double mach_limit=0.1)
- KOKKOS_INLINE_FUNCTION double machPlus1 (double mach)
- KOKKOS_INLINE_FUNCTION double machMinus1 (double mach)
- KOKKOS_INLINE_FUNCTION double machPlus2 (double mach)
- KOKKOS_INLINE_FUNCTION double machMinus2 (double mach)
- KOKKOS_INLINE_FUNCTION double pressurePlus5 (double mach, double alpha)
- KOKKOS_INLINE_FUNCTION double pressureMinus5 (double mach, double alpha)
- KOKKOS_INLINE_FUNCTION Flux ausmPlusUp (const PrimitiveState &left, const PrimitiveState &right, double gamma, double mach_cutoff=0.1)
- KOKKOS_INLINE_FUNCTION Flux flux (RiemannSolver solver, const PrimitiveState &left, const PrimitiveState &right, double gamma)

4.11.1 Function Documentation

4.11.1.1 ausmPlusUp()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::ausmPlusUp (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma,
            double mach_cutoff = 0.1)
```

4.11.1.2 averageFlux()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::averageFlux (
            const Flux & left,
            const Flux & right)
```

4.11.1.3 flux()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::flux (
            RiemannSolver solver,
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma)
```

4.11.1.4 hllc()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllc (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma)
```

4.11.1.5 hllcImpl()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllcImpl (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma)
```

4.11.1.6 hllcLowMach()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllcLowMach (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma,
            double mach_limit = 0.1)
```

4.11.1.7 llf()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::llf (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma)
```

4.11.1.8 machMinus1()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::machMinus1 (
            double mach)
```

4.11.1.9 machMinus2()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::machMinus2 (
            double mach)
```

4.11.1.10 machPlus1()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::machPlus1 (
            double mach)
```

4.11.1.11 machPlus2()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::machPlus2 (
            double mach)
```

4.11.1.12 maxSignalSpeed()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::maxSignalSpeed (
            const PrimitiveState & left,
            const PrimitiveState & right,
            double gamma)
```

4.11.1.13 physicalFlux()

```
KOKKOS_INLINE_FUNCTION Flux saturn::riemann::physicalFlux (
            const PrimitiveState & state,
            double gamma)
```

4.11.1.14 pressureMinus5()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::pressureMinus5 (
            double mach,
            double alpha)
```

4.11.1.15 pressurePlus5()

```
KOKKOS_INLINE_FUNCTION double saturn::riemann::pressurePlus5 (
            double mach,
            double alpha)
```

4.11.1.16 toConservative()

```
KOKKOS_INLINE_FUNCTION ConservativeState saturn::riemann::toConservative (
            const PrimitiveState & state,
            double gamma)
```

4.11.1.17 toPrimitive()

```
KOKKOS_INLINE_FUNCTION PrimitiveState saturn::riemann::toPrimitive (
            const ConservativeState & state,
            double gamma)
```

4.11.1.18 valid()

```
KOKKOS_INLINE_FUNCTION bool saturn::riemann::valid (
            const PrimitiveState & state)
```

4.12 saturn::vof Namespace Reference

Classes

- struct Curvature2D
- struct Curvature3D
- class Grid2D
- class Grid3D
- struct LevelSetCurvature
- struct Point
- struct Reconstruction
- struct Reconstruction3D
- struct Vec3

Typedefs

- using Face = std::vector<Vec3>
- using Polyhedron = std::vector<Face>

Functions

- double polygonArea (const std::vector< Point > &polygon)
- std::vector< Point > clipAxis (const std::vector< Point > &polygon, int axis, double value, double sign)
- std::vector< Point > clipHalfPlane (const std::vector< Point > &polygon, double nx, double ny, double alpha)
- std::vector< Point > unitSquare ()
- Reconstruction reconstruct (double c, double nx, double ny)
- std::vector< Point > fluidPolygon (const Reconstruction &r)
- Vec3 operator- (const Vec3 &a, const Vec3 &b)
- Vec3 operator+ (const Vec3 &a, const Vec3 &b)
- Vec3 operator* (double s, const Vec3 &a)
- double dot (const Vec3 &a, const Vec3 &b)
- Vec3 cross (const Vec3 &a, const Vec3 &b)
- Polyhedron unitCube ()
- double polyhedronVolume (const Polyhedron &poly)
- Polyhedron clipHalfPlane (const Polyhedron &poly, const Vec3 &n, double alpha)
- Reconstruction3D reconstruct (double c, double nx, double ny, double nz)
- Polyhedron fluidPolyhedron (const Reconstruction3D &r)
- double columnHeight (const Grid2D &g, int i)
- double rowHeight (const Grid2D &g, int j)
- Curvature2D curvatureY (const Grid2D &g, int i, int stencil=2)
- Curvature2D curvatureX (const Grid2D &g, int j, int stencil=2)
- double columnHeight (const Grid3D &g, int i, int j)
- Curvature3D curvatureZ (const Grid3D &g, int i, int j)
- void smoothBinomial (const Grid2D &in, Grid2D &out, int passes)
- LevelSetCurvature curvatureLevelset (const Grid2D &smoothed, int i, int j)
- void redistribute (Grid2D &g, int sweeps=20)
- void redistribute (Grid3D &g, int sweeps=20)

4.12.1 Typedef Documentation

4.12.1.1 Face

```
using saturn::vof::Face = std::vector<Vec3>
```

4.12.1.2 Polyhedron

```
using saturn::vof::Polyhedron = std::vector<Face>
```

4.12.2 Function Documentation

4.12.2.1 clipAxis()

```
std::vector< Point > saturn::vof::clipAxis (
            const std::vector< Point > & polygon,
            int axis,
            double value,
            double sign)  [inline]
```

4.12.2.2 clipHalfPlane() [1/2]

```
Polyhedron saturn::vof::clipHalfPlane (
            const Polyhedron & poly,
            const Vec3 & n,
            double alpha)  [inline]
```

4.12.2.3 clipHalfPlane() [2/2]

```
std::vector< Point > saturn::vof::clipHalfPlane (
            const std::vector< Point > & polygon,
            double nx,
            double ny,
            double alpha)  [inline]
```

4.12.2.4 columnHeight() [1/2]

```
double saturn::vof::columnHeight (
            const Grid2D & g,
            int i)  [inline]
```

4.12.2.5 columnHeight() [2/2]

```
double saturn::vof::columnHeight (
            const Grid3D & g,
            int i,
            int j)  [inline]
```

4.12.2.6 cross()

```
Vec3 saturn::vof::cross (
            const Vec3 & a,
            const Vec3 & b)  [inline]
```

4.12.2.7 curvatureLevelset()

```
LevelSetCurvature saturn::vof::curvatureLevelset (
            const Grid2D & smoothed,
            int i,
            int j)  [inline]
```

4.12.2.8 curvatureX()

```
Curvature2D saturn::vof::curvatureX (
            const Grid2D & g,
            int j,
            int stencil = 2)  [inline]
```

4.12.2.9 curvatureY()

```
Curvature2D saturn::vof::curvatureY (
            const Grid2D & g,
            int i,
            int stencil = 2)  [inline]
```

4.12.2.10 curvatureZ()

```
Curvature3D saturn::vof::curvatureZ (
            const Grid3D & g,
            int i,
            int j)  [inline]
```

4.12.2.11 dot()

```
double saturn::vof::dot (
            const Vec3 & a,
            const Vec3 & b)  [inline]
```

4.12.2.12 fluidPolygon()

```
std::vector< Point > saturn::vof::fluidPolygon (
            const Reconstruction & r)  [inline]
```

4.12.2.13 fluidPolyhedron()

```
Polyhedron saturn::vof::fluidPolyhedron (
            const Reconstruction3D & r)  [inline]
```

4.12.2.14 operator*()

```
Vec3 saturn::vof::operator* (
            double s,
            const Vec3 & a)  [inline]
```

4.12.2.15 operator+()

```
Vec3 saturn::vof::operator+ (
            const Vec3 & a,
            const Vec3 & b)  [inline]
```

4.12.2.16 operator-()

```
Vec3 saturn::vof::operator- (
            const Vec3 & a,
            const Vec3 & b)  [inline]
```

4.12.2.17 polygonArea()

```
double saturn::vof::polygonArea (
            const std::vector< Point > & polygon)  [inline]
```

4.12.2.18 polyhedronVolume()

```
double saturn::vof::polyhedronVolume (
            const Polyhedron & poly)  [inline]
```

4.12.2.19 reconstruct() [1/2]

```
Reconstruction saturn::vof::reconstruct (
            double c,
            double nx,
            double ny)  [inline]
```

4.12.2.20 reconstruct() [2/2]

```
Reconstruction3D saturn::vof::reconstruct (
            double c,
            double nx,
            double ny,
            double nz)  [inline]
```

4.12.2.21 redistribute() [1/2]

```
void saturn::vof::redistribute (
            Grid2D & g,
            int sweeps = 20)  [inline]
```

4.12.2.22 redistribute() [2/2]

```
void saturn::vof::redistribute (
            Grid3D & g,
            int sweeps = 20)  [inline]
```

4.12.2.23 rowHeight()

```
double saturn::vof::rowHeight (
            const Grid2D & g,
            int j)  [inline]
```

4.12.2.24 smoothBinomial()

```
void saturn::vof::smoothBinomial (
            const Grid2D & in,
            Grid2D & out,
            int passes)  [inline]
```

4.12.2.25 unitCube()

```
Polyhedron saturn::vof::unitCube ()  [inline]
```

4.12.2.26 unitSquare()

```
std::vector< Point > saturn::vof::unitSquare ()  [inline]
```

Chapter 5

Class Documentation

5.1 saturn::Box Struct Reference

`#include <box.hpp>`

Public Member Functions

- KOKKOS_INLINE_FUNCTION bool empty () const
- KOKKOS_INLINE_FUNCTION int length (int d) const
- KOKKOS_INLINE_FUNCTION long long numCells () const
- KOKKOS_INLINE_FUNCTION bool contains (int i, int j, int k) const
- KOKKOS_INLINE_FUNCTION Box grow (int n) const
- KOKKOS_INLINE_FUNCTION Box shrink (int n) const
- KOKKOS_INLINE_FUNCTION Box shift (int di, int dj, int dk) const
- KOKKOS_INLINE_FUNCTION Box intersect (const Box &other) const
- KOKKOS_INLINE_FUNCTION bool operator== (const Box &other) const
- KOKKOS_INLINE_FUNCTION bool operator!= (const Box &other) const

Static Public Member Functions

- static Box fromExtents (int nx, int ny, int nz)

Public Attributes

- int lo [3] {0, 0, 0}
- int hi [3] {-1, -1, -1}

5.1.1 Member Function Documentation

5.1.1.1 contains()

```
KOKKOS_INLINE_FUNCTION bool saturn::Box::contains (
            int i,
            int j,
            int k) const  [inline]
```

5.1.1.2 empty()

```
KOKKOS_INLINE_FUNCTION bool saturn::Box::empty () const  [inline]
```

5.1.1.3 fromExtents()

```
static Box saturn::Box::fromExtents (
            int nx,
            int ny,
            int nz)  [inline], [static]
```

5.1.1.4 grow()

```
KOKKOS_INLINE_FUNCTION Box saturn::Box::grow (
            int n) const  [inline]
```

5.1.1.5 intersect()

```
KOKKOS_INLINE_FUNCTION Box saturn::Box::intersect (
            const Box & other) const  [inline]
```

5.1.1.6 length()

```
KOKKOS_INLINE_FUNCTION int saturn::Box::length (
            int d) const  [inline]
```

5.1.1.7 numCells()

```
KOKKOS_INLINE_FUNCTION long long saturn::Box::numCells () const  [inline]
```

5.1.1.8 operator"!=()

```
KOKKOS_INLINE_FUNCTION bool saturn::Box::operator!= (
            const Box & other) const  [inline]
```

5.1.1.9 operator==()

```
KOKKOS_INLINE_FUNCTION bool saturn::Box::operator== (
            const Box & other) const  [inline]
```

5.1.1.10 shift()

```
KOKKOS_INLINE_FUNCTION Box saturn::Box::shift (
            int di,
            int dj,
            int dk) const  [inline]
```

5.1.1.11 shrink()

```
KOKKOS_INLINE_FUNCTION Box saturn::Box::shrink (
            int n) const  [inline]
```

5.1.2 Member Data Documentation

5.1.2.1 hi

```
int saturn::Box::hi[3] {-1, -1, -1}
```

5.1.2.2 lo

```
int saturn::Box::lo[3] {0, 0, 0}
```

The documentation for this struct was generated from the following file:

- src/mesh/box.hpp

5.2 saturn::BoxExchange< ExecSpace > Class Template Reference

```
#include <box_exchange.hpp>
```

Public Member Functions

- BoxExchange (const Decomposition &decomposition, int nvar, int ng)
- void exchange (PatchField< ExecSpace > &q)

5.2.1 Constructor & Destructor Documentation

5.2.1.1 BoxExchange()

```
template<class ExecSpace >
saturn::BoxExchange< ExecSpace >::BoxExchange (
            const Decomposition & decomposition,
            int nvar,
            int ng)  [inline]
```

5.2.2 Member Function Documentation

5.2.2.1 exchange()

```
template<class ExecSpace >
void saturn::BoxExchange< ExecSpace >::exchange (
            PatchField< ExecSpace > & q)  [inline]
```

The documentation for this class was generated from the following file:

- src/halo/box_exchange.hpp

5.3 saturn::CheckpointHeader Struct Reference

`#include <checkpoint.hpp>`

Public Attributes

- char magic [8] = {'S', 'A', 'T', 'U', 'R', 'N', 'C', 'K'}
- int version = 1
- int rank = 0
- int size = 1
- int nvar = 1
- int ng = 1
- int local_lo [3] = {0, 0, 0}
- int local_hi [3] = {-1, -1, -1}
- int global_lo [3] = {0, 0, 0}
- int global_hi [3] = {-1, -1, -1}
- int step = 0
- double time = 0.0

5.3.1 Member Data Documentation

5.3.1.1 global_hi

```
int saturn::CheckpointHeader::global_hi[3] = {-1, -1, -1}
```

5.3.1.2 global_lo

```
int saturn::CheckpointHeader::global_lo[3] = {0, 0, 0}
```

5.3.1.3 local_hi

```
int saturn::CheckpointHeader::local_hi[3] = {-1, -1, -1}
```

5.3.1.4 local_lo

```
int saturn::CheckpointHeader::local_lo[3] = {0, 0, 0}
```

5.3.1.5 magic

```
char saturn::CheckpointHeader::magic[8] = {'S', 'A', 'T', 'U', 'R', 'N', 'C', 'K'}
```

5.3.1.6 ng

```
int saturn::CheckpointHeader::ng = 1
```

5.3.1.7 nvar

```
int saturn::CheckpointHeader::nvar = 1
```

5.3.1.8 rank

```
int saturn::CheckpointHeader::rank = 0
```

5.3.1.9 size

```
int saturn::CheckpointHeader::size = 1
```

5.3.1.10 step

```
int saturn::CheckpointHeader::step = 0
```

5.3.1.11 time

```
double saturn::CheckpointHeader::time = 0.0
```

5.3.1.12 version

```
int saturn::CheckpointHeader::version = 1
```

The documentation for this struct was generated from the following file:

- src/io/checkpoint.hpp

5.4 saturn::cavity::ClusteredPoisson2D Class Reference

Host variable-coefficient Poisson solver for the clustered 2D grid.

```
#include <cavity.hpp>
```

Public Member Functions

- ClusteredPoisson2D (int n0, int n1, const Geometry &g)
- void apply (const std::vector< double > &p, std::vector< double > &out) const
- void solve (std::vector< double > &p, const std::vector< double > &f, double tolerance=1.0e-8, int max_↩ iterations=500) const

5.4.1 Detailed Description

Host variable-coefficient Poisson solver for the clustered 2D grid.

Solves the symmetric negative-Laplacian system with Neumann walls. Cell volumes are folded into the right-hand side.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 ClusteredPoisson2D()

```
saturn::cavity::ClusteredPoisson2D::ClusteredPoisson2D (
            int n0,
            int n1,
            const Geometry & g)  [inline]
```

5.4.3 Member Function Documentation

5.4.3.1 apply()

```
void saturn::cavity::ClusteredPoisson2D::apply (
            const std::vector< double > & p,
            std::vector< double > & out) const  [inline]
```

5.4.3.2 solve()

```
void saturn::cavity::ClusteredPoisson2D::solve (
            std::vector< double > & p,
            const std::vector< double > & f,
            double tolerance = 1.0e-8,
            int max_iterations = 500) const  [inline]
```

The documentation for this class was generated from the following file:

- src/numerics/incompressible/cavity.hpp

5.5 saturn::compressible::CompressibleWorkspace< ExecSpace > Class Template Reference

```
#include <compressible_ns.hpp>
```

Public Types

- using memory_space = typename ExecSpace::memory_space
- using face_view = Kokkos::View<double****, Kokkos::LayoutRight, memory_space>

Public Member Functions

- CompressibleWorkspace (const Box &box)

Public Attributes

- PatchField< ExecSpace > grad
- PatchField< ExecSpace > stress
- face_view fx
- face_view fy
- face_view fz

5.5.1 Member Typedef Documentation

5.5.1.1 face_view

```
template<class ExecSpace >
using saturn::compressible::CompressibleWorkspace< ExecSpace >::face_view = Kokkos::View<double****,
Kokkos::LayoutRight, memory_space>
```

5.5.1.2 memory_space

```
template<class ExecSpace >
using saturn::compressible::CompressibleWorkspace< ExecSpace >::memory_space = typename Exec↩
Space::memory_space
```

5.5.2 Constructor & Destructor Documentation

5.5.2.1 CompressibleWorkspace()

```
template<class ExecSpace >
saturn::compressible::CompressibleWorkspace< ExecSpace >::CompressibleWorkspace (
            const Box & box)  [inline], [explicit]
```

5.5.3 Member Data Documentation

5.5.3.1 fx

```
template<class ExecSpace >
face_view saturn::compressible::CompressibleWorkspace< ExecSpace >::fx
```

5.5.3.2 fy

```
template<class ExecSpace >
face_view saturn::compressible::CompressibleWorkspace< ExecSpace >::fy
```

5.5.3.3 fz

```
template<class ExecSpace >
face_view saturn::compressible::CompressibleWorkspace< ExecSpace >::fz
```

5.5.3.4 grad

```
template<class ExecSpace >
PatchField<ExecSpace> saturn::compressible::CompressibleWorkspace< ExecSpace >::grad
```

5.5.3.5 stress

```
template<class ExecSpace >
PatchField<ExecSpace> saturn::compressible::CompressibleWorkspace< ExecSpace >::stress
```

The documentation for this class was generated from the following file:

- src/numerics/compressible/compressible_ns.hpp

5.6 saturn::ConservativeState Struct Reference

```
#include <riemann.hpp>
```

Public Attributes

- double rho
- double mom_n
- double mom_t1
- double mom_t2
- double energy

5.6.1 Member Data Documentation

5.6.1.1 energy

```
double saturn::ConservativeState::energy
```

5.6.1.2 mom_n

```
double saturn::ConservativeState::mom_n
```

5.6.1.3 mom_t1

```
double saturn::ConservativeState::mom_t1
```

5.6.1.4 mom_t2

```
double saturn::ConservativeState::mom_t2
```

5.6.1.5 rho

```
double saturn::ConservativeState::rho
```

The documentation for this struct was generated from the following file:

- src/numerics/compressible/riemann.hpp

5.7 saturn::vof::Curvature2D Struct Reference

```
#include <vof_curvature2d.hpp>
```

Public Attributes

- double kappa = 0.0
- double slope = 0.0
- bool valid = false

5.7.1 Member Data Documentation

5.7.1.1 kappa

```
double saturn::vof::Curvature2D::kappa = 0.0
```

5.7.1.2 slope

```
double saturn::vof::Curvature2D::slope = 0.0
```

5.7.1.3 valid

```
bool saturn::vof::Curvature2D::valid = false
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof_curvature2d.hpp

5.8 saturn::vof::Curvature3D Struct Reference

```
#include <vof_curvature3d.hpp>
```

Public Attributes

- double kappa = 0.0
- double hx = 0.0
- double hy = 0.0
- bool valid = false

5.8.1 Member Data Documentation

5.8.1.1 hx

```
double saturn::vof::Curvature3D::hx = 0.0
```

5.8.1.2 hy

```
double saturn::vof::Curvature3D::hy = 0.0
```

5.8.1.3 kappa

```
double saturn::vof::Curvature3D::kappa = 0.0
```

5.8.1.4 valid

```
bool saturn::vof::Curvature3D::valid = false
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof_curvature3d.hpp

5.9 saturn::Decomposition Struct Reference

```
#include <decomposition.hpp>
```

Public Member Functions

- Box boxForCoords (const int coords_in[3]) const
- int partner (int d, int side) const

Static Public Member Functions

- static Decomposition create (const Box &global_box, const bool periodic_in[3])

Public Attributes

- int rank = 0
- int size = 1
- int dims [3] {1, 1, 1}
- int coords [3] {0, 0, 0}
- bool periodic [3] {true, true, true}
- Box global {}
- Box local {}

5.9.1 Member Function Documentation

5.9.1.1 boxForCoords()

```
Box saturn::Decomposition::boxForCoords (
            const int coords_in[3]) const  [inline]
```

5.9.1.2 create()

```
static Decomposition saturn::Decomposition::create (
            const Box & global_box,
            const bool periodic_in[3])  [inline], [static]
```

5.9.1.3 partner()

```
int saturn::Decomposition::partner (
            int d,
            int side) const  [inline]
```

5.9.2 Member Data Documentation

5.9.2.1 coords

```
int saturn::Decomposition::coords[3] {0, 0, 0}
```

5.9.2.2 dims

```
int saturn::Decomposition::dims[3] {1, 1, 1}
```

5.9.2.3 global

```
Box saturn::Decomposition::global {}
```

5.9.2.4 local

```
Box saturn::Decomposition::local {}
```

5.9.2.5 periodic

```
bool saturn::Decomposition::periodic[3] {true, true, true}
```

5.9.2.6 rank

```
int saturn::Decomposition::rank = 0
```

5.9.2.7 size

```
int saturn::Decomposition::size = 1
```

The documentation for this struct was generated from the following file:

- src/mesh/decomposition.hpp

5.10 saturn::FaceExchange< ExecSpace > Class Template Reference

```
#include <face_exchange.hpp>
```

Public Types

- using memory_space = typename ExecSpace::memory_space
- using buffer_type = Kokkos::View<double*, memory_space>
- using host_buffer_type = Kokkos::View<double*, Kokkos::HostSpace>

Public Member Functions

- FaceExchange (const Decomposition &decomposition, int nvar, int ng)
- void begin (PatchField< ExecSpace > &q)
- void end (PatchField< ExecSpace > &q)
- void exchange (PatchField< ExecSpace > &q)
- void pack (const PatchField< ExecSpace > &q, int d, int side_sign, buffer_type &buffer)
- void unpack (PatchField< ExecSpace > &q, int d, int side_sign, const buffer_type &buffer)

5.10.1 Member Typedef Documentation

5.10.1.1 buffer_type

```
template<class ExecSpace >
using saturn::FaceExchange< ExecSpace >::buffer_type = Kokkos::View<double*, memory_space>
```

5.10.1.2 host_buffer_type

```
template<class ExecSpace >
using saturn::FaceExchange< ExecSpace >::host_buffer_type = Kokkos::View<double*, Kokkos::↩
HostSpace>
```

5.10.1.3 memory_space

```
template<class ExecSpace >
using saturn::FaceExchange< ExecSpace >::memory_space = typename ExecSpace::memory_space
```

5.10.2 Constructor & Destructor Documentation

5.10.2.1 FaceExchange()

```
template<class ExecSpace >
saturn::FaceExchange< ExecSpace >::FaceExchange (
            const Decomposition & decomposition,
            int nvar,
            int ng)  [inline]
```

5.10.3 Member Function Documentation

5.10.3.1 begin()

```
template<class ExecSpace >
void saturn::FaceExchange< ExecSpace >::begin (
            PatchField< ExecSpace > & q)  [inline]
```

5.10.3.2 end()

```
template<class ExecSpace >
void saturn::FaceExchange< ExecSpace >::end (
            PatchField< ExecSpace > & q)  [inline]
```

5.10.3.3 exchange()

```
template<class ExecSpace >
void saturn::FaceExchange< ExecSpace >::exchange (
            PatchField< ExecSpace > & q)  [inline]
```

5.10.3.4 pack()

```
template<class ExecSpace >
void saturn::FaceExchange< ExecSpace >::pack (
            const PatchField< ExecSpace > & q,
            int d,
            int side_sign,
            buffer_type & buffer)  [inline]
```

5.10.3.5 unpack()

```
template<class ExecSpace >
void saturn::FaceExchange< ExecSpace >::unpack (
             PatchField< ExecSpace > & q,
             int d,
             int side_sign,
             const buffer_type & buffer)  [inline]
```

The documentation for this class was generated from the following file:

- src/halo/face_exchange.hpp

5.11 saturn::Flux Struct Reference

```
#include <riemann.hpp>
```

Public Attributes

- double rho
- double mom_n
- double mom_t1
- double mom_t2
- double energy

5.11.1 Member Data Documentation

5.11.1.1 energy

```
double saturn::Flux::energy
```

5.11.1.2 mom_n

```
double saturn::Flux::mom_n
```

5.11.1.3 mom_t1

```
double saturn::Flux::mom_t1
```

5.11.1.4 mom_t2

```
double saturn::Flux::mom_t2
```

5.11.1.5 rho

```
double saturn::Flux::rho
```

The documentation for this struct was generated from the following file:

- src/numerics/compressible/riemann.hpp

5.12 saturn::compressible::GasProperties Struct Reference

```
#include <compressible_ns.hpp>
```

Public Attributes

- double gamma = 1.4
- double gas_constant = 1.0
- double mu = 0.0
- double prandtl = 0.72

5.12.1 Member Data Documentation

5.12.1.1 gamma

```
double saturn::compressible::GasProperties::gamma = 1.4
```

5.12.1.2 gas_constant

```
double saturn::compressible::GasProperties::gas_constant = 1.0
```

5.12.1.3 mu

```
double saturn::compressible::GasProperties::mu = 0.0
```

5.12.1.4 prandtl

```
double saturn::compressible::GasProperties::prandtl = 0.72
```

The documentation for this struct was generated from the following file:

- src/numerics/compressible/compressible_ns.hpp

5.13 saturn::poisson::GeometricMultigrid< ExecSpace > Class Template Reference

Geometric multigrid for the uniform-grid Poisson equation.

`#include <poisson.hpp>`

Classes

- struct Level

Public Types

- using memory_space = typename ExecSpace::memory_space
- using view_type = Kokkos::View<double***, Kokkos::LayoutRight, memory_space>

Public Member Functions

- GeometricMultigrid (const std::array< int, 3 > &cells, const std::array< double, 3 > &spacing)
- GeometricMultigrid (const std::array< int, 3 > &cells, const std::array< double, 3 > &spacing, const std::array< Boundary, 3 > &bc_low, const std::array< Boundary, 3 > &bc_high)
- Result solve (view_type x, view_type b, const Options &options)
- int levels () const
- void vcycle (int level_index, const Options &options)
- void smooth (const Level &level, int sweeps) const
- void computeResidual (const Level &level) const
- void restrict (const Level &fine, Level &coarse) const
- void prolongAdd (const Level &coarse, Level &fine) const
- double norm (const view_type &v) const
- double residualNorm (int level_index) const

Static Public Member Functions

- static void projectMeanImpl (const view_type &v)

5.13.1 Detailed Description

template<class ExecSpace>
class saturn::poisson::GeometricMultigrid< ExecSpace >

Geometric multigrid for the uniform-grid Poisson equation.

Supports periodic, homogeneous-Neumann, and homogeneous-Dirichlet boundaries. The operator is the standard 7-point finite-volume Laplacian.

5.13.2 Member Typedef Documentation

5.13.2.1 memory_space

```
template<class ExecSpace >
using saturn::poisson::GeometricMultigrid< ExecSpace >::memory_space = typename ExecSpace↩
::memory_space
```

5.13.2.2 view_type

```
template<class ExecSpace >
using saturn::poisson::GeometricMultigrid< ExecSpace >::view_type = Kokkos::View<double***,
Kokkos::LayoutRight, memory_space>
```

5.13.3 Constructor & Destructor Documentation

5.13.3.1 GeometricMultigrid() [1/2]

```
template<class ExecSpace >
saturn::poisson::GeometricMultigrid< ExecSpace >::GeometricMultigrid (
            const std::array< int, 3 > & cells,
            const std::array< double, 3 > & spacing)  [inline]
```

5.13.3.2 GeometricMultigrid() [2/2]

```
template<class ExecSpace >
saturn::poisson::GeometricMultigrid< ExecSpace >::GeometricMultigrid (
            const std::array< int, 3 > & cells,
            const std::array< double, 3 > & spacing,
            const std::array< Boundary, 3 > & bc_low,
            const std::array< Boundary, 3 > & bc_high)  [inline]
```

5.13.4 Member Function Documentation

5.13.4.1 computeResidual()

```
template<class ExecSpace >
void saturn::poisson::GeometricMultigrid< ExecSpace >::computeResidual (
            const Level & level) const  [inline]
```

5.13.4.2 levels()

```
template<class ExecSpace >
int saturn::poisson::GeometricMultigrid< ExecSpace >::levels () const  [inline]
```

5.13.4.3 norm()

```
template<class ExecSpace >
double saturn::poisson::GeometricMultigrid< ExecSpace >::norm (
            const view_type & v) const  [inline]
```

5.13.4.4 projectMeanImpl()

```
template<class ExecSpace >
static void saturn::poisson::GeometricMultigrid< ExecSpace >::projectMeanImpl (
            const view_type & v)  [inline], [static]
```

5.13.4.5 prolongAdd()

```
template<class ExecSpace >
void saturn::poisson::GeometricMultigrid< ExecSpace >::prolongAdd (
            const Level & coarse,
            Level & fine) const  [inline]
```

5.13.4.6 residualNorm()

```
template<class ExecSpace >
double saturn::poisson::GeometricMultigrid< ExecSpace >::residualNorm (
            int level_index) const  [inline]
```

5.13.4.7 restrict()

```
template<class ExecSpace >
void saturn::poisson::GeometricMultigrid< ExecSpace >::restrict (
            const Level & fine,
            Level & coarse) const  [inline]
```

5.13.4.8 smooth()

```
template<class ExecSpace >
void saturn::poisson::GeometricMultigrid< ExecSpace >::smooth (
            const Level & level,
            int sweeps) const  [inline]
```

5.13.4.9 solve()

```
template<class ExecSpace >
Result saturn::poisson::GeometricMultigrid< ExecSpace >::solve (
            view_type x,
            view_type b,
            const Options & options)  [inline]
```

5.13.4.10 vcycle()

```
template<class ExecSpace >
void saturn::poisson::GeometricMultigrid< ExecSpace >::vcycle (
            int level_index,
            const Options & options)  [inline]
```

The documentation for this class was generated from the following file:

- src/numerics/incompressible/poisson.hpp

5.14 saturn::cavity::Geometry Struct Reference

Grid, lid speed, advection scheme, and optional wall-clustering map.

`#include <cavity.hpp>`

Public Member Functions

- KOKKOS_INLINE_FUNCTION double mapX (double xi) const
- KOKKOS_INLINE_FUNCTION double dmapX (double xi) const
- KOKKOS_INLINE_FUNCTION double xf (int i) const
- KOKKOS_INLINE_FUNCTION double xc (int i) const
- KOKKOS_INLINE_FUNCTION double yf (int j) const
- KOKKOS_INLINE_FUNCTION double yc (int j) const

Public Attributes

- int n0 = 1
- int n1 = 1
- int n2 = 1
- double hx = 1.0
- double hy = 1.0
- double hz = 1.0
- double u_lid = 1.0
- int adv = 0
- double adv_blend = 0.25
- bool z_wall = false
- double beta = 0.0

5.14.1 Detailed Description

Grid, lid speed, advection scheme, and optional wall-clustering map.

5.14.2 Member Function Documentation

5.14.2.1 dmapX()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::dmapX (
            double xi) const  [inline]
```

5.14.2.2 mapX()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::mapX (
            double xi) const  [inline]
```

5.14.2.3 xc()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::xc (
            int i) const  [inline]
```

5.14.2.4 xf()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::xf (
            int i) const  [inline]
```

5.14.2.5 yc()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::yc (
            int j) const  [inline]
```

5.14.2.6 yf()

```
KOKKOS_INLINE_FUNCTION double saturn::cavity::Geometry::yf (
            int j) const  [inline]
```

5.14.3 Member Data Documentation

5.14.3.1 adv

```
int saturn::cavity::Geometry::adv = 0
```

5.14.3.2 adv_blend

```
double saturn::cavity::Geometry::adv_blend = 0.25
```

5.14.3.3 beta

```
double saturn::cavity::Geometry::beta = 0.0
```

5.14.3.4 hx

```
double saturn::cavity::Geometry::hx = 1.0
```

5.14.3.5 hy

```
double saturn::cavity::Geometry::hy = 1.0
```

5.14.3.6 hz

```
double saturn::cavity::Geometry::hz = 1.0
```

5.14.3.7 n0

```
int saturn::cavity::Geometry::n0 = 1
```

5.14.3.8 n1

```
int saturn::cavity::Geometry::n1 = 1
```

5.14.3.9 n2

```
int saturn::cavity::Geometry::n2 = 1
```

5.14.3.10 u_lid

```
double saturn::cavity::Geometry::u_lid = 1.0
```

5.14.3.11 z_wall

```
bool saturn::cavity::Geometry::z_wall = false
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/cavity.hpp

5.15 saturn::channel::Geometry Struct Reference

`#include <channel.hpp>`

Public Attributes

- int n0 = 1
- int n1 = 1
- int n2 = 1
- double hx = 1.0
- double hy = 1.0
- double hz = 1.0
- double ly = 1.0
- double u_top = 0.0
- double gx = 0.0
- double nu = 1.0

5.15.1 Member Data Documentation

5.15.1.1 gx

```
double saturn::channel::Geometry::gx = 0.0
```

5.15.1.2 hx

```
double saturn::channel::Geometry::hx = 1.0
```

5.15.1.3 hy

```
double saturn::channel::Geometry::hy = 1.0
```

5.15.1.4 hz

```
double saturn::channel::Geometry::hz = 1.0
```

5.15.1.5 ly

```
double saturn::channel::Geometry::ly = 1.0
```

5.15.1.6 n0

```
int saturn::channel::Geometry::n0 = 1
```

5.15.1.7 n1

```
int saturn::channel::Geometry::n1 = 1
```

5.15.1.8 n2

```
int saturn::channel::Geometry::n2 = 1
```

5.15.1.9 nu

```
double saturn::channel::Geometry::nu = 1.0
```

5.15.1.10 u_top

```
double saturn::channel::Geometry::u_top = 0.0
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/channel.hpp

5.16 saturn::GlobalExchange< ExecSpace > Class Template Reference

```
#include <global_exchange.hpp>
```

Public Member Functions

- GlobalExchange (const Decomposition &decomposition, int nvar, int ng)
- void exchange (PatchField< ExecSpace > &q, const BoundaryType bc[3][2])

5.16.1 Constructor & Destructor Documentation

5.16.1.1 GlobalExchange()

```
template<class ExecSpace >
saturn::GlobalExchange< ExecSpace >::GlobalExchange (
            const Decomposition & decomposition,
            int nvar,
            int ng)  [inline]
```

5.16.2 Member Function Documentation

5.16.2.1 exchange()

```
template<class ExecSpace >
void saturn::GlobalExchange< ExecSpace >::exchange (
            PatchField< ExecSpace > & q,
            const BoundaryType bc[3][2])  [inline]
```

The documentation for this class was generated from the following file:

- src/halo/global_exchange.hpp

5.17 saturn::vof::Grid2D Class Reference

```
#include <vof2d.hpp>
```

Public Member Functions

- Grid2D (int nx, int ny, double lx=1.0, double ly=1.0)
- int nx () const
- int ny () const
- double hx () const
- double hy () const
- double & operator() (int i, int j)
- double operator() (int i, int j) const
- std::vector< double > & data ()
- const std::vector< double > & data () const
- std::vector< Reconstruction > reconstructions () const
- void sweepX (const std::vector< double > &u, double dt)
- void sweepY (const std::vector< double > &v, double dt)
- void advance (const std::vector< double > &u, const std::vector< double > &v, double dt)
- double mass () const

5.17.1 Constructor & Destructor Documentation

5.17.1.1 Grid2D()

```
saturn::vof::Grid2D::Grid2D (
            int nx,
            int ny,
            double lx = 1.0,
            double ly = 1.0)  [inline]
```

5.17.2 Member Function Documentation

5.17.2.1 advance()

```
void saturn::vof::Grid2D::advance (
            const std::vector< double > & u,
            const std::vector< double > & v,
            double dt)  [inline]
```

5.17.2.2 data() [1/2]

```
std::vector< double > & saturn::vof::Grid2D::data ()  [inline]
```

5.17.2.3 data() [2/2]

```
const std::vector< double > & saturn::vof::Grid2D::data () const  [inline]
```

5.17.2.4 hx()

```
double saturn::vof::Grid2D::hx () const  [inline]
```

5.17.2.5 hy()

```
double saturn::vof::Grid2D::hy () const  [inline]
```

5.17.2.6 mass()

```
double saturn::vof::Grid2D::mass () const  [inline]
```

5.17.2.7 nx()

```
int saturn::vof::Grid2D::nx () const  [inline]
```

5.17.2.8 ny()

```
int saturn::vof::Grid2D::ny () const  [inline]
```

5.17.2.9 operator()() [1/2]

```
double & saturn::vof::Grid2D::operator() (
            int i,
            int j)  [inline]
```

5.17.2.10 operator()() [2/2]

```
double saturn::vof::Grid2D::operator() (
            int i,
            int j) const  [inline]
```

5.17.2.11 reconstructions()

```
std::vector< Reconstruction > saturn::vof::Grid2D::reconstructions () const  [inline]
```

5.17.2.12 sweepX()

```
void saturn::vof::Grid2D::sweepX (
            const std::vector< double > & u,
            double dt)  [inline]
```

5.17.2.13 sweepY()

```
void saturn::vof::Grid2D::sweepY (
            const std::vector< double > & v,
            double dt)  [inline]
```

The documentation for this class was generated from the following file:

- src/numerics/vof/vof2d.hpp

5.18 saturn::vof::Grid3D Class Reference

```
#include <vof3d.hpp>
```

Public Member Functions

- Grid3D (int nx, int ny, int nz, double lx=1.0, double ly=1.0, double lz=1.0)
- int nx () const
- int ny () const
- int nz () const
- double hx () const
- double hy () const
- double hz () const
- double & operator() (int i, int j, int k)
- double operator() (int i, int j, int k) const
- std::vector< double > & data ()
- const std::vector< double > & data () const
- double cellVolume () const
- std::vector< Reconstruction3D > reconstructions () const
- void sweepX (const std::vector< double > &u, double dt)
- void sweepY (const std::vector< double > &v, double dt)
- void sweepZ (const std::vector< double > &w, double dt)
- void advance (const std::vector< double > &u, const std::vector< double > &v, const std::vector< double > &w, double dt)
- double mass () const

5.18.1 Constructor & Destructor Documentation

5.18.1.1 Grid3D()

```
saturn::vof::Grid3D::Grid3D (
            int nx,
            int ny,
            int nz,
            double lx = 1.0,
            double ly = 1.0,
            double lz = 1.0)  [inline]
```

5.18.2 Member Function Documentation

5.18.2.1 advance()

```
void saturn::vof::Grid3D::advance (
            const std::vector< double > & u,
            const std::vector< double > & v,
            const std::vector< double > & w,
            double dt)  [inline]
```

5.18.2.2 cellVolume()

```
double saturn::vof::Grid3D::cellVolume () const  [inline]
```

5.18.2.3 data() [1/2]

```
std::vector< double > & saturn::vof::Grid3D::data ()  [inline]
```

5.18.2.4 data() [2/2]

```
const std::vector< double > & saturn::vof::Grid3D::data () const  [inline]
```

5.18.2.5 hx()

```
double saturn::vof::Grid3D::hx () const  [inline]
```

5.18.2.6 hy()

```
double saturn::vof::Grid3D::hy () const  [inline]
```

5.18.2.7 hz()

```
double saturn::vof::Grid3D::hz () const  [inline]
```

5.18.2.8 mass()

```
double saturn::vof::Grid3D::mass () const  [inline]
```

5.18.2.9 nx()

```
int saturn::vof::Grid3D::nx () const  [inline]
```

5.18.2.10 ny()

```
int saturn::vof::Grid3D::ny () const  [inline]
```

5.18.2.11 nz()

```
int saturn::vof::Grid3D::nz () const  [inline]
```

5.18.2.12 operator()() [1/2]

```
double & saturn::vof::Grid3D::operator() (
            int i,
            int j,
            int k)  [inline]
```

5.18.2.13 operator()() [2/2]

```
double saturn::vof::Grid3D::operator() (
            int i,
            int j,
            int k) const  [inline]
```

5.18.2.14 reconstructions()

```
std::vector< Reconstruction3D > saturn::vof::Grid3D::reconstructions () const  [inline]
```

5.18.2.15 sweepX()

```
void saturn::vof::Grid3D::sweepX (
            const std::vector< double > & u,
            double dt)  [inline]
```

5.18.2.16 sweepY()

```
void saturn::vof::Grid3D::sweepY (
            const std::vector< double > & v,
            double dt)  [inline]
```

5.18.2.17 sweepZ()

```
void saturn::vof::Grid3D::sweepZ (
            const std::vector< double > & w,
            double dt)  [inline]
```

The documentation for this class was generated from the following file:

- src/numerics/vof/vof3d.hpp

5.19 saturn::poisson::GeometricMultigrid< ExecSpace >::Level Struct Reference

```
#include <poisson.hpp>
```

Public Attributes

- std::array< int, 3 > n {{0, 0, 0}}
- std::array< double, 3 > h {{1.0, 1.0, 1.0}}
- view_type x {}
- view_type b {}
- view_type r {}

5.19.1 Member Data Documentation

5.19.1.1 b

```
template<class ExecSpace >
view_type saturn::poisson::GeometricMultigrid< ExecSpace >::Level::b {}
```

5.19.1.2 h

```
template<class ExecSpace >
std::array<double, 3> saturn::poisson::GeometricMultigrid< ExecSpace >::Level::h {{1.0, 1.0,
1.0}}
```

5.19.1.3 n

```
template<class ExecSpace >
std::array<int, 3> saturn::poisson::GeometricMultigrid< ExecSpace >::Level::n {{0, 0, 0}}
```

5.19.1.4 r

```
template<class ExecSpace >
view_type saturn::poisson::GeometricMultigrid< ExecSpace >::Level::r {}
```

5.19.1.5 x

```
template<class ExecSpace >
view_type saturn::poisson::GeometricMultigrid< ExecSpace >::Level::x {}
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/poisson.hpp

5.20 saturn::vof::LevelSetCurvature Struct Reference

```
#include <vof_curvature_levelset2d.hpp>
```

Public Attributes

- double kappa = 0.0
- bool valid = false

5.20.1 Member Data Documentation

5.20.1.1 kappa

```
double saturn::vof::LevelSetCurvature::kappa = 0.0
```

5.20.1.2 valid

```
bool saturn::vof::LevelSetCurvature::valid = false
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof_curvature_levelset2d.hpp

5.21 saturn::poisson::Options Struct Reference

Multigrid cycle and tolerance controls.

```
#include <poisson.hpp>
```

Public Attributes

- int max_cycles = 100
- int pre_smooth = 2
- int post_smooth = 2
- int coarse_smooth = 60
- double tolerance = 1.0e-8
- bool verbose = false

5.21.1 Detailed Description

Multigrid cycle and tolerance controls.

5.21.2 Member Data Documentation

5.21.2.1 coarse_smooth

```
int saturn::poisson::Options::coarse_smooth = 60
```

5.21.2.2 max_cycles

```
int saturn::poisson::Options::max_cycles = 100
```

5.21.2.3 post_smooth

```
int saturn::poisson::Options::post_smooth = 2
```

5.21.2.4 pre_smooth

```
int saturn::poisson::Options::pre_smooth = 2
```

5.21.2.5 tolerance

```
double saturn::poisson::Options::tolerance = 1.0e-8
```

5.21.2.6 verbose

```
bool saturn::poisson::Options::verbose = false
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/poisson.hpp

5.22 saturn::edac::Parameters Struct Reference

```
#include <edac.hpp>
```

Public Attributes

- double nu = 0.01
- double c = 10.0
- double nu_p = 0.01

5.22.1 Member Data Documentation

5.22.1.1 c

```
double saturn::edac::Parameters::c = 10.0
```

5.22.1.2 nu

```
double saturn::edac::Parameters::nu = 0.01
```

5.22.1.3 nu_p

```
double saturn::edac::Parameters::nu_p = 0.01
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/edac.hpp

5.23 saturn::PatchField< ExecSpace > Class Template Reference

```
#include <patch_field.hpp>
```

Public Types

- using execution_space = ExecSpace
- using memory_space = typename ExecSpace::memory_space
- using view_type = Kokkos::View<double****, Kokkos::LayoutRight, memory_space>

Public Member Functions

- PatchField ()=default
- PatchField (const Box &box, int nvar, int ng, const std::string &label="q")
- KOKKOS_INLINE_FUNCTION double & operator() (int v, int i, int j, int k) const
- const Box & box () const
- int nvar () const
- int ng () const
- view_type & data ()
- const view_type & data () const
- void fillPeriodic ()

5.23.1 Member Typedef Documentation

5.23.1.1 execution_space

```
template<class ExecSpace >
using saturn::PatchField< ExecSpace >::execution_space = ExecSpace
```

5.23.1.2 memory_space

```
template<class ExecSpace >
using saturn::PatchField< ExecSpace >::memory_space = typename ExecSpace::memory_space
```

5.23.1.3 view_type

```
template<class ExecSpace >
using saturn::PatchField< ExecSpace >::view_type = Kokkos::View<double****, Kokkos::Layout↩
Right, memory_space>
```

5.23.2 Constructor & Destructor Documentation

5.23.2.1 PatchField() [1/2]

```
template<class ExecSpace >
saturn::PatchField< ExecSpace >::PatchField ()  [default]
```

5.23.2.2 PatchField() [2/2]

```
template<class ExecSpace >
saturn::PatchField< ExecSpace >::PatchField (
            const Box & box,
            int nvar,
            int ng,
            const std::string & label = "q")  [inline]
```

5.23.3 Member Function Documentation

5.23.3.1 box()

```
template<class ExecSpace >
const Box & saturn::PatchField< ExecSpace >::box () const  [inline]
```

5.23.3.2 data() [1/2]

```
template<class ExecSpace >
view_type & saturn::PatchField< ExecSpace >::data ()  [inline]
```

5.23.3.3 data() [2/2]

```
template<class ExecSpace >
const view_type & saturn::PatchField< ExecSpace >::data () const  [inline]
```

5.23.3.4 fillPeriodic()

```
template<class ExecSpace >
void saturn::PatchField< ExecSpace >::fillPeriodic ()  [inline]
```

5.23.3.5 ng()

```
template<class ExecSpace >
int saturn::PatchField< ExecSpace >::ng () const  [inline]
```

5.23.3.6 nvar()

```
template<class ExecSpace >
int saturn::PatchField< ExecSpace >::nvar () const  [inline]
```

5.23.3.7 operator()()

```
template<class ExecSpace >
KOKKOS_INLINE_FUNCTION double & saturn::PatchField< ExecSpace >::operator() (
            int v,
            int i,
            int j,
            int k) const  [inline]
```

The documentation for this class was generated from the following file:

- src/fields/patch_field.hpp

5.24 saturn::vof::Point Struct Reference

```
#include <vof2d.hpp>
```

Public Attributes

- double x
- double y

5.24.1 Member Data Documentation

5.24.1.1 x

```
double saturn::vof::Point::x
```

5.24.1.2 y

```
double saturn::vof::Point::y
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof2d.hpp

5.25 saturn::PrimitiveState Struct Reference

```
#include <riemann.hpp>
```

Public Attributes

- double rho
- double u
- double v
- double w
- double p

5.25.1 Member Data Documentation

5.25.1.1 p

```
double saturn::PrimitiveState::p
```

5.25.1.2 rho

```
double saturn::PrimitiveState::rho
```

5.25.1.3 u

```
double saturn::PrimitiveState::u
```

5.25.1.4 v

```
double saturn::PrimitiveState::v
```

5.25.1.5 w

```
double saturn::PrimitiveState::w
```

The documentation for this struct was generated from the following file:

- src/numerics/compressible/riemann.hpp

5.26 saturn::vof::Reconstruction Struct Reference

`#include <vof2d.hpp>`

Public Attributes

- double nx = 1.0
- double ny = 0.0
- double alpha = 0.0

5.26.1 Member Data Documentation

5.26.1.1 alpha

```
double saturn::vof::Reconstruction::alpha = 0.0
```

5.26.1.2 nx

```
double saturn::vof::Reconstruction::nx = 1.0
```

5.26.1.3 ny

```
double saturn::vof::Reconstruction::ny = 0.0
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof2d.hpp

5.27 saturn::vof::Reconstruction3D Struct Reference

`#include <vof3d.hpp>`

Public Attributes

- double nx = 1.0
- double ny = 0.0
- double nz = 0.0
- double alpha = 0.0

5.27.1 Member Data Documentation

5.27.1.1 alpha

```
double saturn::vof::Reconstruction3D::alpha = 0.0
```

5.27.1.2 nx

```
double saturn::vof::Reconstruction3D::nx = 1.0
```

5.27.1.3 ny

```
double saturn::vof::Reconstruction3D::ny = 0.0
```

5.27.1.4 nz

```
double saturn::vof::Reconstruction3D::nz = 0.0
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof3d.hpp

5.28 saturn::poisson::Result Struct Reference

```
#include <poisson.hpp>
```

Public Attributes

- int cycles = 0
- double initial_residual = 0.0
- double final_residual = 0.0
- bool converged = false

5.28.1 Member Data Documentation

5.28.1.1 converged

```
bool saturn::poisson::Result::converged = false
```

5.28.1.2 cycles

```
int saturn::poisson::Result::cycles = 0
```

5.28.1.3 final_residual

```
double saturn::poisson::Result::final_residual = 0.0
```

5.28.1.4 initial_residual

```
double saturn::poisson::Result::initial_residual = 0.0
```

The documentation for this struct was generated from the following file:

- src/numerics/incompressible/poisson.hpp

5.29 saturn::UniformGrid Struct Reference

```
#include <uniform_grid.hpp>
```

Public Member Functions

- Box localBox () const
- KOKKOS_INLINE_FUNCTION long long numCells () const

Static Public Member Functions

- static UniformGrid fromCells (int nx, int ny, int nz, double lx, double ly, double lz, int ng)
- static UniformGrid fromCellsLocal (int nx, int ny, int nz, double lx, double ly, double lz, int ng, const Box &local)

Public Attributes

- Box global
- Box local
- std::array< double, 3 > lo {{0.0, 0.0, 0.0}}
- std::array< double, 3 > hi {{1.0, 1.0, 1.0}}
- std::array< double, 3 > h {{1.0, 1.0, 1.0}}
- int ng = 2

5.29.1 Member Function Documentation

5.29.1.1 fromCells()

```
static UniformGrid saturn::UniformGrid::fromCells (
            int nx,
            int ny,
            int nz,
            double lx,
            double ly,
            double lz,
            int ng)  [inline], [static]
```

5.29.1.2 fromCellsLocal()

```
static UniformGrid saturn::UniformGrid::fromCellsLocal (
            int nx,
            int ny,
            int nz,
            double lx,
            double ly,
            double lz,
            int ng,
            const Box & local)  [inline], [static]
```

5.29.1.3 localBox()

```
Box saturn::UniformGrid::localBox () const  [inline]
```

5.29.1.4 numCells()

```
KOKKOS_INLINE_FUNCTION long long saturn::UniformGrid::numCells () const  [inline]
```

5.29.2 Member Data Documentation

5.29.2.1 global

```
Box saturn::UniformGrid::global
```

5.29.2.2 h

```
std::array<double, 3> saturn::UniformGrid::h {{1.0, 1.0, 1.0}}
```

5.29.2.3 hi

```
std::array<double, 3> saturn::UniformGrid::hi {{1.0, 1.0, 1.0}}
```

5.29.2.4 lo

```
std::array<double, 3> saturn::UniformGrid::lo {{0.0, 0.0, 0.0}}
```

5.29.2.5 local

```
Box saturn::UniformGrid::local
```

5.29.2.6 ng

```
int saturn::UniformGrid::ng = 2
```

The documentation for this struct was generated from the following file:

- src/mesh/uniform_grid.hpp

5.30 saturn::vof::Vec3 Struct Reference

```
#include <vof3d.hpp>
```

Public Attributes

- double x
- double y
- double z

5.30.1 Member Data Documentation

5.30.1.1 x

```
double saturn::vof::Vec3::x
```

5.30.1.2 y

```
double saturn::vof::Vec3::y
```

5.30.1.3 z

```
double saturn::vof::Vec3::z
```

The documentation for this struct was generated from the following file:

- src/numerics/vof/vof3d.hpp

Chapter 6

File Documentation

6.1 apps/cavity_sim.cpp File Reference

Lid-driven cavity driver: Ghia validation, GPU runs, clustering.

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "numerics/incompressible/cavity.hpp"
#include "numerics/incompressible/poisson.hpp"
#include "numerics/incompressible/projection.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <limits>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char ∗∗argv)

6.1.1 Detailed Description

Lid-driven cavity driver: Ghia validation, GPU runs, clustering.

Supports uniform and tanh-clustered grids, uniform or no-slip z walls, and central/upwind/MUSCL/WENO5 advection.

6.1.2 Macro Definition Documentation

6.1.2.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.1.3 Function Documentation

6.1.3.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.2 apps/channel_sim.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "numerics/incompressible/channel.hpp"
#include "numerics/incompressible/channel_poisson.hpp"
#include "numerics/incompressible/poisson.hpp"
#include "numerics/incompressible/projection.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <limits>
#include <string>
#include <type_traits>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char ∗∗argv)

6.2.1 Macro Definition Documentation

6.2.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.2.2 Function Documentation

6.2.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.3 apps/compressible_mms.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "fields/patch_field.hpp"
#include "halo/box_exchange.hpp"
#include "mesh/decomposition.hpp"
#include "mesh/uniform_grid.hpp"
#include "numerics/compressible/compressible_ns.hpp"
#include "numerics/compressible/reconstruction.hpp"
#include "numerics/compressible/riemann.hpp"
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char ∗∗argv)

6.3.1 Macro Definition Documentation

6.3.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.3.2 Function Documentation

6.3.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.4 apps/compressible_sim.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "fields/boundary_conditions.hpp"
#include "fields/patch_field.hpp"
#include "halo/box_exchange.hpp"
#include "halo/global_exchange.hpp"
#include "io/checkpoint.hpp"
#include "io/vtk_writer.hpp"
#include "mesh/decomposition.hpp"
#include "mesh/uniform_grid.hpp"
#include "numerics/compressible/compressible_ns.hpp"
#include "numerics/compressible/reconstruction.hpp"
#include "numerics/compressible/riemann.hpp"
#include "numerics/compressible/time_integration.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.4.1 Macro Definition Documentation

6.4.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.4.2 Function Documentation

6.4.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.5 apps/edac_sim.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "numerics/incompressible/edac.hpp"
#include "numerics/incompressible/projection.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <limits>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.5.1 Macro Definition Documentation

6.5.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.5.2 Function Documentation

6.5.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.6 apps/euler1d.cpp File Reference

```
#include "numerics/compressible/reconstruction.hpp"
#include "numerics/compressible/riemann.hpp"
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>
```

Functions

- int main (int argc, char **argv)

6.6.1 Function Documentation

6.6.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.7 apps/halo_bench.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "fields/patch_field.hpp"
#include "halo/face_exchange.hpp"
#include "mesh/decomposition.hpp"
#include "mesh/uniform_grid.hpp"
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.7.1 Macro Definition Documentation

6.7.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.7.2 Function Documentation

6.7.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.8 apps/incompressible_sim.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "numerics/incompressible/fft_poisson.hpp"
#include "numerics/incompressible/incompressible.hpp"
#include "numerics/incompressible/poisson.hpp"
#include "numerics/incompressible/projection.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <limits>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char ∗∗argv)

6.8.1 Macro Definition Documentation

6.8.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.8.2 Function Documentation

6.8.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.9 apps/scalar_mms.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include "fields/patch_field.hpp"
#include "halo/face_exchange.hpp"
#include "io/vtk_writer.hpp"
#include "mesh/decomposition.hpp"
#include "mesh/uniform_grid.hpp"
#include "numerics/compressible/scalar_advection.hpp"
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.9.1 Macro Definition Documentation

6.9.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.9.2 Function Documentation

6.9.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.10 apps/stencil_bench.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/parallel.hpp"
#include "core/version.hpp"
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.10.1 Macro Definition Documentation

6.10.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.10.2 Function Documentation

6.10.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.11 apps/stream_bench.cpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/execution_space.hpp"
#include "core/version.hpp"
#include <cstdio>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
```

Macros

- #define SATURN_DEFAULT_SPACE "default"

Functions

- int main (int argc, char **argv)

6.11.1 Macro Definition Documentation

6.11.1.1 SATURN_DEFAULT_SPACE

```
#define SATURN_DEFAULT_SPACE "default"
```

6.11.2 Function Documentation

6.11.2.1 main()

```
int main (
            int argc,
            char ** argv)
```

6.12 src/core/execution_space.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <cstdio>
#include <string>
#include <utility>
```

Namespaces

- namespace saturn

Functions

- int saturn::unavailableBackend (const std::string &name)
- template<class F >
  int saturn::dispatchExecutionSpace (const std::string &requested, const char *default_space, F &&f)

6.13 execution_space.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <cstdio>
00006 #include <string>
00007 #include <utility>
00008
00009 namespace saturn {
00010
00011 inline int unavailableBackend(const std::string& name) {
00012   std::fprintf(stderr, "backend '%s' is not enabled in this Kokkos build\n",
00013                name.c_str());
00014   return 2;
00015 }
00016
00017 // Dispatch a callable on the requested execution-space type.
00018 //
00019 // The callable is invoked as f(Kokkos::SomeExecutionSpace{}), so a generic
00020 // lambda can recover the type with decltype and instantiate a template:
00021 //
00022 //   saturn::dispatchExecutionSpace(space, SATURN_DEFAULT_SPACE, [&](auto tag)
00023 //   {
00024 //     using space_type = decltype(tag);
00025 //     return runImpl<space_type>(params);
00026 //   });
00027 template <class F>
00028 int dispatchExecutionSpace(const std::string& requested,
00029                            const char* default_space, F&& f) {
00030   std::string space = requested;
00031   if (space == "default") {
00032     space = default_space;
00033   }
00034
00035   if (space == "default") {
00036     return std::forward<F>(f)(Kokkos::DefaultExecutionSpace{});
00037   }
00038   if (space == "serial") {
00039 #ifdef KOKKOS_ENABLE_SERIAL
00040     return std::forward<F>(f)(Kokkos::Serial{});
00041 #else
00042     return unavailableBackend(space);
00043 #endif
00044   }
00045   if (space == "openmp") {
00046 #ifdef KOKKOS_ENABLE_OPENMP
00047     return std::forward<F>(f)(Kokkos::OpenMP{});
00048 #else
00049     return unavailableBackend(space);
00050 #endif
00051   }
00052   if (space == "cuda") {
00053 #ifdef KOKKOS_ENABLE_CUDA
00054     return std::forward<F>(f)(Kokkos::Cuda{});
00055 #else
00056     return unavailableBackend(space);
00057 #endif
00058   }
00059   if (space == "hip") {
00060 #ifdef KOKKOS_ENABLE_HIP
00061     return std::forward<F>(f)(Kokkos::HIP{});
00062 #else
00063     return unavailableBackend(space);
00064 #endif
00065   }
00066   if (space == "sycl") {
00067 #ifdef KOKKOS_ENABLE_SYCL
00068     return std::forward<F>(f)(Kokkos::SYCL{});
00069 #else
00070     return unavailableBackend(space);
00071 #endif
00072   }
00073
00074   std::fprintf(stderr, "unknown execution space '%s'\n", space.c_str());
00075   return 2;
00076 }
00077
00078 }  // namespace saturn
```

6.14 src/core/mpi.hpp File Reference

Namespaces

- namespace saturn

Functions

- int saturn::mpiRank ()
- int saturn::mpiSize ()
- double saturn::mpiSum (double value)
- double saturn::mpiMax (double value)
- bool saturn::mpiAll (bool value)

6.15 mpi.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #ifdef SATURN_ENABLE_MPI
00004 #include <mpi.h>
00005 #endif
00006
00007 namespace saturn {
00008
00009 inline int mpiRank() {
00010 #ifdef SATURN_ENABLE_MPI
00011   int rank = 0;
00012   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
00013   return rank;
00014 #else
00015   return 0;
00016 #endif
00017 }
00018
00019 inline int mpiSize() {
00020 #ifdef SATURN_ENABLE_MPI
00021   int size = 1;
00022   MPI_Comm_size(MPI_COMM_WORLD, &size);
00023   return size;
00024 #else
00025   return 1;
00026 #endif
00027 }
00028
00029 inline double mpiSum(double value) {
00030 #ifdef SATURN_ENABLE_MPI
00031   double result = 0.0;
00032   MPI_Allreduce(&value, &result, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
00033   return result;
00034 #else
00035   return value;
00036 #endif
00037 }
00038
00039 inline double mpiMax(double value) {
00040 #ifdef SATURN_ENABLE_MPI
00041   double result = 0.0;
00042   MPI_Allreduce(&value, &result, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
00043   return result;
00044 #else
00045   return value;
00046 #endif
00047 }
00048
00049 inline bool mpiAll(bool value) {
00050 #ifdef SATURN_ENABLE_MPI
00051   int local = value ? 1 : 0;
00052   int result = 0;
00053   MPI_Allreduce(&local, &result, 1, MPI_INT, MPI_LAND, MPI_COMM_WORLD);
00054   return result != 0;
00055 #else
00056   return value;
00057 #endif
00058 }
00059
00060 }  // namespace saturn
```

6.16 src/core/parallel.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <array>
```

Namespaces

- namespace saturn

Typedefs

- template<class ExecSpace >
  using saturn::MDRange3 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<3>>
- template<class ExecSpace >
  using saturn::MDRange4 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<4>>
- using saturn::Int3 = Kokkos::Array<int, 3>
- using saturn::Int4 = Kokkos::Array<int, 4>

Functions

- template<class ExecSpace >
  MDRange3< ExecSpace > saturn::makeMdrange3 (Int3 lower, Int3 upper)
- template<class ExecSpace >
  MDRange4< ExecSpace > saturn::makeMdrange4 (Int4 lower, Int4 upper)

Variables

- constexpr int saturn::kInnerTile = 32
- template<class ExecSpace >
  constexpr bool saturn::is_host_execution_space

6.17 parallel.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <array>
00006
00007 namespace saturn {
00008
00009 // Inner-dimension tile for 3D MDRangePolicy kernels on device backends.
00010 // Kokkos's default tile was poorly coalesced on the Quadro P600; 32 contiguous
00011 // cells per warp fixed it (5 -> 28 GB/s). Host backends prefer Kokkos's own
00012 // default, so the helper below only applies a tile on device spaces.
00013 inline constexpr int kInnerTile = 32;
00014
00015 template <class ExecSpace>
00016 inline constexpr bool is_host_execution_space =
00017     Kokkos::SpaceAccessibility<Kokkos::HostSpace,
00018                                typename ExecSpace::memory_space>::accessible;
00019
00020 template <class ExecSpace>
00021 using MDRange3 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<3»;
00022
```

```
00023 template <class ExecSpace>
00024 using MDRange4 = Kokkos::MDRangePolicy<ExecSpace, Kokkos::Rank<4»;
00025
00026 using Int3 = Kokkos::Array<int, 3>;
00027 using Int4 = Kokkos::Array<int, 4>;
00028
00029 template <class ExecSpace>
00030 MDRange3<ExecSpace> makeMdrange3(Int3 lower, Int3 upper) {
00031   if constexpr (is_host_execution_space<ExecSpace>) {
00032     return MDRange3<ExecSpace>(lower, upper);
00033   } else {
00034     return MDRange3<ExecSpace>(lower, upper, Int3{{1, 1, kInnerTile}});
00035   }
00036 }
00037
00038 template <class ExecSpace>
00039 MDRange4<ExecSpace> makeMdrange4(Int4 lower, Int4 upper) {
00040   if constexpr (is_host_execution_space<ExecSpace>) {
00041     return MDRange4<ExecSpace>(lower, upper);
00042   } else {
00043     return MDRange4<ExecSpace>(lower, upper, Int4{{1, 1, 1, kInnerTile}});
00044   }
00045 }
00046
00047 }  // namespace saturn
```

6.18 src/core/version.hpp File Reference

Namespaces

- namespace saturn

Variables

- constexpr const char * saturn::kProjectName = "saturnCFD"
- constexpr const char * saturn::kAuthor = "Vinaykanth"
- constexpr const char * saturn::kVersion = "0.1.0"

6.19 version.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 namespace saturn {
00004
00005 inline constexpr const char* kProjectName = "saturnCFD";
00006 inline constexpr const char* kAuthor = "Vinaykanth";
00007 inline constexpr const char* kVersion = "0.1.0";
00008
00009 }  // namespace saturn
```

6.20 src/fields/boundary_conditions.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/parallel.hpp"
#include "fields/patch_field.hpp"
#include "mesh/box.hpp"
#include <string>
```

Namespaces

- namespace saturn

Enumerations

- enum class saturn::BoundaryType { saturn::Periodic , saturn::Transmissive , saturn::SlipWall }

Functions

- BoundaryType saturn::parseBoundaryType (const std::string &name)
- template<class ExecSpace >
  void saturn::applyBoundaryConditions (PatchField< ExecSpace > &q, const BoundaryType bc[3][2])

6.21 boundary_conditions.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "core/parallel.hpp"
00006 #include "fields/patch_field.hpp"
00007 #include "mesh/box.hpp"
00008
00009 #include <string>
00010
00011 namespace saturn {
00012
00013 // Ghost-cell boundary treatment for the compressible solver. `bc[d][0]` is
00014 // the low side and `bc[d][1]` the high side of dimension d.
00015 //
00016 // Periodic ghosts are filled by wrapping the global coordinate. This routine
00017 // is used on a single rank (or a rank whose local box is the whole domain);
00018 // multi-rank periodic exchange is handled by BoxExchange. Transmissive and
00019 // slip-wall boundaries are suitable for the first single-rank shock cases.
00020 enum class BoundaryType {
00021   Periodic,
00022   Transmissive,
00023   SlipWall,
00024 };
00025
00026 inline BoundaryType parseBoundaryType(const std::string& name) {
00027   if (name == "periodic") {
00028     return BoundaryType::Periodic;
00029   }
00030   if (name == "slip" || name == "wall" || name == "reflective") {
00031     return BoundaryType::SlipWall;
00032   }
00033   return BoundaryType::Transmissive;
00034 }
00035
00036 // Fill all ghost cells of a five-variable compressible field. The kernel
00037 // treats each coordinate independently, so edges and corners are filled too.
00038 template <class ExecSpace>
00039 void applyBoundaryConditions(PatchField<ExecSpace>& q,
00040                              const BoundaryType bc[3][2]) {
00041   const Box box = q.box();
00042   const int ng = q.ng();
00043   // Copy the boundary types into scalar captures. Capturing a C array or a
00044   // std::array in a CUDA lambda can bind host memory / misbehave on device.
00045   const BoundaryType bc00 = bc[0][0];
00046   const BoundaryType bc01 = bc[0][1];
00047   const BoundaryType bc10 = bc[1][0];
00048   const BoundaryType bc11 = bc[1][1];
00049   const BoundaryType bc20 = bc[2][0];
00050   const BoundaryType bc21 = bc[2][1];
00051   const int np0 = box.length(0) + 2 * ng;
00052   const int np1 = box.length(1) + 2 * ng;
00053   const int np2 = box.length(2) + 2 * ng;
00054   const auto policy =
00055       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{np2, np1, np0}});
```

```
00056   auto data = q.data();
00057   const int nvar = q.nvar();
00058
00059   Kokkos::parallel_for(
00060       "compressible_boundary", policy, KOKKOS_LAMBDA(int kp, int jp, int ip) {
00061         const int i = box.lo[0] - ng + ip;
00062         const int j = box.lo[1] - ng + jp;
00063         const int k = box.lo[2] - ng + kp;
00064         const bool outside = i < box.lo[0] || i > box.hi[0] || j < box.lo[1] ||
00065                              j > box.hi[1] || k < box.lo[2] || k > box.hi[2];
00066         if (!outside) {
00067           return;
00068         }
00069
00070         int si = i;
00071         int sj = j;
00072         int sk = k;
00073         bool flip_x = false;
00074         bool flip_y = false;
00075         bool flip_z = false;
00076
00077         const int coordinate[3] = {i, j, k};
00078         int source[3] = {i, j, k};
00079         bool flip[3] = {false, false, false};
00080         for (int d = 0; d < 3; ++d) {
00081           const int lo = box.lo[d];
00082           const int hi = box.hi[d];
00083           if (coordinate[d] >= lo && coordinate[d] <= hi) {
00084             continue;
00085           }
00086           const int side = coordinate[d] < lo ? 0 : 1;
00087           const BoundaryType type =
00088               (d == 0) ? (side == 0 ? bc00 : bc01)
00089                        : ((d == 1) ? (side == 0 ? bc10 : bc11)
00090                                    : (side == 0 ? bc20 : bc21));
00091           if (type == BoundaryType::Periodic) {
00092             const int n = hi - lo + 1;
00093             int wrapped = (coordinate[d] - lo) % n;
00094             if (wrapped < 0) {
00095               wrapped += n;
00096             }
00097             source[d] = lo + wrapped;
00098           } else if (type == BoundaryType::SlipWall) {
00099             if (coordinate[d] < lo) {
00100               source[d] = 2 * lo - coordinate[d] - 1;
00101             } else {
00102               source[d] = 2 * hi - coordinate[d] + 1;
00103             }
00104             flip[d] = true;
00105           } else {
00106             source[d] = coordinate[d] < lo ? lo : hi;
00107           }
00108         }
00109         si = source[0];
00110         sj = source[1];
00111         sk = source[2];
00112         flip_x = flip[0];
00113         flip_y = flip[1];
00114         flip_z = flip[2];
00115
00116         const int source_ip = si - box.lo[0] + ng;
00117         const int source_jp = sj - box.lo[1] + ng;
00118         const int source_kp = sk - box.lo[2] + ng;
00119         for (int v = 0; v < nvar; ++v) {
00120           double value = data(v, source_kp, source_jp, source_ip);
00121           if ((v == 1 && flip_x) || (v == 2 && flip_y) || (v == 3 && flip_z)) {
00122             value = -value;
00123           }
00124           data(v, kp, jp, ip) = value;
00125         }
00126       });
00127 }
00128
00129 }  // namespace saturn
```

6.22 src/fields/patch_field.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <string>
#include "core/parallel.hpp"
```

```
#include "mesh/box.hpp"
```

Classes

- class saturn::PatchField< ExecSpace >

Namespaces

- namespace saturn

6.23 patch_field.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <string>
00006
00007 #include "core/parallel.hpp"
00008 #include "mesh/box.hpp"
00009
00010 namespace saturn {
00011
00012 // Cell-centered field on one box, with ng ghost layers on every side.
00013 //
00014 // Layout: [variable][k][j][i], i fastest (LayoutRight), so the inner dimension
00015 // is contiguous and vectorizes/coalesces.
00016 template <class ExecSpace>
00017 class PatchField {
00018  public:
00019   using execution_space = ExecSpace;
00020   using memory_space = typename ExecSpace::memory_space;
00021   using view_type = Kokkos::View<double****, Kokkos::LayoutRight, memory_space>;
00022
00023   PatchField() = default;
00024
00025   PatchField(const Box& box, int nvar, int ng, const std::string& label = "q")
00026       : box_(box),
00027         nvar_(nvar),
00028         ng_(ng),
00029         data_(label, nvar, box.length(2) + 2 * ng, box.length(1) + 2 * ng,
00030               box.length(0) + 2 * ng) {}
00031
00032   KOKKOS_INLINE_FUNCTION
00033   double& operator()(int v, int i, int j, int k) const {
00034     return data_(v, k - box_.lo[2] + ng_, j - box_.lo[1] + ng_,
00035                  i - box_.lo[0] + ng_);
00036   }
00037
00038   const Box& box() const { return box_; }
00039   int nvar() const { return nvar_; }
00040   int ng() const { return ng_; }
00041   view_type& data() { return data_; }
00042   const view_type& data() const { return data_; }
00043
00044   // Periodic ghost fill for a patch that spans the full periodic domain.
00045   // Multi-rank exchanges will replace this with halo messages.
00046   void fillPeriodic() {
00047     const int n0 = box_.length(0);
00048     const int n1 = box_.length(1);
00049     const int n2 = box_.length(2);
00050     const int np0 = n0 + 2 * ng_;
00051     const int np1 = n1 + 2 * ng_;
00052     const int np2 = n2 + 2 * ng_;
00053
00054     auto data = data_;
00055     const int ng = ng_;
00056     const int nvar = nvar_;
00057     const auto policy =
00058         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{np2, np1, np0}});
00059
```

```
00060     Kokkos::parallel_for(
00061         "periodic_fill", policy, KOKKOS_LAMBDA(int kp, int jp, int ip) {
00062           const int ii = wrap(ip - ng, n0);
00063           const int jj = wrap(jp - ng, n1);
00064           const int kk = wrap(kp - ng, n2);
00065           for (int v = 0; v < nvar; ++v) {
00066             data(v, kp, jp, ip) = data(v, kk + ng, jj + ng, ii + ng);
00067           }
00068         });
00069   }
00070
00071  private:
00072   KOKKOS_INLINE_FUNCTION
00073   static int wrap(int i, int n) { return ((i % n) + n) % n; }
00074
00075   Box box_{};
00076   int nvar_ = 1;
00077   int ng_ = 0;
00078   view_type data_{};
00079 };
00080
00081 }  // namespace saturn
```

6.24 src/halo/box_exchange.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "fields/patch_field.hpp"
#include "mesh/decomposition.hpp"
#include <vector>
```

Classes

- class saturn::BoxExchange< ExecSpace >

Namespaces

- namespace saturn

6.25 box_exchange.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "fields/patch_field.hpp"
00006 #include "mesh/decomposition.hpp"
00007
00008 #include <vector>
00009
00010 #ifdef SATURN_ENABLE_MPI
00011 #include <mpi.h>
00012 #endif
00013
00014 namespace saturn {
00015
00016 // Full 3D halo exchange for cell-centered fields.
00017 //
00018 // Unlike FaceExchange, this fills edges and corners as well as faces, which is
00019 // required by the fourth-order viscous stencil. It exchanges all 26 Cartesian
00020 // neighbor blocks. The implementation is host-staged (MPI is not assumed to be
00021 // GPU-aware) and currently uses one MPI_Sendrecv per block; it is correct for
00022 // moderate grids and can be replaced by a fused neighbor exchange later.
00023 //
00024 // Non-periodic boundaries are skipped; apply the physical boundary conditions
```

```
00025 // after this exchange.
00026 template <class ExecSpace>
00027 class BoxExchange {
00028  public:
00029   BoxExchange(const Decomposition& decomposition, int nvar, int ng)
00030       : decomposition_(decomposition), nvar_(nvar), ng_(ng) {}
00031
00032   void exchange(PatchField<ExecSpace>& q) {
00033     const bool all_periodic = decomposition_.periodic[0] &&
00034                               decomposition_.periodic[1] &&
00035                               decomposition_.periodic[2];
00036 #ifndef SATURN_ENABLE_MPI
00037     if (all_periodic) {
00038       q.fillPeriodic();
00039     }
00040     (void)q;
00041 #else
00042     if (decomposition_.size == 1) {
00043       if (all_periodic) {
00044         q.fillPeriodic();
00045       }
00046       return;
00047     }
00048     (void)all_periodic;
00049
00050     auto host =
00051         Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), q.data());
00052
00053     struct Pending {
00054       int neighbor = 0;
00055       int count = 0;
00056       int send_tag = 0;
00057       int recv_tag = 0;
00058       int offset[3] = {0, 0, 0};
00059       int extent[3] = {0, 0, 0};
00060       std::vector<double> send;
00061       std::vector<double> recv;
00062       MPI_Request requests[2]{MPI_REQUEST_NULL, MPI_REQUEST_NULL};
00063     };
00064
00065     std::vector<Pending> pending;
00066     pending.reserve(26);
00067     for (int ox = -1; ox <= 1; ++ox) {
00068       for (int oy = -1; oy <= 1; ++oy) {
00069         for (int oz = -1; oz <= 1; ++oz) {
00070           if (ox == 0 && oy == 0 && oz == 0) {
00071             continue;
00072           }
00073           const Box box = decomposition_.local;
00074           const int off[3] = {ox, oy, oz};
00075           int extent[3];
00076           for (int d = 0; d < 3; ++d) {
00077             extent[d] = (off[d] == 0) ? box.length(d) : ng_;
00078           }
00079
00080           int neighbor_coords[3];
00081           bool skip = false;
00082           for (int d = 0; d < 3; ++d) {
00083             int coordinate = decomposition_.coords[d] + off[d];
00084             if (coordinate < 0 || coordinate >= decomposition_.dims[d]) {
00085               if (!decomposition_.periodic[d]) {
00086                 skip = true;
00087                 break;
00088               }
00089               coordinate = (coordinate + decomposition_.dims[d]) %
00090                            decomposition_.dims[d];
00091             }
00092             neighbor_coords[d] = coordinate;
00093           }
00094           if (skip) {
00095             continue;
00096           }
00097
00098           Pending block;
00099           int neighbor = decomposition_.rank;
00100           MPI_Cart_rank(decomposition_.comm, neighbor_coords, &neighbor);
00101           block.neighbor = neighbor;
00102           block.offset[0] = ox;
00103           block.offset[1] = oy;
00104           block.offset[2] = oz;
00105           block.extent[0] = extent[0];
00106           block.extent[1] = extent[1];
00107           block.extent[2] = extent[2];
00108           block.count = nvar_ * extent[0] * extent[1] * extent[2];
00109           block.send_tag = tagFor(ox, oy, oz);
00110           block.recv_tag = tagFor(-ox, -oy, -oz);
00111           block.send.resize(static_cast<std::size_t>(block.count));
```

```
00112           block.recv.resize(static_cast<std::size_t>(block.count));
00113           if (neighbor == decomposition_.rank) {
00114             // A periodic dimension with one rank maps a boundary to its own
00115             // opposite side, not to the same side. Pack the mirror block.
00116             const int mirror[3] = {-ox, -oy, -oz};
00117             pack(host, box, mirror, extent, block.send);
00118             unpack(host, box, off, extent, block.send);
00119             continue;
00120           }
00121           pack(host, box, off, extent, block.send);
00122           pending.push_back(std::move(block));
00123         }
00124       }
00125     }
00126
00127     for (auto& block : pending) {
00128       MPI_Isend(block.send.data(), block.count, MPI_DOUBLE, block.neighbor,
00129                 block.send_tag, decomposition_.comm, &block.requests[0]);
00130       MPI_Irecv(block.recv.data(), block.count, MPI_DOUBLE, block.neighbor,
00131                 block.recv_tag, decomposition_.comm, &block.requests[1]);
00132     }
00133     for (auto& block : pending) {
00134       MPI_Waitall(2, block.requests, MPI_STATUSES_IGNORE);
00135     }
00136     for (auto& block : pending) {
00137       unpack(host, decomposition_.local, block.offset, block.extent,
00138              block.recv);
00139     }
00140     Kokkos::deep_copy(q.data(), host);
00141 #endif
00142   }
00143
00144  private:
00145   static int tagFor(int ox, int oy, int oz) {
00146     return 100 + (ox + 1) * 9 + (oy + 1) * 3 + (oz + 1);
00147   }
00148
00149 #ifdef SATURN_ENABLE_MPI
00150   template <class HostView>
00151   void pack(const HostView& host, const Box& box, const int off[3],
00152             const int extent[3], std::vector<double>& buffer) const {
00153     int index = 0;
00154     for (int v = 0; v < nvar_; ++v) {
00155       for (int kk = 0; kk < extent[2]; ++kk) {
00156         const int k = (off[2] > 0) ? box.hi[2] - ng_ + 1 + kk : box.lo[2] + kk;
00157         for (int jj = 0; jj < extent[1]; ++jj) {
00158           const int j =
00159               (off[1] > 0) ? box.hi[1] - ng_ + 1 + jj : box.lo[1] + jj;
00160           for (int ii = 0; ii < extent[0]; ++ii) {
00161             const int i =
00162                 (off[0] > 0) ? box.hi[0] - ng_ + 1 + ii : box.lo[0] + ii;
00163             buffer[static_cast<std::size_t>(index++)] =
00164                 host(v, k - box.lo[2] + ng_, j - box.lo[1] + ng_,
00165                      i - box.lo[0] + ng_);
00166           }
00167         }
00168       }
00169     }
00170   }
00171
00172   template <class HostView>
00173   void unpack(HostView& host, const Box& box, const int off[3],
00174               const int extent[3], const std::vector<double>& buffer) const {
00175     int index = 0;
00176     for (int v = 0; v < nvar_; ++v) {
00177       for (int kk = 0; kk < extent[2]; ++kk) {
00178         const int k =
00179             (off[2] > 0) ? box.hi[2] + 1 + kk
00180                          : (off[2] < 0 ? box.lo[2] - ng_ + kk : box.lo[2] + kk);
00181         for (int jj = 0; jj < extent[1]; ++jj) {
00182           const int j = (off[1] > 0) ? box.hi[1] + 1 + jj
00183                                      : (off[1] < 0 ? box.lo[1] - ng_ + jj
00184                                                    : box.lo[1] + jj);
00185           for (int ii = 0; ii < extent[0]; ++ii) {
00186             const int i = (off[0] > 0) ? box.hi[0] + 1 + ii
00187                                        : (off[0] < 0 ? box.lo[0] - ng_ + ii
00188                                                      : box.lo[0] + ii);
00189             host(v, k - box.lo[2] + ng_, j - box.lo[1] + ng_,
00190                  i - box.lo[0] + ng_) =
00191                 buffer[static_cast<std::size_t>(index++)];
00192           }
00193         }
00194       }
00195     }
00196   }
00197 #endif
00198
```

```
00199   Decomposition decomposition_{};
00200   int nvar_ = 1;
00201   int ng_ = 3;
00202 };
00203
00204 }  // namespace saturn
```

6.26 src/halo/face_exchange.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/parallel.hpp"
#include "fields/patch_field.hpp"
#include "mesh/decomposition.hpp"
#include <cstdio>
```

Classes

- class saturn::FaceExchange< ExecSpace >

Namespaces

- namespace saturn

6.27 face_exchange.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "core/parallel.hpp"
00006 #include "fields/patch_field.hpp"
00007 #include "mesh/decomposition.hpp"
00008
00009 #include <cstdio>
00010
00011 namespace saturn {
00012
00013 // Face-only halo exchange for cell-centered fields with ng ghost layers.
00014 //
00015 // Only the six face neighbors are needed by dimension-by-dimension stencils;
00016 // edges and corners are not exchanged. The exchange is host-staged on device
00017 // backends (MPI is not assumed to be GPU-aware). GPU-aware MPI can replace the
00018 // staging later without changing the interface.
00019 template <class ExecSpace>
00020 class FaceExchange {
00021  public:
00022   using memory_space = typename ExecSpace::memory_space;
00023   using buffer_type = Kokkos::View<double*, memory_space>;
00024   using host_buffer_type = Kokkos::View<double*, Kokkos::HostSpace>;
00025
00026   FaceExchange(const Decomposition& decomposition, int nvar, int ng)
00027       : decomposition_(decomposition), nvar_(nvar), ng_(ng) {
00028     const Box& box = decomposition_.local;
00029     for (int d = 0; d < 3; ++d) {
00030       const int n1 = (d == 0) ? box.length(1) : box.length(0);
00031       const int n2 =
00032           (d == 0) ? box.length(2) : ((d == 1) ? box.length(2) : box.length(1));
00033       n1_[d] = n1;
00034       n2_[d] = n2;
00035       size_[d] = nvar_ * ng_ * n1 * n2;
00036       for (int side = 0; side < 2; ++side) {
00037         send_[d][side] = buffer_type("send", size_[d]);
00038         recv_[d][side] = buffer_type("recv", size_[d]);
```

```
00039         host_send_[d][side] = Kokkos::create_mirror_view(send_[d][side]);
00040         host_recv_[d][side] = Kokkos::create_mirror_view(recv_[d][side]);
00041       }
00042     }
00043   }
00044
00045   // Start a non-blocking exchange. Ghost cells on MPI faces are not valid
00046   // until end() is called; interior cells (one layer inside the local box)
00047   // are safe to compute in between.
00048   void begin(PatchField<ExecSpace>& q) {
00049 #ifndef SATURN_ENABLE_MPI
00050     q.fillPeriodic();
00051     (void)q;
00052 #else
00053     nrequests_ = 0;
00054     npending_ = 0;
00055
00056     for (int d = 0; d < 3; ++d) {
00057       for (int side_sign = -1; side_sign <= 1; side_sign += 2) {
00058         const int side = (side_sign < 0) ? 0 : 1;
00059         const int partner = decomposition_.partner(d, side_sign);
00060         if (partner == MPI_PROC_NULL) {
00061           continue;  // physical boundary; handled by BC kernels later
00062         }
00063         if (partner == decomposition_.rank) {
00064           // Periodic dimension with one rank: pack the opposite face and
00065           // unpack it into this side's ghosts immediately.
00066           pack(q, d, -side_sign, send_[d][side]);
00067           Kokkos::deep_copy(recv_[d][side], send_[d][side]);
00068           unpack(q, d, side_sign, recv_[d][side]);
00069           continue;
00070         }
00071         pack(q, d, side_sign, send_[d][side]);
00072         Kokkos::deep_copy(host_send_[d][side], send_[d][side]);
00073         Kokkos::fence();
00074
00075         const int send_tag = 100 + d * 2 + side;
00076         const int recv_tag = 100 + d * 2 + (1 - side);
00077         MPI_Isend(host_send_[d][side].data(), size_[d], MPI_DOUBLE, partner,
00078                   send_tag, decomposition_.comm, &requests_[nrequests_++]);
00079         MPI_Irecv(host_recv_[d][side].data(), size_[d], MPI_DOUBLE, partner,
00080                   recv_tag, decomposition_.comm, &requests_[nrequests_++]);
00081         pending_dim_[npending_] = d;
00082         pending_side_[npending_] = side;
00083         ++npending_;
00084       }
00085     }
00086 #endif
00087   }
00088
00089   // Wait for the outstanding messages and fill the MPI-face ghosts.
00090   void end(PatchField<ExecSpace>& q) {
00091 #ifdef SATURN_ENABLE_MPI
00092     if (nrequests_ > 0) {
00093       MPI_Waitall(nrequests_, requests_, MPI_STATUSES_IGNORE);
00094     }
00095     for (int k = 0; k < npending_; ++k) {
00096       const int d = pending_dim_[k];
00097       const int side = pending_side_[k];
00098       Kokkos::deep_copy(recv_[d][side], host_recv_[d][side]);
00099       Kokkos::fence();
00100       unpack(q, d, (side == 0) ? -1 : 1, recv_[d][side]);
00101     }
00102     nrequests_ = 0;
00103     npending_ = 0;
00104 #else
00105     (void)q;
00106 #endif
00107   }
00108
00109   // Blocking convenience wrapper.
00110   void exchange(PatchField<ExecSpace>& q) {
00111     begin(q);
00112     end(q);
00113   }
00114
00115  public:
00116   // pack/unpack must be public: nvcc's extended-lambda support rejects
00117   // __host__ __device__ lambdas inside private member functions.
00118   void pack(const PatchField<ExecSpace>& q, int d, int side_sign,
00119             buffer_type& buffer) {
00120     const Box box = decomposition_.local;
00121     const int ng = ng_;
00122     const int n1 = n1_[d];
00123     const int n2 = n2_[d];
00124     const int nvar = nvar_;
00125     const int lo0 = box.lo[0];
```

```
00126     const int lo1 = box.lo[1];
00127     const int lo2 = box.lo[2];
00128     const int hi0 = box.hi[0];
00129     const int hi1 = box.hi[1];
00130     const int hi2 = box.hi[2];
00131
00132     const auto policy =
00133         makeMdrange4<ExecSpace>(Int4{{0, 0, 0, 0}}, Int4{{nvar, ng, n1, n2}});
00134     Kokkos::parallel_for(
00135         "halo_pack", policy, KOKKOS_LAMBDA(int v, int f, int a, int b) {
00136           int i = 0;
00137           int j = 0;
00138           int k = 0;
00139           if (d == 0) {
00140             i = (side_sign < 0) ? (lo0 + f) : (hi0 - ng + 1 + f);
00141             j = lo1 + a;
00142             k = lo2 + b;
00143           } else if (d == 1) {
00144             j = (side_sign < 0) ? (lo1 + f) : (hi1 - ng + 1 + f);
00145             i = lo0 + a;
00146             k = lo2 + b;
00147           } else {
00148             k = (side_sign < 0) ? (lo2 + f) : (hi2 - ng + 1 + f);
00149             i = lo0 + a;
00150             j = lo1 + b;
00151           }
00152           const int idx = ((v * ng + f) * n1 + a) * n2 + b;
00153           buffer(idx) = q(v, i, j, k);
00154         });
00155   }
00156
00157   void unpack(PatchField<ExecSpace>& q, int d, int side_sign,
00158               const buffer_type& buffer) {
00159     const Box box = decomposition_.local;
00160     const int ng = ng_;
00161     const int n1 = n1_[d];
00162     const int n2 = n2_[d];
00163     const int nvar = nvar_;
00164     const int lo0 = box.lo[0];
00165     const int lo1 = box.lo[1];
00166     const int lo2 = box.lo[2];
00167     const int hi0 = box.hi[0];
00168     const int hi1 = box.hi[1];
00169     const int hi2 = box.hi[2];
00170
00171     const auto policy =
00172         makeMdrange4<ExecSpace>(Int4{{0, 0, 0, 0}}, Int4{{nvar, ng, n1, n2}});
00173     Kokkos::parallel_for(
00174         "halo_unpack", policy, KOKKOS_LAMBDA(int v, int f, int a, int b) {
00175           int i = 0;
00176           int j = 0;
00177           int k = 0;
00178           if (d == 0) {
00179             i = (side_sign < 0) ? (lo0 - ng + f) : (hi0 + 1 + f);
00180             j = lo1 + a;
00181             k = lo2 + b;
00182           } else if (d == 1) {
00183             j = (side_sign < 0) ? (lo1 - ng + f) : (hi1 + 1 + f);
00184             i = lo0 + a;
00185             k = lo2 + b;
00186           } else {
00187             k = (side_sign < 0) ? (lo2 - ng + f) : (hi2 + 1 + f);
00188             i = lo0 + a;
00189             j = lo1 + b;
00190           }
00191           const int idx = ((v * ng + f) * n1 + a) * n2 + b;
00192           q(v, i, j, k) = buffer(idx);
00193         });
00194   }
00195
00196  private:
00197   Decomposition decomposition_{};
00198   int nvar_ = 1;
00199   int ng_ = 1;
00200   int n1_[3]{1, 1, 1};
00201   int n2_[3]{1, 1, 1};
00202   int size_[3]{0, 0, 0};
00203   buffer_type send_[3][2]{};
00204   buffer_type recv_[3][2]{};
00205   host_buffer_type host_send_[3][2]{};
00206   host_buffer_type host_recv_[3][2]{};
00207 #ifdef SATURN_ENABLE_MPI
00208   MPI_Request requests_[12]{};
00209   int pending_dim_[6]{0, 0, 0, 0, 0, 0};
00210   int pending_side_[6]{0, 0, 0, 0, 0, 0};
00211   int nrequests_ = 0;
00212   int npending_ = 0;
```

```
00213 #endif
00214 };
00215
00216 }  // namespace saturn
```

6.28 src/halo/global_exchange.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/parallel.hpp"
#include "fields/boundary_conditions.hpp"
#include "fields/patch_field.hpp"
#include "mesh/decomposition.hpp"
#include <cstdio>
#include <vector>
```

Classes

- class saturn::GlobalExchange< ExecSpace >

Namespaces

- namespace saturn

6.29 global_exchange.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "core/parallel.hpp"
00006 #include "fields/boundary_conditions.hpp"
00007 #include "fields/patch_field.hpp"
00008 #include "mesh/decomposition.hpp"
00009
00010 #include <cstdio>
00011 #include <vector>
00012
00013 #ifdef SATURN_ENABLE_MPI
00014 #include <mpi.h>
00015 #endif
00016
00017 namespace saturn {
00018
00019 // Correctness-first global halo/BC fill for multi-rank non-periodic runs.
00020 //
00021 // Every rank all-gathers the interior fields, then each rank fills its ghost
00022 // cells by mapping the global coordinate through the boundary conditions and
00023 // looking up the owning rank's local block. This handles faces, edges, and
00024 // corners and mixed periodic/transmissive/slip boundaries in one pass.
00025 //
00026 // It is O(N) communication per exchange and is intended as a transitional
00027 // fallback; all-periodic runs use the scalable BoxExchange.
00028 template <class ExecSpace>
00029 class GlobalExchange {
00030  public:
00031   GlobalExchange(const Decomposition& decomposition, int nvar, int ng)
00032       : decomposition_(decomposition), nvar_(nvar), ng_(ng) {}
00033
00034   void exchange(PatchField<ExecSpace>& q, const BoundaryType bc[3][2]) {
00035 #ifndef SATURN_ENABLE_MPI
00036     applyBoundaryConditions(q, bc);
00037     (void)q;
```

```
00038 #else
00039     if (decomposition_.size == 1) {
00040       applyBoundaryConditions(q, bc);
00041       return;
00042     }
00043
00044     const Box local = decomposition_.local;
00045     const int local_count =
00046         nvar_ * local.length(0) * local.length(1) * local.length(2);
00047
00048     int meta[6] = {local.lo[0], local.lo[1], local.lo[2],
00049                    local.hi[0], local.hi[1], local.hi[2]};
00050     std::vector<int> all_meta(static_cast<std::size_t>(6) *
00051                               decomposition_.size);
00052     MPI_Allgather(meta, 6, MPI_INT, all_meta.data(), 6, MPI_INT,
00053                   decomposition_.comm);
00054
00055     std::vector<int> counts(static_cast<std::size_t>(decomposition_.size));
00056     std::vector<int> displacements(
00057         static_cast<std::size_t>(decomposition_.size));
00058     int total = 0;
00059     for (int rank = 0; rank < decomposition_.size; ++rank) {
00060       const int* m = &all_meta[static_cast<std::size_t>(rank) * 6];
00061       const int len0 = m[3] - m[0] + 1;
00062       const int len1 = m[4] - m[1] + 1;
00063       const int len2 = m[5] - m[2] + 1;
00064       counts[static_cast<std::size_t>(rank)] = nvar_ * len0 * len1 * len2;
00065       displacements[static_cast<std::size_t>(rank)] = total;
00066       total += counts[static_cast<std::size_t>(rank)];
00067     }
00068
00069     auto host =
00070         Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), q.data());
00071     std::vector<double> send(static_cast<std::size_t>(local_count));
00072     int index = 0;
00073     for (int v = 0; v < nvar_; ++v) {
00074       for (int k = local.lo[2]; k <= local.hi[2]; ++k) {
00075         for (int j = local.lo[1]; j <= local.hi[1]; ++j) {
00076           for (int i = local.lo[0]; i <= local.hi[0]; ++i) {
00077             send[static_cast<std::size_t>(index++)] =
00078                 host(v, k - local.lo[2] + ng_, j - local.lo[1] + ng_,
00079                      i - local.lo[0] + ng_);
00080           }
00081         }
00082       }
00083     }
00084     std::vector<double> global(static_cast<std::size_t>(total));
00085     MPI_Allgatherv(send.data(), local_count, MPI_DOUBLE, global.data(),
00086                    counts.data(), displacements.data(), MPI_DOUBLE,
00087                    decomposition_.comm);
00088
00089     const Box global_box = decomposition_.global;
00090     const int np0 = local.length(0) + 2 * ng_;
00091     const int np1 = local.length(1) + 2 * ng_;
00092     const int np2 = local.length(2) + 2 * ng_;
00093     for (int kp = 0; kp < np2; ++kp) {
00094       for (int jp = 0; jp < np1; ++jp) {
00095         for (int ip = 0; ip < np0; ++ip) {
00096           const int i = local.lo[0] - ng_ + ip;
00097           const int j = local.lo[1] - ng_ + jp;
00098           const int k = local.lo[2] - ng_ + kp;
00099           const bool outside = i < local.lo[0] || i > local.hi[0] ||
00100                                j < local.lo[1] || j > local.hi[1] ||
00101                                k < local.lo[2] || k > local.hi[2];
00102           if (!outside) {
00103             continue;
00104           }
00105
00106           const int coordinate[3] = {i, j, k};
00107           int source[3] = {i, j, k};
00108           bool flip[3] = {false, false, false};
00109           for (int d = 0; d < 3; ++d) {
00110             const int lo = global_box.lo[d];
00111             const int hi = global_box.hi[d];
00112             if (coordinate[d] >= lo && coordinate[d] <= hi) {
00113               continue;
00114             }
00115             const int side = coordinate[d] < lo ? 0 : 1;
00116             if (bc[d][side] == BoundaryType::Periodic) {
00117               const int n = hi - lo + 1;
00118               int wrapped = (coordinate[d] - lo) % n;
00119               if (wrapped < 0) {
00120                 wrapped += n;
00121               }
00122               source[d] = lo + wrapped;
00123             } else if (bc[d][side] == BoundaryType::SlipWall) {
00124               source[d] = coordinate[d] < lo ? 2 * lo - coordinate[d] - 1
```

```
00125                                             : 2 * hi - coordinate[d] + 1;
00126             flip[d] = true;
00127           } else {
00128             source[d] = coordinate[d] < lo ? lo : hi;
00129           }
00130         }
00131
00132         int owner = -1;
00133         for (int rank = 0; rank < decomposition_.size; ++rank) {
00134           const int* m = &all_meta[static_cast<std::size_t>(rank) * 6];
00135           if (source[0] >= m[0] && source[0] <= m[3] && source[1] >= m[1] &&
00136               source[1] <= m[4] && source[2] >= m[2] && source[2] <= m[5]) {
00137             owner = rank;
00138             break;
00139           }
00140         }
00141         if (owner < 0) {
00142           continue;  // should not happen for a valid BC mapping
00143         }
00144         const int* m = &all_meta[static_cast<std::size_t>(owner) * 6];
00145         const int len0 = m[3] - m[0] + 1;
00146         const int len1 = m[4] - m[1] + 1;
00147         const int base = displacements[static_cast<std::size_t>(owner)];
00148         for (int v = 0; v < nvar_; ++v) {
00149           const int source_index =
00150               base +
00151               ((v * (m[5] - m[2] + 1) + (source[2] - m[2])) * len1 +
00152                (source[1] - m[1])) *
00153                   len0 +
00154               (source[0] - m[0]);
00155           double value = global[static_cast<std::size_t>(source_index)];
00156           if ((v == 1 && flip[0]) || (v == 2 && flip[1]) ||
00157               (v == 3 && flip[2])) {
00158             value = -value;
00159           }
00160           host(v, kp, jp, ip) = value;
00161         }
00162       }
00163     }
00164   }
00165   Kokkos::deep_copy(q.data(), host);
00166 #endif
00167   }
00168
00169  private:
00170   Decomposition decomposition_{};
00171   int nvar_ = 5;
00172   int ng_ = 3;
00173 };
00174
00175 }  // namespace saturn
```

6.30 src/io/checkpoint.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "fields/patch_field.hpp"
#include "io/vtk_writer.hpp"
#include "mesh/decomposition.hpp"
#include <cstdio>
#include <fstream>
#include <string>
#include <type_traits>
```

Classes

- struct saturn::CheckpointHeader

Namespaces

- namespace saturn

Functions

- template<class ExecSpace >
  bool saturn::writeCheckpoint (const std::string &base, const PatchField< ExecSpace > &q, int step, double time, const Decomposition &decomposition)
- template<class ExecSpace >
  bool saturn::readCheckpoint (const std::string &base, PatchField< ExecSpace > &q, int &step, double &time, const Decomposition &decomposition)

6.31 checkpoint.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "fields/patch_field.hpp"
00006 #include "io/vtk_writer.hpp"
00007 #include "mesh/decomposition.hpp"
00008
00009 #include <cstdio>
00010 #include <fstream>
00011 #include <string>
00012 #include <type_traits>
00013
00014 namespace saturn {
00015
00016 struct CheckpointHeader {
00017   char magic[8] = {'S', 'A', 'T', 'U', 'R', 'N', 'C', 'K'};
00018   int version = 1;
00019   int rank = 0;
00020   int size = 1;
00021   int nvar = 1;
00022   int ng = 1;
00023   int local_lo[3] = {0, 0, 0};
00024   int local_hi[3] = {-1, -1, -1};
00025   int global_lo[3] = {0, 0, 0};
00026   int global_hi[3] = {-1, -1, -1};
00027   int step = 0;
00028   double time = 0.0;
00029 };
00030 static_assert(std::is_trivially_copyable_v<CheckpointHeader>);
00031
00032 template <class ExecSpace>
00033 bool writeCheckpoint(const std::string& base, const PatchField<ExecSpace>& q,
00034                      int step, double time,
00035                      const Decomposition& decomposition) {
00036   const Box box = q.box();
00037   CheckpointHeader header;
00038   header.rank = decomposition.rank;
00039   header.size = decomposition.size;
00040   header.nvar = q.nvar();
00041   header.ng = q.ng();
00042   for (int d = 0; d < 3; ++d) {
00043     header.local_lo[d] = box.lo[d];
00044     header.local_hi[d] = box.hi[d];
00045     header.global_lo[d] = decomposition.global.lo[d];
00046     header.global_hi[d] = decomposition.global.hi[d];
00047   }
00048   header.step = step;
00049   header.time = time;
00050
00051   const std::string path = base + rankSuffix(decomposition.rank) + ".bin";
00052   ensureParent(path);
00053   std::ofstream out(path, std::ios::binary);
00054   if (!out) {
00055     std::fprintf(stderr, "could not open %s for writing\n", path.c_str());
00056     return false;
00057   }
00058   out.write(reinterpret_cast<const char*>(&header), sizeof(header));
00059   if (!out) {
00060     return false;
00061   }
00062
00063   const auto host =
00064       Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), q.data());
00065   for (int v = 0; v < q.nvar(); ++v) {
00066     for (int k = box.lo[2]; k <= box.hi[2]; ++k) {
```

```
00067       for (int j = box.lo[1]; j <= box.hi[1]; ++j) {
00068         for (int i = box.lo[0]; i <= box.hi[0]; ++i) {
00069           const double value =
00070               host(v, k - box.lo[2] + q.ng(), j - box.lo[1] + q.ng(),
00071                    i - box.lo[0] + q.ng());
00072           out.write(reinterpret_cast<const char*>(&value), sizeof(value));
00073         }
00074       }
00075     }
00076   }
00077   return static_cast<bool>(out);
00078 }
00079
00080 template <class ExecSpace>
00081 bool readCheckpoint(const std::string& base, PatchField<ExecSpace>& q,
00082                     int& step, double& time,
00083                     const Decomposition& decomposition) {
00084   const std::string path = base + rankSuffix(decomposition.rank) + ".bin";
00085   std::ifstream in(path, std::ios::binary);
00086   if (!in) {
00087     return false;
00088   }
00089
00090   CheckpointHeader header;
00091   in.read(reinterpret_cast<char*>(&header), sizeof(header));
00092   if (!in) {
00093     return false;
00094   }
00095   const bool magic_ok = header.magic[0] == 'S' && header.magic[1] == 'A' &&
00096                         header.magic[2] == 'T' && header.magic[3] == 'U' &&
00097                         header.magic[4] == 'R' && header.magic[5] == 'N' &&
00098                         header.magic[6] == 'C' && header.magic[7] == 'K';
00099   if (!magic_ok || header.version != 1 || header.rank != decomposition.rank ||
00100       header.size != decomposition.size || header.nvar != q.nvar() ||
00101       header.ng != q.ng()) {
00102     return false;
00103   }
00104   const Box box = q.box();
00105   for (int d = 0; d < 3; ++d) {
00106     if (header.local_lo[d] != box.lo[d] || header.local_hi[d] != box.hi[d] ||
00107         header.global_lo[d] != decomposition.global.lo[d] ||
00108         header.global_hi[d] != decomposition.global.hi[d]) {
00109       return false;
00110     }
00111   }
00112
00113   auto host = Kokkos::create_mirror_view(q.data());
00114   for (int v = 0; v < q.nvar(); ++v) {
00115     for (int k = box.lo[2]; k <= box.hi[2]; ++k) {
00116       for (int j = box.lo[1]; j <= box.hi[1]; ++j) {
00117         for (int i = box.lo[0]; i <= box.hi[0]; ++i) {
00118           double value = 0.0;
00119           in.read(reinterpret_cast<char*>(&value), sizeof(value));
00120           if (!in) {
00121             return false;
00122           }
00123           host(v, k - box.lo[2] + q.ng(), j - box.lo[1] + q.ng(),
00124                i - box.lo[0] + q.ng()) = value;
00125         }
00126       }
00127     }
00128   }
00129   Kokkos::deep_copy(q.data(), host);
00130   step = header.step;
00131   time = header.time;
00132   return true;
00133 }
00134
00135 }  // namespace saturn
```

6.32 src/io/vtk_writer.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "fields/patch_field.hpp"
#include "mesh/box.hpp"
#include <array>
#include <cstdio>
#include <filesystem>
```

```
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>
```

Namespaces

- namespace saturn

Functions

- std::string saturn::rankSuffix (int rank)
- void saturn::ensureParent (const std::string &path)
- template<class ExecSpace >
  bool saturn::writeVtiFields (const std::string &base, const PatchField< ExecSpace > &q, const std::vector< std::string > &names, const std::array< double, 3 > &origin, const std::array< double, 3 > &spacing, const Box &global, double time)
- template<class ExecSpace >
  bool saturn::writeVti (const std::string &base, const PatchField< ExecSpace > &q, const std::array< double, 3 > &origin, const std::array< double, 3 > &spacing, const Box &global, double time)

6.33 vtk_writer.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "fields/patch_field.hpp"
00006 #include "mesh/box.hpp"
00007
00008 #include <array>
00009 #include <cstdio>
00010 #include <filesystem>
00011 #include <fstream>
00012 #include <iomanip>
00013 #include <string>
00014 #include <vector>
00015
00016 #ifdef SATURN_ENABLE_MPI
00017 #include <mpi.h>
00018 #endif
00019
00020 namespace saturn {
00021
00022 inline std::string rankSuffix(int rank) {
00023   char buffer[32];
00024   std::snprintf(buffer, sizeof(buffer), ".rank%04d", rank);
00025   return buffer;
00026 }
00027
00028 inline void ensureParent(const std::string& path) {
00029   const std::filesystem::path p(path);
00030   if (p.has_parent_path()) {
00031     std::filesystem::create_directories(p.parent_path());
00032   }
00033 }
00034
00035 // Write a cell-centered multi-variable field as VTK ImageData.
00036 //
00037 // Single rank:   <base>.vti
00038 // Multiple ranks: <base>.rankNNNN.vti + <base>.pvti (rank 0)
00039 //
00040 // `names` must contain at least q.nvar() entries.
00041 template <class ExecSpace>
00042 bool writeVtiFields(const std::string& base, const PatchField<ExecSpace>& q,
```

```
00043                     const std::vector<std::string>& names,
00044                     const std::array<double, 3>& origin,
00045                     const std::array<double, 3>& spacing, const Box& global,
00046                     double time) {
00047   (void)time;  // a time-series .pvd can be added later
00048   const Box local = q.box();
00049   const int ng = q.ng();
00050   const int nvar = q.nvar();
00051   if (nvar < 1) {
00052     return false;
00053   }
00054
00055   const auto host =
00056       Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), q.data());
00057
00058   int rank = 0;
00059   int size = 1;
00060 #ifdef SATURN_ENABLE_MPI
00061   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
00062   MPI_Comm_size(MPI_COMM_WORLD, &size);
00063 #endif
00064
00065   const std::string vti =
00066       (size > 1) ? (base + rankSuffix(rank) + ".vti") : (base + ".vti");
00067   ensureParent(vti);
00068   std::ofstream out(vti);
00069   if (!out) {
00070     std::fprintf(stderr, "could not open %s for writing\n", vti.c_str());
00071     return false;
00072   }
00073
00074   const int whole[6] = {global.lo[0],     global.hi[0] + 1, global.lo[1],
00075                         global.hi[1] + 1, global.lo[2],     global.hi[2] + 1};
00076   const int piece[6] = {local.lo[0],     local.hi[0] + 1, local.lo[1],
00077                         local.hi[1] + 1, local.lo[2],     local.hi[2] + 1};
00078
00079   auto nameOf = [&](int v) -> const std::string& {
00080     static const std::string fallback = "var";
00081     if (v < static_cast<int>(names.size())) {
00082       return names[static_cast<std::size_t>(v)];
00083     }
00084     return fallback;
00085   };
00086
00087   out « std::setprecision(17);
00088   out « "<?xml version=\"1.0\"?>\n"
00089       « "<VTKFile type=\"ImageData\" version=\"0.1\" "
00090         "byte_order=\"LittleEndian\">\n"
00091       « "  <ImageData WholeExtent=\"" « whole[0] « " " « whole[1] « " "
00092       « whole[2] « " " « whole[3] « " " « whole[4] « " " « whole[5]
00093       « "\" Origin=\"" « origin[0] « " " « origin[1] « " " « origin[2]
00094       « "\" Spacing=\"" « spacing[0] « " " « spacing[1] « " " « spacing[2]
00095       « "\">\n"
00096       « "    <Piece Extent=\"" « piece[0] « " " « piece[1] « " "
00097       « piece[2] « " " « piece[3] « " " « piece[4] « " " « piece[5]
00098       « "\">\n"
00099       « "      <CellData Scalars=\"" « nameOf(0) « "\">\n";
00100
00101   for (int v = 0; v < nvar; ++v) {
00102     out « "        <DataArray type=\"Float64\" Name=\"" « nameOf(v)
00103         « "\" format=\"ascii\">\n          ";
00104     for (int k = local.lo[2]; k <= local.hi[2]; ++k) {
00105       for (int j = local.lo[1]; j <= local.hi[1]; ++j) {
00106         for (int i = local.lo[0]; i <= local.hi[0]; ++i) {
00107           out « " "
00108               « host(v, k - local.lo[2] + ng, j - local.lo[1] + ng,
00109                      i - local.lo[0] + ng);
00110         }
00111       }
00112     }
00113     out « "\n        </DataArray>\n";
00114   }
00115   out « "      </CellData>\n"
00116       « "    </Piece>\n"
00117       « "  </ImageData>\n"
00118       « "</VTKFile>\n";
00119
00120   if (size == 1) {
00121     return true;
00122   }
00123
00124   std::vector<int> extents;
00125 #ifdef SATURN_ENABLE_MPI
00126   int local_extent[6] = {piece[0], piece[1], piece[2],
00127                          piece[3], piece[4], piece[5]};
00128   if (rank == 0) {
00129     extents.resize(static_cast<std::size_t>(6) * size);
```

```
00130   }
00131   MPI_Gather(local_extent, 6, MPI_INT, (rank == 0) ? extents.data() : nullptr,
00132              6, MPI_INT, 0, MPI_COMM_WORLD);
00133 #endif
00134
00135   if (rank == 0) {
00136     const std::string pvti = base + ".pvti";
00137     std::ofstream collection(pvti);
00138     if (!collection) {
00139       std::fprintf(stderr, "could not open %s for writing\n", pvti.c_str());
00140       return false;
00141     }
00142     collection « std::setprecision(17);
00143     collection « "<?xml version=\"1.0\"?>\n"
00144                « "<VTKFile type=\"PImageData\" version=\"0.1\" "
00145                   "byte_order=\"LittleEndian\">\n"
00146                « "  <PImageData WholeExtent=\"" « whole[0] « " " « whole[1]
00147                « " " « whole[2] « " " « whole[3] « " " « whole[4] « " "
00148                « whole[5] « "\" GhostLevel=\"0\" "
00149                « "Origin=\"" « origin[0] « " " « origin[1] « " "
00150                « origin[2] « "\" Spacing=\"" « spacing[0] « " "
00151                « spacing[1] « " " « spacing[2] « "\">\n"
00152                « "    <PCellData Scalars=\"" « nameOf(0) « "\">\n";
00153     for (int v = 0; v < nvar; ++v) {
00154       collection « "      <PDataArray type=\"Float64\" Name=\"" « nameOf(v)
00155                  « "\"/>\n";
00156     }
00157     collection « "    </PCellData>\n";
00158     for (int r = 0; r < size; ++r) {
00159       const int* e = &extents[static_cast<std::size_t>(r) * 6];
00160       collection « "    <Piece Extent=\"" « e[0] « " " « e[1] « " " « e[2]
00161                  « " " « e[3] « " " « e[4] « " " « e[5] « "\" Source=\""
00162                  « base « rankSuffix(r) « ".vti\"/>\n";
00163     }
00164     collection « "  </PImageData>\n"
00165                « "</VTKFile>\n";
00166   }
00167
00168   return true;
00169 }
00170
00171 // Scalar convenience wrapper.
00172 template <class ExecSpace>
00173 bool writeVti(const std::string& base, const PatchField<ExecSpace>& q,
00174               const std::array<double, 3>& origin,
00175               const std::array<double, 3>& spacing, const Box& global,
00176               double time) {
00177   return writeVtiFields(base, q, std::vector<std::string>{"c"}, origin, spacing,
00178                         global, time);
00179 }
00180
00181 }  // namespace saturn
```

6.34 src/mesh/box.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <algorithm>
```

Classes

- struct saturn::Box

Namespaces

- namespace saturn

6.35 box.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <algorithm>
00006
00007 namespace saturn {
00008
00009 // A 3D index box with inclusive bounds: [lo, hi].
00010 // Plain C arrays are used instead of std::array so the accessors are safe in
00011 // CUDA device code without relying on --expt-relaxed-constexpr.
00012 struct Box {
00013   int lo[3]{0, 0, 0};
00014   int hi[3]{-1, -1, -1};
00015
00016   KOKKOS_INLINE_FUNCTION
00017   bool empty() const { return hi[0] < lo[0] || hi[1] < lo[1] || hi[2] < lo[2]; }
00018
00019   KOKKOS_INLINE_FUNCTION
00020   int length(int d) const { return hi[d] - lo[d] + 1; }
00021
00022   KOKKOS_INLINE_FUNCTION
00023   long long numCells() const {
00024     if (empty()) {
00025       return 0;
00026     }
00027     return static_cast<long long>(length(0)) * length(1) * length(2);
00028   }
00029
00030   KOKKOS_INLINE_FUNCTION
00031   bool contains(int i, int j, int k) const {
00032     return i >= lo[0] && i <= hi[0] && j >= lo[1] && j <= hi[1] && k >= lo[2] &&
00033            k <= hi[2];
00034   }
00035
00036   KOKKOS_INLINE_FUNCTION
00037   Box grow(int n) const {
00038     Box result;
00039     for (int d = 0; d < 3; ++d) {
00040       result.lo[d] = lo[d] - n;
00041       result.hi[d] = hi[d] + n;
00042     }
00043     return result;
00044   }
00045
00046   KOKKOS_INLINE_FUNCTION
00047   Box shrink(int n) const { return grow(-n); }
00048
00049   KOKKOS_INLINE_FUNCTION
00050   Box shift(int di, int dj, int dk) const {
00051     Box result;
00052     result.lo[0] = lo[0] + di;
00053     result.lo[1] = lo[1] + dj;
00054     result.lo[2] = lo[2] + dk;
00055     result.hi[0] = hi[0] + di;
00056     result.hi[1] = hi[1] + dj;
00057     result.hi[2] = hi[2] + dk;
00058     return result;
00059   }
00060
00061   KOKKOS_INLINE_FUNCTION
00062   Box intersect(const Box& other) const {
00063     Box result;
00064     for (int d = 0; d < 3; ++d) {
00065       result.lo[d] = (lo[d] > other.lo[d]) ? lo[d] : other.lo[d];
00066       result.hi[d] = (hi[d] < other.hi[d]) ? hi[d] : other.hi[d];
00067     }
00068     return result;
00069   }
00070
00071   KOKKOS_INLINE_FUNCTION
00072   bool operator==(const Box& other) const {
00073     return lo[0] == other.lo[0] && lo[1] == other.lo[1] &&
00074            lo[2] == other.lo[2] && hi[0] == other.hi[0] &&
00075            hi[1] == other.hi[1] && hi[2] == other.hi[2];
00076   }
00077
00078   KOKKOS_INLINE_FUNCTION
00079   bool operator!=(const Box& other) const { return !(*this == other); }
00080
00081   static Box fromExtents(int nx, int ny, int nz) {
00082     Box result;
```

```
00083     result.lo[0] = 0;
00084     result.lo[1] = 0;
00085     result.lo[2] = 0;
00086     result.hi[0] = nx - 1;
00087     result.hi[1] = ny - 1;
00088     result.hi[2] = nz - 1;
00089     return result;
00090   }
00091 };
00092
00093 }  // namespace saturn
```

6.36 src/mesh/decomposition.hpp File Reference

```
#include "mesh/box.hpp"
#include <algorithm>
#include <array>
#include <cstdio>
#include <cstdlib>
```

Classes

- struct saturn::Decomposition

Namespaces

- namespace saturn

6.37 decomposition.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "mesh/box.hpp"
00004
00005 #include <algorithm>
00006 #include <array>
00007 #include <cstdio>
00008 #include <cstdlib>
00009
00010 #ifdef SATURN_ENABLE_MPI
00011 #include <mpi.h>
00012 #endif
00013
00014 namespace saturn {
00015
00016 // Cartesian MPI decomposition of a global box.
00017 struct Decomposition {
00018   int rank = 0;
00019   int size = 1;
00020   int dims[3]{1, 1, 1};
00021   int coords[3]{0, 0, 0};
00022   bool periodic[3]{true, true, true};
00023   Box global{};
00024   Box local{};
00025
00026 #ifdef SATURN_ENABLE_MPI
00027   MPI_Comm comm = MPI_COMM_NULL;
00028 #endif
00029
00030   static Decomposition create(const Box& global_box,
00031                               const bool periodic_in[3]) {
00032     Decomposition dec;
00033     dec.global = global_box;
```

```
00034     for (int d = 0; d < 3; ++d) {
00035       dec.periodic[d] = periodic_in[d];
00036     }
00037
00038 #ifdef SATURN_ENABLE_MPI
00039     MPI_Comm_rank(MPI_COMM_WORLD, &dec.rank);
00040     MPI_Comm_size(MPI_COMM_WORLD, &dec.size);
00041
00042     int dims[3] = {0, 0, 0};
00043     if (dec.size > 1) {
00044       MPI_Dims_create(dec.size, 3, dims);
00045       for (int d = 0; d < 3; ++d) {
00046         dims[d] = std::max(1, std::min(dims[d], global_box.length(d)));
00047       }
00048       if (dims[0] * dims[1] * dims[2] != dec.size) {
00049         // The grid is too thin for the MPI_Dims_create factorization; fall
00050         // back to a greedy factorization that respects each dimension.
00051         int remaining = dec.size;
00052         for (int d = 0; d < 3; ++d) {
00053           int f = std::min(remaining, global_box.length(d));
00054           while (f > 1 && remaining % f != 0) {
00055             --f;
00056           }
00057           dims[d] = f;
00058           remaining /= f;
00059         }
00060         if (remaining != 1) {
00061           std::fprintf(stderr,
00062                        "too many MPI ranks (%d) for grid %d x %d x %d\n",
00063                        dec.size, global_box.length(0), global_box.length(1),
00064                        global_box.length(2));
00065 #ifdef SATURN_ENABLE_MPI
00066           MPI_Abort(MPI_COMM_WORLD, 2);
00067 #else
00068           std::exit(2);
00069 #endif
00070         }
00071       }
00072     } else {
00073       dims[0] = dims[1] = dims[2] = 1;
00074     }
00075     for (int d = 0; d < 3; ++d) {
00076       dec.dims[d] = dims[d];
00077     }
00078
00079     int periods[3] = {dec.periodic[0] ? 1 : 0, dec.periodic[1] ? 1 : 0,
00080                       dec.periodic[2] ? 1 : 0};
00081     MPI_Cart_create(MPI_COMM_WORLD, 3, dec.dims, periods, 0, &dec.comm);
00082     MPI_Cart_coords(dec.comm, dec.rank, 3, dec.coords);
00083 #else
00084     dec.dims[0] = dec.dims[1] = dec.dims[2] = 1;
00085     dec.coords[0] = dec.coords[1] = dec.coords[2] = 0;
00086 #endif
00087
00088     for (int d = 0; d < 3; ++d) {
00089       const int n = global_box.length(d);
00090       const int base = n / dec.dims[d];
00091       const int rem = n % dec.dims[d];
00092       const int start = dec.coords[d] * base + std::min(dec.coords[d], rem);
00093       const int len = base + (dec.coords[d] < rem ? 1 : 0);
00094       dec.local.lo[d] = global_box.lo[d] + start;
00095       dec.local.hi[d] = dec.local.lo[d] + len - 1;
00096     }
00097     return dec;
00098   }
00099
00100   // Box owned by the given Cartesian coordinates. The same split rule is
00101   // used for every rank, so this can reconstruct any rank's interior box.
00102   Box boxForCoords(const int coords_in[3]) const {
00103     Box result;
00104     for (int d = 0; d < 3; ++d) {
00105       const int n = global.length(d);
00106       const int base = n / dims[d];
00107       const int rem = n % dims[d];
00108       const int start = coords_in[d] * base + std::min(coords_in[d], rem);
00109       const int len = base + (coords_in[d] < rem ? 1 : 0);
00110       result.lo[d] = global.lo[d] + start;
00111       result.hi[d] = result.lo[d] + len - 1;
00112     }
00113     return result;
00114   }
00115
00116   // Neighbor rank on the given side (-1 = low, +1 = high). Returns
00117   // MPI_PROC_NULL for a non-periodic physical boundary (or -1 without MPI).
00118   int partner(int d, int side) const {
00119 #ifdef SATURN_ENABLE_MPI
00120     int source = MPI_PROC_NULL;
```

```
00121     int dest = MPI_PROC_NULL;
00122     MPI_Cart_shift(comm, d, side, &source, &dest);
00123     return source;
00124 #else
00125     (void)d;
00126     (void)side;
00127     return -1;
00128 #endif
00129   }
00130 };
00131
00132 }  // namespace saturn
```

6.38 src/mesh/uniform_grid.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include <string>
#include "mesh/box.hpp"
```

Classes

- struct saturn::UniformGrid

Namespaces

- namespace saturn

6.39 uniform_grid.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <array>
00006 #include <cmath>
00007 #include <string>
00008
00009 #include "mesh/box.hpp"
00010
00011 namespace saturn {
00012
00013 struct UniformGrid {
00014   Box global;
00015   Box local;
00016   std::array<double, 3> lo{{0.0, 0.0, 0.0}};
00017   std::array<double, 3> hi{{1.0, 1.0, 1.0}};
00018   std::array<double, 3> h{{1.0, 1.0, 1.0}};
00019   int ng = 2;
00020
00021   static UniformGrid fromCells(int nx, int ny, int nz, double lx, double ly,
00022                                double lz, int ng) {
00023     UniformGrid grid;
00024     grid.global = Box::fromExtents(nx, ny, nz);
00025     grid.local = grid.global;
00026     grid.lo = {0.0, 0.0, 0.0};
00027     grid.hi = {lx, ly, lz};
00028     grid.h = {lx / nx, ly / ny, lz / nz};
00029     grid.ng = ng;
00030     return grid;
00031   }
00032
```

```
00033   static UniformGrid fromCellsLocal(int nx, int ny, int nz, double lx,
00034                                     double ly, double lz, int ng,
00035                                     const Box& local) {
00036     UniformGrid grid = fromCells(nx, ny, nz, lx, ly, lz, ng);
00037     grid.local = local;
00038     return grid;
00039   }
00040
00041   Box localBox() const { return local; }
00042
00043   KOKKOS_INLINE_FUNCTION
00044   long long numCells() const { return global.numCells(); }
00045 };
00046
00047 }  // namespace saturn
```

6.40 src/numerics/compressible/compressible_ns.hpp File Reference

Compressible Navier-Stokes fluxes and source terms.

```
#include <Kokkos_Core.hpp>
#include "core/mpi.hpp"
#include "core/parallel.hpp"
#include "fields/patch_field.hpp"
#include "mesh/box.hpp"
#include "mesh/uniform_grid.hpp"
#include "numerics/compressible/reconstruction.hpp"
#include "numerics/compressible/riemann.hpp"
```

Classes

- struct saturn::compressible::GasProperties
- class saturn::compressible::CompressibleWorkspace< ExecSpace >

Namespaces

- namespace saturn
- namespace saturn::compressible

Functions

- KOKKOS_INLINE_FUNCTION double saturn::compressible::soundSpeed (const PrimitiveState &state, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION double saturn::compressible::temperature (const PrimitiveState &state, const GasProperties &gas)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION PrimitiveState saturn::compressible::loadPrimitive (const PatchType &q, int i, int j, int k, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION PrimitiveState saturn::compressible::toNormal (const PrimitiveState &state, int direction)
- KOKKOS_INLINE_FUNCTION Flux saturn::compressible::fromNormal (const Flux &flux, int direction)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION void saturn::compressible::reconstructFace (const PatchType &q, int i, int j, int k, int direction, Reconstruction scheme, const GasProperties &gas, PrimitiveState &left, PrimitiveState &right)

- template<class PatchType >
  KOKKOS_INLINE_FUNCTION Flux saturn::compressible::inviscidFaceFlux (const PatchType &q, int i, int j, int k, int direction, Reconstruction scheme, RiemannSolver solver, const GasProperties &gas)
- KOKKOS_INLINE_FUNCTION double saturn::compressible::central4Derivative (double minus2, double minus1, double plus1, double plus2, double h)
- template<class PatchType >
  KOKKOS_INLINE_FUNCTION void saturn::compressible::gradientLine (const PatchType &q, int i, int j, int k, int direction, double h, const GasProperties &gas, double derivative[4])
- template<class ExecSpace >
  void saturn::compressible::computeGradients (const PatchField< ExecSpace > &q, PatchField< ExecSpace > &grad, const UniformGrid &grid, const GasProperties &gas, const Box &region)
- template<class ExecSpace >
  void saturn::compressible::computeStress (const PatchField< ExecSpace > &grad, PatchField< ExecSpace > &stress, const GasProperties &gas, const Box &region)
- template<class ExecSpace >
  void saturn::compressible::computeFaceFluxesX (const PatchField< ExecSpace > &q, const CompressibleWorkspace< ExecSpace > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class ExecSpace >
  void saturn::compressible::computeFaceFluxesY (const PatchField< ExecSpace > &q, const CompressibleWorkspace< ExecSpace > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class ExecSpace >
  void saturn::compressible::computeFaceFluxesZ (const PatchField< ExecSpace > &q, const CompressibleWorkspace< ExecSpace > &workspace, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver)
- template<class FieldType >
  KOKKOS_INLINE_FUNCTION double saturn::compressible::fieldDerivative (const FieldType &field, int component, int i, int j, int k, int di, int dj, int dk, double h)
- template<class ExecSpace >
  void saturn::compressible::assembleRhs (const PatchField< ExecSpace > &q, const CompressibleWorkspace< ExecSpace > &workspace, PatchField< ExecSpace > &rhs, const UniformGrid &grid, const GasProperties &gas, bool dissipation_sink)
- template<class ExecSpace >
  void saturn::compressible::compressibleRhs (const PatchField< ExecSpace > &q, CompressibleWorkspace< ExecSpace > &workspace, PatchField< ExecSpace > &rhs, const UniformGrid &grid, const GasProperties &gas, Reconstruction scheme, RiemannSolver solver, bool dissipation_sink=false)
- template<class ExecSpace >
  double saturn::compressible::maxWaveSpeed (const PatchField< ExecSpace > &q, const GasProperties &gas)
- template<class ExecSpace >
  double saturn::compressible::maxKinematicDiffusivity (const PatchField< ExecSpace > &q, const GasProperties &gas)

6.40.1 Detailed Description

Compressible Navier-Stokes fluxes and source terms.

6.41 compressible_ns.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
```

```
00007 #include <Kokkos_Core.hpp>
00008
00009 #include "core/mpi.hpp"
00010 #include "core/parallel.hpp"
00011 #include "fields/patch_field.hpp"
00012 #include "mesh/box.hpp"
00013 #include "mesh/uniform_grid.hpp"
00014 #include "numerics/compressible/reconstruction.hpp"
00015 #include "numerics/compressible/riemann.hpp"
00016
00017 namespace saturn {
00018 namespace compressible {
00019
00020 // Ideal-gas compressible Navier-Stokes in conservative variables:
00021 //
00022 //   U = [rho, rho u, rho v, rho w, E]
00023 //   E = p/(gamma-1) + 0.5 rho (u^2 + v^2 + w^2)
00024 //   p = rho R T
00025 //
00026 // Inviscid fluxes come from the reconstruction + Riemann modules. Viscous
00027 // fluxes use fourth-order cell-centered gradients averaged to faces:
00028 //
00029 //   tau_ij = mu (du_i/dx_j + du_j/dx_i - 2/3 delta_ij div u)
00030 //   q_j    = -k dT/dx_j,   k = mu cp / Pr,
00031 //   Fv_x   = [0, tau_xx, tau_xy, tau_xz,
00032 //             u tau_xx + v tau_xy + w tau_xz - q_x]
00033 //
00034 // with cp = gamma R/(gamma-1). Constant mu and Pr are assumed for now.
00035 struct GasProperties {
00036   double gamma = 1.4;
00037   double gas_constant = 1.0;
00038   double mu = 0.0;
00039   double prandtl = 0.72;
00040 };
00041
00042 KOKKOS_INLINE_FUNCTION
00043 double soundSpeed(const PrimitiveState& state, const GasProperties& gas) {
00044   return Kokkos::sqrt(gas.gamma * state.p / state.rho);
00045 }
00046
00047 KOKKOS_INLINE_FUNCTION
00048 double temperature(const PrimitiveState& state, const GasProperties& gas) {
00049   return state.p / (state.rho * gas.gas_constant);
00050 }
00051
00052 template <class PatchType>
00053 KOKKOS_INLINE_FUNCTION PrimitiveState loadPrimitive(const PatchType& q, int i,
00054                                                     int j, int k,
00055                                                     const GasProperties& gas) {
00056   const ConservativeState state{q(0, i, j, k), q(1, i, j, k), q(2, i, j, k),
00057                                 q(3, i, j, k), q(4, i, j, k)};
00058   return riemann::toPrimitive(state, gas.gamma);
00059 }
00060
00061 // Rotate a state so that its first velocity is normal to the face.
00062 KOKKOS_INLINE_FUNCTION
00063 PrimitiveState toNormal(const PrimitiveState& state, int direction) {
00064   if (direction == 0) {
00065     return state;
00066   }
00067   if (direction == 1) {
00068     return PrimitiveState{state.rho, state.v, state.u, state.w, state.p};
00069   }
00070   return PrimitiveState{state.rho, state.w, state.u, state.v, state.p};
00071 }
00072
00073 // Rotate a normal-coordinate flux back to Cartesian components.
00074 KOKKOS_INLINE_FUNCTION
00075 Flux fromNormal(const Flux& flux, int direction) {
00076   if (direction == 0) {
00077     return flux;
00078   }
00079   if (direction == 1) {
00080     return Flux{flux.rho, flux.mom_t1, flux.mom_n, flux.mom_t2, flux.energy};
00081   }
00082   return Flux{flux.rho, flux.mom_t1, flux.mom_t2, flux.mom_n, flux.energy};
00083 }
00084
00085 // Reconstruct the left/right primitive states at the face between cell
00086 // (i,j,k) and its neighbor in `direction`.
00087 template <class PatchType>
00088 KOKKOS_INLINE_FUNCTION void reconstructFace(const PatchType& q, int i, int j,
00089                                             int k, int direction,
00090                                             Reconstruction scheme,
00091                                             const GasProperties& gas,
00092                                             PrimitiveState& left,
00093                                             PrimitiveState& right) {
```

```
00094   const int di = direction == 0 ? 1 : 0;
00095   const int dj = direction == 1 ? 1 : 0;
00096   const int dk = direction == 2 ? 1 : 0;
00097
00098   double rho[6];
00099   double u[6];
00100   double v[6];
00101   double w[6];
00102   double p[6];
00103   for (int offset = -2; offset <= 3; ++offset) {
00104     const PrimitiveState value = loadPrimitive(
00105         q, i + offset * di, j + offset * dj, k + offset * dk, gas);
00106     const int index = offset + 2;
00107     rho[index] = value.rho;
00108     u[index] = value.u;
00109     v[index] = value.v;
00110     w[index] = value.w;
00111     p[index] = value.p;
00112   }
00113
00114   left = PrimitiveState{
00115       recon::reconstructLeft(scheme, rho[0], rho[1], rho[2], rho[3], rho[4],
00116                              rho[5]),
00117       recon::reconstructLeft(scheme, u[0], u[1], u[2], u[3], u[4], u[5]),
00118       recon::reconstructLeft(scheme, v[0], v[1], v[2], v[3], v[4], v[5]),
00119       recon::reconstructLeft(scheme, w[0], w[1], w[2], w[3], w[4], w[5]),
00120       recon::reconstructLeft(scheme, p[0], p[1], p[2], p[3], p[4], p[5])};
00121   right = PrimitiveState{
00122       recon::reconstructRight(scheme, rho[0], rho[1], rho[2], rho[3], rho[4],
00123                               rho[5]),
00124       recon::reconstructRight(scheme, u[0], u[1], u[2], u[3], u[4], u[5]),
00125       recon::reconstructRight(scheme, v[0], v[1], v[2], v[3], v[4], v[5]),
00126       recon::reconstructRight(scheme, w[0], w[1], w[2], w[3], w[4], w[5]),
00127       recon::reconstructRight(scheme, p[0], p[1], p[2], p[3], p[4], p[5])};
00128
00129   if (!riemann::valid(left) || !riemann::valid(right)) {
00130     left = loadPrimitive(q, i, j, k, gas);
00131     right = loadPrimitive(q, i + di, j + dj, k + dk, gas);
00132   }
00133 }
00134
00135 template <class PatchType>
00136 KOKKOS_INLINE_FUNCTION Flux inviscidFaceFlux(const PatchType& q, int i, int j,
00137                                              int k, int direction,
00138                                              Reconstruction scheme,
00139                                              RiemannSolver solver,
00140                                              const GasProperties& gas) {
00141   PrimitiveState left;
00142   PrimitiveState right;
00143   reconstructFace(q, i, j, k, direction, scheme, gas, left, right);
00144   const Flux normal_flux = riemann::flux(solver, toNormal(left, direction),
00145                                          toNormal(right, direction), gas.gamma);
00146   return fromNormal(normal_flux, direction);
00147 }
00148
00149 KOKKOS_INLINE_FUNCTION
00150 double central4Derivative(double minus2, double minus1, double plus1,
00151                           double plus2, double h) {
00152   return (minus2 - 8.0 * minus1 + 8.0 * plus1 - plus2) / (12.0 * h);
00153 }
00154
00155 // Cell-centered 4th-order gradients of (u,v,w,T) along one direction.
00156 template <class PatchType>
00157 KOKKOS_INLINE_FUNCTION void gradientLine(const PatchType& q, int i, int j,
00158                                          int k, int direction, double h,
00159                                          const GasProperties& gas,
00160                                          double derivative[4]) {
00161   const int di = direction == 0 ? 1 : 0;
00162   const int dj = direction == 1 ? 1 : 0;
00163   const int dk = direction == 2 ? 1 : 0;
00164   const PrimitiveState minus2 =
00165       loadPrimitive(q, i - 2 * di, j - 2 * dj, k - 2 * dk, gas);
00166   const PrimitiveState minus1 = loadPrimitive(q, i - di, j - dj, k - dk, gas);
00167   const PrimitiveState plus1 = loadPrimitive(q, i + di, j + dj, k + dk, gas);
00168   const PrimitiveState plus2 =
00169       loadPrimitive(q, i + 2 * di, j + 2 * dj, k + 2 * dk, gas);
00170
00171   derivative[0] = central4Derivative(minus2.u, minus1.u, plus1.u, plus2.u, h);
00172   derivative[1] = central4Derivative(minus2.v, minus1.v, plus1.v, plus2.v, h);
00173   derivative[2] = central4Derivative(minus2.w, minus1.w, plus1.w, plus2.w, h);
00174   derivative[3] =
00175       central4Derivative(temperature(minus2, gas), temperature(minus1, gas),
00176                          temperature(plus1, gas), temperature(plus2, gas), h);
00177 }
00178
00179 // Gradient layout:
00180 //   0..2  du/dx, du/dy, du/dz
```

```
00181 //   3..5  dv/dx, dv/dy, dv/dz
00182 //   6..8  dw/dx, dw/dy, dw/dz
00183 //   9..11 dT/dx, dT/dy, dT/dz
00184 template <class ExecSpace>
00185 void computeGradients(const PatchField<ExecSpace>& q,
00186                       PatchField<ExecSpace>& grad, const UniformGrid& grid,
00187                       const GasProperties& gas, const Box& region) {
00188   if (region.empty()) {
00189     return;
00190   }
00191   const auto policy = makeMdrange3<ExecSpace>(
00192       Int3{{region.lo[2], region.lo[1], region.lo[0]}},
00193       Int3{{region.hi[2] + 1, region.hi[1] + 1, region.hi[0] + 1}});
00194   const double hx = grid.h[0];
00195   const double hy = grid.h[1];
00196   const double hz = grid.h[2];
00197
00198   Kokkos::parallel_for(
00199       "compressible_gradients", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00200         double derivative[4];
00201         gradientLine(q, i, j, k, 0, hx, gas, derivative);
00202         grad(0, i, j, k) = derivative[0];
00203         grad(3, i, j, k) = derivative[1];
00204         grad(6, i, j, k) = derivative[2];
00205         grad(9, i, j, k) = derivative[3];
00206
00207         gradientLine(q, i, j, k, 1, hy, gas, derivative);
00208         grad(1, i, j, k) = derivative[0];
00209         grad(4, i, j, k) = derivative[1];
00210         grad(7, i, j, k) = derivative[2];
00211         grad(10, i, j, k) = derivative[3];
00212
00213         gradientLine(q, i, j, k, 2, hz, gas, derivative);
00214         grad(2, i, j, k) = derivative[0];
00215         grad(5, i, j, k) = derivative[1];
00216         grad(8, i, j, k) = derivative[2];
00217         grad(11, i, j, k) = derivative[3];
00218       });
00219 }
00220
00221 // Cell-centered viscous stress. Layout:
00222 //   0 tau_xx, 1 tau_yy, 2 tau_zz, 3 tau_xy, 4 tau_xz, 5 tau_yz
00223 template <class ExecSpace>
00224 void computeStress(const PatchField<ExecSpace>& grad,
00225                    PatchField<ExecSpace>& stress, const GasProperties& gas,
00226                    const Box& region) {
00227   if (region.empty()) {
00228     return;
00229   }
00230   const auto policy = makeMdrange3<ExecSpace>(
00231       Int3{{region.lo[2], region.lo[1], region.lo[0]}},
00232       Int3{{region.hi[2] + 1, region.hi[1] + 1, region.hi[0] + 1}});
00233   const double mu = gas.mu;
00234
00235   Kokkos::parallel_for(
00236       "compressible_stress", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00237         const double du_dx = grad(0, i, j, k);
00238         const double du_dy = grad(1, i, j, k);
00239         const double du_dz = grad(2, i, j, k);
00240         const double dv_dx = grad(3, i, j, k);
00241         const double dv_dy = grad(4, i, j, k);
00242         const double dv_dz = grad(5, i, j, k);
00243         const double dw_dx = grad(6, i, j, k);
00244         const double dw_dy = grad(7, i, j, k);
00245         const double dw_dz = grad(8, i, j, k);
00246         const double div = du_dx + dv_dy + dw_dz;
00247         stress(0, i, j, k) = mu * (2.0 * du_dx - (2.0 / 3.0) * div);
00248         stress(1, i, j, k) = mu * (2.0 * dv_dy - (2.0 / 3.0) * div);
00249         stress(2, i, j, k) = mu * (2.0 * dw_dz - (2.0 / 3.0) * div);
00250         stress(3, i, j, k) = mu * (du_dy + dv_dx);
00251         stress(4, i, j, k) = mu * (du_dz + dw_dx);
00252         stress(5, i, j, k) = mu * (dv_dz + dw_dy);
00253       });
00254 }
00255
00256 // Workspace holding gradients and one face-flux array per direction.
00257 template <class ExecSpace>
00258 class CompressibleWorkspace {
00259  public:
00260   using memory_space = typename ExecSpace::memory_space;
00261   using face_view = Kokkos::View<double****, Kokkos::LayoutRight, memory_space>;
00262
00263   explicit CompressibleWorkspace(const Box& box)
00264       : grad(box, 12, 3, "compressible_grad"),
00265         stress(box, 6, 2, "compressible_stress"),
00266         fx("compressible_fx", 5, box.length(2), box.length(1),
00267            box.length(0) + 1),
```

```
00268         fy("compressible_fy", 5, box.length(2), box.length(1) + 1,
00269            box.length(0)),
00270         fz("compressible_fz", 5, box.length(2) + 1, box.length(1),
00271            box.length(0)) {}
00272
00273   PatchField<ExecSpace> grad;
00274   PatchField<ExecSpace> stress;
00275   face_view fx;
00276   face_view fy;
00277   face_view fz;
00278 };
00279
00280 template <class ExecSpace>
00281 void computeFaceFluxesX(const PatchField<ExecSpace>& q,
00282                         const CompressibleWorkspace<ExecSpace>& workspace,
00283                         const UniformGrid& grid, const GasProperties& gas,
00284                         Reconstruction scheme, RiemannSolver solver) {
00285   const Box box = grid.local;
00286   const int nx = box.length(0);
00287   const int ny = box.length(1);
00288   const int nz = box.length(2);
00289   const auto policy =
00290       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{nz, ny, nx + 1}});
00291   auto fx = workspace.fx;
00292
00293   Kokkos::parallel_for(
00294       "compressible_faces_x", policy, KOKKOS_LAMBDA(int k, int j, int face) {
00295         const int i_left = box.lo[0] + face - 1;
00296         const int j_global = box.lo[1] + j;
00297         const int k_global = box.lo[2] + k;
00298
00299         const Flux inviscid = inviscidFaceFlux(q, i_left, j_global, k_global, 0,
00300                                                scheme, solver, gas);
00301         fx(0, k, j, face) = inviscid.rho;
00302         fx(1, k, j, face) = inviscid.mom_n;
00303         fx(2, k, j, face) = inviscid.mom_t1;
00304         fx(3, k, j, face) = inviscid.mom_t2;
00305         fx(4, k, j, face) = inviscid.energy;
00306       });
00307 }
00308
00309 template <class ExecSpace>
00310 void computeFaceFluxesY(const PatchField<ExecSpace>& q,
00311                         const CompressibleWorkspace<ExecSpace>& workspace,
00312                         const UniformGrid& grid, const GasProperties& gas,
00313                         Reconstruction scheme, RiemannSolver solver) {
00314   const Box box = grid.local;
00315   const int nx = box.length(0);
00316   const int ny = box.length(1);
00317   const int nz = box.length(2);
00318   const auto policy =
00319       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{nz, ny + 1, nx}});
00320   auto fy = workspace.fy;
00321
00322   Kokkos::parallel_for(
00323       "compressible_faces_y", policy, KOKKOS_LAMBDA(int k, int face, int i) {
00324         const int j_left = box.lo[1] + face - 1;
00325         const int i_global = box.lo[0] + i;
00326         const int k_global = box.lo[2] + k;
00327
00328         const Flux inviscid = inviscidFaceFlux(q, i_global, j_left, k_global, 1,
00329                                                scheme, solver, gas);
00330         fy(0, k, face, i) = inviscid.rho;
00331         fy(1, k, face, i) = inviscid.mom_n;
00332         fy(2, k, face, i) = inviscid.mom_t1;
00333         fy(3, k, face, i) = inviscid.mom_t2;
00334         fy(4, k, face, i) = inviscid.energy;
00335       });
00336 }
00337
00338 template <class ExecSpace>
00339 void computeFaceFluxesZ(const PatchField<ExecSpace>& q,
00340                         const CompressibleWorkspace<ExecSpace>& workspace,
00341                         const UniformGrid& grid, const GasProperties& gas,
00342                         Reconstruction scheme, RiemannSolver solver) {
00343   const Box box = grid.local;
00344   const int nx = box.length(0);
00345   const int ny = box.length(1);
00346   const int nz = box.length(2);
00347   const auto policy =
00348       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{nz + 1, ny, nx}});
00349   auto fz = workspace.fz;
00350
00351   Kokkos::parallel_for(
00352       "compressible_faces_z", policy, KOKKOS_LAMBDA(int face, int j, int i) {
00353         const int k_left = box.lo[2] + face - 1;
00354         const int i_global = box.lo[0] + i;
```

```
00355         const int j_global = box.lo[1] + j;
00356
00357         const Flux inviscid = inviscidFaceFlux(q, i_global, j_global, k_left, 2,
00358                                                scheme, solver, gas);
00359         fz(0, face, j, i) = inviscid.rho;
00360         fz(1, face, j, i) = inviscid.mom_n;
00361         fz(2, face, j, i) = inviscid.mom_t1;
00362         fz(3, face, j, i) = inviscid.mom_t2;
00363         fz(4, face, j, i) = inviscid.energy;
00364       });
00365 }
00366
00367 // Fourth-order central derivative of one component of a cell-centered field.
00368 template <class FieldType>
00369 KOKKOS_INLINE_FUNCTION double fieldDerivative(const FieldType& field,
00370                                               int component, int i, int j,
00371                                               int k, int di, int dj, int dk,
00372                                               double h) {
00373   return central4Derivative(
00374       field(component, i - 2 * di, j - 2 * dj, k - 2 * dk),
00375       field(component, i - di, j - dj, k - dk),
00376       field(component, i + di, j + dj, k + dk),
00377       field(component, i + 2 * di, j + 2 * dj, k + 2 * dk), h);
00378 }
00379
00380 // Assemble -div(F_inviscid) + div(tau) and the viscous energy terms. The
00381 // viscous operator is cell-centered and fourth-order:
00382 //
00383 //   div(tau)_i = d tau_ij / dx_j
00384 //   energy     = u . div(tau) + tau : grad(u) - div(q)
00385 //
00386 // If `dissipation_sink` is true, tau : grad(u) is removed from the energy
00387 // equation, making the manufactured shear test isothermal.
00388 template <class ExecSpace>
00389 void assembleRhs(const PatchField<ExecSpace>& q,
00390                  const CompressibleWorkspace<ExecSpace>& workspace,
00391                  PatchField<ExecSpace>& rhs, const UniformGrid& grid,
00392                  const GasProperties& gas, bool dissipation_sink) {
00393   const Box box = grid.local;
00394   const double hx = grid.h[0];
00395   const double hy = grid.h[1];
00396   const double hz = grid.h[2];
00397   auto fx = workspace.fx;
00398   auto fy = workspace.fy;
00399   auto fz = workspace.fz;
00400   const auto grad_patch = workspace.grad;
00401   const auto stress = workspace.stress;
00402   const double conductivity =
00403       gas.mu * gas.gamma * gas.gas_constant / ((gas.gamma - 1.0) * gas.prandtl);
00404
00405   const auto policy = makeMdrange3<ExecSpace>(
00406       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00407       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00408
00409   Kokkos::parallel_for(
00410       "compressible_assemble_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00411         const int il = i - box.lo[0];
00412         const int jl = j - box.lo[1];
00413         const int kl = k - box.lo[2];
00414
00415         const double div_x = (fx(0, kl, jl, il + 1) - fx(0, kl, jl, il)) / hx;
00416         const double div_y = (fy(0, kl, jl + 1, il) - fy(0, kl, jl, il)) / hy;
00417         const double div_z = (fz(0, kl + 1, jl, il) - fz(0, kl, jl, il)) / hz;
00418         rhs(0, i, j, k) = -(div_x + div_y + div_z);
00419
00420         const double mom_x = (fx(1, kl, jl, il + 1) - fx(1, kl, jl, il)) / hx +
00421                              (fy(1, kl, jl + 1, il) - fy(1, kl, jl, il)) / hy +
00422                              (fz(1, kl + 1, jl, il) - fz(1, kl, jl, il)) / hz;
00423         rhs(1, i, j, k) = -mom_x;
00424
00425         const double mom_y = (fx(2, kl, jl, il + 1) - fx(2, kl, jl, il)) / hx +
00426                              (fy(2, kl, jl + 1, il) - fy(2, kl, jl, il)) / hy +
00427                              (fz(2, kl + 1, jl, il) - fz(2, kl, jl, il)) / hz;
00428         rhs(2, i, j, k) = -mom_y;
00429
00430         const double mom_z = (fx(3, kl, jl, il + 1) - fx(3, kl, jl, il)) / hx +
00431                              (fy(3, kl, jl + 1, il) - fy(3, kl, jl, il)) / hy +
00432                              (fz(3, kl + 1, jl, il) - fz(3, kl, jl, il)) / hz;
00433         rhs(3, i, j, k) = -mom_z;
00434
00435         const double energy = (fx(4, kl, jl, il + 1) - fx(4, kl, jl, il)) / hx +
00436                               (fy(4, kl, jl + 1, il) - fy(4, kl, jl, il)) / hy +
00437                               (fz(4, kl + 1, jl, il) - fz(4, kl, jl, il)) / hz;
00438         rhs(4, i, j, k) = -energy;
00439
00440         const double div_tau_x =
00441             fieldDerivative(stress, 0, i, j, k, 1, 0, 0, hx) +
```

```
00442             fieldDerivative(stress, 3, i, j, k, 0, 1, 0, hy) +
00443             fieldDerivative(stress, 4, i, j, k, 0, 0, 1, hz);
00444         const double div_tau_y =
00445             fieldDerivative(stress, 3, i, j, k, 1, 0, 0, hx) +
00446             fieldDerivative(stress, 1, i, j, k, 0, 1, 0, hy) +
00447             fieldDerivative(stress, 5, i, j, k, 0, 0, 1, hz);
00448         const double div_tau_z =
00449             fieldDerivative(stress, 4, i, j, k, 1, 0, 0, hx) +
00450             fieldDerivative(stress, 5, i, j, k, 0, 1, 0, hy) +
00451             fieldDerivative(stress, 2, i, j, k, 0, 0, 1, hz);
00452
00453         const double du_dx = grad_patch(0, i, j, k);
00454         const double du_dy = grad_patch(1, i, j, k);
00455         const double du_dz = grad_patch(2, i, j, k);
00456         const double dv_dx = grad_patch(3, i, j, k);
00457         const double dv_dy = grad_patch(4, i, j, k);
00458         const double dv_dz = grad_patch(5, i, j, k);
00459         const double dw_dx = grad_patch(6, i, j, k);
00460         const double dw_dy = grad_patch(7, i, j, k);
00461         const double dw_dz = grad_patch(8, i, j, k);
00462         const double dissipation =
00463             stress(0, i, j, k) * du_dx + stress(1, i, j, k) * dv_dy +
00464             stress(2, i, j, k) * dw_dz + stress(3, i, j, k) * (du_dy + dv_dx) +
00465             stress(4, i, j, k) * (du_dz + dw_dx) +
00466             stress(5, i, j, k) * (dv_dz + dw_dy);
00467
00468         const double lap_temperature =
00469             fieldDerivative(grad_patch, 9, i, j, k, 1, 0, 0, hx) +
00470             fieldDerivative(grad_patch, 10, i, j, k, 0, 1, 0, hy) +
00471             fieldDerivative(grad_patch, 11, i, j, k, 0, 0, 1, hz);
00472
00473         const PrimitiveState state = loadPrimitive(q, i, j, k, gas);
00474         rhs(1, i, j, k) += div_tau_x;
00475         rhs(2, i, j, k) += div_tau_y;
00476         rhs(3, i, j, k) += div_tau_z;
00477         rhs(4, i, j, k) += state.u * div_tau_x + state.v * div_tau_y +
00478                            state.w * div_tau_z + dissipation +
00479                            conductivity * lap_temperature;
00480         if (dissipation_sink) {
00481           rhs(4, i, j, k) -= dissipation;
00482         }
00483       });
00484 }
00485
00486 // Full compressible RHS. The caller must have filled q's ghost cells.
00487 template <class ExecSpace>
00488 void compressibleRhs(const PatchField<ExecSpace>& q,
00489                      CompressibleWorkspace<ExecSpace>& workspace,
00490                      PatchField<ExecSpace>& rhs, const UniformGrid& grid,
00491                      const GasProperties& gas, Reconstruction scheme,
00492                      RiemannSolver solver, bool dissipation_sink = false) {
00493   const Box box = grid.local;
00494   computeGradients(q, workspace.grad, grid, gas, box.grow(2));
00495   computeStress(workspace.grad, workspace.stress, gas, box.grow(2));
00496   computeFaceFluxesX(q, workspace, grid, gas, scheme, solver);
00497   computeFaceFluxesY(q, workspace, grid, gas, scheme, solver);
00498   computeFaceFluxesZ(q, workspace, grid, gas, scheme, solver);
00499   assembleRhs(q, workspace, rhs, grid, gas, dissipation_sink);
00500 }
00501
00502 template <class ExecSpace>
00503 double maxWaveSpeed(const PatchField<ExecSpace>& q, const GasProperties& gas) {
00504   const Box box = q.box();
00505   const auto policy = makeMdrange3<ExecSpace>(
00506       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00507       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00508   double local_max = 0.0;
00509   Kokkos::parallel_reduce(
00510       "compressible_max_speed", policy,
00511       KOKKOS_LAMBDA(int k, int j, int i, double& acc) {
00512         const PrimitiveState state = loadPrimitive(q, i, j, k, gas);
00513         double normal_speed = Kokkos::fabs(state.u);
00514         const double speed_v = Kokkos::fabs(state.v);
00515         const double speed_w = Kokkos::fabs(state.w);
00516         normal_speed = speed_v > normal_speed ? speed_v : normal_speed;
00517         normal_speed = speed_w > normal_speed ? speed_w : normal_speed;
00518         const double speed = normal_speed + soundSpeed(state, gas);
00519         acc = speed > acc ? speed : acc;
00520       },
00521       Kokkos::Max<double>{local_max});
00522   return mpiMax(local_max);
00523 }
00524
00525 template <class ExecSpace>
00526 double maxKinematicDiffusivity(const PatchField<ExecSpace>& q,
00527                                const GasProperties& gas) {
00528   if (gas.mu <= 0.0) {
```

```
00529     return 0.0;
00530   }
00531   const Box box = q.box();
00532   const auto policy = makeMdrange3<ExecSpace>(
00533       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00534       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00535   double local_max = 0.0;
00536   Kokkos::parallel_reduce(
00537       "compressible_max_diffusivity", policy,
00538       KOKKOS_LAMBDA(int k, int j, int i, double& acc) {
00539         const PrimitiveState state = loadPrimitive(q, i, j, k, gas);
00540         const double nu_momentum = (4.0 / 3.0) * gas.mu / state.rho;
00541         const double nu_thermal = gas.mu / (state.rho * gas.prandtl);
00542         const double nu = nu_momentum > nu_thermal ? nu_momentum : nu_thermal;
00543         acc = nu > acc ? nu : acc;
00544       },
00545       Kokkos::Max<double>{local_max});
00546   return mpiMax(local_max);
00547 }
00548
00549 }  // namespace compressible
00550 }  // namespace saturn
```

6.42 src/numerics/compressible/reconstruction.hpp File Reference

Face reconstruction: central, WENO5-Z, and TENO6.

```
#include <Kokkos_Core.hpp>
```

Namespaces

- namespace saturn
- namespace saturn::recon

Enumerations

- enum class saturn::Reconstruction {
  saturn::Central2 , saturn::Central4 , saturn::Central6 , saturn::Weno5Z ,
  saturn::Teno6 }

Functions

- constexpr int saturn::reconstructionStencilWidth (Reconstruction scheme) noexcept
- KOKKOS_INLINE_FUNCTION double saturn::recon::central2 (double, double, double v0, double vp1, double, double)
- KOKKOS_INLINE_FUNCTION double saturn::recon::central4 (double, double vm1, double v0, double vp1, double vp2, double)
- KOKKOS_INLINE_FUNCTION double saturn::recon::central6 (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION void saturn::recon::weno5Smoothness (double vm2, double vm1, double v0, double vp1, double vp2, double &beta0, double &beta1, double &beta2)
- KOKKOS_INLINE_FUNCTION double saturn::recon::weno5z (double vm2, double vm1, double v0, double vp1, double vp2, double)
- KOKKOS_INLINE_FUNCTION double saturn::recon::stencil6Smoothness (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double saturn::recon::tenoInverseSmoothness (double normalized_beta)

- KOKKOS_INLINE_FUNCTION double saturn::recon::teno6 (double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructLeft (Reconstruction scheme, double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructRight (Reconstruction scheme, double vm2, double vm1, double v0, double vp1, double vp2, double vp3)
- template<class Accessor >
  KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructLeft (Reconstruction scheme, const Accessor &accessor)
- template<class Accessor >
  KOKKOS_INLINE_FUNCTION double saturn::recon::reconstructRight (Reconstruction scheme, const Accessor &accessor)

Variables

- constexpr double saturn::recon::kEpsilon = 1.0e-40
- constexpr double saturn::recon::kTenoCutoff = 1.0e-5
- constexpr double saturn::recon::kTenoExponent = 7.0
- constexpr double saturn::recon::kWenoWeight0 = 1.0 / 10.0
- constexpr double saturn::recon::kWenoWeight1 = 3.0 / 5.0
- constexpr double saturn::recon::kWenoWeight2 = 3.0 / 10.0

6.42.1 Detailed Description

Face reconstruction: central, WENO5-Z, and TENO6.

6.43 reconstruction.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #include <Kokkos_Core.hpp>
00008
00009 namespace saturn {
00010
00011 // Face-reconstruction schemes for cell-centered finite volumes.
00012 //
00013 // All schemes reconstruct the state at the face between cells i and i+1 from
00014 // the six-point stencil
00015 //
00016 //   v[-2] v[-1] v[0] | v[1] v[2] v[3]
00017 //                    ^ face
00018 //
00019 // `reconstructLeft` returns the state on the left of the face (upwind for a
00020 // positive velocity); `reconstructRight` mirrors the stencil around the face.
00021 // The caller supplies ghost cells; no boundary handling is done here.
00022 //
00023 // Schemes:
00024 //   Central2  - two-point average, 2nd order.
00025 //   Central4  - symmetric four-point reconstruction, 4th order.
00026 //   Central6  - symmetric six-point reconstruction, 6th order.
00027 //   Weno5Z    - WENO5 with Borges et al. (2008) Z weights, 5th order.
00028 //   Teno6     - TENO6: three 3-point WENO candidates plus a symmetric
00029 //               6-point candidate selected by an inverse-smoothness cutoff.
00030 //               In smooth regions the 6-point candidate is retained, so the
00031 //               scheme reduces to Central6; near discontinuities the cutoff
00032 //               falls back to an ENO-like upwind candidate.
00033 //
00034 // TENO6 follows the incremental-stencil selection of Fu et al. (2021):
00035 // gamma_k = (beta_k + eps)^-7, chi_k = gamma_k / sum(gamma), and a candidate
00036 // is active when chi_k >= C_T. The six-point candidate is tried first; if it
00037 // is rejected, the three small candidates are renormalized with their WENO5
00038 // linear weights.
```

```
00039 enum class Reconstruction {
00040   Central2,
00041   Central4,
00042   Central6,
00043   Weno5Z,
00044   Teno6,
00045 };
00046
00047 // Number of cells touched by a scheme (the largest offset is 3 on each side).
00048 constexpr int reconstructionStencilWidth(Reconstruction scheme) noexcept {
00049   switch (scheme) {
00050     case Reconstruction::Central2:
00051       return 2;
00052     case Reconstruction::Central4:
00053       return 4;
00054     case Reconstruction::Central6:
00055     case Reconstruction::Teno6:
00056       return 6;
00057     case Reconstruction::Weno5Z:
00058       return 5;
00059   }
00060   return 6;
00061 }
00062
00063 namespace recon {
00064
00065 // WENO/TENO parameters. The inverse-smoothness exponent and cutoff are the
00066 // values recommended for TENO6 (Fu et al., JCP 2021).
00067 inline constexpr double kEpsilon = 1.0e-40;
00068 inline constexpr double kTenoCutoff = 1.0e-5;
00069 inline constexpr double kTenoExponent = 7.0;  // applied as x^7
00070
00071 // WENO5 finite-volume linear weights for the backward, central, and forward
00072 // 3-point candidates. These combine to the 5th-order face value.
00073 inline constexpr double kWenoWeight0 = 1.0 / 10.0;
00074 inline constexpr double kWenoWeight1 = 3.0 / 5.0;
00075 inline constexpr double kWenoWeight2 = 3.0 / 10.0;
00076
00077 KOKKOS_INLINE_FUNCTION
00078 double central2(double /*vm2*/, double /*vm1*/, double v0, double vp1,
00079                 double /*vp2*/, double /*vp3*/) {
00080   return 0.5 * (v0 + vp1);
00081 }
00082
00083 KOKKOS_INLINE_FUNCTION
00084 double central4(double /*vm2*/, double vm1, double v0, double vp1, double vp2,
00085                 double /*vp3*/) {
00086   return (-vm1 + 7.0 * v0 + 7.0 * vp1 - vp2) / 12.0;
00087 }
00088
00089 KOKKOS_INLINE_FUNCTION
00090 double central6(double vm2, double vm1, double v0, double vp1, double vp2,
00091                 double vp3) {
00092   return (vm2 - 8.0 * vm1 + 37.0 * v0 + 37.0 * vp1 - 8.0 * vp2 + vp3) / 60.0;
00093 }
00094
00095 // Jiang-Shu smoothness indicators for the three 3-point WENO5 candidates.
00096 // Ordered backward ({-2,-1,0}), central ({-1,0,1}), forward ({0,1,2}).
00097 KOKKOS_INLINE_FUNCTION
00098 void weno5Smoothness(double vm2, double vm1, double v0, double vp1, double vp2,
00099                      double& beta0, double& beta1, double& beta2) {
00100   const double d0 = vm2 - 2.0 * vm1 + v0;
00101   const double d1 = vm1 - 2.0 * v0 + vp1;
00102   const double d2 = v0 - 2.0 * vp1 + vp2;
00103   beta0 = (13.0 / 12.0) * d0 * d0 +
00104           0.25 * (vm2 - 4.0 * vm1 + 3.0 * v0) * (vm2 - 4.0 * vm1 + 3.0 * v0);
00105   beta1 = (13.0 / 12.0) * d1 * d1 + 0.25 * (vm1 - vp1) * (vm1 - vp1);
00106   beta2 = (13.0 / 12.0) * d2 * d2 +
00107           0.25 * (3.0 * v0 - 4.0 * vp1 + vp2) * (3.0 * v0 - 4.0 * vp1 + vp2);
00108 }
00109
00110 // WENO5-Z (Borges et al. 2008) with finite-volume linear weights.
00111 KOKKOS_INLINE_FUNCTION
00112 double weno5z(double vm2, double vm1, double v0, double vp1, double vp2,
00113               double /*vp3*/) {
00114   const double q0 = (2.0 * vm2 - 7.0 * vm1 + 11.0 * v0) / 6.0;
00115   const double q1 = (-vm1 + 5.0 * v0 + 2.0 * vp1) / 6.0;
00116   const double q2 = (2.0 * v0 + 5.0 * vp1 - vp2) / 6.0;
00117
00118   double beta0 = 0.0;
00119   double beta1 = 0.0;
00120   double beta2 = 0.0;
00121   weno5Smoothness(vm2, vm1, v0, vp1, vp2, beta0, beta1, beta2);
00122
00123   // Normalize by the largest indicator so that the nonlinear weights are
00124   // scale invariant and cannot overflow for very large jumps.
00125   double beta_max = beta0 > beta1 ? beta0 : beta1;
```

```
00126   beta_max = beta_max > beta2 ? beta_max : beta2;
00127   double n0 = 0.0;
00128   double n1 = 0.0;
00129   double n2 = 0.0;
00130   double tau = 0.0;
00131   if (beta_max > 0.0) {
00132     n0 = beta0 / beta_max;
00133     n1 = beta1 / beta_max;
00134     n2 = beta2 / beta_max;
00135     tau = n0 > n2 ? n0 - n2 : n2 - n0;
00136   }
00137
00138   const double a0 = kWenoWeight0 * (1.0 + tau / (n0 + kEpsilon)) *
00139                     (1.0 + tau / (n0 + kEpsilon));
00140   const double a1 = kWenoWeight1 * (1.0 + tau / (n1 + kEpsilon)) *
00141                     (1.0 + tau / (n1 + kEpsilon));
00142   const double a2 = kWenoWeight2 * (1.0 + tau / (n2 + kEpsilon)) *
00143                     (1.0 + tau / (n2 + kEpsilon));
00144
00145   const double sum = a0 + a1 + a2;
00146   if (!(sum > 0.0)) {
00147     return q1;
00148   }
00149   return (a0 * q0 + a1 * q1 + a2 * q2) / sum;
00150 }
00151
00152 // Jiang-Shu smoothness indicator for the symmetric six-point stencil
00153 // {-2,...,3}, integrated over the cell adjacent to the face. The rational
00154 // matrix is the exact derivative integral (denominator 120960).
00155 KOKKOS_INLINE_FUNCTION
00156 double stencil6Smoothness(double vm2, double vm1, double v0, double vp1,
00157                           double vp2, double vp3) {
00158   const double v[6] = {vm2, vm1, v0, vp1, vp2, vp3};
00159   const double kNumerator[6][6] = {
00160       {271779.0, -1190400.0, 2043176.0, -1731126.0, 729381.0, -122810.0},
00161       {-1190400.0, 5653317.0, -10213942.0, 8952516.0, -3863994.0, 662503.0},
00162       {2043176.0, -10213942.0, 19510972.0, -17908832.0, 7964956.0, -1396330.0},
00163       {-1731126.0, 8952516.0, -17908832.0, 17195652.0, -7940202.0, 1431992.0},
00164       {729381.0, -3863994.0, 7964956.0, -7940202.0, 3824847.0, -714988.0},
00165       {-122810.0, 662503.0, -1396330.0, 1431992.0, -714988.0, 139633.0}};
00166
00167   double sum = 0.0;
00168   for (int i = 0; i < 6; ++i) {
00169     for (int j = 0; j < 6; ++j) {
00170       sum += kNumerator[i][j] * v[i] * v[j];
00171     }
00172   }
00173   return sum / 120960.0;
00174 }
00175
00176 // gamma = (normalized_beta + eps)^-7, evaluated with multiplications only.
00177 KOKKOS_INLINE_FUNCTION
00178 double tenoInverseSmoothness(double normalized_beta) {
00179   const double inv = 1.0 / (normalized_beta + kEpsilon);
00180   const double inv2 = inv * inv;
00181   const double inv4 = inv2 * inv2;
00182   return inv4 * inv2 * inv;
00183 }
00184
00185 KOKKOS_INLINE_FUNCTION
00186 double teno6(double vm2, double vm1, double v0, double vp1, double vp2,
00187              double vp3) {
00188   const double q0 = (2.0 * vm2 - 7.0 * vm1 + 11.0 * v0) / 6.0;
00189   const double q1 = (-vm1 + 5.0 * v0 + 2.0 * vp1) / 6.0;
00190   const double q2 = (2.0 * v0 + 5.0 * vp1 - vp2) / 6.0;
00191   const double q6 = central6(vm2, vm1, v0, vp1, vp2, vp3);
00192
00193   double beta0 = 0.0;
00194   double beta1 = 0.0;
00195   double beta2 = 0.0;
00196   weno5Smoothness(vm2, vm1, v0, vp1, vp2, beta0, beta1, beta2);
00197   const double beta6 = stencil6Smoothness(vm2, vm1, v0, vp1, vp2, vp3);
00198
00199   double beta_max = beta0 > beta1 ? beta0 : beta1;
00200   beta_max = beta_max > beta2 ? beta_max : beta2;
00201   beta_max = beta_max > beta6 ? beta_max : beta6;
00202   if (!(beta_max > 0.0)) {
00203     return q6;
00204   }
00205
00206   // gamma_k = (beta_k / beta_max + eps)^-7. The common beta_max^-7 factor
00207   // cancels in the normalized chi_k, so this is scale invariant and safe.
00208   const double g0 = tenoInverseSmoothness(beta0 / beta_max);
00209   const double g1 = tenoInverseSmoothness(beta1 / beta_max);
00210   const double g2 = tenoInverseSmoothness(beta2 / beta_max);
00211   const double g6 = tenoInverseSmoothness(beta6 / beta_max);
00212
```

```
00213   const double sum6 = g0 + g1 + g2 + g6;
00214   if (!(sum6 > 0.0)) {
00215     return q6;
00216   }
00217   if (g6 / sum6 >= kTenoCutoff) {
00218     return q6;
00219   }
00220
00221   // Large stencil rejected: apply the TENO cutoff to the small candidates.
00222   const double sum3 = g0 + g1 + g2;
00223   if (!(sum3 > 0.0)) {
00224     return q1;
00225   }
00226   const double delta0 = (g0 / sum3 >= kTenoCutoff) ? 1.0 : 0.0;
00227   const double delta1 = (g1 / sum3 >= kTenoCutoff) ? 1.0 : 0.0;
00228   const double delta2 = (g2 / sum3 >= kTenoCutoff) ? 1.0 : 0.0;
00229
00230   const double w0 = kWenoWeight0 * delta0;
00231   const double w1 = kWenoWeight1 * delta1;
00232   const double w2 = kWenoWeight2 * delta2;
00233   const double sum_w = w0 + w1 + w2;
00234   if (!(sum_w > 0.0)) {
00235     return q1;
00236   }
00237   return (w0 * q0 + w1 * q1 + w2 * q2) / sum_w;
00238 }
00239
00240 // Reconstruct the left state at the face between v0 and vp1.
00241 KOKKOS_INLINE_FUNCTION
00242 double reconstructLeft(Reconstruction scheme, double vm2, double vm1, double v0,
00243                        double vp1, double vp2, double vp3) {
00244   switch (scheme) {
00245     case Reconstruction::Central2:
00246       return central2(vm2, vm1, v0, vp1, vp2, vp3);
00247     case Reconstruction::Central4:
00248       return central4(vm2, vm1, v0, vp1, vp2, vp3);
00249     case Reconstruction::Central6:
00250       return central6(vm2, vm1, v0, vp1, vp2, vp3);
00251     case Reconstruction::Weno5Z:
00252       return weno5z(vm2, vm1, v0, vp1, vp2, vp3);
00253     case Reconstruction::Teno6:
00254       return teno6(vm2, vm1, v0, vp1, vp2, vp3);
00255   }
00256   return central2(vm2, vm1, v0, vp1, vp2, vp3);
00257 }
00258
00259 // Reconstruct the right state at the same face by mirroring the stencil.
00260 KOKKOS_INLINE_FUNCTION
00261 double reconstructRight(Reconstruction scheme, double vm2, double vm1,
00262                         double v0, double vp1, double vp2, double vp3) {
00263   return reconstructLeft(scheme, vp3, vp2, vp1, v0, vm1, vm2);
00264 }
00265
00266 // Convenience overload for a callable indexed by an offset in [-2, 3], e.g.
00267 //
00268 //   reconstructLeft(scheme, [&](int d) { return c(0, i + d, j, k); });
00269 //
00270 // The callable may be a device lambda.
00271 template <class Accessor>
00272 KOKKOS_INLINE_FUNCTION double reconstructLeft(Reconstruction scheme,
00273                                               const Accessor& accessor) {
00274   return reconstructLeft(scheme, accessor(-2), accessor(-1), accessor(0),
00275                          accessor(1), accessor(2), accessor(3));
00276 }
00277
00278 template <class Accessor>
00279 KOKKOS_INLINE_FUNCTION double reconstructRight(Reconstruction scheme,
00280                                                const Accessor& accessor) {
00281   return reconstructRight(scheme, accessor(-2), accessor(-1), accessor(0),
00282                           accessor(1), accessor(2), accessor(3));
00283 }
00284
00285 }  // namespace recon
00286 }  // namespace saturn
```

6.44 src/numerics/compressible/riemann.hpp File Reference

```
#include <Kokkos_Core.hpp>
```

Classes

- struct saturn::PrimitiveState
- struct saturn::ConservativeState
- struct saturn::Flux

Namespaces

- namespace saturn
- namespace saturn::riemann

Enumerations

- enum class saturn::RiemannSolver { saturn::Llf , saturn::Hllc , saturn::HllcLowMach , saturn::AusmPlusUp }

Functions

- KOKKOS_INLINE_FUNCTION bool saturn::riemann::valid (const PrimitiveState &state)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::maxSignalSpeed (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION ConservativeState saturn::riemann::toConservative (const PrimitiveState &state, double gamma)
- KOKKOS_INLINE_FUNCTION PrimitiveState saturn::riemann::toPrimitive (const ConservativeState &state, double gamma)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::physicalFlux (const PrimitiveState &state, double gamma)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::averageFlux (const Flux &left, const Flux &right)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::llf (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllcImpl (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllc (const PrimitiveState &left, const PrimitiveState &right, double gamma)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::hllcLowMach (const PrimitiveState &left, const PrimitiveState &right, double gamma, double mach_limit=0.1)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::machPlus1 (double mach)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::machMinus1 (double mach)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::machPlus2 (double mach)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::machMinus2 (double mach)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::pressurePlus5 (double mach, double alpha)
- KOKKOS_INLINE_FUNCTION double saturn::riemann::pressureMinus5 (double mach, double alpha)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::ausmPlusUp (const PrimitiveState &left, const PrimitiveState &right, double gamma, double mach_cutoff=0.1)
- KOKKOS_INLINE_FUNCTION Flux saturn::riemann::flux (RiemannSolver solver, const PrimitiveState &left, const PrimitiveState &right, double gamma)

6.45 riemann.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 namespace saturn {
00006
00007 // Inviscid fluxes for the ideal-gas Euler equations in normal coordinates:
00008 //
00009 //   U = [rho, rho u, rho v, rho w, E]
00010 //   F = [rho u, rho u^2 + p, rho u v, rho u w, u (E + p)]
00011 //
00012 // Here u is the velocity normal to the face and v, w are tangential. The caller
00013 // rotates the state into this frame before calling a solver and rotates the
00014 // returned flux back. All functions are device-callable.
00015 enum class RiemannSolver {
00016   Llf,
00017   Hllc,
00018   HllcLowMach,
00019   AusmPlusUp,
00020 };
00021
00022 struct PrimitiveState {
00023   double rho;
00024   double u;  // normal velocity
00025   double v;  // first tangential velocity
00026   double w;  // second tangential velocity
00027   double p;
00028 };
00029
00030 struct ConservativeState {
00031   double rho;
00032   double mom_n;
00033   double mom_t1;
00034   double mom_t2;
00035   double energy;
00036 };
00037
00038 struct Flux {
00039   double rho;
00040   double mom_n;
00041   double mom_t1;
00042   double mom_t2;
00043   double energy;
00044 };
00045
00046 namespace riemann {
00047
00048 KOKKOS_INLINE_FUNCTION
00049 bool valid(const PrimitiveState& state) {
00050   return state.rho > 0.0 && state.p > 0.0 && state.rho < 1.0e300 &&
00051          state.p < 1.0e300;
00052 }
00053
00054 // Maximum-entropy wave speed estimate used by the local Lax-Friedrichs flux.
00055 KOKKOS_INLINE_FUNCTION
00056 double maxSignalSpeed(const PrimitiveState& left, const PrimitiveState& right,
00057                       double gamma) {
00058   const double c_left = Kokkos::sqrt(gamma * left.p / left.rho);
00059   const double c_right = Kokkos::sqrt(gamma * right.p / right.rho);
00060   const double a_left = Kokkos::fabs(left.u) + c_left;
00061   const double a_right = Kokkos::fabs(right.u) + c_right;
00062   return a_left > a_right ? a_left : a_right;
00063 }
00064
00065 KOKKOS_INLINE_FUNCTION
00066 ConservativeState toConservative(const PrimitiveState& state, double gamma) {
00067   const double kinetic =
00068       0.5 * state.rho *
00069       (state.u * state.u + state.v * state.v + state.w * state.w);
00070   const double energy = state.p / (gamma - 1.0) + kinetic;
00071   return ConservativeState{state.rho, state.rho * state.u, state.rho * state.v,
00072                            state.rho * state.w, energy};
00073 }
00074
00075 KOKKOS_INLINE_FUNCTION
00076 PrimitiveState toPrimitive(const ConservativeState& state, double gamma) {
00077   const double rho = state.rho;
00078   const double inv_rho = rho > 0.0 ? 1.0 / rho : 0.0;
00079   const double u = state.mom_n * inv_rho;
00080   const double v = state.mom_t1 * inv_rho;
00081   const double w = state.mom_t2 * inv_rho;
00082   const double kinetic =
```

```
00083       0.5 *
00084       (state.mom_n * state.mom_n + state.mom_t1 * state.mom_t1 +
00085        state.mom_t2 * state.mom_t2) *
00086       inv_rho;
00087   const double pressure = (gamma - 1.0) * (state.energy - kinetic);
00088   return PrimitiveState{rho, u, v, w, pressure};
00089 }
00090
00091 KOKKOS_INLINE_FUNCTION
00092 Flux physicalFlux(const PrimitiveState& state, double gamma) {
00093   const double energy =
00094       state.p / (gamma - 1.0) +
00095       0.5 * state.rho *
00096           (state.u * state.u + state.v * state.v + state.w * state.w);
00097   return Flux{state.rho * state.u, state.rho * state.u * state.u + state.p,
00098               state.rho * state.u * state.v, state.rho * state.u * state.w,
00099               state.u * (energy + state.p)};
00100 }
00101
00102 KOKKOS_INLINE_FUNCTION
00103 Flux averageFlux(const Flux& left, const Flux& right) {
00104   return Flux{0.5 * (left.rho + right.rho), 0.5 * (left.mom_n + right.mom_n),
00105               0.5 * (left.mom_t1 + right.mom_t1),
00106               0.5 * (left.mom_t2 + right.mom_t2),
00107               0.5 * (left.energy + right.energy)};
00108 }
00109
00110 // Local Lax-Friedrichs / Rusanov flux. Robust fallback and comparison scheme.
00111 KOKKOS_INLINE_FUNCTION
00112 Flux llf(const PrimitiveState& left, const PrimitiveState& right,
00113          double gamma) {
00114   const Flux flux_left = physicalFlux(left, gamma);
00115   const Flux flux_right = physicalFlux(right, gamma);
00116   if (!valid(left) || !valid(right)) {
00117     return averageFlux(flux_left, flux_right);
00118   }
00119
00120   const ConservativeState u_left = toConservative(left, gamma);
00121   const ConservativeState u_right = toConservative(right, gamma);
00122   const double alpha = maxSignalSpeed(left, right, gamma);
00123   return Flux{0.5 * (flux_left.rho + flux_right.rho) -
00124                   0.5 * alpha * (u_right.rho - u_left.rho),
00125               0.5 * (flux_left.mom_n + flux_right.mom_n) -
00126                   0.5 * alpha * (u_right.mom_n - u_left.mom_n),
00127               0.5 * (flux_left.mom_t1 + flux_right.mom_t1) -
00128                   0.5 * alpha * (u_right.mom_t1 - u_left.mom_t1),
00129               0.5 * (flux_left.mom_t2 + flux_right.mom_t2) -
00130                   0.5 * alpha * (u_right.mom_t2 - u_left.mom_t2),
00131               0.5 * (flux_left.energy + flux_right.energy) -
00132                   0.5 * alpha * (u_right.energy - u_left.energy)};
00133 }
00134
00135 // HLLC with Roe-averaged wave-speed bounds.
00136 KOKKOS_INLINE_FUNCTION
00137 Flux hllcImpl(const PrimitiveState& left, const PrimitiveState& right,
00138               double gamma) {
00139   const Flux flux_left = physicalFlux(left, gamma);
00140   const Flux flux_right = physicalFlux(right, gamma);
00141   if (!valid(left) || !valid(right)) {
00142     return averageFlux(flux_left, flux_right);
00143   }
00144   const ConservativeState u_left = toConservative(left, gamma);
00145   const ConservativeState u_right = toConservative(right, gamma);
00146
00147   const double c_left = Kokkos::sqrt(gamma * left.p / left.rho);
00148   const double c_right = Kokkos::sqrt(gamma * right.p / right.rho);
00149   const double h_left = (u_left.energy + left.p) / left.rho;
00150   const double h_right = (u_right.energy + right.p) / right.rho;
00151
00152   // Roe averages give tighter wave-speed bounds than the plain Davis
00153   // estimate while remaining cheap.
00154   const double sqrt_left = Kokkos::sqrt(left.rho);
00155   const double sqrt_right = Kokkos::sqrt(right.rho);
00156   const double inv_sum = 1.0 / (sqrt_left + sqrt_right);
00157   const double u_roe = (sqrt_left * left.u + sqrt_right * right.u) * inv_sum;
00158   const double v_roe = (sqrt_left * left.v + sqrt_right * right.v) * inv_sum;
00159   const double w_roe = (sqrt_left * left.w + sqrt_right * right.w) * inv_sum;
00160   const double h_roe = (sqrt_left * h_left + sqrt_right * h_right) * inv_sum;
00161   double c_roe_sq =
00162       (gamma - 1.0) *
00163       (h_roe - 0.5 * (u_roe * u_roe + v_roe * v_roe + w_roe * w_roe));
00164   if (!(c_roe_sq > 0.0)) {
00165     c_roe_sq = 0.0;
00166   }
00167   const double c_roe = Kokkos::sqrt(c_roe_sq);
00168
00169   double s_left = left.u - c_left;
```

```
00170   const double s_left_roe = u_roe - c_roe;
00171   if (s_left_roe < s_left) {
00172     s_left = s_left_roe;
00173   }
00174   double s_right = right.u + c_right;
00175   const double s_right_roe = u_roe + c_roe;
00176   if (s_right_roe > s_right) {
00177     s_right = s_right_roe;
00178   }
00179
00180   if (!(s_left < s_right)) {
00181     return llf(left, right, gamma);
00182   }
00183
00184   const double denom =
00185       left.rho * (s_left - left.u) - right.rho * (s_right - right.u);
00186   if (Kokkos::fabs(denom) < 1.0e-300) {
00187     return llf(left, right, gamma);
00188   }
00189   const double s_star =
00190       (right.p - left.p + left.rho * left.u * (s_left - left.u) -
00191        right.rho * right.u * (s_right - right.u)) /
00192       denom;
00193
00194   if (s_left >= 0.0) {
00195     return flux_left;
00196   }
00197   if (s_right <= 0.0) {
00198     return flux_right;
00199   }
00200   if (!(s_star > s_left && s_star < s_right)) {
00201     return llf(left, right, gamma);
00202   }
00203
00204   if (s_star >= 0.0) {
00205     const double delta = s_left - s_star;
00206     if (Kokkos::fabs(delta) < 1.0e-300 ||
00207         Kokkos::fabs(s_left - left.u) < 1.0e-300) {
00208       return llf(left, right, gamma);
00209     }
00210     const double factor = left.rho * (s_left - left.u) / delta;
00211     if (!(factor > 0.0)) {
00212       return llf(left, right, gamma);
00213     }
00214     const ConservativeState star{
00215         factor, factor * s_star, factor * left.v, factor * left.w,
00216         factor * (u_left.energy / left.rho +
00217                   (s_star - left.u) *
00218                       (s_star + left.p / (left.rho * (s_left - left.u))))};
00219     return Flux{flux_left.rho + s_left * (star.rho - u_left.rho),
00220                 flux_left.mom_n + s_left * (star.mom_n - u_left.mom_n),
00221                 flux_left.mom_t1 + s_left * (star.mom_t1 - u_left.mom_t1),
00222                 flux_left.mom_t2 + s_left * (star.mom_t2 - u_left.mom_t2),
00223                 flux_left.energy + s_left * (star.energy - u_left.energy)};
00224   }
00225
00226   const double delta = s_right - s_star;
00227   if (Kokkos::fabs(delta) < 1.0e-300 ||
00228       Kokkos::fabs(s_right - right.u) < 1.0e-300) {
00229     return llf(left, right, gamma);
00230   }
00231   const double factor = right.rho * (s_right - right.u) / delta;
00232   if (!(factor > 0.0)) {
00233     return llf(left, right, gamma);
00234   }
00235   const ConservativeState star{
00236       factor, factor * s_star, factor * right.v, factor * right.w,
00237       factor * (u_right.energy / right.rho +
00238                 (s_star - right.u) *
00239                     (s_star + right.p / (right.rho * (s_right - right.u))))};
00240   return Flux{flux_right.rho + s_right * (star.rho - u_right.rho),
00241               flux_right.mom_n + s_right * (star.mom_n - u_right.mom_n),
00242               flux_right.mom_t1 + s_right * (star.mom_t1 - u_right.mom_t1),
00243               flux_right.mom_t2 + s_right * (star.mom_t2 - u_right.mom_t2),
00244               flux_right.energy + s_right * (star.energy - u_right.energy)};
00245 }
00246
00247 KOKKOS_INLINE_FUNCTION
00248 Flux hllc(const PrimitiveState& left, const PrimitiveState& right,
00249           double gamma) {
00250   return hllcImpl(left, right, gamma);
00251 }
00252
00253 // HLLC with a low-Mach correction in the spirit of Fleischmann et al.: the
00254 // acoustic (HLLC) flux is blended with the central flux using
00255 //
00256 //   phi = sin(min(1, Ma_local / Ma_limit) * pi/2),
```

```
00257 //   Ma_local = max(|u_L|/c_L, |u_R|/c_R).
00258 //
00259 // phi = 1 above Ma_limit, so the scheme is exactly HLLC in compressible flow.
00260 // Below Ma_limit the blend removes the excessive acoustic dissipation that
00261 // smears pressure waves at low Mach. A pressure-jump/velocity-jump sensor
00262 // keeps full HLLC upwinding across shocks and strong expansions, where the
00263 // central component would be unsafe. Ma_limit = 0.1 is the recommended value.
00264 KOKKOS_INLINE_FUNCTION
00265 Flux hllcLowMach(const PrimitiveState& left, const PrimitiveState& right,
00266                  double gamma, double mach_limit = 0.1) {
00267   if (!valid(left) || !valid(right) || !(mach_limit > 0.0)) {
00268     return llf(left, right, gamma);
00269   }
00270   const double c_left = Kokkos::sqrt(gamma * left.p / left.rho);
00271   const double c_right = Kokkos::sqrt(gamma * right.p / right.rho);
00272   const double ma_left = Kokkos::fabs(left.u) / c_left;
00273   const double ma_right = Kokkos::fabs(right.u) / c_right;
00274   const double ma_local = ma_left > ma_right ? ma_left : ma_right;
00275   double ratio = ma_local / mach_limit;
00276   if (ratio > 1.0) {
00277     ratio = 1.0;
00278   }
00279   const double phi = Kokkos::sin(0.5 * 3.14159265358979323846 * ratio);
00280   if (phi >= 1.0) {
00281     return hllcImpl(left, right, gamma);
00282   }
00283
00284   const double p_avg = 0.5 * (left.p + right.p);
00285   const double c_avg = 0.5 * (c_left + c_right);
00286   const double pressure_jump = Kokkos::fabs(right.p - left.p);
00287   const double velocity_jump = Kokkos::fabs(right.u - left.u);
00288   if (pressure_jump > 0.1 * p_avg || velocity_jump > 0.1 * c_avg) {
00289     return hllcImpl(left, right, gamma);
00290   }
00291
00292   const Flux hllc_flux = hllcImpl(left, right, gamma);
00293   const Flux central_flux =
00294       averageFlux(physicalFlux(left, gamma), physicalFlux(right, gamma));
00295   return Flux{phi * hllc_flux.rho + (1.0 - phi) * central_flux.rho,
00296               phi * hllc_flux.mom_n + (1.0 - phi) * central_flux.mom_n,
00297               phi * hllc_flux.mom_t1 + (1.0 - phi) * central_flux.mom_t1,
00298               phi * hllc_flux.mom_t2 + (1.0 - phi) * central_flux.mom_t2,
00299               phi * hllc_flux.energy + (1.0 - phi) * central_flux.energy};
00300 }
00301
00302 // AUSM+ splitting polynomials.
00303 KOKKOS_INLINE_FUNCTION
00304 double machPlus1(double mach) { return 0.5 * (mach + Kokkos::fabs(mach)); }
00305
00306 KOKKOS_INLINE_FUNCTION
00307 double machMinus1(double mach) { return 0.5 * (mach - Kokkos::fabs(mach)); }
00308
00309 KOKKOS_INLINE_FUNCTION
00310 double machPlus2(double mach) {
00311   if (Kokkos::fabs(mach) >= 1.0) {
00312     return machPlus1(mach);
00313   }
00314   return 0.25 * (mach + 1.0) * (mach + 1.0);
00315 }
00316
00317 KOKKOS_INLINE_FUNCTION
00318 double machMinus2(double mach) {
00319   if (Kokkos::fabs(mach) >= 1.0) {
00320     return machMinus1(mach);
00321   }
00322   return -0.25 * (mach - 1.0) * (mach - 1.0);
00323 }
00324
00325 KOKKOS_INLINE_FUNCTION
00326 double pressurePlus5(double mach, double alpha) {
00327   if (Kokkos::fabs(mach) >= 1.0) {
00328     return machPlus1(mach) / mach;
00329   }
00330   return machPlus2(mach) *
00331          ((2.0 - mach) - 16.0 * alpha * mach * machMinus2(mach));
00332 }
00333
00334 KOKKOS_INLINE_FUNCTION
00335 double pressureMinus5(double mach, double alpha) {
00336   if (Kokkos::fabs(mach) >= 1.0) {
00337     return machMinus1(mach) / mach;
00338   }
00339   return machMinus2(mach) *
00340          ((-2.0 - mach) + 16.0 * alpha * mach * machPlus2(mach));
00341 }
00342
00343 // AUSM+-up (Liou 2006) with the standard split Mach polynomials and the
```

```
00344 // all-speed pressure/mass diffusion terms. `mach_cutoff` is the free-stream
00345 // Mach number used to floor the low-Mach scaling function f_a.
00346 KOKKOS_INLINE_FUNCTION
00347 Flux ausmPlusUp(const PrimitiveState& left, const PrimitiveState& right,
00348                 double gamma, double mach_cutoff = 0.1) {
00349   if (!valid(left) || !valid(right)) {
00350     return llf(left, right, gamma);
00351   }
00352   const double c_left = Kokkos::sqrt(gamma * left.p / left.rho);
00353   const double c_right = Kokkos::sqrt(gamma * right.p / right.rho);
00354   const double a_face = 0.5 * (c_left + c_right);
00355   const double inv_a = 1.0 / a_face;
00356
00357   const double mach_left = left.u * inv_a;
00358   const double mach_right = right.u * inv_a;
00359
00360   double mbar_sq =
00361       (left.u * left.u + right.u * right.u) / (2.0 * a_face * a_face);
00362   const double cutoff = mach_cutoff > 0.0 ? mach_cutoff : 1.0e-4;
00363   const double cutoff_sq = cutoff * cutoff;
00364   if (mbar_sq < cutoff_sq) {
00365     mbar_sq = cutoff_sq;
00366   }
00367   if (mbar_sq > 1.0) {
00368     mbar_sq = 1.0;
00369   }
00370   const double m0 = Kokkos::sqrt(mbar_sq);
00371   double f_a = m0 * (2.0 - m0);
00372   if (f_a < 1.0e-8) {
00373     f_a = 1.0e-8;
00374   }
00375
00376   const double alpha = (3.0 / 16.0) * (-4.0 + 5.0 * f_a * f_a);
00377   const double p_plus_left = pressurePlus5(mach_left, alpha);
00378   const double p_minus_right = pressureMinus5(mach_right, alpha);
00379
00380   const double rho_face = 0.5 * (left.rho + right.rho);
00381   double pressure_diffusion = -0.75 * p_plus_left * p_minus_right *
00382                               (left.rho + right.rho) * (f_a * a_face) *
00383                               (right.u - left.u);
00384
00385   double mass_diffusion = 0.0;
00386   const double pressure_jump = right.p - left.p;
00387   if (Kokkos::fabs(pressure_jump) > 0.0) {
00388     mass_diffusion = -(0.25 / f_a) * (1.0 - mbar_sq) * pressure_jump /
00389                      (rho_face * a_face * a_face);
00390   }
00391
00392   const double mach_face =
00393       machPlus2(mach_left) + machMinus2(mach_right) + mass_diffusion;
00394   const double mass_flux =
00395       a_face * mach_face * (mach_face > 0.0 ? left.rho : right.rho);
00396   const double pressure_face =
00397       p_plus_left * left.p + p_minus_right * right.p + pressure_diffusion;
00398
00399   const PrimitiveState& upwind = mass_flux > 0.0 ? left : right;
00400   const double energy =
00401       upwind.p / (gamma - 1.0) +
00402       0.5 * upwind.rho *
00403           (upwind.u * upwind.u + upwind.v * upwind.v + upwind.w * upwind.w);
00404   const double enthalpy = (energy + upwind.p) / upwind.rho;
00405
00406   return Flux{mass_flux, mass_flux * upwind.u + pressure_face,
00407               mass_flux * upwind.v, mass_flux * upwind.w, mass_flux * enthalpy};
00408 }
00409
00410 KOKKOS_INLINE_FUNCTION
00411 Flux flux(RiemannSolver solver, const PrimitiveState& left,
00412           const PrimitiveState& right, double gamma) {
00413   switch (solver) {
00414     case RiemannSolver::Llf:
00415       return llf(left, right, gamma);
00416     case RiemannSolver::Hllc:
00417       return hllc(left, right, gamma);
00418     case RiemannSolver::HllcLowMach:
00419       return hllcLowMach(left, right, gamma);
00420     case RiemannSolver::AusmPlusUp:
00421       return ausmPlusUp(left, right, gamma);
00422   }
00423   return llf(left, right, gamma);
00424 }
00425
00426 }  // namespace riemann
00427 }  // namespace saturn
```

6.46 src/numerics/compressible/scalar_advection.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <algorithm>
#include <array>
#include "core/parallel.hpp"
#include "fields/patch_field.hpp"
#include "mesh/box.hpp"
```

Namespaces

- namespace saturn

Functions

- template<class PatchType >
  KOKKOS_INLINE_FUNCTION double saturn::scalarRhsValue (const PatchType &c, int i, int j, int k, double ux, double uy, double uz, double hx, double hy, double hz, double nu)
- template<class ExecSpace >
  void saturn::scalarAdvectionDiffusionRhs (const PatchField< ExecSpace > &c, PatchField< ExecSpace > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h, const Box &region)
- template<class ExecSpace >
  void saturn::scalarAdvectionDiffusionRhs (const PatchField< ExecSpace > &c, PatchField< ExecSpace > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void saturn::scalarAdvectionDiffusionRhsShell (const PatchField< ExecSpace > &c, PatchField< Exec↩ Space > &rhs, const std::array< double, 3 > &velocity, double nu, const std::array< double, 3 > &h, const Box &box)

6.47 scalar_advection.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <algorithm>
00006 #include <array>
00007
00008 #include "core/parallel.hpp"
00009 #include "fields/patch_field.hpp"
00010 #include "mesh/box.hpp"
00011
00012 namespace saturn {
00013
00014 // Pointwise second-order central advection-diffusion operator:
00015 //
00016 //   L(c) = -u . grad(c) + nu * laplacian(c)
00017 //
00018 // The caller must ensure the required neighbors are valid. This helper is
00019 // shared by the interior and boundary-shell kernels.
00020 template <class PatchType>
00021 KOKKOS_INLINE_FUNCTION double scalarRhsValue(const PatchType& c, int i, int j,
00022                                               int k, double ux, double uy,
00023                                               double uz, double hx, double hy,
00024                                               double hz, double nu) {
00025   const double cc = c(0, i, j, k);
00026   const double adv = 0.5 * ux * (c(0, i + 1, j, k) - c(0, i - 1, j, k)) / hx +
00027                      0.5 * uy * (c(0, i, j + 1, k) - c(0, i, j - 1, k)) / hy +
00028                      0.5 * uz * (c(0, i, j, k + 1) - c(0, i, j, k - 1)) / hz;
```

```
00029   const double lap =
00030       (c(0, i + 1, j, k) - 2.0 * cc + c(0, i - 1, j, k)) / (hx * hx) +
00031       (c(0, i, j + 1, k) - 2.0 * cc + c(0, i, j - 1, k)) / (hy * hy) +
00032       (c(0, i, j, k + 1) - 2.0 * cc + c(0, i, j, k - 1)) / (hz * hz);
00033   return -adv + nu * lap;
00034 }
00035
00036 template <class ExecSpace>
00037 void scalarAdvectionDiffusionRhs(const PatchField<ExecSpace>& c,
00038                                  PatchField<ExecSpace>& rhs,
00039                                  const std::array<double, 3>& velocity,
00040                                  double nu, const std::array<double, 3>& h,
00041                                  const Box& region) {
00042   if (region.empty()) {
00043     return;
00044   }
00045   const auto policy = makeMdrange3<ExecSpace>(
00046       Int3{{region.lo[2], region.lo[1], region.lo[0]}},
00047       Int3{{region.hi[2] + 1, region.hi[1] + 1, region.hi[0] + 1}});
00048
00049   const double ux = velocity[0];
00050   const double uy = velocity[1];
00051   const double uz = velocity[2];
00052   const double hx = h[0];
00053   const double hy = h[1];
00054   const double hz = h[2];
00055
00056   Kokkos::parallel_for(
00057       "scalar_advection_diffusion", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00058         rhs(0, i, j, k) =
00059             scalarRhsValue(c, i, j, k, ux, uy, uz, hx, hy, hz, nu);
00060       });
00061 }
00062
00063 // Full-box operator (all interior cells).
00064 template <class ExecSpace>
00065 void scalarAdvectionDiffusionRhs(const PatchField<ExecSpace>& c,
00066                                  PatchField<ExecSpace>& rhs,
00067                                  const std::array<double, 3>& velocity,
00068                                  double nu, const std::array<double, 3>& h) {
00069   scalarAdvectionDiffusionRhs(c, rhs, velocity, nu, h, c.box());
00070 }
00071
00072 // Boundary shell of a box: six faces, partitioned so every shell cell is
00073 // visited exactly once. Requires valid ghosts (call after FaceExchange::end).
00074 template <class ExecSpace>
00075 void scalarAdvectionDiffusionRhsShell(const PatchField<ExecSpace>& c,
00076                                       PatchField<ExecSpace>& rhs,
00077                                       const std::array<double, 3>& velocity,
00078                                       double nu, const std::array<double, 3>& h,
00079                                       const Box& box) {
00080   const int lo0 = box.lo[0];
00081   const int hi0 = box.hi[0];
00082   const int lo1 = box.lo[1];
00083   const int hi1 = box.hi[1];
00084   const int lo2 = box.lo[2];
00085   const int hi2 = box.hi[2];
00086   const int nx = box.length(0);
00087   const int ny = box.length(1);
00088   const int nz = box.length(2);
00089   const int nmax = std::max({nx, ny, nz});
00090
00091   const auto policy =
00092       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{6, nmax, nmax}});
00093
00094   const double ux = velocity[0];
00095   const double uy = velocity[1];
00096   const double uz = velocity[2];
00097   const double hx = h[0];
00098   const double hy = h[1];
00099   const double hz = h[2];
00100
00101   Kokkos::parallel_for(
00102       "scalar_rhs_shell", policy, KOKKOS_LAMBDA(int face, int a, int b) {
00103         int i = 0;
00104         int j = 0;
00105         int k = 0;
00106         bool active = false;
00107         if (face < 2) {
00108           // x faces: all tangential cells.
00109           if (a < ny && b < nz) {
00110             i = (face == 0) ? lo0 : hi0;
00111             j = lo1 + a;
00112             k = lo2 + b;
00113             active = true;
00114           }
00115         } else if (face < 4) {
```

```
00116           // y faces: exclude the x edges.
00117           if (a < nx && b < nz && nx > 2) {
00118             i = lo0 + a;
00119             j = (face == 2) ? lo1 : hi1;
00120             k = lo2 + b;
00121             active = (i > lo0 && i < hi0);
00122           }
00123         } else {
00124           // z faces: exclude the x and y edges.
00125           if (a < nx && b < ny && nx > 2 && ny > 2) {
00126             i = lo0 + a;
00127             j = lo1 + b;
00128             k = (face == 4) ? lo2 : hi2;
00129             active = (i > lo0 && i < hi0 && j > lo1 && j < hi1);
00130           }
00131         }
00132         if (active) {
00133           rhs(0, i, j, k) =
00134               scalarRhsValue(c, i, j, k, ux, uy, uz, hx, hy, hz, nu);
00135         }
00136       });
00137 }
00138
00139 }  // namespace saturn
```

6.48 src/numerics/compressible/time_integration.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include "core/parallel.hpp"
#include "fields/patch_field.hpp"
```

Namespaces

- namespace saturn

Functions

- template<class Field >
  void saturn::fieldCombine (Field &destination, double a, const Field &x, double b, const Field &y)
- template<class Field >
  void saturn::fieldAddScaled (Field &destination, double scale, const Field &x)
- template<class Field >
  void saturn::fieldCopy (const Field &source, Field &destination)
- template<class Field , class ComputeRhs >
  void saturn::advanceSsprk3 (Field &q, Field &stage, Field &next, Field &rhs, double dt, double time, ComputeRhs &computeRhs)
- template<class Field , class ComputeRhs >
  void saturn::advanceRk4 (Field &q, Field &stage, Field &rhs, Field &k1, Field &k2, Field &k3, double dt, double time, ComputeRhs &computeRhs)

6.49 time_integration.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include "core/parallel.hpp"
00006 #include "fields/patch_field.hpp"
```

```
00007
00008 namespace saturn {
00009
00010 template <class Field>
00011 void fieldCombine(Field& destination, double a, const Field& x, double b,
00012                   const Field& y) {
00013   using ExecSpace = typename Field::execution_space;
00014   const Box box = destination.box();
00015   const auto policy = makeMdrange3<ExecSpace>(
00016       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00017       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00018   auto dest = destination.data();
00019   auto xd = x.data();
00020   auto yd = y.data();
00021   const int nvar = destination.nvar();
00022   const int dng = destination.ng();
00023   const int xng = x.ng();
00024   const int yng = y.ng();
00025   const int lo0 = box.lo[0];
00026   const int lo1 = box.lo[1];
00027   const int lo2 = box.lo[2];
00028   Kokkos::parallel_for(
00029       "fieldCombine", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00030         const int ip = i - lo0 + dng;
00031         const int jp = j - lo1 + dng;
00032         const int kp = k - lo2 + dng;
00033         const int xip = i - lo0 + xng;
00034         const int xjp = j - lo1 + xng;
00035         const int xkp = k - lo2 + xng;
00036         const int yip = i - lo0 + yng;
00037         const int yjp = j - lo1 + yng;
00038         const int ykp = k - lo2 + yng;
00039         for (int v = 0; v < nvar; ++v) {
00040           dest(v, kp, jp, ip) =
00041               a * xd(v, xkp, xjp, xip) + b * yd(v, ykp, yjp, yip);
00042         }
00043       });
00044 }
00045
00046 template <class Field>
00047 void fieldAddScaled(Field& destination, double scale, const Field& x) {
00048   using ExecSpace = typename Field::execution_space;
00049   const Box box = destination.box();
00050   const auto policy = makeMdrange3<ExecSpace>(
00051       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00052       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00053   auto dest = destination.data();
00054   auto xd = x.data();
00055   const int nvar = destination.nvar();
00056   const int dng = destination.ng();
00057   const int xng = x.ng();
00058   const int lo0 = box.lo[0];
00059   const int lo1 = box.lo[1];
00060   const int lo2 = box.lo[2];
00061   Kokkos::parallel_for(
00062       "fieldAddScaled", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00063         const int ip = i - lo0 + dng;
00064         const int jp = j - lo1 + dng;
00065         const int kp = k - lo2 + dng;
00066         const int xip = i - lo0 + xng;
00067         const int xjp = j - lo1 + xng;
00068         const int xkp = k - lo2 + xng;
00069         for (int v = 0; v < nvar; ++v) {
00070           dest(v, kp, jp, ip) += scale * xd(v, xkp, xjp, xip);
00071         }
00072       });
00073 }
00074
00075 template <class Field>
00076 void fieldCopy(const Field& source, Field& destination) {
00077   using ExecSpace = typename Field::execution_space;
00078   const Box box = source.box();
00079   const auto policy = makeMdrange3<ExecSpace>(
00080       Int3{{box.lo[2], box.lo[1], box.lo[0]}},
00081       Int3{{box.hi[2] + 1, box.hi[1] + 1, box.hi[0] + 1}});
00082   auto src = source.data();
00083   auto dest = destination.data();
00084   const int nvar = source.nvar();
00085   const int sng = source.ng();
00086   const int dng = destination.ng();
00087   const int lo0 = box.lo[0];
00088   const int lo1 = box.lo[1];
00089   const int lo2 = box.lo[2];
00090   Kokkos::parallel_for(
00091       "fieldCopy", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00092         const int sip = i - lo0 + sng;
00093         const int sjp = j - lo1 + sng;
```

```
00094         const int skp = k - lo2 + sng;
00095         const int dip = i - lo0 + dng;
00096         const int djp = j - lo1 + dng;
00097         const int dkp = k - lo2 + dng;
00098         for (int v = 0; v < nvar; ++v) {
00099           dest(v, dkp, djp, dip) = src(v, skp, sjp, sip);
00100         }
00101       });
00102 }
00103
00104 // SSP-RK3. `computeRhs(state, rhs, time)` must fill rhs with
00105 // L(state, time). Boundary conditions that depend on time are evaluated at
00106 // the stage time.
00107 template <class Field, class ComputeRhs>
00108 void advanceSsprk3(Field& q, Field& stage, Field& next, Field& rhs, double dt,
00109                    double time, ComputeRhs& computeRhs) {
00110   computeRhs(q, rhs, time);
00111   fieldCombine(stage, 1.0, q, dt, rhs);
00112
00113   computeRhs(stage, rhs, time + dt);
00114   fieldCombine(next, 0.75, q, 0.25, stage);
00115   fieldAddScaled(next, 0.25 * dt, rhs);
00116
00117   computeRhs(next, rhs, time + 0.5 * dt);
00118   fieldCombine(q, 1.0 / 3.0, q, 2.0 / 3.0, next);
00119   fieldAddScaled(q, (2.0 / 3.0) * dt, rhs);
00120 }
00121
00122 // Classical RK4. `computeRhs(state, rhs, time)` must fill rhs with
00123 // L(state, time).
00124 template <class Field, class ComputeRhs>
00125 void advanceRk4(Field& q, Field& stage, Field& rhs, Field& k1, Field& k2,
00126                 Field& k3, double dt, double time, ComputeRhs& computeRhs) {
00127   computeRhs(q, rhs, time);
00128   fieldCopy(rhs, k1);
00129   fieldCombine(stage, 1.0, q, 0.5 * dt, k1);
00130
00131   computeRhs(stage, rhs, time + 0.5 * dt);
00132   fieldCopy(rhs, k2);
00133   fieldCombine(stage, 1.0, q, 0.5 * dt, k2);
00134
00135   computeRhs(stage, rhs, time + 0.5 * dt);
00136   fieldCopy(rhs, k3);
00137   fieldCombine(stage, 1.0, q, dt, k3);
00138
00139   computeRhs(stage, rhs, time + dt);
00140   fieldAddScaled(q, dt / 6.0, k1);
00141   fieldAddScaled(q, dt / 3.0, k2);
00142   fieldAddScaled(q, dt / 3.0, k3);
00143   fieldAddScaled(q, dt / 6.0, rhs);
00144 }
00145
00146 }  // namespace saturn
```

6.50 src/numerics/incompressible/cavity.hpp File Reference

Lid-driven cavity on a staggered MAC grid, uniform or wall-clustered.

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include "core/parallel.hpp"
#include "numerics/compressible/reconstruction.hpp"
#include "numerics/incompressible/incompressible.hpp"
#include "numerics/incompressible/poisson.hpp"
#include "numerics/incompressible/projection.hpp"
```

Classes

- struct saturn::cavity::Geometry

  Grid, lid speed, advection scheme, and optional wall-clustering map.
- class saturn::cavity::ClusteredPoisson2D

  Host variable-coefficient Poisson solver for the clustered 2D grid.

Namespaces

- namespace saturn
- namespace saturn::cavity

Functions

- KOKKOS_INLINE_FUNCTION double saturn::cavity::minmod (double a, double b)
- KOKKOS_INLINE_FUNCTION double saturn::cavity::weno5Derivative (double qm3, double qm2, double qm1, double q0, double qp1, double qp2, double qp3, double vel, double h)
- KOKKOS_INLINE_FUNCTION double saturn::cavity::limitedUpwind (double q0, double qm1, double qm2, double qp1, double qp2, double vel, double h)
- template<class ExecSpace >
  void saturn::cavity::divergence (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &div, const Geometry &g)
- template<class ExecSpace >
  void saturn::cavity::momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const Geometry &g, double nu)

  Momentum right-hand side on the MAC faces.
- template<class ExecSpace >
  void saturn::cavity::filterVelocity (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u_out, projection::ScalarView< ExecSpace > &v_out, projection::ScalarView< ExecSpace > &w_out, const Geometry &g, double alpha)

  Weak low-pass/dealiasing filter applied to the velocity field.
- template<class ExecSpace >
  void saturn::cavity::projectClustered (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g)

  Pressure projection on the wall-clustered 2D grid.
- template<class ExecSpace , class PoissonSolver >
  poisson::Result saturn::cavity::project (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g, PoissonSolver &solver, const poisson::Options &options)

  MAC pressure projection on the uniform grid.
- template<class ExecSpace , class Project >
  void saturn::cavity::advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const Geometry &g, double nu, double dt, Project &project)

  SSP-RK3 step with a pressure projection after every stage.

6.50.1 Detailed Description

Lid-driven cavity on a staggered MAC grid, uniform or wall-clustered.

The solver is a second-order MAC projection with SSP-RK3 and selectable central/upwind/MUSCL/WENO5 advection. The 2D clustered path uses a tanh wall map and a variable-coefficient pressure PCG.

6.51 cavity.hpp

Go to the documentation of this file.

```
00001
00009 #pragma once
00010
00011 #include <Kokkos_Core.hpp>
00012
00013 #include <array>
00014 #include <cmath>
00015
00016 #include "core/parallel.hpp"
00017 #include "numerics/compressible/reconstruction.hpp"
00018 #include "numerics/incompressible/incompressible.hpp"
00019 #include "numerics/incompressible/poisson.hpp"
00020 #include "numerics/incompressible/projection.hpp"
00021
00022 namespace saturn {
00023 namespace cavity {
00024
00025 // Lid-driven cavity on a staggered MAC grid.
00026 //
00027 //   x, y: no-slip walls; the top wall moves in x with speed `u_lid`
00028 //   z:    periodic
00029 //
00030 // Layout (same extents as the cell-centered pressure):
00031 //
00032 //   u(i,j,k): x-face; u(0)=0 at the left wall, u(n0)=0 (implicit) at the right
00033 //   v(i,j,k): y-face; v(0)=0 at the bottom, v(n1)=0 (implicit) at the top
00034 //   w(i,j,k): z-face; tangential no-slip ghosts in x and y
00035 //
00036 // Pressure uses homogeneous Neumann conditions on the x/y walls and periodic
00037 // conditions in z.
00038
00041 struct Geometry {
00042   int n0 = 1;
00043   int n1 = 1;
00044   int n2 = 1;
00045   double hx = 1.0;
00046   double hy = 1.0;
00047   double hz = 1.0;
00048   double u_lid = 1.0;
00049   // 0 = central, 1 = first-order upwind, 2 = upwind-biased MUSCL,
00050   // 3 = blended central/upwind, 4 = WENO5.
00051   int adv = 0;
00052   double adv_blend = 0.25;
00053   bool z_wall = false;
00054   // beta = 0: uniform grid. beta > 0: tanh clustering at both walls.
00055   double beta = 0.0;
00056
00057   KOKKOS_INLINE_FUNCTION
00058   double mapX(double xi) const {
00059     if (beta <= 0.0) {
00060       return xi;
00061     }
00062     return 0.5 *
00063            (1.0 + Kokkos::tanh(beta * (2.0 * xi - 1.0)) / Kokkos::tanh(beta));
00064   }
00065   KOKKOS_INLINE_FUNCTION
00066   double dmapX(double xi) const {
00067     if (beta <= 0.0) {
00068       return 1.0;
00069     }
00070     const double t = Kokkos::tanh(beta * (2.0 * xi - 1.0));
00071     return beta / Kokkos::tanh(beta) * (1.0 - t * t);
00072   }
00073   KOKKOS_INLINE_FUNCTION
00074   double xf(int i) const { return mapX(static_cast<double>(i) / n0); }
00075   KOKKOS_INLINE_FUNCTION
00076   double xc(int i) const { return mapX((static_cast<double>(i) + 0.5) / n0); }
00077   KOKKOS_INLINE_FUNCTION
00078   double yf(int j) const { return mapX(static_cast<double>(j) / n1); }
00079   KOKKOS_INLINE_FUNCTION
00080   double yc(int j) const { return mapX((static_cast<double>(j) + 0.5) / n1); }
00081 };
00082
00083 KOKKOS_INLINE_FUNCTION
00084 double minmod(double a, double b) {
00085   if (a * b <= 0.0) {
00086     return 0.0;
00087   }
00088   return Kokkos::abs(a) < Kokkos::abs(b) ? a : b;
00089 }
00090
00091 // WENO5 derivative from six samples q[i-3..i+2]: reconstruct the two
```

```
00092 // midpoints with weno5z and difference them.
00093 KOKKOS_INLINE_FUNCTION
00094 double weno5Derivative(double qm3, double qm2, double qm1, double q0,
00095                        double qp1, double qp2, double qp3, double vel,
00096                        double h) {
00097   if (vel >= 0.0) {
00098     const double left = recon::weno5z(qm3, qm2, qm1, q0, qp1, 0.0);
00099     const double right = recon::weno5z(qm2, qm1, q0, qp1, qp2, 0.0);
00100     return (right - left) / h;
00101   }
00102   // Mirror the stencil for negative advection velocity.
00103   const double left = recon::weno5z(qp3, qp2, qp1, q0, qm1, 0.0);
00104   const double right = recon::weno5z(qp2, qp1, q0, qm1, qm2, 0.0);
00105   return (left - right) / h;
00106 }
00107
00108 // Second-order upwind derivative with a minmod limiter.
00109 KOKKOS_INLINE_FUNCTION
00110 double limitedUpwind(double q0, double qm1, double qm2, double qp1, double qp2,
00111                      double vel, double h) {
00112   if (vel > 0.0) {
00113     const double d1 = (q0 - qm1) / h;
00114     const double d2 = (3.0 * q0 - 4.0 * qm1 + qm2) / (2.0 * h);
00115     return minmod(d1, d2);
00116   }
00117   const double d1 = (qp1 - q0) / h;
00118   const double d2 = (-3.0 * q0 + 4.0 * qp1 - qp2) / (2.0 * h);
00119   return minmod(d1, d2);
00120 }
00121
00122 template <class ExecSpace>
00123 void divergence(const projection::ScalarView<ExecSpace>& u,
00124                 const projection::ScalarView<ExecSpace>& v,
00125                 const projection::ScalarView<ExecSpace>& w,
00126                 projection::ScalarView<ExecSpace>& div, const Geometry& g) {
00127   const int n0 = g.n0;
00128   const int n1 = g.n1;
00129   const int n2 = g.n2;
00130   const auto policy =
00131       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00132   Kokkos::parallel_for(
00133       "cavity_divergence", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00134         const int ip = (i + 1 == n0) ? -1 : i + 1;
00135         const int jp = (j + 1 == n1) ? -1 : j + 1;
00136         const int kp = (k + 1 == n2) ? (g.z_wall ? -1 : 0) : k + 1;
00137         const double up = (ip < 0) ? 0.0 : u(k, j, ip);
00138         const double vp = (jp < 0) ? 0.0 : v(k, jp, i);
00139         const double wp = (kp < 0) ? 0.0 : w(kp, j, i);
00140         const double dx = g.xf(i + 1) - g.xf(i);
00141         const double dy = g.yf(j + 1) - g.yf(j);
00142         div(k, j, i) = (up - u(k, j, i)) / dx + (vp - v(k, j, i)) / dy +
00143                        (wp - w(k, j, i)) / g.hz;
00144       });
00145 }
00146
00154 template <class ExecSpace>
00155 void momentumRhs(const projection::ScalarView<ExecSpace>& u,
00156                  const projection::ScalarView<ExecSpace>& v,
00157                  const projection::ScalarView<ExecSpace>& w,
00158                  projection::ScalarView<ExecSpace>& du,
00159                  projection::ScalarView<ExecSpace>& dv,
00160                  projection::ScalarView<ExecSpace>& dw, const Geometry& g,
00161                  double nu) {
00162   const int n0 = g.n0;
00163   const int n1 = g.n1;
00164   const int n2 = g.n2;
00165   const double hx = g.hx;
00166   const double hy = g.hy;
00167   const double hz = g.hz;
00168   const double hz2 = hz * hz;
00169   const double u_lid = g.u_lid;
00170   const int adv = g.adv;
00171   const double adv_blend = g.adv_blend;
00172   const auto policy =
00173       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00174   Kokkos::parallel_for(
00175       "cavity_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00176         const int im = (i == 0) ? -1 : i - 1;
00177         const int ip = (i + 1 == n0) ? -1 : i + 1;
00178         const int jm = (j == 0) ? -1 : j - 1;
00179         const int jp = (j + 1 == n1) ? -1 : j + 1;
00180         const int km = (k == 0) ? n2 - 1 : k - 1;
00181         const int kp = (k + 1 == n2) ? 0 : k + 1;
00182         const double u_here = u(k, j, i);
00183         const double v_here = v(k, j, i);
00184         const double w_here = w(k, j, i);
00185
```

```
00186         // ----- u-momentum (interior x-faces only) -----
00187         if (i == 0) {
00188           du(k, j, i) = 0.0;
00189         } else {
00190           const double u_im = (im < 0) ? 0.0 : u(k, j, im);
00191           const double u_ip = (ip < 0) ? 0.0 : u(k, j, ip);
00192           const double u_jm = (jm < 0) ? -u_here : u(k, jm, i);
00193           const double u_jp = (jp < 0) ? 2.0 * u_lid - u_here : u(k, jp, i);
00194           const double u_km = (k == 0) ? (g.z_wall ? -u_here : u(n2 - 1, j, i))
00195                                        : u(k - 1, j, i);
00196           const double u_kp = (k + 1 == n2) ? (g.z_wall ? -u_here : u(0, j, i))
00197                                             : u(k + 1, j, i);
00198           double dudx = (u_ip - u_im) / (2.0 * hx);
00199           double dudy = (u_jp - u_jm) / (2.0 * hy);
00200           double dudz = (u_kp - u_km) / (2.0 * hz);
00201           const double v_jm = (jm < 0) ? 0.0 : v(k, jm, i);
00202           const double v_jp = (jp < 0) ? 0.0 : v(k, jp, i);
00203           const double v_jm_im = (jm < 0) ? 0.0 : v(k, jm, im);
00204           const double v_jp_im = (jp < 0) ? 0.0 : v(k, jp, im);
00205           const double v_at_u = 0.25 * (v_jm_im + v_jm + v_jp_im + v_jp);
00206           const double w_at_u =
00207               0.25 * (w(k, j, im) + w(k, j, i) + w(kp, j, im) + w(kp, j, i));
00208           if (adv == 1) {
00209             dudx = u_here > 0.0 ? (u_here - u_im) / hx : (u_ip - u_here) / hx;
00210             dudy = v_at_u > 0.0 ? (u_here - u_jm) / hy : (u_jp - u_here) / hy;
00211             dudz = w_at_u > 0.0 ? (u_here - u_km) / hz : (u_kp - u_here) / hz;
00212           } else if (adv == 2) {
00213             dudx = minmod(u_here - u_im, u_ip - u_here) / hx;
00214             dudy = minmod(u_here - u_jm, u_jp - u_here) / hy;
00215             dudz = minmod(u_here - u_km, u_kp - u_here) / hz;
00216           } else if (adv == 3) {
00217             const double cx = (u_ip - u_im) / (2.0 * hx);
00218             const double cy = (u_jp - u_jm) / (2.0 * hy);
00219             const double cz = (u_kp - u_km) / (2.0 * hz);
00220             const double ux =
00221                 u_here > 0.0 ? (u_here - u_im) / hx : (u_ip - u_here) / hx;
00222             const double uy =
00223                 v_at_u > 0.0 ? (u_here - u_jm) / hy : (u_jp - u_here) / hy;
00224             const double uz =
00225                 w_at_u > 0.0 ? (u_here - u_km) / hz : (u_kp - u_here) / hz;
00226             dudx = (1.0 - adv_blend) * cx + adv_blend * ux;
00227             dudy = (1.0 - adv_blend) * cy + adv_blend * uy;
00228             dudz = (1.0 - adv_blend) * cz + adv_blend * uz;
00229           } else if (adv == 4) {
00230             if (i >= 3 && i + 3 < n0) {
00231               dudx = weno5Derivative(
00232                   u(k, j, i - 3), u(k, j, i - 2), u(k, j, i - 1), u_here,
00233                   u(k, j, i + 1), u(k, j, i + 2), u(k, j, i + 3), u_here, hx);
00234             } else {
00235               dudx = minmod(u_here - u_im, u_ip - u_here) / hx;
00236             }
00237             if (j >= 3 && j + 3 < n1) {
00238               dudy = weno5Derivative(
00239                   u(k, j - 3, i), u(k, j - 2, i), u(k, j - 1, i), u_here,
00240                   u(k, j + 1, i), u(k, j + 2, i), u(k, j + 3, i), v_at_u, hy);
00241             } else {
00242               dudy = minmod(u_here - u_jm, u_jp - u_here) / hy;
00243             }
00244             if (!g.z_wall && n2 >= 5) {
00245               dudz = weno5Derivative(
00246                   u((k + n2 - 3) % n2, j, i), u((k + n2 - 2) % n2, j, i),
00247                   u((k + n2 - 1) % n2, j, i), u_here, u((k + 1) % n2, j, i),
00248                   u((k + 2) % n2, j, i), u((k + 3) % n2, j, i), w_at_u, hz);
00249             }
00250           }
00251           dudx /= g.dmapX(i * hx);
00252           dudy /= g.dmapX((j + 0.5) * hy);
00253           const double dxm_u = g.xf(i) - g.xf(i - 1);
00254           const double dxp_u = g.xf(i + 1) - g.xf(i);
00255           const double dym_u =
00256               (j == 0) ? g.yf(1) - g.yf(0) : g.yc(j) - g.yc(j - 1);
00257           const double dyp_u =
00258               (j + 1 == n1) ? g.yf(n1) - g.yf(n1 - 1) : g.yc(j + 1) - g.yc(j);
00259           const double lap_u =
00260               2.0 / (dxm_u + dxp_u) *
00261                   ((u_ip - u_here) / dxp_u - (u_here - u_im) / dxm_u) +
00262               2.0 / (dym_u + dyp_u) *
00263                   ((u_jp - u_here) / dyp_u - (u_here - u_jm) / dym_u) +
00264               (u_kp - 2.0 * u_here + u_km) / hz2;
00265           du(k, j, i) =
00266               -(u_here * dudx + v_at_u * dudy + w_at_u * dudz) + nu * lap_u;
00267         }
00268
00269         // ----- v-momentum (interior y-faces only) -----
00270         if (j == 0) {
00271           dv(k, j, i) = 0.0;
00272         } else {
```

```
00273           const double v_im = (im < 0) ? -v_here : v(k, j, im);
00274           const double v_ip = (ip < 0) ? -v_here : v(k, j, ip);
00275           const double v_jm = (jm < 0) ? 0.0 : v(k, jm, i);
00276           const double v_jp = (jp < 0) ? 0.0 : v(k, jp, i);
00277           const double v_km = (k == 0) ? (g.z_wall ? -v_here : v(n2 - 1, j, i))
00278                                        : v(k - 1, j, i);
00279           const double v_kp = (k + 1 == n2) ? (g.z_wall ? -v_here : v(0, j, i))
00280                                             : v(k + 1, j, i);
00281           double dvdx = (v_ip - v_im) / (2.0 * hx);
00282           double dvdy = (v_jp - v_jm) / (2.0 * hy);
00283           double dvdz = (v_kp - v_km) / (2.0 * hz);
00284           const double u_here_i = (i == 0) ? 0.0 : u(k, j, i);
00285           const double u_here_ip = (ip < 0) ? 0.0 : u(k, j, ip);
00286           const double u_jm_i = (jm < 0) ? 0.0 : u(k, jm, i);
00287           const double u_jm_ip = (jm < 0 || ip < 0) ? 0.0 : u(k, jm, ip);
00288           const double u_at_v =
00289               0.25 * (u_jm_i + u_here_i + u_jm_ip + u_here_ip);
00290           const double w_at_v =
00291               0.25 * (w(k, jm, i) + w(k, j, i) + w(kp, jm, i) + w(kp, j, i));
00292           if (adv == 1) {
00293             dvdx = u_at_v > 0.0 ? (v_here - v_im) / hx : (v_ip - v_here) / hx;
00294             dvdy = v_here > 0.0 ? (v_here - v_jm) / hy : (v_jp - v_here) / hy;
00295             dvdz = w_at_v > 0.0 ? (v_here - v_km) / hz : (v_kp - v_here) / hz;
00296           } else if (adv == 2) {
00297             dvdx = minmod(v_here - v_im, v_ip - v_here) / hx;
00298             dvdy = minmod(v_here - v_jm, v_jp - v_here) / hy;
00299             dvdz = minmod(v_here - v_km, v_kp - v_here) / hz;
00300           } else if (adv == 3) {
00301             const double cx = (v_ip - v_im) / (2.0 * hx);
00302             const double cy = (v_jp - v_jm) / (2.0 * hy);
00303             const double cz = (v_kp - v_km) / (2.0 * hz);
00304             const double ux =
00305                 u_at_v > 0.0 ? (v_here - v_im) / hx : (v_ip - v_here) / hx;
00306             const double uy =
00307                 v_here > 0.0 ? (v_here - v_jm) / hy : (v_jp - v_here) / hy;
00308             const double uz =
00309                 w_at_v > 0.0 ? (v_here - v_km) / hz : (v_kp - v_here) / hz;
00310             dvdx = (1.0 - adv_blend) * cx + adv_blend * ux;
00311             dvdy = (1.0 - adv_blend) * cy + adv_blend * uy;
00312             dvdz = (1.0 - adv_blend) * cz + adv_blend * uz;
00313           } else if (adv == 4) {
00314             if (i >= 3 && i + 3 < n0) {
00315               dvdx = weno5Derivative(
00316                   v(k, j, i - 3), v(k, j, i - 2), v(k, j, i - 1), v_here,
00317                   v(k, j, i + 1), v(k, j, i + 2), v(k, j, i + 3), u_at_v, hx);
00318             } else {
00319               dvdx = minmod(v_here - v_im, v_ip - v_here) / hx;
00320             }
00321             if (j >= 3 && j + 3 < n1) {
00322               dvdy = weno5Derivative(
00323                   v(k, j - 3, i), v(k, j - 2, i), v(k, j - 1, i), v_here,
00324                   v(k, j + 1, i), v(k, j + 2, i), v(k, j + 3, i), v_here, hy);
00325             } else {
00326               dvdy = minmod(v_here - v_jm, v_jp - v_here) / hy;
00327             }
00328             if (!g.z_wall && n2 >= 5) {
00329               dvdz = weno5Derivative(
00330                   v((k + n2 - 3) % n2, j, i), v((k + n2 - 2) % n2, j, i),
00331                   v((k + n2 - 1) % n2, j, i), v_here, v((k + 1) % n2, j, i),
00332                   v((k + 2) % n2, j, i), v((k + 3) % n2, j, i), w_at_v, hz);
00333             }
00334           }
00335           dvdx /= g.dmapX((i + 0.5) * hx);
00336           dvdy /= g.dmapX(j * hy);
00337           const double dxm_v =
00338               (i == 0) ? g.xf(1) - g.xf(0) : g.xc(i) - g.xc(i - 1);
00339           const double dxp_v =
00340               (i + 1 == n0) ? g.xf(n0) - g.xf(n0 - 1) : g.xc(i + 1) - g.xc(i);
00341           const double dym_v = g.yf(j) - g.yf(j - 1);
00342           const double dyp_v =
00343               (j + 1 == n1) ? g.yf(n1) - g.yf(n1 - 1) : g.yf(j + 1) - g.yf(j);
00344           const double lap_v =
00345               2.0 / (dxm_v + dxp_v) *
00346                   ((v_ip - v_here) / dxp_v - (v_here - v_im) / dxm_v) +
00347               2.0 / (dym_v + dyp_v) *
00348                   ((v_jp - v_here) / dyp_v - (v_here - v_jm) / dym_v) +
00349               (v_kp - 2.0 * v_here + v_km) / hz2;
00350           dv(k, j, i) =
00351               -(u_at_v * dvdx + v_here * dvdy + w_at_v * dvdz) + nu * lap_v;
00352         }
00353
00354         // ----- w-momentum -----
00355         const double w_im = (im < 0) ? -w_here : w(k, j, im);
00356         const double w_ip = (ip < 0) ? -w_here : w(k, j, ip);
00357         const double w_jm = (jm < 0) ? -w_here : w(k, jm, i);
00358         const double w_jp = (jp < 0) ? -w_here : w(k, jp, i);
00359         const double w_km =
```

```
00360             (k == 0) ? 0.0 : ((g.z_wall) ? w(k - 1, j, i) : w(km, j, i));
00361         const double w_kp =
00362             (k + 1 == n2) ? 0.0 : ((g.z_wall) ? w(k + 1, j, i) : w(kp, j, i));
00363         double dwdx = (w_ip - w_im) / (2.0 * hx);
00364         double dwdy = (w_jp - w_jm) / (2.0 * hy);
00365         double dwdz = (w_kp - w_km) / (2.0 * hz);
00366         const double u_ip = (ip < 0) ? 0.0 : u(k, j, ip);
00367         const double u_km_ip = (ip < 0) ? 0.0 : u(km, j, ip);
00368         const double u_at_w =
00369             0.25 * (u(km, j, i) + u(k, j, i) + u_km_ip + u_ip);
00370         const double v_jp_local = (jp < 0) ? 0.0 : v(k, jp, i);
00371         const double v_km_jp = (jp < 0) ? 0.0 : v(km, jp, i);
00372         const double v_at_w =
00373             0.25 * (v(km, j, i) + v(k, j, i) + v_km_jp + v_jp_local);
00374         if (adv == 1) {
00375           dwdx = u_at_w > 0.0 ? (w_here - w_im) / hx : (w_ip - w_here) / hx;
00376           dwdy = v_at_w > 0.0 ? (w_here - w_jm) / hy : (w_jp - w_here) / hy;
00377           dwdz = w_here > 0.0 ? (w_here - w_km) / hz : (w_kp - w_here) / hz;
00378         } else if (adv == 2) {
00379           dwdx = minmod(w_here - w_im, w_ip - w_here) / hx;
00380           dwdy = minmod(w_here - w_jm, w_jp - w_here) / hy;
00381           dwdz = minmod(w_here - w_km, w_kp - w_here) / hz;
00382         } else if (adv == 3) {
00383           const double cx = (w_ip - w_im) / (2.0 * hx);
00384           const double cy = (w_jp - w_jm) / (2.0 * hy);
00385           const double cz = (w_kp - w_km) / (2.0 * hz);
00386           const double ux =
00387               u_at_w > 0.0 ? (w_here - w_im) / hx : (w_ip - w_here) / hx;
00388           const double uy =
00389               v_at_w > 0.0 ? (w_here - w_jm) / hy : (w_jp - w_here) / hy;
00390           const double uz =
00391               w_here > 0.0 ? (w_here - w_km) / hz : (w_kp - w_here) / hz;
00392           dwdx = (1.0 - adv_blend) * cx + adv_blend * ux;
00393           dwdy = (1.0 - adv_blend) * cy + adv_blend * uy;
00394           dwdz = (1.0 - adv_blend) * cz + adv_blend * uz;
00395         } else if (adv == 4) {
00396           if (i >= 3 && i + 3 < n0) {
00397             dwdx = weno5Derivative(w(k, j, i - 3), w(k, j, i - 2),
00398                                    w(k, j, i - 1), w_here, w(k, j, i + 1),
00399                                    w(k, j, i + 2), w(k, j, i + 3), u_at_w, hx);
00400           } else {
00401             dwdx = minmod(w_here - w_im, w_ip - w_here) / hx;
00402           }
00403           if (j >= 3 && j + 3 < n1) {
00404             dwdy = weno5Derivative(w(k, j - 3, i), w(k, j - 2, i),
00405                                    w(k, j - 1, i), w_here, w(k, j + 1, i),
00406                                    w(k, j + 2, i), w(k, j + 3, i), v_at_w, hy);
00407           } else {
00408             dwdy = minmod(w_here - w_jm, w_jp - w_here) / hy;
00409           }
00410           if (!g.z_wall && n2 >= 5) {
00411             dwdz = weno5Derivative(
00412                 w((k + n2 - 3) % n2, j, i), w((k + n2 - 2) % n2, j, i),
00413                 w((k + n2 - 1) % n2, j, i), w_here, w((k + 1) % n2, j, i),
00414                 w((k + 2) % n2, j, i), w((k + 3) % n2, j, i), w_here, hz);
00415           }
00416         }
00417         dwdx /= g.dmapX((i + 0.5) * hx);
00418         dwdy /= g.dmapX((j + 0.5) * hy);
00419         const double dxm_w =
00420             (i == 0) ? g.xf(1) - g.xf(0) : g.xc(i) - g.xc(i - 1);
00421         const double dxp_w =
00422             (i + 1 == n0) ? g.xf(n0) - g.xf(n0 - 1) : g.xc(i + 1) - g.xc(i);
00423         const double dym_w =
00424             (j == 0) ? g.yf(1) - g.yf(0) : g.yc(j) - g.yc(j - 1);
00425         const double dyp_w =
00426             (j + 1 == n1) ? g.yf(n1) - g.yf(n1 - 1) : g.yc(j + 1) - g.yc(j);
00427         const double lap_w =
00428             2.0 / (dxm_w + dxp_w) *
00429                 ((w_ip - w_here) / dxp_w - (w_here - w_im) / dxm_w) +
00430             2.0 / (dym_w + dyp_w) *
00431                 ((w_jp - w_here) / dyp_w - (w_here - w_jm) / dym_w) +
00432             (w_kp - 2.0 * w_here + w_km) / hz2;
00433         dw(k, j, i) =
00434             -(u_at_w * dwdx + v_at_w * dwdy + w_here * dwdz) + nu * lap_w;
00435         if (g.z_wall && k == 0) {
00436           dw(k, j, i) = 0.0;
00437         }
00438       });
00439 }
00440
00441 // Weak low-pass (dealiasing) filter: q <- q + alpha * laplacian(q).
00442 // Applied to the velocity components after a time step; it damps the
00443 // grid-scale content that a high-order reconstruction can alias.
00444 template <class ExecSpace>
00446 void filterVelocity(const projection::ScalarView<ExecSpace>& u,
00447                     const projection::ScalarView<ExecSpace>& v,
```

```
00448                     const projection::ScalarView<ExecSpace>& w,
00449                     projection::ScalarView<ExecSpace>& u_out,
00450                     projection::ScalarView<ExecSpace>& v_out,
00451                     projection::ScalarView<ExecSpace>& w_out, const Geometry& g,
00452                     double alpha) {
00453   const int n0 = g.n0;
00454   const int n1 = g.n1;
00455   const int n2 = g.n2;
00456   const auto policy =
00457       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00458   Kokkos::parallel_for(
00459       "cavity_filter", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00460         auto at = [&](const projection::ScalarView<ExecSpace>& q, int kk,
00461                       int jj, int ii) {
00462           const int ci = ii < 0 ? 0 : (ii >= n0 ? n0 - 1 : ii);
00463           const int cj = jj < 0 ? 0 : (jj >= n1 ? n1 - 1 : jj);
00464           int ck = ((kk % n2) + n2) % n2;
00465           if (g.z_wall) {
00466             ck = kk < 0 ? 0 : (kk >= n2 ? n2 - 1 : kk);
00467           }
00468           return q(ck, cj, ci);
00469         };
00470         const double uu = u(k, j, i);
00471         const double vv = v(k, j, i);
00472         const double ww = w(k, j, i);
00473         const double lu = at(u, k, j, i - 1) + at(u, k, j, i + 1) - 2.0 * uu +
00474                           at(u, k, j - 1, i) + at(u, k, j + 1, i) - 2.0 * uu +
00475                           at(u, k - 1, j, i) + at(u, k + 1, j, i) - 2.0 * uu;
00476         const double lv = at(v, k, j, i - 1) + at(v, k, j, i + 1) - 2.0 * vv +
00477                           at(v, k, j - 1, i) + at(v, k, j + 1, i) - 2.0 * vv +
00478                           at(v, k - 1, j, i) + at(v, k + 1, j, i) - 2.0 * vv;
00479         const double lw = at(w, k, j, i - 1) + at(w, k, j, i + 1) - 2.0 * ww +
00480                           at(w, k, j - 1, i) + at(w, k, j + 1, i) - 2.0 * ww +
00481                           at(w, k - 1, j, i) + at(w, k + 1, j, i) - 2.0 * ww;
00482         u_out(k, j, i) = (i == 0) ? 0.0 : uu + alpha * lu;
00483         v_out(k, j, i) = (j == 0) ? 0.0 : vv + alpha * lv;
00484         w_out(k, j, i) = (k == 0) ? 0.0 : ww + alpha * lw;
00485       });
00486 }
00487
00488 // Host-side variable-coefficient Poisson for the clustered 2D grid:
00489 //   B p = f,  B = -(variable-coefficient Laplacian)
00490 // with Neumann walls. The cell volumes are folded into f, so B is symmetric.
00497 class ClusteredPoisson2D {
00498  public:
00499   ClusteredPoisson2D(int n0, int n1, const Geometry& g)
00500       : n0_(n0), n1_(n1), bx_(n0 * n1, 0.0), by_(n0 * n1, 0.0) {
00501     for (int j = 0; j < n1; ++j) {
00502       for (int i = 0; i < n0; ++i) {
00503         const int index = j * n0 + i;
00504         if (i > 0) {
00505           bx_[index] = (g.yf(j + 1) - g.yf(j)) / (g.xc(i) - g.xc(i - 1));
00506         }
00507         if (j > 0) {
00508           by_[index] = (g.xf(i + 1) - g.xf(i)) / (g.yc(j) - g.yc(j - 1));
00509         }
00510       }
00511     }
00512   }
00513
00514   void apply(const std::vector<double>& p, std::vector<double>& out) const {
00515     for (int j = 0; j < n1_; ++j) {
00516       for (int i = 0; i < n0_; ++i) {
00517         const int index = j * n0_ + i;
00518         const double bl = (i > 0) ? bx_[index] : 0.0;
00519         const double br = (i + 1 < n0_) ? bx_[index + 1] : 0.0;
00520         const double bo = (j > 0) ? by_[index] : 0.0;
00521         const double bt = (j + 1 < n1_) ? by_[index + n0_] : 0.0;
00522         const double pl = (i > 0) ? p[index - 1] : p[index];
00523         const double pr = (i + 1 < n0_) ? p[index + 1] : p[index];
00524         const double pd = (j > 0) ? p[index - n0_] : p[index];
00525         const double pu = (j + 1 < n1_) ? p[index + n0_] : p[index];
00526         out[index] = br * (p[index] - pr) + bl * (p[index] - pl) +
00527                      bt * (p[index] - pu) + bo * (p[index] - pd);
00528       }
00529     }
00530   }
00531
00532   void solve(std::vector<double>& p, const std::vector<double>& f,
00533              double tolerance = 1.0e-8, int max_iterations = 500) const {
00534     const int total = n0_ * n1_;
00535     p.assign(total, 0.0);
00536     std::vector<double> rhs = f;
00537     double mean = 0.0;
00538     for (double value : rhs) {
00539       mean += value;
00540     }
```

```
00541     mean /= static_cast<double>(total);
00542     for (double& value : rhs) {
00543       value -= mean;
00544     }
00545     std::vector<double> r = rhs;
00546     std::vector<double> z(total, 0.0);
00547     std::vector<double> dir(total, 0.0);
00548     std::vector<double> ap(total, 0.0);
00549     std::vector<double> diag(total, 0.0);
00550     for (int j = 0; j < n1_; ++j) {
00551       for (int i = 0; i < n0_; ++i) {
00552         const int index = j * n0_ + i;
00553         const double bl = (i > 0) ? bx_[index] : 0.0;
00554         const double br = (i + 1 < n0_) ? bx_[index + 1] : 0.0;
00555         const double bo = (j > 0) ? by_[index] : 0.0;
00556         const double bt = (j + 1 < n1_) ? by_[index + n0_] : 0.0;
00557         diag[index] = bl + br + bo + bt;
00558       }
00559     }
00560     auto dot = [&](const std::vector<double>& a, const std::vector<double>& b) {
00561       double sum = 0.0;
00562       for (int q = 0; q < total; ++q) {
00563         sum += a[q] * b[q];
00564       }
00565       return sum;
00566     };
00567     auto norm = [&](const std::vector<double>& a) {
00568       return std::sqrt(dot(a, a) / static_cast<double>(total));
00569     };
00570     const double target = tolerance * std::max(1.0, norm(rhs));
00571     for (int q = 0; q < total; ++q) {
00572       z[q] = diag[q] != 0.0 ? r[q] / diag[q] : 0.0;
00573     }
00574     dir = z;
00575     double rz = dot(r, z);
00576     for (int it = 0; it < max_iterations; ++it) {
00577       if (norm(r) <= target) {
00578         break;
00579       }
00580       apply(dir, ap);
00581       const double denom = dot(dir, ap);
00582       if (std::abs(denom) < 1.0e-300) {
00583         break;
00584       }
00585       const double alpha = rz / denom;
00586       for (int q = 0; q < total; ++q) {
00587         p[q] += alpha * dir[q];
00588         r[q] -= alpha * ap[q];
00589       }
00590       for (int q = 0; q < total; ++q) {
00591         z[q] = diag[q] != 0.0 ? r[q] / diag[q] : 0.0;
00592       }
00593       const double rz_new = dot(r, z);
00594       const double beta = rz_new / rz;
00595       for (int q = 0; q < total; ++q) {
00596         dir[q] = z[q] + beta * dir[q];
00597       }
00598       rz = rz_new;
00599     }
00600   }
00601
00602  private:
00603   int n0_;
00604   int n1_;
00605   std::vector<double> bx_;
00606   std::vector<double> by_;
00607 };
00608
00614 template <class ExecSpace>
00615 void projectClustered(projection::ScalarView<ExecSpace>& u,
00616                       projection::ScalarView<ExecSpace>& v,
00617                       projection::ScalarView<ExecSpace>& w,
00618                       projection::ScalarView<ExecSpace>& phi,
00619                       projection::ScalarView<ExecSpace>& scratch,
00620                       const Geometry& g) {
00621   divergence<ExecSpace>(u, v, w, scratch, g);
00622   auto hdiv = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), scratch);
00623   auto hphi = Kokkos::create_mirror_view(phi);
00624   const int n0 = g.n0;
00625   const int n1 = g.n1;
00626   std::vector<double> f(static_cast<std::size_t>(n0) * n1, 0.0);
00627   for (int j = 0; j < n1; ++j) {
00628     for (int i = 0; i < n0; ++i) {
00629       const double dx = g.xf(i + 1) - g.xf(i);
00630       const double dy = g.yf(j + 1) - g.yf(j);
00631       f[static_cast<std::size_t>(j) * n0 + i] = hdiv(0, j, i) * dx * dy;
00632     }
```

```
00633   }
00634   ClusteredPoisson2D solver(n0, n1, g);
00635   std::vector<double> p;
00636   solver.solve(p, f, 1.0e-12, 2000);
00637   for (int j = 0; j < n1; ++j) {
00638     for (int i = 0; i < n0; ++i) {
00639       hphi(0, j, i) = p[static_cast<std::size_t>(j) * n0 + i];
00640     }
00641   }
00642   Kokkos::deep_copy(phi, hphi);
00643
00644   const auto policy =
00645       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{g.n2, n1, n0}});
00646   Kokkos::parallel_for(
00647       "cavity_clustered_project", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00648         if (i > 0) {
00649           u(k, j, i) +=
00650               (phi(k, j, i) - phi(k, j, i - 1)) / (g.xc(i) - g.xc(i - 1));
00651         } else {
00652           u(k, j, i) = 0.0;
00653         }
00654         if (j > 0) {
00655           v(k, j, i) +=
00656               (phi(k, j, i) - phi(k, j - 1, i)) / (g.yc(j) - g.yc(j - 1));
00657         } else {
00658           v(k, j, i) = 0.0;
00659         }
00660         if (g.n2 == 1) {
00661           w(k, j, i) = 0.0;
00662         } else {
00663           const int km = (k == 0) ? (g.z_wall ? -1 : g.n2 - 1) : k - 1;
00664           if (k > 0) {
00665             w(k, j, i) += (phi(k, j, i) - phi(km, j, i)) / g.hz;
00666           } else if (g.z_wall) {
00667             w(k, j, i) = 0.0;
00668           }
00669         }
00670       });
00671 }
00672
00677 template <class ExecSpace, class PoissonSolver>
00678 poisson::Result project(projection::ScalarView<ExecSpace>& u,
00679                         projection::ScalarView<ExecSpace>& v,
00680                         projection::ScalarView<ExecSpace>& w,
00681                         projection::ScalarView<ExecSpace>& phi,
00682                         projection::ScalarView<ExecSpace>& scratch,
00683                         const Geometry& g, PoissonSolver& solver,
00684                         const poisson::Options& options) {
00685   divergence<ExecSpace>(u, v, w, scratch, g);
00686   const int n0 = g.n0;
00687   const int n1 = g.n1;
00688   const int n2 = g.n2;
00689   const auto policy =
00690       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00691   const double dt = 1.0;
00692   Kokkos::parallel_for(
00693       "cavity_projection_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00694         scratch(k, j, i) = -scratch(k, j, i) / dt;
00695       });
00696   const poisson::Result result = solver.solve(phi, scratch, options);
00697
00698   Kokkos::parallel_for(
00699       "cavity_projection_update", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00700         const int im = (i == 0) ? n0 - 1 : i - 1;
00701         const int jm = (j == 0) ? n1 - 1 : j - 1;
00702         const int km = (k == 0) ? (g.z_wall ? -1 : n2 - 1) : k - 1;
00703         if (i > 0) {
00704           u(k, j, i) += dt * (phi(k, j, i) - phi(k, j, im)) / g.hx;
00705         } else {
00706           u(k, j, i) = 0.0;
00707         }
00708         if (j > 0) {
00709           v(k, j, i) += dt * (phi(k, j, i) - phi(k, jm, i)) / g.hy;
00710         } else {
00711           v(k, j, i) = 0.0;
00712         }
00713         if (k > 0) {
00714           w(k, j, i) += dt * (phi(k, j, i) - phi(km, j, i)) / g.hz;
00715         } else if (g.z_wall) {
00716           w(k, j, i) = 0.0;
00717         } else {
00718           w(k, j, i) += dt * (phi(k, j, i) - phi(km, j, i)) / g.hz;
00719         }
00720       });
00721   return result;
00722 }
00723
```

```
00725 template <class ExecSpace, class Project>
00726 void advanceSsprk3(projection::ScalarView<ExecSpace>& u,
00727                    projection::ScalarView<ExecSpace>& v,
00728                    projection::ScalarView<ExecSpace>& w,
00729                    projection::ScalarView<ExecSpace>& u1,
00730                    projection::ScalarView<ExecSpace>& v1,
00731                    projection::ScalarView<ExecSpace>& w1,
00732                    projection::ScalarView<ExecSpace>& u2,
00733                    projection::ScalarView<ExecSpace>& v2,
00734                    projection::ScalarView<ExecSpace>& w2,
00735                    projection::ScalarView<ExecSpace>& ku,
00736                    projection::ScalarView<ExecSpace>& kv,
00737                    projection::ScalarView<ExecSpace>& kw, const Geometry& g,
00738                    double nu, double dt, Project& project) {
00739   momentumRhs<ExecSpace>(u, v, w, ku, kv, kw, g, nu);
00740   incompressible::combine<ExecSpace>(u, 1.0, ku, dt, ku, 0.0, u1);
00741   incompressible::combine<ExecSpace>(v, 1.0, kv, dt, ku, 0.0, v1);
00742   incompressible::combine<ExecSpace>(w, 1.0, kw, dt, ku, 0.0, w1);
00743   project(u1, v1, w1);
00744
00745   momentumRhs<ExecSpace>(u1, v1, w1, ku, kv, kw, g, nu);
00746   incompressible::combine<ExecSpace>(u, 0.75, u1, 0.25, ku, 0.25 * dt, u2);
00747   incompressible::combine<ExecSpace>(v, 0.75, v1, 0.25, kv, 0.25 * dt, v2);
00748   incompressible::combine<ExecSpace>(w, 0.75, w1, 0.25, kw, 0.25 * dt, w2);
00749   project(u2, v2, w2);
00750
00751   momentumRhs<ExecSpace>(u2, v2, w2, ku, kv, kw, g, nu);
00752   incompressible::combine<ExecSpace>(u, 1.0 / 3.0, u2, 2.0 / 3.0, ku,
00753                                      (2.0 / 3.0) * dt, u);
00754   incompressible::combine<ExecSpace>(v, 1.0 / 3.0, v2, 2.0 / 3.0, kv,
00755                                      (2.0 / 3.0) * dt, v);
00756   incompressible::combine<ExecSpace>(w, 1.0 / 3.0, w2, 2.0 / 3.0, kw,
00757                                      (2.0 / 3.0) * dt, w);
00758   project(u, v, w);
00759 }
00760
00761 }  // namespace cavity
00762 }  // namespace saturn
```

6.52 src/numerics/incompressible/channel.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include "core/parallel.hpp"
#include "numerics/incompressible/incompressible.hpp"
#include "numerics/incompressible/poisson.hpp"
#include "numerics/incompressible/projection.hpp"
```

Classes

- struct saturn::channel::Geometry

Namespaces

- namespace saturn
- namespace saturn::channel

Functions

- template<class ExecSpace >
  void saturn::channel::divergence (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &div, const Geometry &g)
- template<class ExecSpace >
  void saturn::channel::momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const Geometry &g, double nu)
- template<class ExecSpace , class PoissonSolver >
  poisson::Result saturn::channel::project (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &phi, projection::ScalarView< ExecSpace > &scratch, const Geometry &g, PoissonSolver &solver, const poisson::Options &options)
- template<class ExecSpace , class Project >
  void saturn::channel::advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const Geometry &g, double nu, double dt, Project &project)
- template<class ExecSpace >
  void saturn::channel::filterDimension (const projection::ScalarView< ExecSpace > &in, projection::ScalarView< ExecSpace > &out, double alpha, int dimension, int component, const Geometry &g)
- template<class ExecSpace >
  void saturn::channel::filter4Channel (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &tu, projection::ScalarView< ExecSpace > &tv, projection::ScalarView< ExecSpace > &tw, const Geometry &g, double alpha)

6.53 channel.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <array>
00006 #include <cmath>
00007
00008 #include "core/parallel.hpp"
00009 #include "numerics/incompressible/incompressible.hpp"
00010 #include "numerics/incompressible/poisson.hpp"
00011 #include "numerics/incompressible/projection.hpp"
00012
00013 namespace saturn {
00014 namespace channel {
00015
00016 // Wall-bounded channel on a staggered MAC grid.
00017 //
00018 //   x, z: periodic
00019 //   y:    no-slip walls at y=0 and y=Ly
00020 //
00021 // Velocity layout (same extents as the cell-centered pressure):
00022 //
00023 //   u(i,j,k): x-face, y and z at cell centers
00024 //   v(i,j,k): y-face; v(j=0) is the bottom wall, v(j=n1) (implicit) the top
00025 //   w(i,j,k): z-face, x and y at cell centers
00026 //
00027 // The top wall can move in x with speed `u_top` (Couette flow); the bottom
00028 // wall is stationary. A constant body force `gx` drives plane Poiseuille flow.
00029 // Pressure uses homogeneous Neumann conditions at both walls.
00030
00031 struct Geometry {
```

```
00032   int n0 = 1;
00033   int n1 = 1;
00034   int n2 = 1;
00035   double hx = 1.0;
00036   double hy = 1.0;
00037   double hz = 1.0;
00038   double ly = 1.0;
00039   double u_top = 0.0;
00040   double gx = 0.0;
00041   double nu = 1.0;
00042 };
00043
00044 template <class ExecSpace>
00045 void divergence(const projection::ScalarView<ExecSpace>& u,
00046                 const projection::ScalarView<ExecSpace>& v,
00047                 const projection::ScalarView<ExecSpace>& w,
00048                 projection::ScalarView<ExecSpace>& div, const Geometry& g) {
00049   const int n0 = g.n0;
00050   const int n1 = g.n1;
00051   const int n2 = g.n2;
00052   const auto policy =
00053       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00054   Kokkos::parallel_for(
00055       "channel_divergence", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00056         const int ip = (i + 1 == n0) ? 0 : i + 1;
00057         const int kp = (k + 1 == n2) ? 0 : k + 1;
00058         const double vp = (j + 1 == n1) ? 0.0 : v(k, j + 1, i);
00059         div(k, j, i) = (u(k, j, ip) - u(k, j, i)) / g.hx +
00060                        (vp - v(k, j, i)) / g.hy +
00061                        (w(kp, j, i) - w(k, j, i)) / g.hz;
00062       });
00063 }
00064
00065 template <class ExecSpace>
00066 void momentumRhs(const projection::ScalarView<ExecSpace>& u,
00067                  const projection::ScalarView<ExecSpace>& v,
00068                  const projection::ScalarView<ExecSpace>& w,
00069                  projection::ScalarView<ExecSpace>& du,
00070                  projection::ScalarView<ExecSpace>& dv,
00071                  projection::ScalarView<ExecSpace>& dw, const Geometry& g,
00072                  double nu) {
00073   const int n0 = g.n0;
00074   const int n1 = g.n1;
00075   const int n2 = g.n2;
00076   const double hx = g.hx;
00077   const double hy = g.hy;
00078   const double hz = g.hz;
00079   const double hx2 = hx * hx;
00080   const double hy2 = hy * hy;
00081   const double hz2 = hz * hz;
00082   const double u_top = g.u_top;
00083   const auto policy =
00084       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00085   Kokkos::parallel_for(
00086       "channel_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00087         const int im = (i == 0) ? n0 - 1 : i - 1;
00088         const int ip = (i + 1 == n0) ? 0 : i + 1;
00089         const int jm = (j == 0) ? -1 : j - 1;
00090         const int jp = (j + 1 == n1) ? -1 : j + 1;
00091         const int km = (k == 0) ? n2 - 1 : k - 1;
00092         const int kp = (k + 1 == n2) ? 0 : k + 1;
00093
00094         const double u_here = u(k, j, i);
00095         const double u_jm = (j == 0) ? -u_here : u(k, jm, i);
00096         const double u_jp = (j + 1 == n1) ? 2.0 * u_top - u_here : u(k, jp, i);
00097         const double w_here = w(k, j, i);
00098         const double w_jm = (j == 0) ? -w_here : w(k, jm, i);
00099         const double w_jp = (j + 1 == n1) ? -w_here : w(k, jp, i);
00100         const double v_jm = (j == 0) ? 0.0 : v(k, j - 1, i);
00101         const double v_jp = (j + 1 == n1) ? 0.0 : v(k, j + 1, i);
00102         const double v_jp_im = (j + 1 == n1) ? 0.0 : v(k, j + 1, im);
00103         const double v_jp_km = (j + 1 == n1) ? 0.0 : v(km, j + 1, i);
00104
00105         // u-momentum.
00106         const double dudx = (u(k, j, ip) - u(k, j, im)) / (2.0 * hx);
00107         const double dudy = (u_jp - u_jm) / (2.0 * hy);
00108         const double dudz = (u(kp, j, i) - u(km, j, i)) / (2.0 * hz);
00109         const double v_at_u =
00110             0.25 * (v(k, j, im) + v(k, j, i) + v_jp_im + v_jp);
00111         const double w_at_u =
00112             0.25 * (w(k, j, im) + w(k, j, i) + w(kp, j, im) + w(kp, j, i));
00113         const double lap_u = (u(k, j, ip) - 2.0 * u_here + u(k, j, im)) / hx2 +
00114                              (u_jp - 2.0 * u_here + u_jm) / hy2 +
00115                              (u(kp, j, i) - 2.0 * u_here + u(km, j, i)) / hz2;
00116         du(k, j, i) = -(u_here * dudx + v_at_u * dudy + w_at_u * dudz) +
00117                       nu * lap_u + g.gx;
00118
```

```
00119         // v-momentum at interior y-faces; the wall faces stay zero.
00120         if (j == 0) {
00121           dv(k, j, i) = 0.0;
00122         } else {
00123           const double dvdx = (v(k, j, ip) - v(k, j, im)) / (2.0 * hx);
00124           const double dvdy = (v_jp - v_jm) / (2.0 * hy);
00125           const double dvdz = (v(kp, j, i) - v(km, j, i)) / (2.0 * hz);
00126           const double u_at_v =
00127               0.25 * (u(k, jm, i) + u_here + u(k, jm, ip) + u(k, j, ip));
00128           const double w_at_v =
00129               0.25 * (w(k, jm, i) + w_here + w(kp, jm, i) + w(kp, j, i));
00130           const double lap_v =
00131               (v(k, j, ip) - 2.0 * v(k, j, i) + v(k, j, im)) / hx2 +
00132               (v_jp - 2.0 * v(k, j, i) + v_jm) / hy2 +
00133               (v(kp, j, i) - 2.0 * v(k, j, i) + v(km, j, i)) / hz2;
00134           dv(k, j, i) =
00135               -(u_at_v * dvdx + v(k, j, i) * dvdy + w_at_v * dvdz) + nu * lap_v;
00136         }
00137
00138         // w-momentum.
00139         const double dwdx = (w(k, j, ip) - w(k, j, im)) / (2.0 * hx);
00140         const double dwdy = (w_jp - w_jm) / (2.0 * hy);
00141         const double dwdz = (w(kp, j, i) - w(km, j, i)) / (2.0 * hz);
00142         const double u_at_w =
00143             0.25 * (u(km, j, i) + u_here + u(km, j, ip) + u(k, j, ip));
00144         const double v_at_w =
00145             0.25 * (v(km, j, i) + v(k, j, i) + v_jp_km + v_jp);
00146         const double lap_w = (w(k, j, ip) - 2.0 * w_here + w(k, j, im)) / hx2 +
00147                              (w_jp - 2.0 * w_here + w_jm) / hy2 +
00148                              (w(kp, j, i) - 2.0 * w_here + w(km, j, i)) / hz2;
00149         dw(k, j, i) =
00150             -(u_at_w * dwdx + v_at_w * dwdy + w_here * dwdz) + nu * lap_w;
00151       });
00152 }
00153
00154 // Pressure projection with Neumann walls in y. `phi` and `scratch` are
00155 // workspace; `solver` must use periodic x/z and Neumann y.
00156 template <class ExecSpace, class PoissonSolver>
00157 poisson::Result project(projection::ScalarView<ExecSpace>& u,
00158                         projection::ScalarView<ExecSpace>& v,
00159                         projection::ScalarView<ExecSpace>& w,
00160                         projection::ScalarView<ExecSpace>& phi,
00161                         projection::ScalarView<ExecSpace>& scratch,
00162                         const Geometry& g, PoissonSolver& solver,
00163                         const poisson::Options& options) {
00164   divergence<ExecSpace>(u, v, w, scratch, g);
00165   const int n0 = g.n0;
00166   const int n1 = g.n1;
00167   const int n2 = g.n2;
00168   const auto policy =
00169       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00170   const double dt = 1.0;
00171   Kokkos::parallel_for(
00172       "channel_projection_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00173         scratch(k, j, i) = -scratch(k, j, i) / dt;
00174       });
00175   const poisson::Result result = solver.solve(phi, scratch, options);
00176
00177   Kokkos::parallel_for(
00178       "channel_projection_update", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00179         const int im = (i == 0) ? n0 - 1 : i - 1;
00180         const int km = (k == 0) ? n2 - 1 : k - 1;
00181         u(k, j, i) += dt * (phi(k, j, i) - phi(k, j, im)) / g.hx;
00182         if (j > 0) {
00183           v(k, j, i) += dt * (phi(k, j, i) - phi(k, j - 1, i)) / g.hy;
00184         } else {
00185           v(k, j, i) = 0.0;
00186         }
00187         w(k, j, i) += dt * (phi(k, j, i) - phi(km, j, i)) / g.hz;
00188       });
00189   return result;
00190 }
00191
00192 // SSP-RK3 with a projection after every stage. The callback takes the three
00193 // velocity views and must make them divergence-free.
00194 template <class ExecSpace, class Project>
00195 void advanceSsprk3(projection::ScalarView<ExecSpace>& u,
00196                    projection::ScalarView<ExecSpace>& v,
00197                    projection::ScalarView<ExecSpace>& w,
00198                    projection::ScalarView<ExecSpace>& u1,
00199                    projection::ScalarView<ExecSpace>& v1,
00200                    projection::ScalarView<ExecSpace>& w1,
00201                    projection::ScalarView<ExecSpace>& u2,
00202                    projection::ScalarView<ExecSpace>& v2,
00203                    projection::ScalarView<ExecSpace>& w2,
00204                    projection::ScalarView<ExecSpace>& ku,
00205                    projection::ScalarView<ExecSpace>& kv,
```

```
00206                   projection::ScalarView<ExecSpace>& kw, const Geometry& g,
00207                   double nu, double dt, Project& project) {
00208   momentumRhs<ExecSpace>(u, v, w, ku, kv, kw, g, nu);
00209   incompressible::combine<ExecSpace>(u, 1.0, ku, dt, ku, 0.0, u1);
00210   incompressible::combine<ExecSpace>(v, 1.0, kv, dt, ku, 0.0, v1);
00211   incompressible::combine<ExecSpace>(w, 1.0, kw, dt, ku, 0.0, w1);
00212   project(u1, v1, w1);
00213
00214   momentumRhs<ExecSpace>(u1, v1, w1, ku, kv, kw, g, nu);
00215   incompressible::combine<ExecSpace>(u, 0.75, u1, 0.25, ku, 0.25 * dt, u2);
00216   incompressible::combine<ExecSpace>(v, 0.75, v1, 0.25, kv, 0.25 * dt, v2);
00217   incompressible::combine<ExecSpace>(w, 0.75, w1, 0.25, kw, 0.25 * dt, w2);
00218   project(u2, v2, w2);
00219
00220   momentumRhs<ExecSpace>(u2, v2, w2, ku, kv, kw, g, nu);
00221   incompressible::combine<ExecSpace>(u, 1.0 / 3.0, u2, 2.0 / 3.0, ku,
00222                                      (2.0 / 3.0) * dt, u);
00223   incompressible::combine<ExecSpace>(v, 1.0 / 3.0, v2, 2.0 / 3.0, kv,
00224                                      (2.0 / 3.0) * dt, v);
00225   incompressible::combine<ExecSpace>(w, 1.0 / 3.0, w2, 2.0 / 3.0, kw,
00226                                      (2.0 / 3.0) * dt, w);
00227   project(u, v, w);
00228 }
00229
00230 // Wall-aware fourth-order filter for the channel (periodic x/z, walls in y).
00231 // `component`: 0 = u, 1 = v, 2 = w. Temps must be distinct from the inputs.
00232 template <class ExecSpace>
00233 void filterDimension(const projection::ScalarView<ExecSpace>& in,
00234                      projection::ScalarView<ExecSpace>& out, double alpha,
00235                      int dimension, int component, const Geometry& g) {
00236   const int n0 = g.n0;
00237   const int n1 = g.n1;
00238   const int n2 = g.n2;
00239   const double u_top = (component == 0) ? g.u_top : 0.0;
00240   const auto policy =
00241       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00242   Kokkos::parallel_for(
00243       "channel_filter", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00244         auto at = [&](int jj) {
00245           if (jj >= 0 && jj < n1) {
00246             return in(k, jj, i);
00247           }
00248           if (jj < 0) {
00249             if (component == 1) {
00250               return -in(k, -jj, i);
00251             }
00252             return -in(k, -jj - 1, i);
00253           }
00254           if (component == 1) {
00255             if (jj == n1) {
00256               return 0.0;
00257             }
00258             return -in(k, 2 * n1 - jj, i);
00259           }
00260           const int mirror = 2 * n1 - jj - 1;
00261           if (component == 0) {
00262             return 2.0 * u_top - in(k, mirror, i);
00263           }
00264           return -in(k, mirror, i);
00265         };
00266         double value = 0.0;
00267         if (dimension == 0) {
00268           const int im2 = (i - 2 + n0) % n0;
00269           const int im1 = (i - 1 + n0) % n0;
00270           const int ip1 = (i + 1) % n0;
00271           const int ip2 = (i + 2) % n0;
00272           value = in(k, j, im2) - 4.0 * in(k, j, im1) + 6.0 * in(k, j, i) -
00273                   4.0 * in(k, j, ip1) + in(k, j, ip2);
00274         } else if (dimension == 1) {
00275           value = at(j - 2) - 4.0 * at(j - 1) + 6.0 * in(k, j, i) -
00276                   4.0 * at(j + 1) + at(j + 2);
00277         } else {
00278           const int km2 = (k - 2 + n2) % n2;
00279           const int km1 = (k - 1 + n2) % n2;
00280           const int kp1 = (k + 1) % n2;
00281           const int kp2 = (k + 2) % n2;
00282           value = in(km2, j, i) - 4.0 * in(km1, j, i) + 6.0 * in(k, j, i) -
00283                   4.0 * in(kp1, j, i) + in(kp2, j, i);
00284         }
00285         out(k, j, i) = in(k, j, i) - (alpha / 16.0) * value;
00286       });
00287 }
00288
00289 template <class ExecSpace>
00290 void filter4Channel(projection::ScalarView<ExecSpace>& u,
00291                     projection::ScalarView<ExecSpace>& v,
00292                     projection::ScalarView<ExecSpace>& w,
```

```
00293                     projection::ScalarView<ExecSpace>& tu,
00294                     projection::ScalarView<ExecSpace>& tv,
00295                     projection::ScalarView<ExecSpace>& tw, const Geometry& g,
00296                     double alpha) {
00297   if (alpha <= 0.0) {
00298     return;
00299   }
00300   filterDimension<ExecSpace>(u, tu, alpha, 0, 0, g);
00301   filterDimension<ExecSpace>(tu, u, alpha, 1, 0, g);
00302   filterDimension<ExecSpace>(u, tu, alpha, 2, 0, g);
00303   Kokkos::deep_copy(u, tu);
00304
00305   filterDimension<ExecSpace>(v, tv, alpha, 0, 1, g);
00306   filterDimension<ExecSpace>(tv, v, alpha, 1, 1, g);
00307   filterDimension<ExecSpace>(v, tv, alpha, 2, 1, g);
00308   Kokkos::deep_copy(v, tv);
00309
00310   filterDimension<ExecSpace>(w, tw, alpha, 0, 2, g);
00311   filterDimension<ExecSpace>(tw, w, alpha, 1, 2, g);
00312   filterDimension<ExecSpace>(w, tw, alpha, 2, 2, g);
00313   Kokkos::deep_copy(w, tw);
00314
00315   const int n0 = g.n0;
00316   const int n1 = g.n1;
00317   const int n2 = g.n2;
00318   const auto policy =
00319       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00320   Kokkos::parallel_for(
00321       "channel_filter_walls", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00322         (void)k;
00323         if (i == 0) {
00324           u(k, j, i) = 0.0;
00325         }
00326         if (j == 0) {
00327           v(k, j, i) = 0.0;
00328         }
00329       });
00330 }
00331
00332 }  // namespace channel
00333 }  // namespace saturn
```

6.54 src/numerics/incompressible/channel_poisson.hpp File Reference

6.55 channel_poisson.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #ifdef SATURN_HAVE_FFTW
00004
00005 #include <fftw3.h>
00006 #include <Kokkos_Core.hpp>
00007
00008 #include <array>
00009 #include <cmath>
00010 #include <complex>
00011 #include <vector>
00012
00013 #include "numerics/incompressible/poisson.hpp"
00014
00015 namespace saturn {
00016 namespace poisson {
00017
00018 // Fast Poisson solver for the channel configuration:
00019 //
00020 //   x, z: periodic        y: homogeneous Neumann walls
00021 //
00022 // The x/z directions are diagonalized with 2D FFTs; for every Fourier mode a
00023 // tridiagonal system in y is solved with the Thomas algorithm. This is much
00024 // faster than multigrid for the wall-bounded channel and makes long turbulent
00025 // runs practical. It is host-side (FFTW plus a serial tridiagonal solve).
00026
00027 template <class ExecSpace>
00028 class ChannelFftPoisson {
00029  public:
00030   using memory_space = typename ExecSpace::memory_space;
00031   using view_type = Kokkos::View<double***, Kokkos::LayoutRight, memory_space>;
```

```
00032
00033   ChannelFftPoisson(const std::array<int, 3>& cells,
00034                     const std::array<double, 3>& spacing)
00035       : n0_(cells[0]), n1_(cells[1]), n2_(cells[2]), h_(spacing) {
00036     plane_ = n0_ * n2_;
00037     in_ = fftw_alloc_complex(static_cast<std::size_t>(plane_));
00038     out_ = fftw_alloc_complex(static_cast<std::size_t>(plane_));
00039     forward_ =
00040         fftw_plan_dft_2d(n2_, n0_, in_, out_, FFTW_FORWARD, FFTW_ESTIMATE);
00041     inverse_ =
00042         fftw_plan_dft_2d(n2_, n0_, in_, out_, FFTW_BACKWARD, FFTW_ESTIMATE);
00043     spectrum_.resize(static_cast<std::size_t>(n1_) * plane_);
00044   }
00045
00046   ~ChannelFftPoisson() {
00047     if (forward_ != nullptr) {
00048       fftw_destroy_plan(forward_);
00049     }
00050     if (inverse_ != nullptr) {
00051       fftw_destroy_plan(inverse_);
00052     }
00053     if (in_ != nullptr) {
00054       fftw_free(in_);
00055     }
00056     if (out_ != nullptr) {
00057       fftw_free(out_);
00058     }
00059   }
00060
00061   ChannelFftPoisson(const ChannelFftPoisson&) = delete;
00062   ChannelFftPoisson& operator=(const ChannelFftPoisson&) = delete;
00063
00064   Result solve(view_type x, view_type b, const Options& options) {
00065     (void)options;
00066     auto hb = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), b);
00067
00068     // Forward transform each y plane.
00069     for (int j = 0; j < n1_; ++j) {
00070       for (int k = 0; k < n2_; ++k) {
00071         for (int i = 0; i < n0_; ++i) {
00072           const int index = k * n0_ + i;
00073           in_[index][0] = hb(k, j, i);
00074           in_[index][1] = 0.0;
00075         }
00076       }
00077       fftw_execute_dft(forward_, in_, out_);
00078       for (int m = 0; m < plane_; ++m) {
00079         spectrum_[static_cast<std::size_t>(j) * plane_ + m] =
00080             std::complex<double>(out_[m][0], out_[m][1]);
00081       }
00082     }
00083
00084     const double hy2 = h_[1] * h_[1];
00085     std::vector<double> lower(n1_), diag(n1_), upper(n1_), cp(n1_);
00086     std::vector<std::complex<double» rhs(n1_), sol(n1_);
00087
00088     for (int kz = 0; kz < n2_; ++kz) {
00089       const double kzz = 2.0 * std::sin(kPi * kz / n2_) / h_[2];
00090       for (int kx = 0; kx < n0_; ++kx) {
00091         const double kxx = 2.0 * std::sin(kPi * kx / n0_) / h_[0];
00092         const double lam = -(kxx * kxx + kzz * kzz);
00093         const int mode = kz * n0_ + kx;
00094         if (kx == 0 && kz == 0) {
00095           for (int j = 0; j < n1_; ++j) {
00096             spectrum_[static_cast<std::size_t>(j) * plane_ + mode] = 0.0;
00097           }
00098           continue;
00099         }
00100         for (int j = 0; j < n1_; ++j) {
00101           lower[j] = (j == 0) ? 0.0 : 1.0 / hy2;
00102           upper[j] = (j == n1_ - 1) ? 0.0 : 1.0 / hy2;
00103           diag[j] = -2.0 / hy2 + lam;
00104           if (j == 0 || j == n1_ - 1) {
00105             diag[j] += 1.0 / hy2;
00106           }
00107           rhs[j] = spectrum_[static_cast<std::size_t>(j) * plane_ + mode];
00108         }
00109         cp[0] = upper[0] / diag[0];
00110         for (int j = 1; j < n1_; ++j) {
00111           const double denom = diag[j] - lower[j] * cp[j - 1];
00112           cp[j] = upper[j] / denom;
00113         }
00114         sol[0] = rhs[0] / diag[0];
00115         for (int j = 1; j < n1_; ++j) {
00116           const double denom = diag[j] - lower[j] * cp[j - 1];
00117           sol[j] = (rhs[j] - lower[j] * sol[j - 1]) / denom;
00118         }
```

```
00119         for (int j = n1_ - 2; j >= 0; --j) {
00120           sol[j] -= cp[j] * sol[j + 1];
00121         }
00122         for (int j = 0; j < n1_; ++j) {
00123           spectrum_[static_cast<std::size_t>(j) * plane_ + mode] = sol[j];
00124         }
00125       }
00126     }
00127
00128     // Inverse transform each y plane.
00129     auto hx = Kokkos::create_mirror_view(x);
00130     const double scale = 1.0 / static_cast<double>(plane_);
00131     for (int j = 0; j < n1_; ++j) {
00132       for (int m = 0; m < plane_; ++m) {
00133         const auto value = spectrum_[static_cast<std::size_t>(j) * plane_ + m];
00134         in_[m][0] = value.real();
00135         in_[m][1] = value.imag();
00136       }
00137       fftw_execute_dft(inverse_, in_, out_);
00138       for (int k = 0; k < n2_; ++k) {
00139         for (int i = 0; i < n0_; ++i) {
00140           const int m = k * n0_ + i;
00141           hx(k, j, i) = out_[m][0] * scale;
00142         }
00143       }
00144     }
00145     Kokkos::deep_copy(x, hx);
00146
00147     Result result;
00148     result.cycles = 1;
00149     result.converged = true;
00150     return result;
00151   }
00152
00153   int levels() const { return 1; }
00154
00155  private:
00156   static constexpr double kPi = 3.14159265358979323846;
00157   int n0_ = 1;
00158   int n1_ = 1;
00159   int n2_ = 1;
00160   int plane_ = 1;
00161   std::array<double, 3> h_{{1.0, 1.0, 1.0}};
00162   fftw_complex* in_ = nullptr;
00163   fftw_complex* out_ = nullptr;
00164   fftw_plan forward_ = nullptr;
00165   fftw_plan inverse_ = nullptr;
00166   std::vector<std::complex<double>> spectrum_;
00167 };
00168
00169 }  // namespace poisson
00170 }  // namespace saturn
00171
00172 #endif  // SATURN_HAVE_FFTW
```

6.56 src/numerics/incompressible/edac.hpp File Reference

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include "core/parallel.hpp"
#include "numerics/incompressible/incompressible.hpp"
#include "numerics/incompressible/projection.hpp"
```

Classes

- struct saturn::edac::Parameters

Namespaces

- namespace saturn
- namespace saturn::edac

Functions

- template<class ExecSpace >
  void saturn::edac::laplacian (const projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void saturn::edac::subtract (projection::ScalarView< ExecSpace > &a, const projection::ScalarView< ExecSpace > &b)
- template<class ExecSpace >
  void saturn::edac::velocityRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, const projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, projection::ScalarView< ExecSpace > &gx, projection::ScalarView< ExecSpace > &gy, projection::ScalarView< ExecSpace > &gz, const std::array< double, 3 > &h, double nu)
- template<class ExecSpace >
  void saturn::edac::pressureRhs (const projection::ScalarView< ExecSpace > &p, const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &dp, projection::ScalarView< ExecSpace > &div, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h, const Parameters &params)
- template<class ExecSpace >
  void saturn::edac::advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &p, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &p1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &p2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, projection::ScalarView< ExecSpace > &kp, projection::ScalarView< ExecSpace > &gx, projection::ScalarView< ExecSpace > &gy, projection::ScalarView< ExecSpace > &gz, projection::ScalarView< ExecSpace > &div, projection::ScalarView< ExecSpace > &lap, const std::array< double, 3 > &h, const Parameters &params, double dt)

6.57 edac.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <Kokkos_Core.hpp>
00004
00005 #include <array>
00006 #include <cmath>
00007
00008 #include "core/parallel.hpp"
00009 #include "numerics/incompressible/incompressible.hpp"
00010 #include "numerics/incompressible/projection.hpp"
00011
00012 namespace saturn {
00013 namespace edac {
00014
00015 // Entropically Damped Artificial Compressibility (Clausen 2013) on a
00016 // staggered MAC grid:
00017 //
00018 //   du/dt = -(u.grad)u + nu laplacian(u) - grad(p)
00019 //   dp/dt = -c^2 div(u) + nu_p laplacian(p),   nu_p = c^2 tau,  c = 1/M_c
00020 //
00021 // No pressure Poisson solve is performed. `tau` is set to tau_factor*h/c, so
00022 // nu_p = c*tau_factor*h. All fields are periodic in this prototype.
00023
00024 struct Parameters {
00025   double nu = 0.01;
00026   double c = 10.0;
00027   double nu_p = 0.01;
00028 };
00029
```

```
00030 template <class ExecSpace>
00031 void laplacian(const projection::ScalarView<ExecSpace>& p,
00032                 projection::ScalarView<ExecSpace>& lap,
00033                 const std::array<double, 3>& h) {
00034   const int n0 = static_cast<int>(p.extent(2));
00035   const int n1 = static_cast<int>(p.extent(1));
00036   const int n2 = static_cast<int>(p.extent(0));
00037   const double hx2 = h[0] * h[0];
00038   const double hy2 = h[1] * h[1];
00039   const double hz2 = h[2] * h[2];
00040   const auto policy =
00041       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00042   Kokkos::parallel_for(
00043       "edac_laplacian", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00044         const int im = (i == 0) ? n0 - 1 : i - 1;
00045         const int ip = (i + 1 == n0) ? 0 : i + 1;
00046         const int jm = (j == 0) ? n1 - 1 : j - 1;
00047         const int jp = (j + 1 == n1) ? 0 : j + 1;
00048         const int km = (k == 0) ? n2 - 1 : k - 1;
00049         const int kp = (k + 1 == n2) ? 0 : k + 1;
00050         lap(k, j, i) = (p(k, j, ip) - 2.0 * p(k, j, i) + p(k, j, im)) / hx2 +
00051                        (p(k, jp, i) - 2.0 * p(k, j, i) + p(k, jm, i)) / hy2 +
00052                        (p(kp, j, i) - 2.0 * p(k, j, i) + p(km, j, i)) / hz2;
00053       });
00054 }
00055
00056 template <class ExecSpace>
00057 void subtract(projection::ScalarView<ExecSpace>& a,
00058               const projection::ScalarView<ExecSpace>& b) {
00059   const int n0 = static_cast<int>(a.extent(2));
00060   const int n1 = static_cast<int>(a.extent(1));
00061   const int n2 = static_cast<int>(a.extent(0));
00062   const auto policy =
00063       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00064   Kokkos::parallel_for(
00065       "edac_subtract", policy,
00066       KOKKOS_LAMBDA(int k, int j, int i) { a(k, j, i) -= b(k, j, i); });
00067 }
00068
00069 // Velocity RHS: advection + diffusion from the shared momentum operator, then
00070 // subtract grad(p) evaluated at the faces.
00071 template <class ExecSpace>
00072 void velocityRhs(const projection::ScalarView<ExecSpace>& u,
00073                  const projection::ScalarView<ExecSpace>& v,
00074                  const projection::ScalarView<ExecSpace>& w,
00075                  const projection::ScalarView<ExecSpace>& p,
00076                  projection::ScalarView<ExecSpace>& du,
00077                  projection::ScalarView<ExecSpace>& dv,
00078                  projection::ScalarView<ExecSpace>& dw,
00079                  projection::ScalarView<ExecSpace>& gx,
00080                  projection::ScalarView<ExecSpace>& gy,
00081                  projection::ScalarView<ExecSpace>& gz,
00082                  const std::array<double, 3>& h, double nu) {
00083   incompressible::momentumRhs<ExecSpace>(u, v, w, du, dv, dw, h, nu);
00084   projection::gradientX<ExecSpace>(p, gx, h[0]);
00085   projection::gradientY<ExecSpace>(p, gy, h[1]);
00086   projection::gradientZ<ExecSpace>(p, gz, h[2]);
00087   subtract<ExecSpace>(du, gx);
00088   subtract<ExecSpace>(dv, gy);
00089   subtract<ExecSpace>(dw, gz);
00090 }
00091
00092 template <class ExecSpace>
00093 void pressureRhs(const projection::ScalarView<ExecSpace>& p,
00094                  const projection::ScalarView<ExecSpace>& u,
00095                  const projection::ScalarView<ExecSpace>& v,
00096                  const projection::ScalarView<ExecSpace>& w,
00097                  projection::ScalarView<ExecSpace>& dp,
00098                  projection::ScalarView<ExecSpace>& div,
00099                  projection::ScalarView<ExecSpace>& lap,
00100                  const std::array<double, 3>& h, const Parameters& params) {
00101   projection::divergence<ExecSpace>(u, v, w, div, h);
00102   laplacian<ExecSpace>(p, lap, h);
00103   const int n0 = static_cast<int>(p.extent(2));
00104   const int n1 = static_cast<int>(p.extent(1));
00105   const int n2 = static_cast<int>(p.extent(0));
00106   const double c2 = params.c * params.c;
00107   const auto policy =
00108       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00109   Kokkos::parallel_for(
00110       "edac_pressure_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00111         dp(k, j, i) = -c2 * div(k, j, i) + params.nu_p * lap(k, j, i);
00112       });
00113 }
00114
00115 // SSP-RK3 for (u,v,w,p).
00116 template <class ExecSpace>
```

```
00117 void advanceSsprk3(projection::ScalarView<ExecSpace>& u,
00118                    projection::ScalarView<ExecSpace>& v,
00119                    projection::ScalarView<ExecSpace>& w,
00120                    projection::ScalarView<ExecSpace>& p,
00121                    projection::ScalarView<ExecSpace>& u1,
00122                    projection::ScalarView<ExecSpace>& v1,
00123                    projection::ScalarView<ExecSpace>& w1,
00124                    projection::ScalarView<ExecSpace>& p1,
00125                    projection::ScalarView<ExecSpace>& u2,
00126                    projection::ScalarView<ExecSpace>& v2,
00127                    projection::ScalarView<ExecSpace>& w2,
00128                    projection::ScalarView<ExecSpace>& p2,
00129                    projection::ScalarView<ExecSpace>& ku,
00130                    projection::ScalarView<ExecSpace>& kv,
00131                    projection::ScalarView<ExecSpace>& kw,
00132                    projection::ScalarView<ExecSpace>& kp,
00133                    projection::ScalarView<ExecSpace>& gx,
00134                    projection::ScalarView<ExecSpace>& gy,
00135                    projection::ScalarView<ExecSpace>& gz,
00136                    projection::ScalarView<ExecSpace>& div,
00137                    projection::ScalarView<ExecSpace>& lap,
00138                    const std::array<double, 3>& h, const Parameters& params,
00139                    double dt) {
00140   auto rhs = [&](auto& su, auto& sv, auto& sw, auto& sp, auto& du, auto& dv,
00141                  auto& dw, auto& dp) {
00142     velocityRhs<ExecSpace>(su, sv, sw, sp, du, dv, dw, gx, gy, gz, h,
00143                            params.nu);
00144     pressureRhs<ExecSpace>(sp, su, sv, sw, dp, div, lap, h, params);
00145   };
00146
00147   rhs(u, v, w, p, ku, kv, kw, kp);
00148   incompressible::combine<ExecSpace>(u, 1.0, ku, dt, ku, 0.0, u1);
00149   incompressible::combine<ExecSpace>(v, 1.0, kv, dt, ku, 0.0, v1);
00150   incompressible::combine<ExecSpace>(w, 1.0, kw, dt, ku, 0.0, w1);
00151   incompressible::combine<ExecSpace>(p, 1.0, kp, dt, ku, 0.0, p1);
00152
00153   rhs(u1, v1, w1, p1, ku, kv, kw, kp);
00154   incompressible::combine<ExecSpace>(u, 0.75, u1, 0.25, ku, 0.25 * dt, u2);
00155   incompressible::combine<ExecSpace>(v, 0.75, v1, 0.25, kv, 0.25 * dt, v2);
00156   incompressible::combine<ExecSpace>(w, 0.75, w1, 0.25, kw, 0.25 * dt, w2);
00157   incompressible::combine<ExecSpace>(p, 0.75, p1, 0.25, kp, 0.25 * dt, p2);
00158
00159   rhs(u2, v2, w2, p2, ku, kv, kw, kp);
00160   incompressible::combine<ExecSpace>(u, 1.0 / 3.0, u2, 2.0 / 3.0, ku,
00161                                      (2.0 / 3.0) * dt, u);
00162   incompressible::combine<ExecSpace>(v, 1.0 / 3.0, v2, 2.0 / 3.0, kv,
00163                                      (2.0 / 3.0) * dt, v);
00164   incompressible::combine<ExecSpace>(w, 1.0 / 3.0, w2, 2.0 / 3.0, kw,
00165                                      (2.0 / 3.0) * dt, w);
00166   incompressible::combine<ExecSpace>(p, 1.0 / 3.0, p2, 2.0 / 3.0, kp,
00167                                      (2.0 / 3.0) * dt, p);
00168 }
00169
00170 }  // namespace edac
00171 }  // namespace saturn
```

6.58 src/numerics/incompressible/fft_poisson.hpp File Reference

6.59 fft_poisson.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #ifdef SATURN_HAVE_FFTW
00004
00005 #include <fftw3.h>
00006 #include <Kokkos_Core.hpp>
00007
00008 #include <array>
00009 #include <cmath>
00010 #include <complex>
00011 #include <vector>
00012
00013 #include "numerics/incompressible/poisson.hpp"
00014
00015 namespace saturn {
00016 namespace poisson {
00017
```

```
00018 // Direct FFT Poisson solver for a periodic uniform box.
00019 //
00020 //   laplacian(p) = f,   mean(f) = 0
00021 //
00022 // The right-hand side is transformed with a complex-to-complex 3D FFT, divided
00023 // by the discrete eigenvalue
00024 //
00025 //   lambda_k = -4 [ sin^2(pi kx/nx)/hx^2 + ... ],
00026 //
00027 // and transformed back. The zero mode is set to zero (the constant null
00028 // space). This is exact for the 7-point operator up to round-off and is much
00029 // faster than multigrid on periodic power-of-two grids; FFTW handles arbitrary
00030 // sizes.
00031 //
00032 // The implementation is host-side: the field is copied to a host mirror,
00033 // transformed, and copied back. It is intended for CPU runs and as a reference
00034 // for the multigrid solver; a device FFT would be the next step.
00035
00036 template <class ExecSpace>
00037 class FftPoisson {
00038  public:
00039   using memory_space = typename ExecSpace::memory_space;
00040   using view_type = Kokkos::View<double***, Kokkos::LayoutRight, memory_space>;
00041
00042   FftPoisson(const std::array<int, 3>& cells,
00043              const std::array<double, 3>& spacing)
00044       : n0_(cells[0]), n1_(cells[1]), n2_(cells[2]), h_(spacing) {
00045     const long long total = static_cast<long long>(n0_) * n1_ * n2_;
00046     in_ = fftw_alloc_complex(static_cast<std::size_t>(total));
00047     out_ = fftw_alloc_complex(static_cast<std::size_t>(total));
00048     forward_ =
00049         fftw_plan_dft_3d(n2_, n1_, n0_, in_, out_, FFTW_FORWARD, FFTW_ESTIMATE);
00050     inverse_ = fftw_plan_dft_3d(n2_, n1_, n0_, in_, out_, FFTW_BACKWARD,
00051                                 FFTW_ESTIMATE);
00052   }
00053
00054   ~FftPoisson() {
00055     if (forward_ != nullptr) {
00056       fftw_destroy_plan(forward_);
00057     }
00058     if (inverse_ != nullptr) {
00059       fftw_destroy_plan(inverse_);
00060     }
00061     if (in_ != nullptr) {
00062       fftw_free(in_);
00063     }
00064     if (out_ != nullptr) {
00065       fftw_free(out_);
00066     }
00067   }
00068
00069   FftPoisson(const FftPoisson&) = delete;
00070   FftPoisson& operator=(const FftPoisson&) = delete;
00071
00072   Result solve(view_type x, view_type b, const Options& options) {
00073     (void)options;
00074     const long long total = static_cast<long long>(n0_) * n1_ * n2_;
00075     auto hb = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), b);
00076     for (int k = 0; k < n2_; ++k) {
00077       for (int j = 0; j < n1_; ++j) {
00078         for (int i = 0; i < n0_; ++i) {
00079           const long long index =
00080               (static_cast<long long>(k) * n1_ + j) * n0_ + i;
00081           in_[index][0] = hb(k, j, i);
00082           in_[index][1] = 0.0;
00083         }
00084       }
00085     }
00086
00087     fftw_execute_dft(forward_, in_, out_);
00088     for (int k = 0; k < n2_; ++k) {
00089       const double kz = 2.0 * std::sin(kPi * k / n2_) / h_[2];
00090       for (int j = 0; j < n1_; ++j) {
00091         const double ky = 2.0 * std::sin(kPi * j / n1_) / h_[1];
00092         for (int i = 0; i < n0_; ++i) {
00093           const double kx = 2.0 * std::sin(kPi * i / n0_) / h_[0];
00094           const long long index =
00095               (static_cast<long long>(k) * n1_ + j) * n0_ + i;
00096           const double lambda = -(kx * kx + ky * ky + kz * kz);
00097           double re = 0.0;
00098           double im = 0.0;
00099           if (lambda != 0.0) {
00100             re = out_[index][0] / lambda;
00101             im = out_[index][1] / lambda;
00102           }
00103           in_[index][0] = re;
00104           in_[index][1] = im;
```

```
00105         }
00106       }
00107     }
00108
00109     fftw_execute_dft(inverse_, in_, out_);
00110     auto hx = Kokkos::create_mirror_view(x);
00111     const double scale = 1.0 / static_cast<double>(total);
00112     for (int k = 0; k < n2_; ++k) {
00113       for (int j = 0; j < n1_; ++j) {
00114         for (int i = 0; i < n0_; ++i) {
00115           const long long index =
00116               (static_cast<long long>(k) * n1_ + j) * n0_ + i;
00117           hx(k, j, i) = out_[index][0] * scale;
00118         }
00119       }
00120     }
00121     Kokkos::deep_copy(x, hx);
00122
00123     Result result;
00124     result.cycles = 1;
00125     result.initial_residual = 0.0;
00126     result.final_residual = 0.0;
00127     result.converged = true;
00128     return result;
00129   }
00130
00131   int levels() const { return 1; }
00132
00133  private:
00134   static constexpr double kPi = 3.14159265358979323846;
00135   int n0_ = 1;
00136   int n1_ = 1;
00137   int n2_ = 1;
00138   std::array<double, 3> h_{{1.0, 1.0, 1.0}};
00139   fftw_complex* in_ = nullptr;
00140   fftw_complex* out_ = nullptr;
00141   fftw_plan forward_ = nullptr;
00142   fftw_plan inverse_ = nullptr;
00143 };
00144
00145 }  // namespace poisson
00146 }  // namespace saturn
00147
00148 #endif  // SATURN_HAVE_FFTW
```

6.60 src/numerics/incompressible/incompressible.hpp File Reference

Incompressible MAC momentum, filtering, and SSP-RK3 helpers.

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include "core/parallel.hpp"
#include "numerics/incompressible/projection.hpp"
```

Namespaces

- namespace saturn
- namespace saturn::incompressible

Functions

- template<class ExecSpace >
  void saturn::incompressible::momentumRhs (const projection::ScalarView< ExecSpace > &u, const projection::ScalarView< ExecSpace > &v, const projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &du, projection::ScalarView< ExecSpace > &dv, projection::ScalarView< ExecSpace > &dw, const std::array< double, 3 > &h, double nu)

- template<class ExecSpace >
  void saturn::incompressible::combine (const projection::ScalarView< ExecSpace > &a, double alpha, const projection::ScalarView< ExecSpace > &b, double beta, const projection::ScalarView< ExecSpace > &c, double gamma, projection::ScalarView< ExecSpace > &out)
- template<class ExecSpace >
  void saturn::incompressible::filterDimension (const projection::ScalarView< ExecSpace > &in, projection::ScalarView< ExecSpace > &out, double alpha, int dimension)
- template<class ExecSpace >
  void saturn::incompressible::filter4 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &tu, projection::ScalarView< ExecSpace > &tv, projection::ScalarView< ExecSpace > &tw, double alpha)
- template<class ExecSpace , class Project >
  void saturn::incompressible::advanceSsprk3 (projection::ScalarView< ExecSpace > &u, projection::ScalarView< ExecSpace > &v, projection::ScalarView< ExecSpace > &w, projection::ScalarView< ExecSpace > &u1, projection::ScalarView< ExecSpace > &v1, projection::ScalarView< ExecSpace > &w1, projection::ScalarView< ExecSpace > &u2, projection::ScalarView< ExecSpace > &v2, projection::ScalarView< ExecSpace > &w2, projection::ScalarView< ExecSpace > &ku, projection::ScalarView< ExecSpace > &kv, projection::ScalarView< ExecSpace > &kw, const std::array< double, 3 > &h, double nu, double dt, Project &project)

6.60.1 Detailed Description

Incompressible MAC momentum, filtering, and SSP-RK3 helpers.

6.61 incompressible.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #include <Kokkos_Core.hpp>
00008
00009 #include <array>
00010 #include <cmath>
00011
00012 #include "core/parallel.hpp"
00013 #include "numerics/incompressible/projection.hpp"
00014
00015 namespace saturn {
00016 namespace incompressible {
00017
00018 // Explicit momentum right-hand side for the MAC (staggered) formulation:
00019 //
00020 //   du/dt = -(u.grad)u + nu laplacian(u)
00021 //
00022 // with second-order central differences. Pressure is applied separately by
00023 // `projection::project`. The advecting transverse velocities are interpolated
00024 // to the face from the four surrounding faces.
00025
00026 template <class ExecSpace>
00027 void momentumRhs(const projection::ScalarView<ExecSpace>& u,
00028                  const projection::ScalarView<ExecSpace>& v,
00029                  const projection::ScalarView<ExecSpace>& w,
00030                  projection::ScalarView<ExecSpace>& du,
00031                  projection::ScalarView<ExecSpace>& dv,
00032                  projection::ScalarView<ExecSpace>& dw,
00033                  const std::array<double, 3>& h, double nu) {
00034   const int n0 = static_cast<int>(u.extent(2));
00035   const int n1 = static_cast<int>(u.extent(1));
00036   const int n2 = static_cast<int>(u.extent(0));
00037   const double hx = h[0];
00038   const double hy = h[1];
00039   const double hz = h[2];
00040   const double hx2 = hx * hx;
00041   const double hy2 = hy * hy;
00042   const double hz2 = hz * hz;
00043   const auto policy =
```

```
00044       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00045   Kokkos::parallel_for(
00046       "incompressible_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00047         const int im = (i == 0) ? n0 - 1 : i - 1;
00048         const int ip = (i + 1 == n0) ? 0 : i + 1;
00049         const int jm = (j == 0) ? n1 - 1 : j - 1;
00050         const int jp = (j + 1 == n1) ? 0 : j + 1;
00051         const int km = (k == 0) ? n2 - 1 : k - 1;
00052         const int kp = (k + 1 == n2) ? 0 : k + 1;
00053
00054         // u-momentum at the x-face (i,j,k).
00055         const double dudx = (u(k, j, ip) - u(k, j, im)) / (2.0 * hx);
00056         const double dudy = (u(k, jp, i) - u(k, jm, i)) / (2.0 * hy);
00057         const double dudz = (u(kp, j, i) - u(km, j, i)) / (2.0 * hz);
00058         const double v_at_u =
00059             0.25 * (v(k, j, im) + v(k, j, i) + v(k, jp, im) + v(k, jp, i));
00060         const double w_at_u =
00061             0.25 * (w(k, j, im) + w(k, j, i) + w(kp, j, im) + w(kp, j, i));
00062         const double lap_u =
00063             (u(k, j, ip) - 2.0 * u(k, j, i) + u(k, j, im)) / hx2 +
00064             (u(k, jp, i) - 2.0 * u(k, j, i) + u(k, jm, i)) / hy2 +
00065             (u(kp, j, i) - 2.0 * u(k, j, i) + u(km, j, i)) / hz2;
00066         du(k, j, i) =
00067             -(u(k, j, i) * dudx + v_at_u * dudy + w_at_u * dudz) + nu * lap_u;
00068
00069         // v-momentum at the y-face (i,j,k).
00070         const double dvdx = (v(k, j, ip) - v(k, j, im)) / (2.0 * hx);
00071         const double dvdy = (v(k, jp, i) - v(k, jm, i)) / (2.0 * hy);
00072         const double dvdz = (v(kp, j, i) - v(km, j, i)) / (2.0 * hz);
00073         const double u_at_v =
00074             0.25 * (u(k, jm, i) + u(k, j, i) + u(k, jm, ip) + u(k, j, ip));
00075         const double w_at_v =
00076             0.25 * (w(k, jm, i) + w(k, j, i) + w(kp, jm, i) + w(kp, j, i));
00077         const double lap_v =
00078             (v(k, j, ip) - 2.0 * v(k, j, i) + v(k, j, im)) / hx2 +
00079             (v(k, jp, i) - 2.0 * v(k, j, i) + v(k, jm, i)) / hy2 +
00080             (v(kp, j, i) - 2.0 * v(k, j, i) + v(km, j, i)) / hz2;
00081         dv(k, j, i) =
00082             -(u_at_v * dvdx + v(k, j, i) * dvdy + w_at_v * dvdz) + nu * lap_v;
00083
00084         // w-momentum at the z-face (i,j,k).
00085         const double dwdx = (w(k, j, ip) - w(k, j, im)) / (2.0 * hx);
00086         const double dwdy = (w(k, jp, i) - w(k, jm, i)) / (2.0 * hy);
00087         const double dwdz = (w(kp, j, i) - w(km, j, i)) / (2.0 * hz);
00088         const double u_at_w =
00089             0.25 * (u(km, j, i) + u(k, j, i) + u(km, j, ip) + u(k, j, ip));
00090         const double v_at_w =
00091             0.25 * (v(km, j, i) + v(k, j, i) + v(km, jp, i) + v(k, jp, i));
00092         const double lap_w =
00093             (w(k, j, ip) - 2.0 * w(k, j, i) + w(k, j, im)) / hx2 +
00094             (w(k, jp, i) - 2.0 * w(k, j, i) + w(k, jm, i)) / hy2 +
00095             (w(kp, j, i) - 2.0 * w(k, j, i) + w(km, j, i)) / hz2;
00096         dw(k, j, i) =
00097             -(u_at_w * dwdx + v_at_w * dwdy + w(k, j, i) * dwdz) + nu * lap_w;
00098       });
00099 }
00100
00101 template <class ExecSpace>
00102 void combine(const projection::ScalarView<ExecSpace>& a, double alpha,
00103              const projection::ScalarView<ExecSpace>& b, double beta,
00104              const projection::ScalarView<ExecSpace>& c, double gamma,
00105              projection::ScalarView<ExecSpace>& out) {
00106   const int n0 = static_cast<int>(out.extent(2));
00107   const int n1 = static_cast<int>(out.extent(1));
00108   const int n2 = static_cast<int>(out.extent(0));
00109   const auto policy =
00110       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00111   Kokkos::parallel_for(
00112       "incompressible_combine", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00113         out(k, j, i) =
00114             alpha * a(k, j, i) + beta * b(k, j, i) + gamma * c(k, j, i);
00115       });
00116 }
00117
00118 // Fourth-order periodic filter along one dimension:
00119 //
00120 //   out = in - alpha/16 (in_{d-2} - 4 in_{d-1} + 6 in - 4 in_{d+1} + in_{d+2})
00121 //
00122 // The symbol is 1 - alpha sin^4(k h / 2), so grid-scale modes are damped while
00123 // well-resolved modes are barely touched. This is a numerical stabilization
00124 // for under-resolved transitional flows, not part of the physical model.
00125 template <class ExecSpace>
00126 void filterDimension(const projection::ScalarView<ExecSpace>& in,
00127                      projection::ScalarView<ExecSpace>& out, double alpha,
00128                      int dimension) {
00129   const int n0 = static_cast<int>(in.extent(2));
00130   const int n1 = static_cast<int>(in.extent(1));
```

```
00131   const int n2 = static_cast<int>(in.extent(0));
00132   const auto policy =
00133       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00134   Kokkos::parallel_for(
00135       "incompressible_filter", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00136         double value = 0.0;
00137         if (dimension == 0) {
00138           const int im2 = (i - 2 + n0) % n0;
00139           const int im1 = (i - 1 + n0) % n0;
00140           const int ip1 = (i + 1) % n0;
00141           const int ip2 = (i + 2) % n0;
00142           value = in(k, j, im2) - 4.0 * in(k, j, im1) + 6.0 * in(k, j, i) -
00143                   4.0 * in(k, j, ip1) + in(k, j, ip2);
00144         } else if (dimension == 1) {
00145           const int jm2 = (j - 2 + n1) % n1;
00146           const int jm1 = (j - 1 + n1) % n1;
00147           const int jp1 = (j + 1) % n1;
00148           const int jp2 = (j + 2) % n1;
00149           value = in(k, jm2, i) - 4.0 * in(k, jm1, i) + 6.0 * in(k, j, i) -
00150                   4.0 * in(k, jp1, i) + in(k, jp2, i);
00151         } else {
00152           const int km2 = (k - 2 + n2) % n2;
00153           const int km1 = (k - 1 + n2) % n2;
00154           const int kp1 = (k + 1) % n2;
00155           const int kp2 = (k + 2) % n2;
00156           value = in(km2, j, i) - 4.0 * in(km1, j, i) + 6.0 * in(k, j, i) -
00157                   4.0 * in(kp1, j, i) + in(kp2, j, i);
00158         }
00159         out(k, j, i) = in(k, j, i) - (alpha / 16.0) * value;
00160       });
00161 }
00162
00163 template <class ExecSpace>
00164 void filter4(projection::ScalarView<ExecSpace>& u,
00165              projection::ScalarView<ExecSpace>& v,
00166              projection::ScalarView<ExecSpace>& w,
00167              projection::ScalarView<ExecSpace>& tu,
00168              projection::ScalarView<ExecSpace>& tv,
00169              projection::ScalarView<ExecSpace>& tw, double alpha) {
00170   if (alpha <= 0.0) {
00171     return;
00172   }
00173   filterDimension<ExecSpace>(u, tu, alpha, 0);
00174   filterDimension<ExecSpace>(tu, u, alpha, 1);
00175   filterDimension<ExecSpace>(u, tu, alpha, 2);
00176   Kokkos::deep_copy(u, tu);
00177
00178   filterDimension<ExecSpace>(v, tv, alpha, 0);
00179   filterDimension<ExecSpace>(tv, v, alpha, 1);
00180   filterDimension<ExecSpace>(v, tv, alpha, 2);
00181   Kokkos::deep_copy(v, tv);
00182
00183   filterDimension<ExecSpace>(w, tw, alpha, 0);
00184   filterDimension<ExecSpace>(tw, w, alpha, 1);
00185   filterDimension<ExecSpace>(w, tw, alpha, 2);
00186   Kokkos::deep_copy(w, tw);
00187 }
00188
00189 // SSP-RK3 for the three face-velocity components, with a projection applied
00190 // after every stage. The projection callback takes the three velocity views
00191 // and must make them discretely divergence-free.
00192 template <class ExecSpace, class Project>
00193 void advanceSsprk3(projection::ScalarView<ExecSpace>& u,
00194                    projection::ScalarView<ExecSpace>& v,
00195                    projection::ScalarView<ExecSpace>& w,
00196                    projection::ScalarView<ExecSpace>& u1,
00197                    projection::ScalarView<ExecSpace>& v1,
00198                    projection::ScalarView<ExecSpace>& w1,
00199                    projection::ScalarView<ExecSpace>& u2,
00200                    projection::ScalarView<ExecSpace>& v2,
00201                    projection::ScalarView<ExecSpace>& w2,
00202                    projection::ScalarView<ExecSpace>& ku,
00203                    projection::ScalarView<ExecSpace>& kv,
00204                    projection::ScalarView<ExecSpace>& kw,
00205                    const std::array<double, 3>& h, double nu, double dt,
00206                    Project& project) {
00207   momentumRhs<ExecSpace>(u, v, w, ku, kv, kw, h, nu);
00208   combine<ExecSpace>(u, 1.0, ku, dt, ku, 0.0, u1);
00209   combine<ExecSpace>(v, 1.0, kv, dt, kv, 0.0, v1);
00210   combine<ExecSpace>(w, 1.0, kw, dt, kw, 0.0, w1);
00211   project(u1, v1, w1);
00212
00213   momentumRhs<ExecSpace>(u1, v1, w1, ku, kv, kw, h, nu);
00214   combine<ExecSpace>(u, 0.75, u1, 0.25, ku, 0.25 * dt, u2);
00215   combine<ExecSpace>(v, 0.75, v1, 0.25, kv, 0.25 * dt, v2);
00216   combine<ExecSpace>(w, 0.75, w1, 0.25, kw, 0.25 * dt, w2);
00217   project(u2, v2, w2);
```

```
00218
00219   momentumRhs<ExecSpace>(u2, v2, w2, ku, kv, kw, h, nu);
00220   combine<ExecSpace>(u, 1.0 / 3.0, u2, 2.0 / 3.0, ku, (2.0 / 3.0) * dt, u);
00221   combine<ExecSpace>(v, 1.0 / 3.0, v2, 2.0 / 3.0, kv, (2.0 / 3.0) * dt, v);
00222   combine<ExecSpace>(w, 1.0 / 3.0, w2, 2.0 / 3.0, kw, (2.0 / 3.0) * dt, w);
00223   project(u, v, w);
00224 }
00225
00226 }  // namespace incompressible
00227 }  // namespace saturn
```

6.62 src/numerics/incompressible/poisson.hpp File Reference

Cell-centred geometric-multigrid Poisson solver.

```
#include <Kokkos_Core.hpp>
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <string>
#include <vector>
#include "core/parallel.hpp"
```

Classes

- struct saturn::poisson::Options

  Multigrid cycle and tolerance controls.
- struct saturn::poisson::Result
- class saturn::poisson::GeometricMultigrid< ExecSpace >

  Geometric multigrid for the uniform-grid Poisson equation.
- struct saturn::poisson::GeometricMultigrid< ExecSpace >::Level

Namespaces

- namespace saturn
- namespace saturn::poisson
- namespace saturn::poisson::detail

Enumerations

- enum class saturn::poisson::Boundary { saturn::poisson::Periodic , saturn::poisson::Neumann , saturn::poisson::Dirichlet }

Functions

- KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::plusIndex (int i, int n, Boundary bc, int &sign)
- KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::minusIndex (int i, int n, Boundary bc, int &sign)
- KOKKOS_INLINE_FUNCTION int saturn::poisson::detail::extendedIndex (int index, int n, Boundary low, Boundary high, int &sign)

6.62.1 Detailed Description

Cell-centred geometric-multigrid Poisson solver.

6.63 poisson.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #include <Kokkos_Core.hpp>
00008
00009 #include <algorithm>
00010 #include <array>
00011 #include <cmath>
00012 #include <cstdio>
00013 #include <string>
00014 #include <vector>
00015
00016 #include "core/parallel.hpp"
00017
00018 namespace saturn {
00019 namespace poisson {
00020
00021 // Cell-centered constant-coefficient Poisson solver:
00022 //
00023 //   laplacian(p) = f
00024 //
00025 // on a uniform box with periodic, homogeneous-Neumann, or homogeneous-
00026 // Dirichlet boundaries. The discretization is the standard 7-point
00027 // finite-volume Laplacian with ghost cells:
00028 //
00029 //   Neumann:   p_ghost =  p_interior   (zero normal derivative at the face)
00030 //   Dirichlet: p_ghost = -p_interior   (zero value at the face)
00031 //
00032 // Solutions with only periodic/Neumann boundaries are defined up to a
00033 // constant; the right-hand side and initial guess are projected to zero mean.
00034 //
00035 // The solver is a geometric-multigrid V-cycle with red-black Gauss-Seidel
00036 // smoothing, full-weighting restriction, and trilinear prolongation.
00037
00038 enum class Boundary {
00039   Periodic,
00040   Neumann,
00041   Dirichlet,
00042 };
00043
00045 struct Options {
00046   int max_cycles = 100;
00047   int pre_smooth = 2;
00048   int post_smooth = 2;
00049   int coarse_smooth = 60;
00050   double tolerance = 1.0e-8;
00051   bool verbose = false;
00052 };
00053
00054 struct Result {
00055   int cycles = 0;
00056   double initial_residual = 0.0;
00057   double final_residual = 0.0;
00058   bool converged = false;
00059 };
00060
00061 namespace detail {
00062
00063 KOKKOS_INLINE_FUNCTION
00064 int plusIndex(int i, int n, Boundary bc, int& sign) {
00065   sign = 1;
00066   if (i + 1 < n) {
00067     return i + 1;
00068   }
00069   if (bc == Boundary::Periodic) {
00070     return 0;
00071   }
00072   if (bc == Boundary::Neumann) {
00073     return n - 1;
00074   }
00075   sign = -1;
00076   return n - 1;
```

```
00077 }
00078
00079 KOKKOS_INLINE_FUNCTION
00080 int minusIndex(int i, int n, Boundary bc, int& sign) {
00081   sign = 1;
00082   if (i > 0) {
00083     return i - 1;
00084   }
00085   if (bc == Boundary::Periodic) {
00086     return n - 1;
00087   }
00088   if (bc == Boundary::Neumann) {
00089     return 0;
00090   }
00091   sign = -1;
00092   return 0;
00093 }
00094
00095 // Map an index that may lie one cell outside [0,n) through the boundary.
00096 KOKKOS_INLINE_FUNCTION
00097 int extendedIndex(int index, int n, Boundary low, Boundary high, int& sign) {
00098   sign = 1;
00099   if (index >= 0 && index < n) {
00100     return index;
00101   }
00102   if (index < 0) {
00103     if (low == Boundary::Periodic) {
00104       return ((index % n) + n) % n;
00105     }
00106     if (low == Boundary::Dirichlet) {
00107       sign = -1;
00108     }
00109     return 0;
00110   }
00111   if (high == Boundary::Periodic) {
00112     return index % n;
00113   }
00114   if (high == Boundary::Dirichlet) {
00115     sign = -1;
00116   }
00117   return n - 1;
00118 }
00119
00120 }  // namespace detail
00121
00128 template <class ExecSpace>
00129 class GeometricMultigrid {
00130  public:
00131   using memory_space = typename ExecSpace::memory_space;
00132   using view_type = Kokkos::View<double***, Kokkos::LayoutRight, memory_space>;
00133
00134   GeometricMultigrid(const std::array<int, 3>& cells,
00135                      const std::array<double, 3>& spacing)
00136       : GeometricMultigrid(
00137             cells, spacing,
00138             std::array<Boundary, 3>{Boundary::Periodic, Boundary::Periodic,
00139                                     Boundary::Periodic},
00140             std::array<Boundary, 3>{Boundary::Periodic, Boundary::Periodic,
00141                                     Boundary::Periodic}) {}
00142
00143   GeometricMultigrid(const std::array<int, 3>& cells,
00144                      const std::array<double, 3>& spacing,
00145                      const std::array<Boundary, 3>& bc_low,
00146                      const std::array<Boundary, 3>& bc_high)
00147       : bc_low_(bc_low), bc_high_(bc_high) {
00148     has_dirichlet_ = false;
00149     all_periodic_ = true;
00150     for (int d = 0; d < 3; ++d) {
00151       has_dirichlet_ = has_dirichlet_ || bc_low[d] == Boundary::Dirichlet ||
00152                        bc_high[d] == Boundary::Dirichlet;
00153       all_periodic_ = all_periodic_ && bc_low[d] == Boundary::Periodic &&
00154                       bc_high[d] == Boundary::Periodic;
00155     }
00156
00157     std::array<int, 3> n = cells;
00158     std::array<double, 3> h = spacing;
00159     while (true) {
00160       Level level;
00161       level.n = n;
00162       level.h = h;
00163       level.x = view_type("poisson_x", n[2], n[1], n[0]);
00164       level.b = view_type("poisson_b", n[2], n[1], n[0]);
00165       level.r = view_type("poisson_r", n[2], n[1], n[0]);
00166       levels_.push_back(level);
00167
00168       const bool can_coarsen = n[0] % 2 == 0 && n[1] % 2 == 0 &&
00169                                n[2] % 2 == 0 && n[0] >= 6 && n[1] >= 6 &&
```

```
00170                                n[2] >= 6;
00171       if (!can_coarsen) {
00172         break;
00173       }
00174       for (int d = 0; d < 3; ++d) {
00175         n[d] /= 2;
00176         h[d] *= 2.0;
00177       }
00178     }
00179   }
00180
00181   // Solve laplacian(x) = b with x as the initial guess. With only periodic
00182   // and Neumann boundaries, x and b are projected to zero mean.
00183   Result solve(view_type x, view_type b, const Options& options) {
00184     Kokkos::deep_copy(levels_[0].x, x);
00185     Kokkos::deep_copy(levels_[0].b, b);
00186     if (!has_dirichlet_) {
00187       projectMeanImpl(levels_[0].x);
00188       projectMeanImpl(levels_[0].b);
00189     }
00190
00191     Result result;
00192     result.initial_residual = residualNorm(0);
00193     result.final_residual = result.initial_residual;
00194     const double rhs_norm = norm(levels_[0].b);
00195     const double target = options.tolerance * rhs_norm;
00196     if (result.initial_residual == 0.0 || result.initial_residual <= target) {
00197       result.converged = true;
00198     }
00199
00200     for (int cycle = 0; cycle < options.max_cycles; ++cycle) {
00201       if (result.final_residual <= target) {
00202         result.converged = true;
00203         break;
00204       }
00205       vcycle(0, options);
00206       if (!has_dirichlet_) {
00207         projectMeanImpl(levels_[0].x);
00208       }
00209       result.cycles = cycle + 1;
00210       result.final_residual = residualNorm(0);
00211     }
00212     if (result.final_residual <= target) {
00213       result.converged = true;
00214     }
00215     Kokkos::deep_copy(x, levels_[0].x);
00216     return result;
00217   }
00218
00219   int levels() const { return static_cast<int>(levels_.size()); }
00220
00221  public:
00222   struct Level {
00223     std::array<int, 3> n{{0, 0, 0}};
00224     std::array<double, 3> h{{1.0, 1.0, 1.0}};
00225     view_type x{};
00226     view_type b{};
00227     view_type r{};
00228   };
00229
00230   void vcycle(int level_index, const Options& options) {
00231     Level& level = levels_[static_cast<std::size_t>(level_index)];
00232     if (level_index + 1 == static_cast<int>(levels_.size())) {
00233       smooth(level, options.coarse_smooth);
00234       return;
00235     }
00236
00237     smooth(level, options.pre_smooth);
00238     computeResidual(level);
00239
00240     Level& coarse = levels_[static_cast<std::size_t>(level_index + 1)];
00241     restrict(level, coarse);
00242     Kokkos::deep_copy(coarse.x, 0.0);
00243     vcycle(level_index + 1, options);
00244     prolongAdd(coarse, level);
00245
00246     smooth(level, options.post_smooth);
00247   }
00248
00249   void smooth(const Level& level, int sweeps) const {
00250     const int n0 = level.n[0];
00251     const int n1 = level.n[1];
00252     const int n2 = level.n[2];
00253     const double hx2 = level.h[0] * level.h[0];
00254     const double hy2 = level.h[1] * level.h[1];
00255     const double hz2 = level.h[2] * level.h[2];
00256     const double diag = -2.0 / hx2 - 2.0 / hy2 - 2.0 / hz2;
```

```
00257     const Boundary bl0 = bc_low_[0];
00258     const Boundary bh0 = bc_high_[0];
00259     const Boundary bl1 = bc_low_[1];
00260     const Boundary bh1 = bc_high_[1];
00261     const Boundary bl2 = bc_low_[2];
00262     const Boundary bh2 = bc_high_[2];
00263     auto x = level.x;
00264     auto b = level.b;
00265     const auto policy =
00266         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00267     if (all_periodic_) {
00268       for (int sweep = 0; sweep < sweeps; ++sweep) {
00269         for (int color = 0; color < 2; ++color) {
00270           Kokkos::parallel_for(
00271               "poisson_smooth_periodic", policy,
00272               KOKKOS_LAMBDA(int k, int j, int i) {
00273                 if (((i + j + k) & 1) != color) {
00274                   return;
00275                 }
00276                 const int im = (i == 0) ? n0 - 1 : i - 1;
00277                 const int ip = (i + 1 == n0) ? 0 : i + 1;
00278                 const int jm = (j == 0) ? n1 - 1 : j - 1;
00279                 const int jp = (j + 1 == n1) ? 0 : j + 1;
00280                 const int km = (k == 0) ? n2 - 1 : k - 1;
00281                 const int kp = (k + 1 == n2) ? 0 : k + 1;
00282                 const double off = (x(k, j, im) + x(k, j, ip)) / hx2 +
00283                                    (x(k, jm, i) + x(k, jp, i)) / hy2 +
00284                                    (x(km, j, i) + x(kp, j, i)) / hz2;
00285                 x(k, j, i) = (b(k, j, i) - off) / diag;
00286               });
00287         }
00288       }
00289       return;
00290     }
00291     for (int sweep = 0; sweep < sweeps; ++sweep) {
00292       for (int color = 0; color < 2; ++color) {
00293         Kokkos::parallel_for(
00294             "poisson_smooth", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00295               if (((i + j + k) & 1) != color) {
00296                 return;
00297               }
00298               int sim;
00299               int sip;
00300               int sjm;
00301               int sjp;
00302               int skm;
00303               int skp;
00304               const int im = detail::minusIndex(i, n0, bl0, sim);
00305               const int ip = detail::plusIndex(i, n0, bh0, sip);
00306               const int jm = detail::minusIndex(j, n1, bl1, sjm);
00307               const int jp = detail::plusIndex(j, n1, bh1, sjp);
00308               const int km = detail::minusIndex(k, n2, bl2, skm);
00309               const int kp = detail::plusIndex(k, n2, bh2, skp);
00310               const double off = (sim * x(k, j, im) + sip * x(k, j, ip)) / hx2 +
00311                                  (sjm * x(k, jm, i) + sjp * x(k, jp, i)) / hy2 +
00312                                  (skm * x(km, j, i) + skp * x(kp, j, i)) / hz2;
00313               x(k, j, i) = (b(k, j, i) - off) / diag;
00314             });
00315       }
00316     }
00317   }
00318
00319   void computeResidual(const Level& level) const {
00320     const int n0 = level.n[0];
00321     const int n1 = level.n[1];
00322     const int n2 = level.n[2];
00323     const double hx2 = level.h[0] * level.h[0];
00324     const double hy2 = level.h[1] * level.h[1];
00325     const double hz2 = level.h[2] * level.h[2];
00326     const Boundary bl0 = bc_low_[0];
00327     const Boundary bh0 = bc_high_[0];
00328     const Boundary bl1 = bc_low_[1];
00329     const Boundary bh1 = bc_high_[1];
00330     const Boundary bl2 = bc_low_[2];
00331     const Boundary bh2 = bc_high_[2];
00332     auto x = level.x;
00333     auto b = level.b;
00334     auto r = level.r;
00335     const auto policy =
00336         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00337     if (all_periodic_) {
00338       Kokkos::parallel_for(
00339           "poisson_residual_periodic", policy,
00340           KOKKOS_LAMBDA(int k, int j, int i) {
00341             const int im = (i == 0) ? n0 - 1 : i - 1;
00342             const int ip = (i + 1 == n0) ? 0 : i + 1;
00343             const int jm = (j == 0) ? n1 - 1 : j - 1;
```

```
00344             const int jp = (j + 1 == n1) ? 0 : j + 1;
00345             const int km = (k == 0) ? n2 - 1 : k - 1;
00346             const int kp = (k + 1 == n2) ? 0 : k + 1;
00347             const double lap =
00348                 (x(k, j, ip) - 2.0 * x(k, j, i) + x(k, j, im)) / hx2 +
00349                 (x(k, jp, i) - 2.0 * x(k, j, i) + x(k, jm, i)) / hy2 +
00350                 (x(kp, j, i) - 2.0 * x(k, j, i) + x(km, j, i)) / hz2;
00351             r(k, j, i) = b(k, j, i) - lap;
00352           });
00353       return;
00354     }
00355     Kokkos::parallel_for(
00356         "poisson_residual", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00357           int sim;
00358           int sip;
00359           int sjm;
00360           int sjp;
00361           int skm;
00362           int skp;
00363           const int im = detail::minusIndex(i, n0, bl0, sim);
00364           const int ip = detail::plusIndex(i, n0, bh0, sip);
00365           const int jm = detail::minusIndex(j, n1, bl1, sjm);
00366           const int jp = detail::plusIndex(j, n1, bh1, sjp);
00367           const int km = detail::minusIndex(k, n2, bl2, skm);
00368           const int kp = detail::plusIndex(k, n2, bh2, skp);
00369           const double lap =
00370               (sip * x(k, j, ip) - 2.0 * x(k, j, i) + sim * x(k, j, im)) / hx2 +
00371               (sjp * x(k, jp, i) - 2.0 * x(k, j, i) + sjm * x(k, jm, i)) / hy2 +
00372               (skp * x(kp, j, i) - 2.0 * x(k, j, i) + skm * x(km, j, i)) / hz2;
00373           r(k, j, i) = b(k, j, i) - lap;
00374         });
00375   }
00376
00377   void restrict(const Level& fine, Level& coarse) const {
00378     const int n0 = fine.n[0];
00379     const int n1 = fine.n[1];
00380     const int n2 = fine.n[2];
00381     const int c0 = coarse.n[0];
00382     const int c1 = coarse.n[1];
00383     const int c2 = coarse.n[2];
00384     const Boundary bl0 = bc_low_[0];
00385     const Boundary bh0 = bc_high_[0];
00386     const Boundary bl1 = bc_low_[1];
00387     const Boundary bh1 = bc_high_[1];
00388     const Boundary bl2 = bc_low_[2];
00389     const Boundary bh2 = bc_high_[2];
00390     auto r = fine.r;
00391     auto b = coarse.b;
00392     const auto policy =
00393         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{c2, c1, c0}});
00394     Kokkos::parallel_for(
00395         "poisson_restrict", policy, KOKKOS_LAMBDA(int ck, int cj, int ci) {
00396           double sum = 0.0;
00397           for (int dk = -1; dk <= 1; ++dk) {
00398             const double wz = (dk == 0) ? 0.5 : 0.25;
00399             int sz;
00400             const int k = detail::extendedIndex(2 * ck + dk, n2, bl2, bh2, sz);
00401             for (int dj = -1; dj <= 1; ++dj) {
00402               const double wy = (dj == 0) ? 0.5 : 0.25;
00403               int sy;
00404               const int j =
00405                   detail::extendedIndex(2 * cj + dj, n1, bl1, bh1, sy);
00406               for (int di = -1; di <= 1; ++di) {
00407                 const double wx = (di == 0) ? 0.5 : 0.25;
00408                 int sx;
00409                 const int i =
00410                     detail::extendedIndex(2 * ci + di, n0, bl0, bh0, sx);
00411                 sum += wx * wy * wz * sx * sy * sz * r(k, j, i);
00412               }
00413             }
00414           }
00415           b(ck, cj, ci) = sum;
00416         });
00417   }
00418
00419   void prolongAdd(const Level& coarse, Level& fine) const {
00420     const int c0 = coarse.n[0];
00421     const int c1 = coarse.n[1];
00422     const int c2 = coarse.n[2];
00423     const int n0 = fine.n[0];
00424     const int n1 = fine.n[1];
00425     const int n2 = fine.n[2];
00426     const Boundary bl0 = bc_low_[0];
00427     const Boundary bh0 = bc_high_[0];
00428     const Boundary bl1 = bc_low_[1];
00429     const Boundary bh1 = bc_high_[1];
00430     const Boundary bl2 = bc_low_[2];
```

```
00431     const Boundary bh2 = bc_high_[2];
00432     auto xc = coarse.x;
00433     auto xf = fine.x;
00434     const auto policy =
00435         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00436     Kokkos::parallel_for(
00437         "poisson_prolong", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00438           const int ci = i / 2;
00439           const int cj = j / 2;
00440           const int ck = k / 2;
00441           int s0;
00442           int s1;
00443           const int i0 =
00444               detail::extendedIndex((i & 1) ? ci : ci - 1, c0, bl0, bh0, s0);
00445           const int i1 =
00446               detail::extendedIndex((i & 1) ? ci + 1 : ci, c0, bl0, bh0, s1);
00447           const double wi0 = (i & 1) ? 0.75 : 0.25;
00448           const double wi1 = 1.0 - wi0;
00449           const int j0 =
00450               detail::extendedIndex((j & 1) ? cj : cj - 1, c1, bl1, bh1, s0);
00451           const int j1 =
00452               detail::extendedIndex((j & 1) ? cj + 1 : cj, c1, bl1, bh1, s1);
00453           const double wj0 = (j & 1) ? 0.75 : 0.25;
00454           const double wj1 = 1.0 - wj0;
00455           const int k0 =
00456               detail::extendedIndex((k & 1) ? ck : ck - 1, c2, bl2, bh2, s0);
00457           const int k1 =
00458               detail::extendedIndex((k & 1) ? ck + 1 : ck, c2, bl2, bh2, s1);
00459           const double wk0 = (k & 1) ? 0.75 : 0.25;
00460           const double wk1 = 1.0 - wk0;
00461           const double value =
00462               wk0 * (wj0 * (wi0 * xc(k0, j0, i0) + wi1 * xc(k0, j0, i1)) +
00463                      wj1 * (wi0 * xc(k0, j1, i0) + wi1 * xc(k0, j1, i1))) +
00464               wk1 * (wj0 * (wi0 * xc(k1, j0, i0) + wi1 * xc(k1, j0, i1)) +
00465                      wj1 * (wi0 * xc(k1, j1, i0) + wi1 * xc(k1, j1, i1)));
00466           xf(k, j, i) += value;
00467         });
00468   }
00469
00470   double norm(const view_type& v) const {
00471     const int n0 = static_cast<int>(v.extent(2));
00472     const int n1 = static_cast<int>(v.extent(1));
00473     const int n2 = static_cast<int>(v.extent(0));
00474     const long long cells = static_cast<long long>(n0) * n1 * n2;
00475     double sum = 0.0;
00476     const auto policy =
00477         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00478     Kokkos::parallel_reduce(
00479         "poisson_norm", policy,
00480         KOKKOS_LAMBDA(int k, int j, int i, double& acc) {
00481           acc += v(k, j, i) * v(k, j, i);
00482         },
00483         Kokkos::Sum<double>{sum});
00484     return std::sqrt(sum / static_cast<double>(cells));
00485   }
00486
00487   double residualNorm(int level_index) const {
00488     const Level& level = levels_[static_cast<std::size_t>(level_index)];
00489     const long long cells =
00490         static_cast<long long>(level.n[0]) * level.n[1] * level.n[2];
00491     double sum = 0.0;
00492     const auto policy = makeMdrange3<ExecSpace>(
00493         Int3{{0, 0, 0}}, Int3{{level.n[2], level.n[1], level.n[0]}});
00494     auto r = level.r;
00495     computeResidual(level);
00496     Kokkos::parallel_reduce(
00497         "poisson_residual_norm", policy,
00498         KOKKOS_LAMBDA(int k, int j, int i, double& acc) {
00499           acc += r(k, j, i) * r(k, j, i);
00500         },
00501         Kokkos::Sum<double>{sum});
00502     return std::sqrt(sum / static_cast<double>(cells));
00503   }
00504
00505   static void projectMeanImpl(const view_type& v) {
00506     const int n0 = static_cast<int>(v.extent(2));
00507     const int n1 = static_cast<int>(v.extent(1));
00508     const int n2 = static_cast<int>(v.extent(0));
00509     const long long cells = static_cast<long long>(n0) * n1 * n2;
00510     double sum = 0.0;
00511     const auto policy =
00512         makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00513     Kokkos::parallel_reduce(
00514         "poisson_mean", policy,
00515         KOKKOS_LAMBDA(int k, int j, int i, double& acc) { acc += v(k, j, i); },
00516         Kokkos::Sum<double>{sum});
00517     const double mean = sum / static_cast<double>(cells);
```

```
00518     Kokkos::parallel_for(
00519         "poisson_project", policy,
00520         KOKKOS_LAMBDA(int k, int j, int i) { v(k, j, i) -= mean; });
00521   }
00522
00523  private:
00524   std::vector<Level> levels_;
00525   std::array<Boundary, 3> bc_low_{
00526       {Boundary::Periodic, Boundary::Periodic, Boundary::Periodic}};
00527   std::array<Boundary, 3> bc_high_{
00528       {Boundary::Periodic, Boundary::Periodic, Boundary::Periodic}};
00529   bool has_dirichlet_ = false;
00530   bool all_periodic_ = true;
00531 };
00532
00533 }  // namespace poisson
00534 }  // namespace saturn
```

6.64 src/numerics/incompressible/projection.hpp File Reference

MAC velocity layout and discrete divergence/gradient operators.

```
#include <Kokkos_Core.hpp>
#include <array>
#include <cmath>
#include "core/parallel.hpp"
#include "numerics/incompressible/poisson.hpp"
```

Namespaces

- namespace saturn
- namespace saturn::projection

Typedefs

- template<class ExecSpace >
  using saturn::projection::ScalarView

Functions

- template<class ExecSpace >
  void saturn::projection::divergence (const ScalarView< ExecSpace > &u, const ScalarView< ExecSpace > &v, const ScalarView< ExecSpace > &w, ScalarView< ExecSpace > &div, const std::array< double, 3 > &h)
- template<class ExecSpace >
  void saturn::projection::gradientX (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gx, double hx)
- template<class ExecSpace >
  void saturn::projection::gradientY (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gy, double hy)
- template<class ExecSpace >
  void saturn::projection::gradientZ (const ScalarView< ExecSpace > &p, ScalarView< ExecSpace > &gz, double hz)
- template<class ExecSpace , class PoissonSolver >
  poisson::Result saturn::projection::project (ScalarView< ExecSpace > &u, ScalarView< ExecSpace > &v, ScalarView< ExecSpace > &w, ScalarView< ExecSpace > &phi, ScalarView< ExecSpace > &scratch, const std::array< double, 3 > &h, double dt, PoissonSolver &solver, const poisson::Options &options)

6.64.1 Detailed Description

MAC velocity layout and discrete divergence/gradient operators.

6.65 projection.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #include <Kokkos_Core.hpp>
00008
00009 #include <array>
00010 #include <cmath>
00011
00012 #include "core/parallel.hpp"
00013 #include "numerics/incompressible/poisson.hpp"
00014
00015 namespace saturn {
00016 namespace projection {
00017
00018 // MAC (staggered) velocity layout on a periodic box.
00019 //
00020 // Face index i is the x-face on the low side of cell i, so u,v,w all have the
00021 // same extents as the cell-centered pressure. The discrete divergence and
00022 // gradient are exact adjoints:
00023 //
00024 //   div(u)_ijk = (u_{i+1} - u_i)/hx + (v_{j+1} - v_j)/hy + (w_{k+1} - w_k)/hz
00025 //   grad(p)_i  = (p_i - p_{i-1})/hx
00026 //
00027 // and div(grad(p)) is the 7-point Laplacian used by the Poisson solver.
00028
00029 template <class ExecSpace>
00030 using ScalarView = Kokkos::View<double***, Kokkos::LayoutRight,
00031                                 typename ExecSpace::memory_space>;
00032
00033 template <class ExecSpace>
00034 void divergence(const ScalarView<ExecSpace>& u, const ScalarView<ExecSpace>& v,
00035                 const ScalarView<ExecSpace>& w, ScalarView<ExecSpace>& div,
00036                 const std::array<double, 3>& h) {
00037   const int n0 = static_cast<int>(u.extent(2));
00038   const int n1 = static_cast<int>(u.extent(1));
00039   const int n2 = static_cast<int>(u.extent(0));
00040   const double hx = h[0];
00041   const double hy = h[1];
00042   const double hz = h[2];
00043   const auto policy =
00044       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00045   Kokkos::parallel_for(
00046       "projection_divergence", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00047         const int ip = (i + 1 == n0) ? 0 : i + 1;
00048         const int jp = (j + 1 == n1) ? 0 : j + 1;
00049         const int kp = (k + 1 == n2) ? 0 : k + 1;
00050         div(k, j, i) = (u(k, j, ip) - u(k, j, i)) / hx +
00051                        (v(k, jp, i) - v(k, j, i)) / hy +
00052                        (w(kp, j, i) - w(k, j, i)) / hz;
00053       });
00054 }
00055
00056 template <class ExecSpace>
00057 void gradientX(const ScalarView<ExecSpace>& p, ScalarView<ExecSpace>& gx,
00058                double hx) {
00059   const int n0 = static_cast<int>(p.extent(2));
00060   const int n1 = static_cast<int>(p.extent(1));
00061   const int n2 = static_cast<int>(p.extent(0));
00062   const auto policy =
00063       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00064   Kokkos::parallel_for(
00065       "projection_grad_x", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00066         const int im = (i == 0) ? n0 - 1 : i - 1;
00067         gx(k, j, i) = (p(k, j, i) - p(k, j, im)) / hx;
00068       });
00069 }
00070
00071 template <class ExecSpace>
00072 void gradientY(const ScalarView<ExecSpace>& p, ScalarView<ExecSpace>& gy,
00073                double hy) {
00074   const int n0 = static_cast<int>(p.extent(2));
00075   const int n1 = static_cast<int>(p.extent(1));
```

```
00076   const int n2 = static_cast<int>(p.extent(0));
00077   const auto policy =
00078       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00079   Kokkos::parallel_for(
00080       "projection_grad_y", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00081         const int jm = (j == 0) ? n1 - 1 : j - 1;
00082         gy(k, j, i) = (p(k, j, i) - p(k, jm, i)) / hy;
00083       });
00084 }
00085
00086 template <class ExecSpace>
00087 void gradientZ(const ScalarView<ExecSpace>& p, ScalarView<ExecSpace>& gz,
00088                double hz) {
00089   const int n0 = static_cast<int>(p.extent(2));
00090   const int n1 = static_cast<int>(p.extent(1));
00091   const int n2 = static_cast<int>(p.extent(0));
00092   const auto policy =
00093       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00094   Kokkos::parallel_for(
00095       "projection_grad_z", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00096         const int km = (k == 0) ? n2 - 1 : k - 1;
00097         gz(k, j, i) = (p(k, j, i) - p(km, j, i)) / hz;
00098       });
00099 }
00100
00101 // Remove the discrete divergence from a MAC velocity:
00102 //
00103 //   u* = u + dt grad(phi),   laplacian(phi) = -div(u)/dt.
00104 //
00105 // `scratch` holds the right-hand side and is overwritten. `phi` holds the
00106 // pressure-correction potential. The returned Poisson result describes the
00107 // quality of the projection.
00108 template <class ExecSpace, class PoissonSolver>
00109 poisson::Result project(ScalarView<ExecSpace>& u, ScalarView<ExecSpace>& v,
00110                         ScalarView<ExecSpace>& w, ScalarView<ExecSpace>& phi,
00111                         ScalarView<ExecSpace>& scratch,
00112                         const std::array<double, 3>& h, double dt,
00113                         PoissonSolver& solver,
00114                         const poisson::Options& options) {
00115   divergence<ExecSpace>(u, v, w, scratch, h);
00116   const int n0 = static_cast<int>(u.extent(2));
00117   const int n1 = static_cast<int>(u.extent(1));
00118   const int n2 = static_cast<int>(u.extent(0));
00119   const double hx = h[0];
00120   const double hy = h[1];
00121   const double hz = h[2];
00122   const auto policy =
00123       makeMdrange3<ExecSpace>(Int3{{0, 0, 0}}, Int3{{n2, n1, n0}});
00124   Kokkos::parallel_for(
00125       "projection_rhs", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00126         scratch(k, j, i) = -scratch(k, j, i) / dt;
00127       });
00128   const poisson::Result result = solver.solve(phi, scratch, options);
00129
00130   Kokkos::parallel_for(
00131       "projection_update", policy, KOKKOS_LAMBDA(int k, int j, int i) {
00132         const int im = (i == 0) ? n0 - 1 : i - 1;
00133         const int jm = (j == 0) ? n1 - 1 : j - 1;
00134         const int km = (k == 0) ? n2 - 1 : k - 1;
00135         u(k, j, i) += dt * (phi(k, j, i) - phi(k, j, im)) / hx;
00136         v(k, j, i) += dt * (phi(k, j, i) - phi(k, jm, i)) / hy;
00137         w(k, j, i) += dt * (phi(k, j, i) - phi(km, j, i)) / hz;
00138       });
00139   return result;
00140 }
00141
00142 }  // namespace projection
00143 }  // namespace saturn
```

6.66 src/numerics/multiphase/dambreak2d.hpp File Reference

Wall-bounded two-phase dam-break solver.

6.66.1 Detailed Description

Wall-bounded two-phase dam-break solver.

6.67 dambreak2d.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #ifdef SATURN_HAVE_FFTW
00008
00009 #include "numerics/multiphase/twophase2d.hpp"
00010 #include "numerics/vof/vof2d.hpp"
00011
00012 #include <algorithm>
00013 #include <cmath>
00014 #include <vector>
00015
00016 namespace saturn {
00017 namespace twophase {
00018
00019 // Wall-bounded two-phase solver prototype.
00020 //
00021 // Closed box, no-slip/no-penetration walls, gravity in -y, variable density
00022 // rho(c) and viscosity mu(c), wall-aware PLIC transport, balanced CSF, and a
00023 // face-staggered conservative momentum update. The volume fraction and
00024 // momentum fluxes are built from the same PLIC face fluxes, so the discrete
00025 // density and momentum updates are consistent. The pressure projection uses
00026 // the variable-coefficient Neumann PCG solver. The scheme is first order in
00027 // time; it is a validation prototype, not yet a production solver.
00028 class DamBreak2D {
00029  public:
00030   DamBreak2D(int nx, int ny, double lx = 1.0, double ly = 1.0)
00031       : nx_(nx),
00032         ny_(ny),
00033         hx_(lx / nx),
00034         hy_(ly / ny),
00035         c_(static_cast<std::size_t>(nx) * ny, 0.0),
00036         u_(static_cast<std::size_t>(nx) * ny, 0.0),
00037         v_(static_cast<std::size_t>(nx) * ny, 0.0),
00038         p_(static_cast<std::size_t>(nx) * ny, 0.0) {}
00039
00040   int nx() const { return nx_; }
00041   int ny() const { return ny_; }
00042   double hx() const { return hx_; }
00043   double hy() const { return hy_; }
00044   std::vector<double>& c() { return c_; }
00045   const std::vector<double>& c() const { return c_; }
00046   std::vector<double>& u() { return u_; }
00047   std::vector<double>& v() { return v_; }
00048   const std::vector<double>& p() const { return p_; }
00049
00050   double mass() const {
00051     double sum = 0.0;
00052     for (double value : c_) {
00053       sum += value;
00054     }
00055     return sum * hx_ * hy_;
00056   }
00057
00058   void step(double dt, double rho_l, double rho_g, double mu, double gy,
00059             bool project = true, double sigma = 0.0) {
00060     step(dt, rho_l, rho_g, mu, mu, gy, project, sigma);
00061   }
00062
00063   // Face-staggered conservative step. c and the momentum use the same PLIC
00064   // mass fluxes; momentum lives on the MAC faces, so no cell-centred
00065   // velocity re-averaging is needed before the projection.
00066   void step(double dt, double rho_l, double rho_g, double mu_l, double mu_g,
00067             double gy, bool project, double sigma) {
00068     const std::size_t total = static_cast<std::size_t>(nx_) * ny_;
00069     std::vector<double> rho_old(total, 0.0);
00070     for (std::size_t q = 0; q < total; ++q) {
00071       rho_old[q] = rho_l * c_[q] + rho_g * (1.0 - c_[q]);
00072     }
00073
00074     std::vector<double> vof_x;
00075     std::vector<double> vof_y;
00076     plicFluxX(dt, vof_x);
00077     plicFluxY(dt, vof_y);
00078
00079     for (int j = 0; j < ny_; ++j) {
00080       for (int i = 0; i < nx_; ++i) {
00081         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00082         const double right = (i + 1 == nx_) ? 0.0 : vof_x[index + 1];
00083         const double high = (j + 1 == ny_) ? 0.0 : vof_y[index + nx_];
00084         c_[index] -= (right - vof_x[index]) / (hx_ * hy_) +
00085                      (high - vof_y[index]) / (hx_ * hy_);
```

```
00086       }
00087     }
00088     restoreBounds();
00089
00090     std::vector<double> rho_new(total, 0.0);
00091     std::vector<double> mu_cell(total, 0.0);
00092     for (std::size_t q = 0; q < total; ++q) {
00093       rho_new[q] = rho_l * c_[q] + rho_g * (1.0 - c_[q]);
00094       mu_cell[q] = mu_l * c_[q] + mu_g * (1.0 - c_[q]);
00095     }
00096
00097     std::vector<double> rho_u_old(total, 0.0);
00098     std::vector<double> rho_v_old(total, 0.0);
00099     std::vector<double> rho_u_new(total, 0.0);
00100     std::vector<double> rho_v_new(total, 0.0);
00101     std::vector<double> m_u(total, 0.0);
00102     std::vector<double> m_v(total, 0.0);
00103     std::vector<double> mass_x(total, 0.0);
00104     std::vector<double> mass_y(total, 0.0);
00105     for (int j = 0; j < ny_; ++j) {
00106       for (int i = 0; i < nx_; ++i) {
00107         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00108         rho_u_old[index] = (i == 0)
00109                                ? rho_old[index]
00110                                : 0.5 * (rho_old[index - 1] + rho_old[index]);
00111         rho_v_old[index] = (j == 0)
00112                                ? rho_old[index]
00113                                : 0.5 * (rho_old[index - nx_] + rho_old[index]);
00114         rho_u_new[index] = (i == 0)
00115                                ? rho_new[index]
00116                                : 0.5 * (rho_new[index - 1] + rho_new[index]);
00117         rho_v_new[index] = (j == 0)
00118                                ? rho_new[index]
00119                                : 0.5 * (rho_new[index - nx_] + rho_new[index]);
00120         m_u[index] = rho_u_old[index] * u_[index];
00121         m_v[index] = rho_v_old[index] * v_[index];
00122         if (i > 0) {
00123           mass_x[index] = rho_l * vof_x[index] +
00124                           rho_g * (u_[index] * dt * hy_ - vof_x[index]);
00125         }
00126         if (j > 0) {
00127           mass_y[index] = rho_l * vof_y[index] +
00128                           rho_g * (v_[index] * dt * hx_ - vof_y[index]);
00129         }
00130       }
00131     }
00132
00133     std::vector<double> bx_new(total, 0.0);
00134     std::vector<double> by_new(total, 0.0);
00135     for (int j = 0; j < ny_; ++j) {
00136       for (int i = 0; i < nx_; ++i) {
00137         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00138         if (i > 0) {
00139           const double rl = rho_new[index - 1];
00140           bx_new[index] = (rl + rho_new[index]) / (2.0 * rl * rho_new[index]);
00141         }
00142         if (j > 0) {
00143           const double rd = rho_new[index - nx_];
00144           by_new[index] = (rd + rho_new[index]) / (2.0 * rd * rho_new[index]);
00145         }
00146       }
00147     }
00148
00149     // u-momentum on x-faces.
00150     std::vector<double> u_new(total, 0.0);
00151     for (int j = 0; j < ny_; ++j) {
00152       for (int i = 1; i < nx_; ++i) {
00153         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00154         const double mass_left =
00155             0.5 * (((i > 1) ? mass_x[index - 1] : 0.0) + mass_x[index]);
00156         const double mass_right =
00157             0.5 * (mass_x[index] + ((i + 1 < nx_) ? mass_x[index + 1] : 0.0));
00158         const double u_left_boundary =
00159             0.5 * (((i > 1) ? u_[index - 1] : 0.0) + u_[index]);
00160         const double u_right_boundary =
00161             0.5 * (u_[index] + ((i + 1 < nx_) ? u_[index + 1] : 0.0));
00162         const double f_left =
00163             mass_left * (u_left_boundary > 0.0 ? u_[index - 1] : u_[index]);
00164         const double f_right =
00165             mass_right * (u_right_boundary > 0.0
00166                               ? u_[index]
00167                               : ((i + 1 < nx_) ? u_[index + 1] : 0.0));
00168         const double v_bottom = 0.5 * (v_[index - 1] + v_[index]);
00169         const double v_top =
00170             0.5 * (((j + 1 < ny_) ? v_[index - 1 + nx_] : 0.0) +
00171                    ((j + 1 < ny_) ? v_[index + nx_] : 0.0));
00172         const double mass_bottom = 0.5 * (((j > 0) ? mass_y[index - 1] : 0.0) +
```

```
00173                                       ((j > 0) ? mass_y[index] : 0.0));
00174       const double mass_top =
00175           0.5 * (((j + 1 < ny_) ? mass_y[index - 1 + nx_] : 0.0) +
00176                  ((j + 1 < ny_) ? mass_y[index + nx_] : 0.0));
00177       const double g_bottom =
00178           mass_bottom *
00179           (v_bottom > 0.0 ? ((j > 0) ? u_[index - nx_] : 0.0) : u_[index]);
00180       const double g_top =
00181           mass_top *
00182           (v_top > 0.0 ? u_[index] : ((j + 1 < ny_) ? u_[index + nx_] : 0.0));
00183       const double u_left = (i > 1) ? u_[index - 1] : 0.0;
00184       const double u_right = (i + 1 < nx_) ? u_[index + 1] : 0.0;
00185       const double u_low = (j > 0) ? u_[index - nx_] : -u_[index];
00186       const double u_high = (j + 1 < ny_) ? u_[index + nx_] : -u_[index];
00187       const double lap = (u_right - 2.0 * u_[index] + u_left) / (hx_ * hx_) +
00188                          (u_high - 2.0 * u_[index] + u_low) / (hy_ * hy_);
00189       const double mu_face = 0.5 * (mu_cell[index - 1] + mu_cell[index]);
00190       const double m_new =
00191           m_u[index] -
00192           ((f_right - f_left) + (g_top - g_bottom)) / (hx_ * hy_);
00193       u_new[index] =
00194           m_new / rho_u_new[index] + dt * (mu_face * bx_new[index] * lap);
00195     }
00196     u_new[static_cast<std::size_t>(j) * nx_] = 0.0;
00197   }
00198
00199   // v-momentum on y-faces.
00200   std::vector<double> v_new(total, 0.0);
00201   for (int j = 1; j < ny_; ++j) {
00202     for (int i = 0; i < nx_; ++i) {
00203       const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00204       const double mass_left = mass_x[index];
00205       const double mass_right = (i + 1 < nx_) ? mass_x[index + 1] : 0.0;
00206       const double f_left =
00207           mass_left *
00208           (u_[index] > 0.0 ? ((i > 0) ? v_[index - 1] : 0.0) : v_[index]);
00209       const double f_right =
00210           mass_right * (((i + 1 < nx_) && u_[index + 1] > 0.0)
00211                             ? v_[index]
00212                             : ((i + 1 < nx_) ? v_[index + 1] : 0.0));
00213       const double mass_bottom =
00214           0.5 * (((j > 1) ? mass_y[index - nx_] : 0.0) + mass_y[index]);
00215       const double mass_top =
00216           0.5 * (mass_y[index] + ((j + 1 < ny_) ? mass_y[index + nx_] : 0.0));
00217       const double v_bottom =
00218           0.5 * (((j > 1) ? v_[index - nx_] : 0.0) + v_[index]);
00219       const double v_top =
00220           0.5 * (v_[index] + ((j + 1 < ny_) ? v_[index + nx_] : 0.0));
00221       const double g_bottom =
00222           mass_bottom *
00223           (v_bottom > 0.0 ? ((j > 1) ? v_[index - nx_] : 0.0) : v_[index]);
00224       const double g_top =
00225           mass_top *
00226           (v_top > 0.0 ? v_[index] : ((j + 1 < ny_) ? v_[index + nx_] : 0.0));
00227       const double v_low = (j > 1) ? v_[index - nx_] : 0.0;
00228       const double v_high = (j + 1 < ny_) ? v_[index + nx_] : 0.0;
00229       const double v_left = (i > 0) ? v_[index - 1] : 0.0;
00230       const double v_right = (i + 1 < nx_) ? v_[index + 1] : 0.0;
00231       const double lap = (v_right - 2.0 * v_[index] + v_left) / (hx_ * hx_) +
00232                          (v_high - 2.0 * v_[index] + v_low) / (hy_ * hy_);
00233       const double mu_face = 0.5 * (mu_cell[index - nx_] + mu_cell[index]);
00234       const double m_new =
00235           m_v[index] -
00236           ((f_right - f_left) + (g_top - g_bottom)) / (hx_ * hy_);
00237       v_new[index] = m_new / rho_v_new[index] +
00238                      dt * (mu_face * by_new[index] * lap + gy);
00239     }
00240   }
00241
00242   // CSF acts on faces.
00243   std::vector<double> csf_x;
00244   std::vector<double> csf_y;
00245   csfForces(sigma, csf_x, csf_y);
00246   for (int j = 0; j < ny_; ++j) {
00247     for (int i = 1; i < nx_; ++i) {
00248       const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00249       u_new[index] += dt * bx_new[index] * csf_x[index];
00250     }
00251   }
00252   for (int j = 1; j < ny_; ++j) {
00253     for (int i = 0; i < nx_; ++i) {
00254       const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00255       v_new[index] += dt * by_new[index] * csf_y[index];
00256     }
00257   }
00258
00259   u_ = u_new;
```

```
00260     v_ = v_new;
00261
00262     if (project) {
00263       std::vector<double> div(total, 0.0);
00264       for (int j = 0; j < ny_; ++j) {
00265         for (int i = 0; i < nx_; ++i) {
00266           const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00267           const double u_right = (i + 1 == nx_) ? 0.0 : u_[index + 1];
00268           const double v_high = (j + 1 == ny_) ? 0.0 : v_[index + nx_];
00269           div[index] = (u_right - u_[index]) / hx_ + (v_high - v_[index]) / hy_;
00270         }
00271       }
00272       for (double& value : div) {
00273         value /= dt;
00274       }
00275       VariablePoisson2D solver(nx_, ny_, hx_, hy_, bx_new, by_new, false,
00276                                false);
00277       solver.solve(div, p_);
00278       for (int j = 0; j < ny_; ++j) {
00279         for (int i = 1; i < nx_; ++i) {
00280           const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00281           u_[index] -= dt * bx_new[index] * (p_[index] - p_[index - 1]) / hx_;
00282         }
00283         u_[static_cast<std::size_t>(j) * nx_] = 0.0;
00284       }
00285       for (int j = 1; j < ny_; ++j) {
00286         for (int i = 0; i < nx_; ++i) {
00287           const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00288           v_[index] -= dt * by_new[index] * (p_[index] - p_[index - nx_]) / hy_;
00289         }
00290       }
00291       for (int i = 0; i < nx_; ++i) {
00292         v_[i] = 0.0;
00293       }
00294     }
00295   }
00296
00297   // Wall-aware PLIC advection: Strang split x/y with geometric face fluxes.
00298   void advectPlic(double dt) {
00299     sweepXPlic(0.5 * dt);
00300     sweepYPlic(dt);
00301     sweepXPlic(0.5 * dt);
00302     restoreBounds();
00303   }
00304
00305  private:
00306   std::size_t cellIndex(int i, int j) const {
00307     const int ii = std::min(std::max(i, 0), nx_ - 1);
00308     const int jj = std::min(std::max(j, 0), ny_ - 1);
00309     return static_cast<std::size_t>(jj) * nx_ + ii;
00310   }
00311
00312   // Balanced CSF force on faces: sigma * kappa * grad(c), with a smoothed
00313   // volume fraction and mirror (zero-normal-gradient) ghosts at the walls.
00314   void csfForces(double sigma, std::vector<double>& fx,
00315                  std::vector<double>& fy) const {
00316     const std::size_t total = static_cast<std::size_t>(nx_) * ny_;
00317     fx.assign(total, 0.0);
00318     fy.assign(total, 0.0);
00319     if (sigma <= 0.0) {
00320       return;
00321     }
00322     std::vector<double> smoothed = c_;
00323     for (int pass = 0; pass < 8; ++pass) {
00324       std::vector<double> buffer(smoothed);
00325       for (int j = 0; j < ny_; ++j) {
00326         for (int i = 0; i < nx_; ++i) {
00327           buffer[cellIndex(i, j)] = 0.25 * smoothed[cellIndex(i - 1, j)] +
00328                                     0.5 * smoothed[cellIndex(i, j)] +
00329                                     0.25 * smoothed[cellIndex(i + 1, j)];
00330         }
00331       }
00332       std::vector<double> next(buffer);
00333       for (int j = 0; j < ny_; ++j) {
00334         for (int i = 0; i < nx_; ++i) {
00335           next[cellIndex(i, j)] = 0.25 * buffer[cellIndex(i, j - 1)] +
00336                                   0.5 * buffer[cellIndex(i, j)] +
00337                                   0.25 * buffer[cellIndex(i, j + 1)];
00338         }
00339       }
00340       smoothed = next;
00341     }
00342     auto grad = [&](int i, int j, double& gx, double& gy) {
00343       gx = 0.5 *
00344            (smoothed[cellIndex(i + 1, j)] - smoothed[cellIndex(i - 1, j)]) /
00345            hx_;
00346       gy = 0.5 *
```

```
00347           (smoothed[cellIndex(i, j + 1)] - smoothed[cellIndex(i, j - 1)]) /
00348           hy_;
00349     };
00350     auto normalX = [&](int i, int j) {
00351       double gx = 0.0;
00352       double gy = 0.0;
00353       grad(i, j, gx, gy);
00354       const double norm = std::sqrt(gx * gx + gy * gy);
00355       return norm > 1.0e-10 ? gx / norm : 0.0;
00356     };
00357     auto normalY = [&](int i, int j) {
00358       double gx = 0.0;
00359       double gy = 0.0;
00360       grad(i, j, gx, gy);
00361       const double norm = std::sqrt(gx * gx + gy * gy);
00362       return norm > 1.0e-10 ? gy / norm : 0.0;
00363     };
00364     std::vector<double> kappa(total, 0.0);
00365     for (int j = 0; j < ny_; ++j) {
00366       for (int i = 0; i < nx_; ++i) {
00367         const double div = 0.5 * (normalX(i + 1, j) - normalX(i - 1, j)) / hx_ +
00368                            0.5 * (normalY(i, j + 1) - normalY(i, j - 1)) / hy_;
00369         kappa[cellIndex(i, j)] = -div;
00370       }
00371     }
00372     for (int j = 0; j < ny_; ++j) {
00373       for (int i = 1; i < nx_; ++i) {
00374         const std::size_t index = cellIndex(i, j);
00375         const std::size_t left = cellIndex(i - 1, j);
00376         const double kx = 0.5 * (kappa[index] + kappa[left]);
00377         fx[index] = sigma * kx * (c_[index] - c_[left]) / hx_;
00378       }
00379     }
00380     for (int j = 1; j < ny_; ++j) {
00381       for (int i = 0; i < nx_; ++i) {
00382         const std::size_t index = cellIndex(i, j);
00383         const std::size_t low = cellIndex(i, j - 1);
00384         const double ky = 0.5 * (kappa[index] + kappa[low]);
00385         fy[index] = sigma * ky * (c_[index] - c_[low]) / hy_;
00386       }
00387     }
00388   }
00389
00390   // Strang splitting can leave small overshoots at sharp interfaces even for a
00391   // discretely divergence-free velocity. Clamp and redistribute the resulting
00392   // mass error over available capacity so the update stays conservative and
00393   // bounded (the same construction as vof::redistribute, for a raw vector).
00394   void restoreBounds(int sweeps = 8) {
00395     const double cellVolume = hx_ * hy_;
00396     double target = 0.0;
00397     for (double value : c_) {
00398       target += value * cellVolume;
00399     }
00400     for (int sweep = 0; sweep < sweeps; ++sweep) {
00401       for (double& value : c_) {
00402         value = std::min(1.0, std::max(0.0, value));
00403       }
00404       double current = 0.0;
00405       for (double value : c_) {
00406         current += value * cellVolume;
00407       }
00408       const double delta = target - current;
00409       if (std::abs(delta) <= 1.0e-14 * std::max(1.0, std::abs(target))) {
00410         break;
00411       }
00412       double capacity = 0.0;
00413       for (double value : c_) {
00414         const bool eligible = (delta > 0.0) ? (value > 0.0) : (value < 1.0);
00415         if (eligible) {
00416           capacity += (delta > 0.0) ? (1.0 - value) : value;
00417         }
00418       }
00419       if (capacity * cellVolume < std::abs(delta)) {
00420         // Interface cells cannot absorb the correction; fall back to the whole
00421         // domain.
00422         capacity = 0.0;
00423         for (double value : c_) {
00424           capacity += (delta > 0.0) ? (1.0 - value) : value;
00425         }
00426       }
00427       if (capacity * cellVolume <= 0.0) {
00428         break;
00429       }
00430       const double weight = delta / (capacity * cellVolume);
00431       for (double& value : c_) {
00432         const bool eligible = (delta > 0.0) ? (value > 0.0) : (value < 1.0);
00433         if (!eligible) {
```

```
00434           continue;
00435         }
00436         const double available = (delta > 0.0) ? (1.0 - value) : value;
00437         value += weight * available;
00438         value = std::min(1.0, std::max(0.0, value));
00439       }
00440     }
00441   }
00442
00443   double cval(int i, int j) const {
00444     const int ii = std::min(std::max(i, 0), nx_ - 1);
00445     const int jj = std::min(std::max(j, 0), ny_ - 1);
00446     return c_[static_cast<std::size_t>(jj) * nx_ + ii];
00447   }
00448
00449   // Geometric PLIC flux of fluid 1 through each x-face (indexed like u_).
00450   void plicFluxX(double dt, std::vector<double>& flux) const {
00451     const auto rec = wallReconstructions();
00452     flux.assign(c_.size(), 0.0);
00453     for (int j = 0; j < ny_; ++j) {
00454       for (int i = 1; i < nx_; ++i) {
00455         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00456         const double vel = u_[index];
00457         if (vel == 0.0) {
00458           continue;
00459         }
00460         const int up = vel > 0.0 ? i - 1 : i;
00461         const auto& r = rec[static_cast<std::size_t>(j) * nx_ + up];
00462         const double width = std::min(std::abs(vel) * dt / hx_, 1.0);
00463         const double x0 = vel > 0.0 ? 1.0 - width : 0.0;
00464         const double x1 = vel > 0.0 ? 1.0 : width;
00465         std::vector<vof::Point> poly = vof::fluidPolygon(r);
00466         poly = vof::clipAxis(poly, 0, x0, -1.0);
00467         poly = vof::clipAxis(poly, 0, x1, 1.0);
00468         flux[index] =
00469             (vel > 0.0 ? 1.0 : -1.0) * vof::polygonArea(poly) * hx_ * hy_;
00470       }
00471     }
00472   }
00473
00474   // Geometric PLIC flux of fluid 1 through each y-face (indexed like v_).
00475   void plicFluxY(double dt, std::vector<double>& flux) const {
00476     const auto rec = wallReconstructions();
00477     flux.assign(c_.size(), 0.0);
00478     for (int j = 1; j < ny_; ++j) {
00479       for (int i = 0; i < nx_; ++i) {
00480         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00481         const double vel = v_[index];
00482         if (vel == 0.0) {
00483           continue;
00484         }
00485         const int up = vel > 0.0 ? j - 1 : j;
00486         const auto& r = rec[static_cast<std::size_t>(up) * nx_ + i];
00487         const double width = std::min(std::abs(vel) * dt / hy_, 1.0);
00488         const double y0 = vel > 0.0 ? 1.0 - width : 0.0;
00489         const double y1 = vel > 0.0 ? 1.0 : width;
00490         std::vector<vof::Point> poly = vof::fluidPolygon(r);
00491         poly = vof::clipAxis(poly, 1, y0, -1.0);
00492         poly = vof::clipAxis(poly, 1, y1, 1.0);
00493         flux[index] =
00494             (vel > 0.0 ? 1.0 : -1.0) * vof::polygonArea(poly) * hx_ * hy_;
00495       }
00496     }
00497   }
00498
00499   std::vector<vof::Reconstruction> wallReconstructions() const {
00500     std::vector<vof::Reconstruction> rec(static_cast<std::size_t>(nx_) * ny_);
00501     for (int j = 0; j < ny_; ++j) {
00502       for (int i = 0; i < nx_; ++i) {
00503         const double nx = -0.5 * (cval(i + 1, j) - cval(i - 1, j));
00504         const double ny = -0.5 * (cval(i, j + 1) - cval(i, j - 1));
00505         rec[static_cast<std::size_t>(j) * nx_ + i] =
00506             vof::reconstruct(c_[static_cast<std::size_t>(j) * nx_ + i], nx, ny);
00507       }
00508     }
00509     return rec;
00510   }
00511
00512   void sweepXPlic(double dt) {
00513     const auto rec = wallReconstructions();
00514     std::vector<double> flux(c_.size(), 0.0);
00515     for (int j = 0; j < ny_; ++j) {
00516       for (int i = 1; i < nx_; ++i) {
00517         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00518         const double vel = u_[index];
00519         if (vel == 0.0) {
00520           continue;
```

```
00521         }
00522         const int up = vel > 0.0 ? i - 1 : i;
00523         const auto& r = rec[static_cast<std::size_t>(j) * nx_ + up];
00524         const double width = std::min(std::abs(vel) * dt / hx_, 1.0);
00525         const double x0 = vel > 0.0 ? 1.0 - width : 0.0;
00526         const double x1 = vel > 0.0 ? 1.0 : width;
00527         std::vector<vof::Point> poly = vof::fluidPolygon(r);
00528         poly = vof::clipAxis(poly, 0, x0, -1.0);
00529         poly = vof::clipAxis(poly, 0, x1, 1.0);
00530         flux[index] =
00531             (vel > 0.0 ? 1.0 : -1.0) * vof::polygonArea(poly) * hx_ * hy_;
00532       }
00533     }
00534     for (int j = 0; j < ny_; ++j) {
00535       for (int i = 0; i < nx_; ++i) {
00536         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00537         const double left = flux[index];
00538         const double right = (i + 1 == nx_) ? 0.0 : flux[index + 1];
00539         c_[index] -= (right - left) / (hx_ * hy_);
00540       }
00541     }
00542   }
00543
00544   void sweepYPlic(double dt) {
00545     const auto rec = wallReconstructions();
00546     std::vector<double> flux(c_.size(), 0.0);
00547     for (int j = 1; j < ny_; ++j) {
00548       for (int i = 0; i < nx_; ++i) {
00549         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00550         const double vel = v_[index];
00551         if (vel == 0.0) {
00552           continue;
00553         }
00554         const int up = vel > 0.0 ? j - 1 : j;
00555         const auto& r = rec[static_cast<std::size_t>(up) * nx_ + i];
00556         const double width = std::min(std::abs(vel) * dt / hy_, 1.0);
00557         const double y0 = vel > 0.0 ? 1.0 - width : 0.0;
00558         const double y1 = vel > 0.0 ? 1.0 : width;
00559         std::vector<vof::Point> poly = vof::fluidPolygon(r);
00560         poly = vof::clipAxis(poly, 1, y0, -1.0);
00561         poly = vof::clipAxis(poly, 1, y1, 1.0);
00562         flux[index] =
00563             (vel > 0.0 ? 1.0 : -1.0) * vof::polygonArea(poly) * hx_ * hy_;
00564       }
00565     }
00566     for (int i = 0; i < nx_; ++i) {
00567       for (int j = 0; j < ny_; ++j) {
00568         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00569         const double low = flux[index];
00570         const double high = (j + 1 == ny_) ? 0.0 : flux[index + nx_];
00571         c_[index] -= (high - low) / (hx_ * hy_);
00572       }
00573     }
00574   }
00575
00576  public:
00577   double maxSpeed() const {
00578     double result = 0.0;
00579     for (double x : u_) {
00580       result = std::max(result, std::abs(x));
00581     }
00582     for (double x : v_) {
00583       result = std::max(result, std::abs(x));
00584     }
00585     return result;
00586   }
00587
00588  private:
00589   int nx_;
00590   int ny_;
00591   double hx_;
00592   double hy_;
00593   std::vector<double> c_;
00594   std::vector<double> u_;
00595   std::vector<double> v_;
00596   std::vector<double> p_;
00597 };
00598
00599 }  // namespace twophase
00600 }  // namespace saturn
00601
00602 #endif  // SATURN_HAVE_FFTW
```

6.68 src/numerics/multiphase/twophase2d.hpp File Reference

6.69 twophase2d.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #ifdef SATURN_HAVE_FFTW
00004
00005 #include <fftw3.h>
00006
00007 #include <algorithm>
00008 #include <cmath>
00009 #include <complex>
00010 #include <vector>
00011
00012 #include "numerics/vof/vof2d.hpp"
00013
00014 namespace saturn {
00015 namespace twophase {
00016
00017 // Two-dimensional periodic MAC projection solver with VOF and a balanced
00018 // CSF surface-tension force. Constant density in this prototype.
00019 //
00020 //   u(i,j): x-face at x = i hx, cell-centered in y
00021 //   v(i,j): y-face at y = j hy, cell-centered in x
00022 //   p(i,j): cell center
00023 //
00024 // The CSF force is evaluated on faces from the jump in c,
00025 //   f_x = sigma kappa_face (c_R - c_L)/hx,
00026 // which is a discrete gradient of sigma*kappa*c when kappa is constant. The
00027 // projection then balances it with pressure and leaves a static bubble with
00028 // zero velocity up to solver tolerance (balanced-force CSF).
00029
00030 class FftPoisson2D {
00031  public:
00032   FftPoisson2D(int nx, int ny, double hx, double hy)
00033       : nx_(nx), ny_(ny), hx_(hx), hy_(hy) {
00034     in_ = fftw_alloc_complex(static_cast<std::size_t>(nx) * ny);
00035     out_ = fftw_alloc_complex(static_cast<std::size_t>(nx) * ny);
00036     forward_ = fftw_plan_dft_2d(ny, nx, in_, out_, FFTW_FORWARD, FFTW_ESTIMATE);
00037     inverse_ =
00038         fftw_plan_dft_2d(ny, nx, in_, out_, FFTW_BACKWARD, FFTW_ESTIMATE);
00039   }
00040   ~FftPoisson2D() {
00041     if (forward_ != nullptr) {
00042       fftw_destroy_plan(forward_);
00043     }
00044     if (inverse_ != nullptr) {
00045       fftw_destroy_plan(inverse_);
00046     }
00047     if (in_ != nullptr) {
00048       fftw_free(in_);
00049     }
00050     if (out_ != nullptr) {
00051       fftw_free(out_);
00052     }
00053   }
00054   FftPoisson2D(const FftPoisson2D&) = delete;
00055   FftPoisson2D& operator=(const FftPoisson2D&) = delete;
00056
00057   // Solve laplacian(p) = rhs with zero mean; periodic boundaries.
00058   void solve(const std::vector<double>& rhs, std::vector<double>& p) {
00059     const int total = nx_ * ny_;
00060     for (int j = 0; j < ny_; ++j) {
00061       for (int i = 0; i < nx_; ++i) {
00062         const int index = j * nx_ + i;
00063         in_[index][0] = rhs[static_cast<std::size_t>(index)];
00064         in_[index][1] = 0.0;
00065       }
00066     }
00067     fftw_execute_dft(forward_, in_, out_);
00068     for (int j = 0; j < ny_; ++j) {
00069       const double ky = 2.0 * std::sin(kPi * j / ny_) / hy_;
00070       for (int i = 0; i < nx_; ++i) {
00071         const double kx = 2.0 * std::sin(kPi * i / nx_) / hx_;
00072         const int index = j * nx_ + i;
00073         const double lambda = -(kx * kx + ky * ky);
00074         double re = 0.0;
00075         double im = 0.0;
00076         if (lambda != 0.0) {
00077           re = out_[index][0] / lambda;
```

```
00078           im = out_[index][1] / lambda;
00079         }
00080         in_[index][0] = re;
00081         in_[index][1] = im;
00082       }
00083     }
00084     fftw_execute_dft(inverse_, in_, out_);
00085     p.assign(static_cast<std::size_t>(total), 0.0);
00086     const double scale = 1.0 / static_cast<double>(total);
00087     for (int j = 0; j < ny_; ++j) {
00088       for (int i = 0; i < nx_; ++i) {
00089         const int index = j * nx_ + i;
00090         p[static_cast<std::size_t>(index)] = out_[index][0] * scale;
00091       }
00092     }
00093   }
00094
00095  private:
00096   static constexpr double kPi = 3.14159265358979323846;
00097   int nx_;
00098   int ny_;
00099   double hx_;
00100   double hy_;
00101   fftw_complex* in_ = nullptr;
00102   fftw_complex* out_ = nullptr;
00103   fftw_plan forward_ = nullptr;
00104   fftw_plan inverse_ = nullptr;
00105 };
00106
00107 // Variable-coefficient periodic Poisson solver:
00108 //
00109 //   div(beta grad(p)) = rhs,   beta = 1/rho at faces.
00110 //
00111 // Matrix-free PCG with a Jacobi preconditioner. Used for the variable-density
00112 // projection.
00113 class VariablePoisson2D {
00114  public:
00115   VariablePoisson2D(int nx, int ny, double hx, double hy,
00116                     const std::vector<double>& beta_x,
00117                     const std::vector<double>& beta_y, bool periodic_x = true,
00118                     bool periodic_y = true)
00119       : nx_(nx),
00120         ny_(ny),
00121         hx_(hx),
00122         hy_(hy),
00123         bx_(beta_x),
00124         by_(beta_y),
00125         periodic_x_(periodic_x),
00126         periodic_y_(periodic_y) {}
00127
00128   void apply(const std::vector<double>& p, std::vector<double>& out) const {
00129     for (int j = 0; j < ny_; ++j) {
00130       for (int i = 0; i < nx_; ++i) {
00131         const int im = (i == 0) ? nx_ - 1 : i - 1;
00132         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00133         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00134         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00135         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00136         const bool wall_left = !periodic_x_ && i == 0;
00137         const bool wall_right = !periodic_x_ && i == nx_ - 1;
00138         const bool wall_low = !periodic_y_ && j == 0;
00139         const bool wall_high = !periodic_y_ && j == ny_ - 1;
00140         const double b_left =
00141             wall_left ? 0.0 : bx_[static_cast<std::size_t>(j) * nx_ + i];
00142         const double b_right =
00143             wall_right ? 0.0 : bx_[static_cast<std::size_t>(j) * nx_ + ip];
00144         const double b_low =
00145             wall_low ? 0.0 : by_[static_cast<std::size_t>(j) * nx_ + i];
00146         const double b_high =
00147             wall_high ? 0.0 : by_[static_cast<std::size_t>(jp) * nx_ + i];
00148         const double xflux =
00149             b_right * (p[static_cast<std::size_t>(j) * nx_ + ip] - p[index]) -
00150             b_left * (p[index] - p[static_cast<std::size_t>(j) * nx_ + im]);
00151         const double yflux =
00152             b_high * (p[static_cast<std::size_t>(jp) * nx_ + i] - p[index]) -
00153             b_low * (p[index] - p[static_cast<std::size_t>(jm) * nx_ + i]);
00154         out[index] = xflux / (hx_ * hx_) + yflux / (hy_ * hy_);
00155       }
00156     }
00157   }
00158
00159   void solve(const std::vector<double>& rhs, std::vector<double>& p,
00160              double tolerance = 1.0e-10, int max_iterations = 500) const {
00161     const std::size_t total = static_cast<std::size_t>(nx_) * ny_;
00162     p.assign(total, 0.0);
00163     double mean = 0.0;
00164     for (double value : rhs) {
```

```
00165       mean += value;
00166     }
00167     mean /= static_cast<double>(total);
00168     std::vector<double> r = rhs;
00169     for (double& value : r) {
00170       value -= mean;
00171     }
00172     std::vector<double> diag(total, 0.0);
00173     for (int j = 0; j < ny_; ++j) {
00174       for (int i = 0; i < nx_; ++i) {
00175         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00176         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00177         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00178         const bool wall_left = !periodic_x_ && i == 0;
00179         const bool wall_right = !periodic_x_ && i == nx_ - 1;
00180         const bool wall_low = !periodic_y_ && j == 0;
00181         const bool wall_high = !periodic_y_ && j == ny_ - 1;
00182         const double b_left = wall_left ? 0.0 : bx_[index];
00183         const double b_right =
00184             wall_right ? 0.0 : bx_[static_cast<std::size_t>(j) * nx_ + ip];
00185         const double b_low = wall_low ? 0.0 : by_[index];
00186         const double b_high =
00187             wall_high ? 0.0 : by_[static_cast<std::size_t>(jp) * nx_ + i];
00188         diag[index] =
00189             -(b_left + b_right) / (hx_ * hx_) - (b_low + b_high) / (hy_ * hy_);
00190       }
00191     }
00192     std::vector<double> z(total, 0.0);
00193     std::vector<double> dir(total, 0.0);
00194     std::vector<double> ap(total, 0.0);
00195     auto dot = [&](const std::vector<double>& a, const std::vector<double>& b) {
00196       double sum = 0.0;
00197       for (std::size_t i = 0; i < total; ++i) {
00198         sum += a[i] * b[i];
00199       }
00200       return sum;
00201     };
00202     auto precondition = [&]() {
00203       for (std::size_t i = 0; i < total; ++i) {
00204         z[i] = r[i] / diag[i];
00205       }
00206     };
00207     auto residualNorm = [&]() {
00208       double sum = 0.0;
00209       for (double value : r) {
00210         sum += value * value;
00211       }
00212       return std::sqrt(sum / static_cast<double>(total));
00213     };
00214     const double target =
00215         tolerance * std::max(1.0, std::sqrt(dot(rhs, rhs) / total));
00216     precondition();
00217     dir = z;
00218     double rz = dot(r, z);
00219     for (int it = 0; it < max_iterations; ++it) {
00220       if (residualNorm() <= target) {
00221         break;
00222       }
00223       apply(dir, ap);
00224       const double denom = dot(dir, ap);
00225       if (std::abs(denom) < 1e-300) {
00226         break;
00227       }
00228       const double alpha = rz / denom;
00229       for (std::size_t i = 0; i < total; ++i) {
00230         p[i] += alpha * dir[i];
00231         r[i] -= alpha * ap[i];
00232       }
00233       precondition();
00234       const double rz_new = dot(r, z);
00235       const double beta = rz_new / rz;
00236       for (std::size_t i = 0; i < total; ++i) {
00237         dir[i] = z[i] + beta * dir[i];
00238       }
00239       rz = rz_new;
00240     }
00241   }
00242
00243  private:
00244   int nx_;
00245   int ny_;
00246   double hx_;
00247   double hy_;
00248   std::vector<double> bx_;
00249   std::vector<double> by_;
00250   bool periodic_x_ = true;
00251   bool periodic_y_ = true;
```

```
00252 };
00253
00254 class Solver2D {
00255  public:
00256   Solver2D(int nx, int ny, double lx = 1.0, double ly = 1.0)
00257       : nx_(nx),
00258         ny_(ny),
00259         hx_(lx / nx),
00260         hy_(ly / ny),
00261         c_(nx, ny, lx, ly),
00262         u_(static_cast<std::size_t>(nx) * ny, 0.0),
00263         v_(static_cast<std::size_t>(nx) * ny, 0.0),
00264         p_(static_cast<std::size_t>(nx) * ny, 0.0),
00265         poisson_(nx, ny, hx_, hy_) {}
00266
00267   vof::Grid2D& c() { return c_; }
00268   const vof::Grid2D& c() const { return c_; }
00269   std::vector<double>& u() { return u_; }
00270   std::vector<double>& v() { return v_; }
00271   const std::vector<double>& p() const { return p_; }
00272   int nx() const { return nx_; }
00273   int ny() const { return ny_; }
00274   double hx() const { return hx_; }
00275   double hy() const { return hy_; }
00276
00277   // CSF force faces from a cell-centered curvature field. The force is
00278   // sigma*kappa_face*(c_R-c_L)/h.
00279   void csfFaces(const std::vector<double>& kappa, double sigma,
00280                 std::vector<double>& fx, std::vector<double>& fy) const {
00281     fx.assign(u_.size(), 0.0);
00282     fy.assign(u_.size(), 0.0);
00283     for (int j = 0; j < ny_; ++j) {
00284       for (int i = 0; i < nx_; ++i) {
00285         const int index = j * nx_ + i;
00286         const int im = (i == 0) ? nx_ - 1 : i - 1;
00287         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00288         const double kx = 0.5 * (kappa[static_cast<std::size_t>(j) * nx_ + im] +
00289                                  kappa[static_cast<std::size_t>(index)]);
00290         const double ky = 0.5 * (kappa[static_cast<std::size_t>(jm) * nx_ + i] +
00291                                  kappa[static_cast<std::size_t>(index)]);
00292         fx[static_cast<std::size_t>(index)] =
00293             sigma * kx * (c_(i, j) - c_(im, j)) / hx_;
00294         fy[static_cast<std::size_t>(index)] =
00295             sigma * ky * (c_(i, j) - c_(i, jm)) / hy_;
00296       }
00297     }
00298   }
00299
00300   // One projection step from the CSF force, with an optional explicit
00301   // advection/viscous term omitted in this static-bubble prototype.
00302   void projectForce(const std::vector<double>& fx,
00303                     const std::vector<double>& fy, double dt,
00304                     double rho = 1.0) {
00305     for (int j = 0; j < ny_; ++j) {
00306       for (int i = 0; i < nx_; ++i) {
00307         const int index = j * nx_ + i;
00308         u_[static_cast<std::size_t>(index)] +=
00309             dt * fx[static_cast<std::size_t>(index)] / rho;
00310         v_[static_cast<std::size_t>(index)] +=
00311             dt * fy[static_cast<std::size_t>(index)] / rho;
00312       }
00313     }
00314     std::vector<double> div(static_cast<std::size_t>(nx_) * ny_, 0.0);
00315     for (int j = 0; j < ny_; ++j) {
00316       for (int i = 0; i < nx_; ++i) {
00317         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00318         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00319         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00320         div[index] =
00321             (u_[static_cast<std::size_t>(j) * nx_ + ip] - u_[index]) / hx_ +
00322             (v_[static_cast<std::size_t>(jp) * nx_ + i] - v_[index]) / hy_;
00323       }
00324     }
00325     for (double& d : div) {
00326       d = d / dt;
00327     }
00328     poisson_.solve(div, p_);
00329     for (int j = 0; j < ny_; ++j) {
00330       for (int i = 0; i < nx_; ++i) {
00331         const int im = (i == 0) ? nx_ - 1 : i - 1;
00332         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00333         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00334         u_[index] -= dt *
00335                      (p_[index] - p_[static_cast<std::size_t>(j) * nx_ + im]) /
00336                      (rho * hx_);
00337         v_[index] -= dt *
00338                      (p_[index] - p_[static_cast<std::size_t>(jm) * nx_ + i]) /
```

```
00339                     (rho * hy_);
00340       }
00341     }
00342   }
00343
00344   // Variable-density projection. rho(c) is interpolated arithmetically to
00345   // cell centers and harmonically to faces; the CSF force and the pressure
00346   // gradient use the same face density, which keeps a static bubble balanced.
00347   void projectForceVariable(const std::vector<double>& fx,
00348                             const std::vector<double>& fy, double dt,
00349                             double rho_l, double rho_g) {
00350     const std::size_t total = static_cast<std::size_t>(nx_) * ny_;
00351     std::vector<double> rho_cell(total, 0.0);
00352     for (int j = 0; j < ny_; ++j) {
00353       for (int i = 0; i < nx_; ++i) {
00354         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00355         rho_cell[index] = rho_l * c_(i, j) + rho_g * (1.0 - c_(i, j));
00356       }
00357     }
00358     std::vector<double> beta_x(total, 0.0);
00359     std::vector<double> beta_y(total, 0.0);
00360     for (int j = 0; j < ny_; ++j) {
00361       for (int i = 0; i < nx_; ++i) {
00362         const int im = (i == 0) ? nx_ - 1 : i - 1;
00363         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00364         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00365         const double rl = rho_cell[static_cast<std::size_t>(j) * nx_ + im];
00366         const double rr = rho_cell[index];
00367         const double rd = rho_cell[static_cast<std::size_t>(jm) * nx_ + i];
00368         const double ru = rho_cell[index];
00369         const double rho_x = 2.0 * rl * rr / (rl + rr);
00370         const double rho_y = 2.0 * rd * ru / (rd + ru);
00371         beta_x[index] = 1.0 / rho_x;
00372         beta_y[index] = 1.0 / rho_y;
00373       }
00374     }
00375     for (int j = 0; j < ny_; ++j) {
00376       for (int i = 0; i < nx_; ++i) {
00377         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00378         u_[index] += dt * fx[index] * beta_x[index];
00379         v_[index] += dt * fy[index] * beta_y[index];
00380       }
00381     }
00382     std::vector<double> div(total, 0.0);
00383     for (int j = 0; j < ny_; ++j) {
00384       for (int i = 0; i < nx_; ++i) {
00385         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00386         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00387         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00388         div[index] =
00389             (u_[static_cast<std::size_t>(j) * nx_ + ip] - u_[index]) / hx_ +
00390             (v_[static_cast<std::size_t>(jp) * nx_ + i] - v_[index]) / hy_;
00391       }
00392     }
00393     for (double& d : div) {
00394       d = d / dt;
00395     }
00396     VariablePoisson2D solver(nx_, ny_, hx_, hy_, beta_x, beta_y);
00397     solver.solve(div, p_);
00398     for (int j = 0; j < ny_; ++j) {
00399       for (int i = 0; i < nx_; ++i) {
00400         const int im = (i == 0) ? nx_ - 1 : i - 1;
00401         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00402         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00403         u_[index] -= dt * beta_x[index] *
00404                      (p_[index] - p_[static_cast<std::size_t>(j) * nx_ + im]) /
00405                      hx_;
00406         v_[index] -= dt * beta_y[index] *
00407                      (p_[index] - p_[static_cast<std::size_t>(jm) * nx_ + i]) /
00408                      hy_;
00409       }
00410     }
00411   }
00412
00413   // Full explicit step with variable density, advection, diffusion, gravity,
00414   // CSF, and projection. First-order in time; adequate for the prototype.
00415   void step(double dt, double sigma, const std::vector<double>& kappa,
00416             double rho_l, double rho_g, double mu, double gx, double gy) {
00417     const std::size_t total = static_cast<std::size_t>(nx_) * ny_;
00418     std::vector<double> rho_cell(total, 0.0);
00419     std::vector<double> beta_x(total, 0.0);
00420     std::vector<double> beta_y(total, 0.0);
00421     for (int j = 0; j < ny_; ++j) {
00422       for (int i = 0; i < nx_; ++i) {
00423         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00424         rho_cell[index] = rho_l * c_(i, j) + rho_g * (1.0 - c_(i, j));
00425       }
```

```
00426     }
00427     for (int j = 0; j < ny_; ++j) {
00428       for (int i = 0; i < nx_; ++i) {
00429         const int im = (i == 0) ? nx_ - 1 : i - 1;
00430         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00431         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00432         const double rl = rho_cell[static_cast<std::size_t>(j) * nx_ + im];
00433         const double rr = rho_cell[index];
00434         const double rd = rho_cell[static_cast<std::size_t>(jm) * nx_ + i];
00435         const double ru = rho_cell[index];
00436         beta_x[index] = (rl + rr) / (2.0 * rl * rr);
00437         beta_y[index] = (rd + ru) / (2.0 * rd * ru);
00438       }
00439     }
00440
00441     std::vector<double> du(total, 0.0);
00442     std::vector<double> dv(total, 0.0);
00443     for (int j = 0; j < ny_; ++j) {
00444       for (int i = 0; i < nx_; ++i) {
00445         const int im = (i == 0) ? nx_ - 1 : i - 1;
00446         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00447         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00448         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00449         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00450         const std::size_t xm = static_cast<std::size_t>(j) * nx_ + im;
00451         const std::size_t xp = static_cast<std::size_t>(j) * nx_ + ip;
00452         const std::size_t ym = static_cast<std::size_t>(jm) * nx_ + i;
00453         const std::size_t yp = static_cast<std::size_t>(jp) * nx_ + i;
00454         const double u_here = u_[index];
00455         const double v_here = v_[index];
00456         const double nu_x = mu * beta_x[index];
00457         const double nu_y = mu * beta_y[index];
00458         const double dudx = (u_[xp] - u_[xm]) / (2.0 * hx_);
00459         const double dudy = (u_[yp] - u_[ym]) / (2.0 * hy_);
00460         const double v_at_u =
00461             0.25 *
00462             (v_[xm] + v_[index] + v_[static_cast<std::size_t>(jp) * nx_ + im] +
00463              v_[static_cast<std::size_t>(jp) * nx_ + i]);
00464         const double lap_u = (u_[xp] - 2.0 * u_here + u_[xm]) / (hx_ * hx_) +
00465                              (u_[yp] - 2.0 * u_here + u_[ym]) / (hy_ * hy_);
00466         du[index] = -(u_here * dudx + v_at_u * dudy) + nu_x * lap_u + gx;
00467
00468         const double dvdx = (v_[xp] - v_[xm]) / (2.0 * hx_);
00469         const double dvdy = (v_[yp] - v_[ym]) / (2.0 * hy_);
00470         const double u_at_v =
00471             0.25 * (u_[ym] + u_[index] +
00472                     u_[static_cast<std::size_t>(jm) * nx_ + ip] + u_[xp]);
00473         const double lap_v = (v_[xp] - 2.0 * v_here + v_[xm]) / (hx_ * hx_) +
00474                              (v_[yp] - 2.0 * v_here + v_[ym]) / (hy_ * hy_);
00475         dv[index] = -(u_at_v * dvdx + v_here * dvdy) + nu_y * lap_v + gy;
00476       }
00477     }
00478
00479     std::vector<double> fx;
00480     std::vector<double> fy;
00481     if (sigma > 0.0 && !kappa.empty()) {
00482       csfFaces(kappa, sigma, fx, fy);
00483     } else {
00484       fx.assign(total, 0.0);
00485       fy.assign(total, 0.0);
00486     }
00487     for (std::size_t index = 0; index < total; ++index) {
00488       u_[index] += dt * (du[index] + fx[index] * beta_x[index]);
00489       v_[index] += dt * (dv[index] + fy[index] * beta_y[index]);
00490     }
00491
00492     std::vector<double> div(total, 0.0);
00493     for (int j = 0; j < ny_; ++j) {
00494       for (int i = 0; i < nx_; ++i) {
00495         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00496         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00497         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
00498         div[index] =
00499             (u_[static_cast<std::size_t>(j) * nx_ + ip] - u_[index]) / hx_ +
00500             (v_[static_cast<std::size_t>(jp) * nx_ + i] - v_[index]) / hy_;
00501       }
00502     }
00503     for (double& d : div) {
00504       d = d / dt;
00505     }
00506     VariablePoisson2D solver(nx_, ny_, hx_, hy_, beta_x, beta_y);
00507     solver.solve(div, p_);
00508     for (int j = 0; j < ny_; ++j) {
00509       for (int i = 0; i < nx_; ++i) {
00510         const int im = (i == 0) ? nx_ - 1 : i - 1;
00511         const int jm = (j == 0) ? ny_ - 1 : j - 1;
00512         const std::size_t index = static_cast<std::size_t>(j) * nx_ + i;
```

```
00513         u_[index] -= dt * beta_x[index] *
00514                       (p_[index] - p_[static_cast<std::size_t>(j) * nx_ + im]) /
00515                       hx_;
00516         v_[index] -= dt * beta_y[index] *
00517                       (p_[index] - p_[static_cast<std::size_t>(jm) * nx_ + i]) /
00518                       hy_;
00519       }
00520     }
00521     c_.advance(u_, v_, dt);
00522   }
00523
00524   // Advect the volume fraction with the current face velocities.
00525   void advect(double dt) { c_.advance(u_, v_, dt); }
00526
00527   double maxSpeed() const {
00528     double result = 0.0;
00529     for (double x : u_) {
00530       result = std::max(result, std::abs(x));
00531     }
00532     for (double x : v_) {
00533       result = std::max(result, std::abs(x));
00534     }
00535     return result;
00536   }
00537
00538   double maxDivergence() const {
00539     double result = 0.0;
00540     for (int j = 0; j < ny_; ++j) {
00541       for (int i = 0; i < nx_; ++i) {
00542         const int ip = (i + 1 == nx_) ? 0 : i + 1;
00543         const int jp = (j + 1 == ny_) ? 0 : j + 1;
00544         const double div = (u_[static_cast<std::size_t>(j) * nx_ + ip] -
00545                             u_[static_cast<std::size_t>(j) * nx_ + i]) /
00546                                hx_ +
00547                            (v_[static_cast<std::size_t>(jp) * nx_ + i] -
00548                             v_[static_cast<std::size_t>(j) * nx_ + i]) /
00549                                hy_;
00550         result = std::max(result, std::abs(div));
00551       }
00552     }
00553     return result;
00554   }
00555
00556  private:
00557   int nx_;
00558   int ny_;
00559   double hx_;
00560   double hy_;
00561   vof::Grid2D c_;
00562   std::vector<double> u_;
00563   std::vector<double> v_;
00564   std::vector<double> p_;
00565   FftPoisson2D poisson_;
00566 };
00567
00568 }  // namespace twophase
00569 }  // namespace saturn
00570
00571 #endif  // SATURN_HAVE_FFTW
```

6.70 src/numerics/vof/vof2d.hpp File Reference

Two-dimensional PLIC volume-of-fluid reconstruction and advection.

```
#include <algorithm>
#include <cmath>
#include <vector>
```

Classes

- struct saturn::vof::Point
- struct saturn::vof::Reconstruction
- class saturn::vof::Grid2D

Namespaces

- namespace saturn
- namespace saturn::vof

Functions

- double saturn::vof::polygonArea (const std::vector< Point > &polygon)
- std::vector< Point > saturn::vof::clipAxis (const std::vector< Point > &polygon, int axis, double value, double sign)
- std::vector< Point > saturn::vof::clipHalfPlane (const std::vector< Point > &polygon, double nx, double ny, double alpha)
- std::vector< Point > saturn::vof::unitSquare ()
- Reconstruction saturn::vof::reconstruct (double c, double nx, double ny)
- std::vector< Point > saturn::vof::fluidPolygon (const Reconstruction &r)

6.70.1 Detailed Description

Two-dimensional PLIC volume-of-fluid reconstruction and advection.

6.71 vof2d.hpp

Go to the documentation of this file.

```
00001
00005 #pragma once
00006
00007 #include <algorithm>
00008 #include <cmath>
00009 #include <vector>
00010
00011 namespace saturn {
00012 namespace vof {
00013
00014 // Two-dimensional PLIC volume-of-fluid prototype.
00015 //
00016 // Cell-centered volume fractions on a periodic Cartesian grid. Each cell is
00017 // reconstructed as a half-plane
00018 //
00019 //   n . (x - center) <= alpha
00020 //
00021 // with a Youngs normal from central differences of c and alpha chosen so that
00022 // the clipped area equals c. Advection is dimensionally split (Strang:
00023 // x half-step, y full-step, x half-step) with geometric fluxes: the fluid area
00024 // crossing a face is the intersection of the upwind cell's PLIC polygon with
00025 // the swept strip.
00026
00027 struct Point {
00028   double x;
00029   double y;
00030 };
00031
00032 inline double polygonArea(const std::vector<Point>& polygon) {
00033   double area = 0.0;
00034   const int n = static_cast<int>(polygon.size());
00035   for (int i = 0; i < n; ++i) {
00036     const Point& a = polygon[i];
00037     const Point& b = polygon[(i + 1) % n];
00038     area += a.x * b.y - b.x * a.y;
00039   }
00040   return 0.5 * std::abs(area);
00041 }
00042
00043 // Clip a convex polygon by the half-plane sign*(p.x - c) <= 0 along one axis.
00044 inline std::vector<Point> clipAxis(const std::vector<Point>& polygon, int axis,
00045                                    double value, double sign) {
00046   std::vector<Point> output;
00047   if (polygon.empty()) {
```

```
00048     return output;
00049   }
00050   const int n = static_cast<int>(polygon.size());
00051   for (int i = 0; i < n; ++i) {
00052     const Point& current = polygon[i];
00053     const Point& next = polygon[(i + 1) % n];
00054     auto coord = [axis](const Point& p) { return axis == 0 ? p.x : p.y; };
00055     const double dc = sign * (coord(current) - value);
00056     const double dn = sign * (coord(next) - value);
00057     const bool inside_current = dc <= 0.0;
00058     const bool inside_next = dn <= 0.0;
00059     if (inside_current) {
00060       output.push_back(current);
00061     }
00062     if (inside_current != inside_next) {
00063       const double t = dc / (dc - dn);
00064       output.push_back(Point{current.x + t * (next.x - current.x),
00065                              current.y + t * (next.y - current.y)});
00066     }
00067   }
00068   return output;
00069 }
00070
00071 inline std::vector<Point> clipHalfPlane(const std::vector<Point>& polygon,
00072                                         double nx, double ny, double alpha) {
00073   std::vector<Point> output;
00074   if (polygon.empty()) {
00075     return output;
00076   }
00077   const int n = static_cast<int>(polygon.size());
00078   for (int i = 0; i < n; ++i) {
00079     const Point& current = polygon[i];
00080     const Point& next = polygon[(i + 1) % n];
00081     auto f = [&](const Point& p) {
00082       return nx * (p.x - 0.5) + ny * (p.y - 0.5) - alpha;
00083     };
00084     const double dc = f(current);
00085     const double dn = f(next);
00086     const bool inside_current = dc <= 0.0;
00087     const bool inside_next = dn <= 0.0;
00088     if (inside_current) {
00089       output.push_back(current);
00090     }
00091     if (inside_current != inside_next) {
00092       const double t = dc / (dc - dn);
00093       output.push_back(Point{current.x + t * (next.x - current.x),
00094                              current.y + t * (next.y - current.y)});
00095     }
00096   }
00097   return output;
00098 }
00099
00100 inline std::vector<Point> unitSquare() {
00101   return {Point{0.0, 0.0}, Point{1.0, 0.0}, Point{1.0, 1.0}, Point{0.0, 1.0}};
00102 }
00103
00104 struct Reconstruction {
00105   double nx = 1.0;
00106   double ny = 0.0;
00107   double alpha = 0.0;
00108 };
00109
00110 inline Reconstruction reconstruct(double c, double nx, double ny) {
00111   Reconstruction result;
00112   const double norm = std::sqrt(nx * nx + ny * ny);
00113   if (norm < 1.0e-14 || c <= 0.0 || c >= 1.0) {
00114     result.nx = 1.0;
00115     result.ny = 0.0;
00116     result.alpha = c - 0.5;
00117     return result;
00118   }
00119   result.nx = nx / norm;
00120   result.ny = ny / norm;
00121   double lo = -0.5 * (std::abs(result.nx) + std::abs(result.ny));
00122   double hi = 0.5 * (std::abs(result.nx) + std::abs(result.ny));
00123   const auto square = unitSquare();
00124   for (int iter = 0; iter < 50; ++iter) {
00125     const double mid = 0.5 * (lo + hi);
00126     const double area =
00127         polygonArea(clipHalfPlane(square, result.nx, result.ny, mid));
00128     if (area < c) {
00129       lo = mid;
00130     } else {
00131       hi = mid;
00132     }
00133   }
00134   result.alpha = 0.5 * (lo + hi);
```

```
00135   return result;
00136 }
00137
00138 inline std::vector<Point> fluidPolygon(const Reconstruction& r) {
00139   return clipHalfPlane(unitSquare(), r.nx, r.ny, r.alpha);
00140 }
00141
00142 class Grid2D {
00143  public:
00144   Grid2D(int nx, int ny, double lx = 1.0, double ly = 1.0)
00145       : nx_(nx),
00146         ny_(ny),
00147         hx_(lx / nx),
00148         hy_(ly / ny),
00149         c_(static_cast<std::size_t>(nx) * ny, 0.0) {}
00150
00151   int nx() const { return nx_; }
00152   int ny() const { return ny_; }
00153   double hx() const { return hx_; }
00154   double hy() const { return hy_; }
00155   double& operator()(int i, int j) {
00156     return c_[static_cast<std::size_t>(wrap(i, nx_)) * ny_ + wrap(j, ny_)];
00157   }
00158   double operator()(int i, int j) const {
00159     return c_[static_cast<std::size_t>(wrap(i, nx_)) * ny_ + wrap(j, ny_)];
00160   }
00161   std::vector<double>& data() { return c_; }
00162   const std::vector<double>& data() const { return c_; }
00163
00164   // Reconstruct all cells; normals are Youngs gradients.
00165   std::vector<Reconstruction> reconstructions() const {
00166     std::vector<Reconstruction> r(static_cast<std::size_t>(nx_) * ny_);
00167     for (int i = 0; i < nx_; ++i) {
00168       for (int j = 0; j < ny_; ++j) {
00169         // The normal points from fluid to empty, i.e. opposite the gradient.
00170         const double nx = -0.5 * ((*this)(i + 1, j) - (*this)(i - 1, j));
00171         const double ny = -0.5 * ((*this)(i, j + 1) - (*this)(i, j - 1));
00172         r[static_cast<std::size_t>(i) * ny_ + j] =
00173             reconstruct((*this)(i, j), nx, ny);
00174       }
00175     }
00176     return r;
00177   }
00178
00179   // One directional sweep. `velocity` is face-centered in the sweep direction
00180   // (nx*ny for x, nx*ny for y); periodic indexing.
00181   void sweepX(const std::vector<double>& u, double dt) {
00182     const auto rec = reconstructions();
00183     std::vector<double> flux(static_cast<std::size_t>(nx_) * ny_, 0.0);
00184     for (int i = 0; i < nx_; ++i) {
00185       for (int j = 0; j < ny_; ++j) {
00186         const double vel = u[static_cast<std::size_t>(i) * ny_ + j];
00187         if (vel == 0.0) {
00188           continue;
00189         }
00190         const int up = vel > 0.0 ? i - 1 : i;
00191         const auto& r = rec[static_cast<std::size_t>(wrap(up, nx_)) * ny_ + j];
00192         const double width = std::min(std::abs(vel) * dt / hx_, 1.0);
00193         const double x0 = vel > 0.0 ? 1.0 - width : 0.0;
00194         const double x1 = vel > 0.0 ? 1.0 : width;
00195         std::vector<Point> piece = fluidPolygon(r);
00196         piece = clipAxis(piece, 0, x0, -1.0);
00197         piece = clipAxis(piece, 0, x1, 1.0);
00198         const double area = polygonArea(piece);
00199         flux[static_cast<std::size_t>(i) * ny_ + j] =
00200             (vel > 0.0 ? 1.0 : -1.0) * area * hx_ * hy_;
00201       }
00202     }
00203     for (int i = 0; i < nx_; ++i) {
00204       for (int j = 0; j < ny_; ++j) {
00205         const double left = flux[static_cast<std::size_t>(i) * ny_ + j];
00206         const double right =
00207             flux[static_cast<std::size_t>(wrap(i + 1, nx_)) * ny_ + j];
00208         (*this)(i, j) -= (right - left) / (hx_ * hy_);
00209       }
00210     }
00211   }
00212
00213   void sweepY(const std::vector<double>& v, double dt) {
00214     const auto rec = reconstructions();
00215     std::vector<double> flux(static_cast<std::size_t>(nx_) * ny_, 0.0);
00216     for (int i = 0; i < nx_; ++i) {
00217       for (int j = 0; j < ny_; ++j) {
00218         const double vel = v[static_cast<std::size_t>(i) * ny_ + j];
00219         if (vel == 0.0) {
00220           continue;
00221         }
```

```
00222         const int up = vel > 0.0 ? j - 1 : j;
00223         const auto& r = rec[static_cast<std::size_t>(i) * ny_ + wrap(up, ny_)];
00224         const double width = std::min(std::abs(vel) * dt / hy_, 1.0);
00225         const double y0 = vel > 0.0 ? 1.0 - width : 0.0;
00226         const double y1 = vel > 0.0 ? 1.0 : width;
00227         std::vector<Point> piece = fluidPolygon(r);
00228         piece = clipAxis(piece, 1, y0, -1.0);
00229         piece = clipAxis(piece, 1, y1, 1.0);
00230         const double area = polygonArea(piece);
00231         flux[static_cast<std::size_t>(i) * ny_ + j] =
00232             (vel > 0.0 ? 1.0 : -1.0) * area * hx_ * hy_;
00233       }
00234     }
00235     for (int i = 0; i < nx_; ++i) {
00236       for (int j = 0; j < ny_; ++j) {
00237         const double lower = flux[static_cast<std::size_t>(i) * ny_ + j];
00238         const double upper =
00239             flux[static_cast<std::size_t>(i) * ny_ + wrap(j + 1, ny_)];
00240         (*this)(i, j) -= (upper - lower) / (hx_ * hy_);
00241       }
00242     }
00243   }
00244
00245   // Strang-split advection over dt.
00246   void advance(const std::vector<double>& u, const std::vector<double>& v,
00247                double dt) {
00248     sweepX(u, 0.5 * dt);
00249     sweepY(v, dt);
00250     sweepX(u, 0.5 * dt);
00251   }
00252
00253   double mass() const {
00254     double sum = 0.0;
00255     for (double c : c_) {
00256       sum += c;
00257     }
00258     return sum * hx_ * hy_;
00259   }
00260
00261  private:
00262   static int wrap(int i, int n) { return ((i % n) + n) % n; }
00263
00264   int nx_;
00265   int ny_;
00266   double hx_;
00267   double hy_;
00268   std::vector<double> c_;
00269 };
00270
00271 }  // namespace vof
00272 }  // namespace saturn
```

6.72 src/numerics/vof/vof3d.hpp File Reference

```
#include <algorithm>
#include <cmath>
#include <vector>
```

Classes

- struct saturn::vof::Vec3
- struct saturn::vof::Reconstruction3D
- class saturn::vof::Grid3D

Namespaces

- namespace saturn
- namespace saturn::vof

Typedefs

- using saturn::vof::Face = std::vector<Vec3>
- using saturn::vof::Polyhedron = std::vector<Face>

Functions

- Vec3 saturn::vof::operator- (const Vec3 &a, const Vec3 &b)
- Vec3 saturn::vof::operator+ (const Vec3 &a, const Vec3 &b)
- Vec3 saturn::vof::operator∗ (double s, const Vec3 &a)
- double saturn::vof::dot (const Vec3 &a, const Vec3 &b)
- Vec3 saturn::vof::cross (const Vec3 &a, const Vec3 &b)
- Polyhedron saturn::vof::unitCube ()
- double saturn::vof::polyhedronVolume (const Polyhedron &poly)
- Polyhedron saturn::vof::clipHalfPlane (const Polyhedron &poly, const Vec3 &n, double alpha)
- Reconstruction3D saturn::vof::reconstruct (double c, double nx, double ny, double nz)
- Polyhedron saturn::vof::fluidPolyhedron (const Reconstruction3D &r)

6.73 vof3d.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <algorithm>
00004 #include <cmath>
00005 #include <vector>
00006
00007 namespace saturn {
00008 namespace vof {
00009
00010 // Three-dimensional PLIC volume-of-fluid prototype on a periodic grid.
00011 //
00012 // Each cell is a unit cube reconstructed as a half-plane
00013 //   n . (x - 0.5) <= alpha
00014 // with a Youngs normal from the volume-fraction gradient and alpha chosen so
00015 // that the clipped volume equals c. Advection is Strang-split in x, y, z with
00016 // geometric fluxes: the fluid volume crossing a face is the volume of the
00017 // upwind cell's PLIC polyhedron intersected with the swept slab.
00018
00019 struct Vec3 {
00020   double x;
00021   double y;
00022   double z;
00023 };
00024
00025 using Face = std::vector<Vec3>;
00026 using Polyhedron = std::vector<Face>;
00027
00028 inline Vec3 operator-(const Vec3& a, const Vec3& b) {
00029   return Vec3{a.x - b.x, a.y - b.y, a.z - b.z};
00030 }
00031 inline Vec3 operator+(const Vec3& a, const Vec3& b) {
00032   return Vec3{a.x + b.x, a.y + b.y, a.z + b.z};
00033 }
00034 inline Vec3 operator*(double s, const Vec3& a) {
00035   return Vec3{s * a.x, s * a.y, s * a.z};
00036 }
00037 inline double dot(const Vec3& a, const Vec3& b) {
00038   return a.x * b.x + a.y * b.y + a.z * b.z;
00039 }
00040 inline Vec3 cross(const Vec3& a, const Vec3& b) {
00041   return Vec3{a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z,
00042               a.x * b.y - a.y * b.x};
00043 }
00044
00045 inline Polyhedron unitCube() {
00046   Polyhedron cube;
00047   cube.push_back({{0, 0, 0}, {0, 0, 1}, {0, 1, 1}, {0, 1, 0}});  // -x
00048   cube.push_back({{1, 0, 0}, {1, 1, 0}, {1, 1, 1}, {1, 0, 1}});  // +x
00049   cube.push_back({{0, 0, 0}, {1, 0, 0}, {1, 0, 1}, {0, 0, 1}});  // -y
```

```
00050   cube.push_back({{0, 1, 0}, {0, 1, 1}, {1, 1, 1}, {1, 1, 0}});  // +y
00051   cube.push_back({{0, 0, 0}, {0, 1, 0}, {1, 1, 0}, {1, 0, 0}});  // -z
00052   cube.push_back({{0, 0, 1}, {1, 0, 1}, {1, 1, 1}, {0, 1, 1}});  // +z
00053   return cube;
00054 }
00055
00056 inline double polyhedronVolume(const Polyhedron& poly) {
00057   double volume = 0.0;
00058   for (const auto& face : poly) {
00059     if (face.size() < 3) {
00060       continue;
00061     }
00062     const Vec3& a = face[0];
00063     for (std::size_t i = 1; i + 1 < face.size(); ++i) {
00064       volume += dot(a, cross(face[i], face[i + 1])) / 6.0;
00065     }
00066   }
00067   return std::abs(volume);
00068 }
00069
00070 // Clip a convex polyhedron by n.x <= alpha. Intersection points are collected
00071 // into a new cap face, sorted around the plane normal.
00072 inline Polyhedron clipHalfPlane(const Polyhedron& poly, const Vec3& n,
00073                                 double alpha) {
00074   Polyhedron result;
00075   std::vector<Vec3> cap;
00076   for (const auto& face : poly) {
00077     if (face.size() < 3) {
00078       continue;
00079     }
00080     Face clipped;
00081     const int count = static_cast<int>(face.size());
00082     for (int i = 0; i < count; ++i) {
00083       const Vec3& current = face[i];
00084       const Vec3& next = face[(i + 1) % count];
00085       const double dc = dot(n, current) - alpha;
00086       const double dn = dot(n, next) - alpha;
00087       const bool inside_current = dc <= 0.0;
00088       const bool inside_next = dn <= 0.0;
00089       if (inside_current) {
00090         clipped.push_back(current);
00091       }
00092       if (inside_current != inside_next) {
00093         const double t = dc / (dc - dn);
00094         const Vec3 point = current + t * (next - current);
00095         clipped.push_back(point);
00096         cap.push_back(point);
00097       }
00098     }
00099     if (clipped.size() >= 3) {
00100       result.push_back(std::move(clipped));
00101     }
00102   }
00103
00104   if (cap.size() >= 3) {
00105     Vec3 center{0, 0, 0};
00106     for (const auto& p : cap) {
00107       center = center + p;
00108     }
00109     center = (1.0 / static_cast<double>(cap.size())) * center;
00110     Vec3 axis = n;
00111     const double norm = std::sqrt(dot(axis, axis));
00112     if (norm > 0.0) {
00113       axis = (1.0 / norm) * axis;
00114     }
00115     Vec3 helper = std::abs(axis.x) < 0.9 ? Vec3{1, 0, 0} : Vec3{0, 1, 0};
00116     Vec3 e1 = cross(axis, helper);
00117     const double e1n = std::sqrt(dot(e1, e1));
00118     if (e1n > 1e-14) {
00119       e1 = (1.0 / e1n) * e1;
00120     }
00121     Vec3 e2 = cross(axis, e1);
00122     std::sort(cap.begin(), cap.end(), [&](const Vec3& a, const Vec3& b) {
00123       const double aa = std::atan2(dot(a - center, e2), dot(a - center, e1));
00124       const double bb = std::atan2(dot(b - center, e2), dot(b - center, e1));
00125       return aa < bb;
00126     });
00127     result.push_back(std::move(cap));
00128   }
00129   return result;
00130 }
00131
00132 struct Reconstruction3D {
00133   double nx = 1.0;
00134   double ny = 0.0;
00135   double nz = 0.0;
00136   double alpha = 0.0;
```

```
00137 };
00138
00139 inline Reconstruction3D reconstruct(double c, double nx, double ny, double nz) {
00140   Reconstruction3D result;
00141   const double norm = std::sqrt(nx * nx + ny * ny + nz * nz);
00142   if (norm < 1e-14 || c <= 0.0 || c >= 1.0) {
00143     result.nx = 1.0;
00144     result.ny = 0.0;
00145     result.nz = 0.0;
00146     result.alpha = c - 0.5;
00147     return result;
00148   }
00149   result.nx = nx / norm;
00150   result.ny = ny / norm;
00151   result.nz = nz / norm;
00152   const Vec3 normal{result.nx, result.ny, result.nz};
00153   const double center_offset = 0.5 * (result.nx + result.ny + result.nz);
00154   const Polyhedron cube = unitCube();
00155   double lo =
00156       -0.5 * (std::abs(result.nx) + std::abs(result.ny) + std::abs(result.nz));
00157   double hi = -lo;
00158   for (int iter = 0; iter < 42; ++iter) {
00159     const double mid = 0.5 * (lo + hi);
00160     const double volume =
00161         polyhedronVolume(clipHalfPlane(cube, normal, mid + center_offset));
00162     if (volume < c) {
00163       lo = mid;
00164     } else {
00165       hi = mid;
00166     }
00167   }
00168   result.alpha = 0.5 * (lo + hi);
00169   return result;
00170 }
00171
00172 inline Polyhedron fluidPolyhedron(const Reconstruction3D& r) {
00173   const double plane_alpha = r.alpha + 0.5 * (r.nx + r.ny + r.nz);
00174   return clipHalfPlane(unitCube(), Vec3{r.nx, r.ny, r.nz}, plane_alpha);
00175 }
00176
00177 class Grid3D {
00178  public:
00179   Grid3D(int nx, int ny, int nz, double lx = 1.0, double ly = 1.0,
00180          double lz = 1.0)
00181       : nx_(nx),
00182         ny_(ny),
00183         nz_(nz),
00184         hx_(lx / nx),
00185         hy_(ly / ny),
00186         hz_(lz / nz),
00187         c_(static_cast<std::size_t>(nx) * ny * nz, 0.0) {}
00188
00189   int nx() const { return nx_; }
00190   int ny() const { return ny_; }
00191   int nz() const { return nz_; }
00192   double hx() const { return hx_; }
00193   double hy() const { return hy_; }
00194   double hz() const { return hz_; }
00195
00196   double& operator()(int i, int j, int k) { return c_[index(i, j, k)]; }
00197   double operator()(int i, int j, int k) const { return c_[index(i, j, k)]; }
00198   std::vector<double>& data() { return c_; }
00199   const std::vector<double>& data() const { return c_; }
00200
00201   double cellVolume() const { return hx_ * hy_ * hz_; }
00202
00203   std::vector<Reconstruction3D> reconstructions() const {
00204     std::vector<Reconstruction3D> r(c_.size());
00205     for (int i = 0; i < nx_; ++i) {
00206       for (int j = 0; j < ny_; ++j) {
00207         for (int k = 0; k < nz_; ++k) {
00208           const double nx =
00209               -0.5 * ((*this)(i + 1, j, k) - (*this)(i - 1, j, k));
00210           const double ny =
00211               -0.5 * ((*this)(i, j + 1, k) - (*this)(i, j - 1, k));
00212           const double nz =
00213               -0.5 * ((*this)(i, j, k + 1) - (*this)(i, j, k - 1));
00214           r[index(i, j, k)] = reconstruct((*this)(i, j, k), nx, ny, nz);
00215         }
00216       }
00217     }
00218     return r;
00219   }
00220
00221   void sweepX(const std::vector<double>& u, double dt) {
00222     const auto rec = reconstructions();
00223     std::vector<double> flux(c_.size(), 0.0);
```

```
00224     for (int i = 0; i < nx_; ++i) {
00225       for (int j = 0; j < ny_; ++j) {
00226         for (int k = 0; k < nz_; ++k) {
00227           const double vel = u[index(i, j, k)];
00228           if (vel == 0.0) {
00229             continue;
00230           }
00231           const int up = vel > 0.0 ? i - 1 : i;
00232           const auto& r = rec[index(up, j, k)];
00233           const double width = std::min(std::abs(vel) * dt / hx_, 1.0);
00234           Polyhedron poly = fluidPolyhedron(r);
00235           if (vel > 0.0) {
00236             poly = clipHalfPlane(poly, Vec3{1, 0, 0}, 1.0);
00237             poly = clipHalfPlane(poly, Vec3{-1, 0, 0}, -(1.0 - width));
00238           } else {
00239             poly = clipHalfPlane(poly, Vec3{-1, 0, 0}, 0.0);
00240             poly = clipHalfPlane(poly, Vec3{1, 0, 0}, width);
00241           }
00242           const double fraction = polyhedronVolume(poly);
00243           flux[index(i, j, k)] =
00244               (vel > 0.0 ? 1.0 : -1.0) * fraction * cellVolume();
00245         }
00246       }
00247     }
00248     for (int i = 0; i < nx_; ++i) {
00249       for (int j = 0; j < ny_; ++j) {
00250         for (int k = 0; k < nz_; ++k) {
00251           const double left = flux[index(i, j, k)];
00252           const double right = flux[index(i + 1, j, k)];
00253           (*this)(i, j, k) -= (right - left) / cellVolume();
00254         }
00255       }
00256     }
00257   }
00258
00259   void sweepY(const std::vector<double>& v, double dt) {
00260     const auto rec = reconstructions();
00261     std::vector<double> flux(c_.size(), 0.0);
00262     for (int i = 0; i < nx_; ++i) {
00263       for (int j = 0; j < ny_; ++j) {
00264         for (int k = 0; k < nz_; ++k) {
00265           const double vel = v[index(i, j, k)];
00266           if (vel == 0.0) {
00267             continue;
00268           }
00269           const int up = vel > 0.0 ? j - 1 : j;
00270           const auto& r = rec[index(i, up, k)];
00271           const double width = std::min(std::abs(vel) * dt / hy_, 1.0);
00272           Polyhedron poly = fluidPolyhedron(r);
00273           if (vel > 0.0) {
00274             poly = clipHalfPlane(poly, Vec3{0, 1, 0}, 1.0);
00275             poly = clipHalfPlane(poly, Vec3{0, -1, 0}, -(1.0 - width));
00276           } else {
00277             poly = clipHalfPlane(poly, Vec3{0, -1, 0}, 0.0);
00278             poly = clipHalfPlane(poly, Vec3{0, 1, 0}, width);
00279           }
00280           const double fraction = polyhedronVolume(poly);
00281           flux[index(i, j, k)] =
00282               (vel > 0.0 ? 1.0 : -1.0) * fraction * cellVolume();
00283         }
00284       }
00285     }
00286     for (int i = 0; i < nx_; ++i) {
00287       for (int j = 0; j < ny_; ++j) {
00288         for (int k = 0; k < nz_; ++k) {
00289           const double lower = flux[index(i, j, k)];
00290           const double upper = flux[index(i, j + 1, k)];
00291           (*this)(i, j, k) -= (upper - lower) / cellVolume();
00292         }
00293       }
00294     }
00295   }
00296
00297   void sweepZ(const std::vector<double>& w, double dt) {
00298     const auto rec = reconstructions();
00299     std::vector<double> flux(c_.size(), 0.0);
00300     for (int i = 0; i < nx_; ++i) {
00301       for (int j = 0; j < ny_; ++j) {
00302         for (int k = 0; k < nz_; ++k) {
00303           const double vel = w[index(i, j, k)];
00304           if (vel == 0.0) {
00305             continue;
00306           }
00307           const int up = vel > 0.0 ? k - 1 : k;
00308           const auto& r = rec[index(i, j, up)];
00309           const double width = std::min(std::abs(vel) * dt / hz_, 1.0);
00310           Polyhedron poly = fluidPolyhedron(r);
```

```
00311           if (vel > 0.0) {
00312             poly = clipHalfPlane(poly, Vec3{0, 0, 1}, 1.0);
00313             poly = clipHalfPlane(poly, Vec3{0, 0, -1}, -(1.0 - width));
00314           } else {
00315             poly = clipHalfPlane(poly, Vec3{0, 0, -1}, 0.0);
00316             poly = clipHalfPlane(poly, Vec3{0, 0, 1}, width);
00317           }
00318           const double fraction = polyhedronVolume(poly);
00319           flux[index(i, j, k)] =
00320               (vel > 0.0 ? 1.0 : -1.0) * fraction * cellVolume();
00321         }
00322       }
00323     }
00324     for (int i = 0; i < nx_; ++i) {
00325       for (int j = 0; j < ny_; ++j) {
00326         for (int k = 0; k < nz_; ++k) {
00327           const double lower = flux[index(i, j, k)];
00328           const double upper = flux[index(i, j, k + 1)];
00329           (*this)(i, j, k) -= (upper - lower) / cellVolume();
00330         }
00331       }
00332     }
00333   }
00334
00335   void advance(const std::vector<double>& u, const std::vector<double>& v,
00336                const std::vector<double>& w, double dt) {
00337     sweepX(u, 0.5 * dt);
00338     sweepY(v, 0.5 * dt);
00339     sweepZ(w, dt);
00340     sweepY(v, 0.5 * dt);
00341     sweepX(u, 0.5 * dt);
00342   }
00343
00344   double mass() const {
00345     double sum = 0.0;
00346     for (double c : c_) {
00347       sum += c;
00348     }
00349     return sum * cellVolume();
00350   }
00351
00352  private:
00353   std::size_t index(int i, int j, int k) const {
00354     return (static_cast<std::size_t>(wrap(i, nx_)) * ny_ +
00355             static_cast<std::size_t>(wrap(j, ny_))) *
00356                nz_ +
00357            static_cast<std::size_t>(wrap(k, nz_));
00358   }
00359   static int wrap(int i, int n) { return ((i % n) + n) % n; }
00360
00361   int nx_;
00362   int ny_;
00363   int nz_;
00364   double hx_;
00365   double hy_;
00366   double hz_;
00367   std::vector<double> c_;
00368 };
00369
00370 }  // namespace vof
00371 }  // namespace saturn
```

6.74 src/numerics/vof/vof_curvature2d.hpp File Reference

```
#include "numerics/vof/vof2d.hpp"
#include <algorithm>
#include <cmath>
```

Classes

- struct saturn::vof::Curvature2D

Namespaces

- namespace saturn
- namespace saturn::vof

Functions

- double saturn::vof::columnHeight (const Grid2D &g, int i)
- double saturn::vof::rowHeight (const Grid2D &g, int j)
- Curvature2D saturn::vof::curvatureY (const Grid2D &g, int i, int stencil=2)
- Curvature2D saturn::vof::curvatureX (const Grid2D &g, int j, int stencil=2)

6.75 vof_curvature2d.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "numerics/vof/vof2d.hpp"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007
00008 namespace saturn {
00009 namespace vof {
00010
00011 // Height-function curvature for a mostly horizontal interface y = f(x).
00012 //
00013 // The height function is the column integral of the volume fraction,
00014 //   h_i = sum_j c_ij * hy,
00015 // and the curvature is
00016 //   kappa = h'' / (1 + h'^2)^(3/2).
00017 // A five-point stencil is used for h' and h'' by default.
00018 struct Curvature2D {
00019   double kappa = 0.0;
00020   double slope = 0.0;
00021   bool valid = false;
00022 };
00023
00024 inline double columnHeight(const Grid2D& g, int i) {
00025   double sum = 0.0;
00026   for (int j = 0; j < g.ny(); ++j) {
00027     sum += g(i, j) * g.hy();
00028   }
00029   return sum;
00030 }
00031
00032 inline double rowHeight(const Grid2D& g, int j) {
00033   double sum = 0.0;
00034   for (int i = 0; i < g.nx(); ++i) {
00035     sum += g(i, j) * g.hx();
00036   }
00037   return sum;
00038 }
00039
00040 // Curvature of a y = f(x) interface at column i.
00041 inline Curvature2D curvatureY(const Grid2D& g, int i, int stencil = 2) {
00042   Curvature2D result;
00043   if (i - stencil < 0 || i + stencil >= g.nx()) {
00044     return result;
00045   }
00046   const int s = stencil;
00047   const double hx = g.hx();
00048   double first = 0.0;
00049   double second = 0.0;
00050   if (s == 1) {
00051     const double hm = columnHeight(g, i - 1);
00052     const double h0 = columnHeight(g, i);
00053     const double hp = columnHeight(g, i + 1);
00054     first = (hp - hm) / (2.0 * hx);
00055     second = (hp - 2.0 * h0 + hm) / (hx * hx);
00056   } else {
00057     // 5-point first and second derivatives.
00058     const double hm2 = columnHeight(g, i - 2);
00059     const double hm1 = columnHeight(g, i - 1);
```

```
00060     const double h0 = columnHeight(g, i);
00061     const double hp1 = columnHeight(g, i + 1);
00062     const double hp2 = columnHeight(g, i + 2);
00063     first = (hm2 - 8.0 * hm1 + 8.0 * hp1 - hp2) / (12.0 * hx);
00064     second =
00065         (-hp2 + 16.0 * hp1 - 30.0 * h0 + 16.0 * hm1 - hm2) / (12.0 * hx * hx);
00066   }
00067   result.slope = first;
00068   result.kappa = second / std::pow(1.0 + first * first, 1.5);
00069   result.valid = true;
00070   return result;
00071 }
00072
00073 // Curvature of an x = f(y) interface at row j (transpose of curvatureY).
00074 inline Curvature2D curvatureX(const Grid2D& g, int j, int stencil = 2) {
00075   Curvature2D result;
00076   if (j - stencil < 0 || j + stencil >= g.ny()) {
00077     return result;
00078   }
00079   const int s = stencil;
00080   const double hy = g.hy();
00081   double first = 0.0;
00082   double second = 0.0;
00083   if (s == 1) {
00084     const double hm = rowHeight(g, j - 1);
00085     const double h0 = rowHeight(g, j);
00086     const double hp = rowHeight(g, j + 1);
00087     first = (hp - hm) / (2.0 * hy);
00088     second = (hp - 2.0 * h0 + hm) / (hy * hy);
00089   } else {
00090     const double hm2 = rowHeight(g, j - 2);
00091     const double hm1 = rowHeight(g, j - 1);
00092     const double h0 = rowHeight(g, j);
00093     const double hp1 = rowHeight(g, j + 1);
00094     const double hp2 = rowHeight(g, j + 2);
00095     first = (hm2 - 8.0 * hm1 + 8.0 * hp1 - hp2) / (12.0 * hy);
00096     second =
00097         (-hp2 + 16.0 * hp1 - 30.0 * h0 + 16.0 * hm1 - hm2) / (12.0 * hy * hy);
00098   }
00099   result.slope = first;
00100   result.kappa = second / std::pow(1.0 + first * first, 1.5);
00101   result.valid = true;
00102   return result;
00103 }
00104
00105 }  // namespace vof
00106 }  // namespace saturn
```

6.76 src/numerics/vof/vof_curvature3d.hpp File Reference

```
#include "numerics/vof/vof3d.hpp"
#include <algorithm>
#include <cmath>
```

Classes

- struct saturn::vof::Curvature3D

Namespaces

- namespace saturn
- namespace saturn::vof

Functions

- double saturn::vof::columnHeight (const Grid3D &g, int i, int j)
- Curvature3D saturn::vof::curvatureZ (const Grid3D &g, int i, int j)

6.77 vof_curvature3d.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "numerics/vof/vof3d.hpp"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007
00008 namespace saturn {
00009 namespace vof {
00010
00011 // Height-function curvature for a mostly horizontal interface z = f(x, y).
00012 //
00013 //   h_ij = sum_k c_ijk hz
00014 //   kappa = [h_xx (1+h_y^2) + h_yy (1+h_x^2) - 2 h_x h_y h_xy]
00015 //           / (1 + h_x^2 + h_y^2)^(3/2)
00016 //
00017 // Derivatives use 5-point stencils in x and y.
00018
00019 struct Curvature3D {
00020   double kappa = 0.0;
00021   double hx = 0.0;
00022   double hy = 0.0;
00023   bool valid = false;
00024 };
00025
00026 inline double columnHeight(const Grid3D& g, int i, int j) {
00027   double sum = 0.0;
00028   for (int k = 0; k < g.nz(); ++k) {
00029     sum += g(i, j, k) * g.hz();
00030   }
00031   return sum;
00032 }
00033
00034 inline Curvature3D curvatureZ(const Grid3D& g, int i, int j) {
00035   Curvature3D result;
00036   const int s = 2;
00037   if (i - s < 0 || i + s >= g.nx() || j - s < 0 || j + s >= g.ny()) {
00038     return result;
00039   }
00040   const double hx = g.hx();
00041   const double hy = g.hy();
00042   const double hm2 = columnHeight(g, i - 2, j);
00043   const double hm1 = columnHeight(g, i - 1, j);
00044   const double h0 = columnHeight(g, i, j);
00045   const double hp1 = columnHeight(g, i + 1, j);
00046   const double hp2 = columnHeight(g, i + 2, j);
00047
00048   const double jm2 = columnHeight(g, i, j - 2);
00049   const double jm1 = columnHeight(g, i, j - 1);
00050   const double jp1 = columnHeight(g, i, j + 1);
00051   const double jp2 = columnHeight(g, i, j + 2);
00052
00053   const double hx_der = (hm2 - 8.0 * hm1 + 8.0 * hp1 - hp2) / (12.0 * hx);
00054   const double hy_der = (jm2 - 8.0 * jm1 + 8.0 * jp1 - jp2) / (12.0 * hy);
00055   const double hxx =
00056       (-hp2 + 16.0 * hp1 - 30.0 * h0 + 16.0 * hm1 - hm2) / (12.0 * hx * hx);
00057   const double hyy =
00058       (-jp2 + 16.0 * jp1 - 30.0 * h0 + 16.0 * jm1 - jm2) / (12.0 * hy * hy);
00059   // Mixed derivative from the 4-point central difference of h_x in y.
00060   const double hx_jm =
00061       (columnHeight(g, i + 1, j - 1) - columnHeight(g, i - 1, j - 1)) /
00062       (2.0 * hx);
00063   const double hx_jp =
00064       (columnHeight(g, i + 1, j + 1) - columnHeight(g, i - 1, j + 1)) /
00065       (2.0 * hx);
00066   const double hxy = (hx_jp - hx_jm) / (2.0 * hy);
00067
00068   const double gx2 = hx_der * hx_der;
00069   const double gy2 = hy_der * hy_der;
00070   const double denom = std::pow(1.0 + gx2 + gy2, 1.5);
00071   result.kappa =
00072       (hxx * (1.0 + gy2) + hyy * (1.0 + gx2) - 2.0 * hx_der * hy_der * hxy) /
00073       denom;
00074   result.hx = hx_der;
00075   result.hy = hy_der;
00076   result.valid = true;
00077   return result;
00078 }
00079
00080 }  // namespace vof
00081 }  // namespace saturn
```

6.78 src/numerics/vof/vof_curvature_levelset2d.hpp File Reference

```
#include "numerics/vof/vof2d.hpp"
#include <algorithm>
#include <cmath>
#include <vector>
```

Classes

- struct saturn::vof::LevelSetCurvature

Namespaces

- namespace saturn
- namespace saturn::vof

Functions

- void saturn::vof::smoothBinomial (const Grid2D &in, Grid2D &out, int passes)
- LevelSetCurvature saturn::vof::curvatureLevelset (const Grid2D &smoothed, int i, int j)

6.79 vof_curvature_levelset2d.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "numerics/vof/vof2d.hpp"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007 #include <vector>
00008
00009 namespace saturn {
00010 namespace vof {
00011
00012 // Level-set curvature on a smoothed volume fraction:
00013 //
00014 //   phi = smooth(c) - 1/2
00015 //   n   = grad(phi) / |grad(phi)|
00016 //   kappa = -div(n)
00017 //
00018 // A short separable binomial smoothing removes the grid-scale staircase of a
00019 // sharp PLIC field. This works for closed interfaces (bubbles) where the
00020 // column height function is not single-valued.
00021
00022 inline void smoothBinomial(const Grid2D& in, Grid2D& out, int passes) {
00023   const int nx = in.nx();
00024   const int ny = in.ny();
00025   std::vector<double> buffer(in.data().size(), 0.0);
00026   Grid2D work = in;
00027   for (int pass = 0; pass < passes; ++pass) {
00028     for (int j = 0; j < ny; ++j) {
00029       for (int i = 0; i < nx; ++i) {
00030         buffer[static_cast<std::size_t>(j) * nx + i] =
00031             0.25 * work(i - 1, j) + 0.5 * work(i, j) + 0.25 * work(i + 1, j);
00032       }
00033     }
00034     for (int j = 0; j < ny; ++j) {
00035       for (int i = 0; i < nx; ++i) {
00036         work(i, j) =
00037             0.25 *
00038                 buffer[static_cast<std::size_t>((j - 1 + ny) % ny) * nx + i] +
00039             0.5 * buffer[static_cast<std::size_t>(j) * nx + i] +
```

```
00040             0.25 * buffer[static_cast<std::size_t>((j + 1) % ny) * nx + i];
00041       }
00042     }
00043   }
00044   out = work;
00045 }
00046
00047 struct LevelSetCurvature {
00048   double kappa = 0.0;
00049   bool valid = false;
00050 };
00051
00052 inline LevelSetCurvature curvatureLevelset(const Grid2D& smoothed, int i,
00053                                            int j) {
00054   LevelSetCurvature result;
00055   const double hx = smoothed.hx();
00056   const double hy = smoothed.hy();
00057   const double gx = 0.5 * (smoothed(i + 1, j) - smoothed(i - 1, j)) / hx;
00058   const double gy = 0.5 * (smoothed(i, j + 1) - smoothed(i, j - 1)) / hy;
00059   const double norm = std::sqrt(gx * gx + gy * gy);
00060   if (norm < 1.0e-8) {
00061     return result;
00062   }
00063   // Divergence of the normalized gradient.
00064   auto normalX = [&](int ii, int jj) {
00065     const double x = 0.5 * (smoothed(ii + 1, jj) - smoothed(ii - 1, jj)) / hx;
00066     const double y = 0.5 * (smoothed(ii, jj + 1) - smoothed(ii, jj - 1)) / hy;
00067     const double n = std::sqrt(x * x + y * y);
00068     return n > 1.0e-10 ? x / n : 0.0;
00069   };
00070   auto normalY = [&](int ii, int jj) {
00071     const double x = 0.5 * (smoothed(ii + 1, jj) - smoothed(ii - 1, jj)) / hx;
00072     const double y = 0.5 * (smoothed(ii, jj + 1) - smoothed(ii, jj - 1)) / hy;
00073     const double n = std::sqrt(x * x + y * y);
00074     return n > 1.0e-10 ? y / n : 0.0;
00075   };
00076   const double div = 0.5 * (normalX(i + 1, j) - normalX(i - 1, j)) / hx +
00077                      0.5 * (normalY(i, j + 1) - normalY(i, j - 1)) / hy;
00078   result.kappa = -div;
00079   result.valid = true;
00080   return result;
00081 }
00082
00083 }  // namespace vof
00084 }  // namespace saturn
```

6.80 src/numerics/vof/vof_redistribute.hpp File Reference

```
#include "numerics/vof/vof2d.hpp"
#include "numerics/vof/vof3d.hpp"
#include <algorithm>
#include <cmath>
#include <vector>
```

Namespaces

- namespace saturn
- namespace saturn::vof

Functions

- void saturn::vof::redistribute (Grid2D &g, int sweeps=20)
- void saturn::vof::redistribute (Grid3D &g, int sweeps=20)

6.81 vof_redistribute.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include "numerics/vof/vof2d.hpp"
00004 #include "numerics/vof/vof3d.hpp"
00005
00006 #include <algorithm>
00007 #include <cmath>
00008 #include <vector>
00009
00010 namespace saturn {
00011 namespace vof {
00012
00013 // Conservative bounded redistribution.
00014 //
00015 // Advection can leave small overshoots (c < 0 or c > 1) at sharp interfaces.
00016 // This pass clamps the field and redistributes the resulting mass error to
00017 // cells with available capacity, iterating until the target mass is restored
00018 // (or capacity is exhausted). It is a safety net, not a replacement for a
00019 // bounded advection scheme.
00020
00021 inline void redistribute(Grid2D& g, int sweeps = 20) {
00022   const double target = g.mass();
00023   const double volume = g.hx() * g.hy();
00024   for (int sweep = 0; sweep < sweeps; ++sweep) {
00025     for (double& c : g.data()) {
00026       c = std::min(1.0, std::max(0.0, c));
00027     }
00028     const double current = g.mass();
00029     const double delta = target - current;
00030     if (std::abs(delta) <= 1.0e-15 * std::max(1.0, std::abs(target))) {
00031       break;
00032     }
00033     double capacity = 0.0;
00034     for (double c : g.data()) {
00035       capacity += (delta > 0.0) ? (1.0 - c) : c;
00036     }
00037     if (capacity <= 0.0) {
00038       break;
00039     }
00040     const double amount = delta / (capacity * volume);
00041     for (double& c : g.data()) {
00042       const double available = (delta > 0.0) ? (1.0 - c) : c;
00043       c += amount * available;
00044       c = std::min(1.0, std::max(0.0, c));
00045     }
00046   }
00047 }
00048
00049 inline void redistribute(Grid3D& g, int sweeps = 20) {
00050   const double target = g.mass();
00051   const double volume = g.cellVolume();
00052   for (int sweep = 0; sweep < sweeps; ++sweep) {
00053     for (double& c : g.data()) {
00054       c = std::min(1.0, std::max(0.0, c));
00055     }
00056     const double current = g.mass();
00057     const double delta = target - current;
00058     if (std::abs(delta) <= 1.0e-15 * std::max(1.0, std::abs(target))) {
00059       break;
00060     }
00061     double capacity = 0.0;
00062     for (double c : g.data()) {
00063       capacity += (delta > 0.0) ? (1.0 - c) : c;
00064     }
00065     if (capacity <= 0.0) {
00066       break;
00067     }
00068     const double amount = delta / (capacity * volume);
00069     for (double& c : g.data()) {
00070       const double available = (delta > 0.0) ? (1.0 - c) : c;
00071       c += amount * available;
00072       c = std::min(1.0, std::max(0.0, c));
00073     }
00074   }
00075 }
00076
00077 }  // namespace vof
00078 }  // namespace saturn
```

Index

adv
    saturn::cavity::Geometry, 66
adv_blend
    saturn::cavity::Geometry, 66
advance
    saturn::vof::Grid2D, 71
    saturn::vof::Grid3D, 73
advanceRk4
    saturn, 10
advanceSsprk3
    saturn, 10
    saturn::cavity, 16
    saturn::channel, 20
    saturn::edac, 27
    saturn::incompressible, 29
alpha
    saturn::vof::Reconstruction, 82
    saturn::vof::Reconstruction3D, 82
apply
    saturn::cavity::ClusteredPoisson2D, 52
applyBoundaryConditions
    saturn, 11
apps/cavity_sim.cpp, 87
apps/channel_sim.cpp, 88
apps/compressible_mms.cpp, 89
apps/compressible_sim.cpp, 90
apps/edac_sim.cpp, 91
apps/euler1d.cpp, 92
apps/halo_bench.cpp, 93
apps/incompressible_sim.cpp, 94
apps/scalar_mms.cpp, 95
apps/stencil_bench.cpp, 96
apps/stream_bench.cpp, 96
assembleRhs
    saturn::compressible, 23
AusmPlusUp
    saturn, 10
ausmPlusUp
    saturn::riemann, 38
averageFlux
    saturn::riemann, 38

b
    saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
begin
    saturn::FaceExchange< ExecSpace >, 59
beta
    saturn::cavity::Geometry, 66
Boundary
    saturn::poisson, 31
BoundaryType
    saturn, 9
box
    saturn::PatchField< ExecSpace >, 79
BoxExchange
    saturn::BoxExchange< ExecSpace >, 49
boxForCoords
    saturn::Decomposition, 57
buffer_type
    saturn::FaceExchange< ExecSpace >, 58

c
    saturn::edac::Parameters, 78
cavity_sim.cpp
    main, 88
    SATURN_DEFAULT_SPACE, 88
cellVolume
    saturn::vof::Grid3D, 73
Central2
    saturn, 10
central2
    saturn::recon, 34
Central4
    saturn, 10
central4
    saturn::recon, 34
central4Derivative
    saturn::compressible, 23
Central6
    saturn, 10
central6
    saturn::recon, 35
channel_sim.cpp
    main, 89
    SATURN_DEFAULT_SPACE, 89
clipAxis
    saturn::vof, 42
clipHalfPlane
    saturn::vof, 42
ClusteredPoisson2D
    saturn::cavity::ClusteredPoisson2D, 52
coarse_smooth
    saturn::poisson::Options, 77
columnHeight
    saturn::vof, 42
combine
    saturn::incompressible, 29
compressible_mms.cpp
    main, 90

SATURN_DEFAULT_SPACE, 90
compressible_sim.cpp
main, 91
SATURN_DEFAULT_SPACE, 91
compressibleRhs
saturn::compressible, 23
CompressibleWorkspace
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53
computeFaceFluxesX
saturn::compressible, 23
computeFaceFluxesY
saturn::compressible, 23
computeFaceFluxesZ
saturn::compressible, 24
computeGradients
saturn::compressible, 24
computeResidual
saturn::poisson::GeometricMultigrid< ExecSpace >, 63
computeStress
saturn::compressible, 24
contains
saturn::Box, 47
converged
saturn::poisson::Result, 83
coords
saturn::Decomposition, 57
create
saturn::Decomposition, 57
cross
saturn::vof, 42
curvatureLevelset
saturn::vof, 42
curvatureX
saturn::vof, 43
curvatureY
saturn::vof, 43
curvatureZ
saturn::vof, 43
cycles
saturn::poisson::Result, 83

data
saturn::PatchField< ExecSpace >, 79
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 73
dims
saturn::Decomposition, 57
Dirichlet
saturn::poisson, 31
dispatchExecutionSpace
saturn, 11
divergence
saturn::cavity, 17
saturn::channel, 20
saturn::projection, 33
dmapX
saturn::cavity::Geometry, 66
dot
saturn::vof, 43

edac_sim.cpp
main, 92
SATURN_DEFAULT_SPACE, 92
empty
saturn::Box, 47
end
saturn::FaceExchange< ExecSpace >, 59
energy
saturn::ConservativeState, 54
saturn::Flux, 60
ensureParent
saturn, 11
euler1d.cpp
main, 92
exchange
saturn::BoxExchange< ExecSpace >, 49
saturn::FaceExchange< ExecSpace >, 59
saturn::GlobalExchange< ExecSpace >, 70
execution_space
saturn::PatchField< ExecSpace >, 78
extendedIndex
saturn::poisson::detail, 31

Face
saturn::vof, 41
face_view
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53
FaceExchange
saturn::FaceExchange< ExecSpace >, 59
fieldAddScaled
saturn, 11
fieldCombine
saturn, 11
fieldCopy
saturn, 11
fieldDerivative
saturn::compressible, 24
fillPeriodic
saturn::PatchField< ExecSpace >, 79
filter4
saturn::incompressible, 30
filter4Channel
saturn::channel, 20
filterDimension
saturn::channel, 21
saturn::incompressible, 30
filterVelocity
saturn::cavity, 17
final_residual
saturn::poisson::Result, 83
fluidPolygon
saturn::vof, 43
fluidPolyhedron
saturn::vof, 43
flux

saturn::riemann, 38
fromCells
saturn::UniformGrid, 84
fromCellsLocal
saturn::UniformGrid, 84
fromExtents
saturn::Box, 48
fromNormal
saturn::compressible, 24
fx
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53
fy
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53
fz
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53

gamma
saturn::compressible::GasProperties, 61
gas_constant
saturn::compressible::GasProperties, 61
GeometricMultigrid
saturn::poisson::GeometricMultigrid< ExecSpace >, 63
global
saturn::Decomposition, 57
saturn::UniformGrid, 85
global_hi
saturn::CheckpointHeader, 50
global_lo
saturn::CheckpointHeader, 50
GlobalExchange
saturn::GlobalExchange< ExecSpace >, 69
grad
saturn::compressible::CompressibleWorkspace< ExecSpace >, 54
gradientLine
saturn::compressible, 25
gradientX
saturn::projection, 33
gradientY
saturn::projection, 33
gradientZ
saturn::projection, 33
Grid2D
saturn::vof::Grid2D, 70
Grid3D
saturn::vof::Grid3D, 73
grow
saturn::Box, 48
gx
saturn::channel::Geometry, 68

h
saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
saturn::UniformGrid, 85
halo_bench.cpp
main, 93
SATURN_DEFAULT_SPACE, 93
hi
saturn::Box, 49
saturn::UniformGrid, 85
Hllc
saturn, 10
hllc
saturn::riemann, 38
hllcImpl
saturn::riemann, 39
HllcLowMach
saturn, 10
hllcLowMach
saturn::riemann, 39
host_buffer_type
saturn::FaceExchange< ExecSpace >, 58
hx
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 68
saturn::vof::Curvature3D, 56
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 73
hy
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 68
saturn::vof::Curvature3D, 56
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 73
hz
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 68
saturn::vof::Grid3D, 73

incompressible_sim.cpp
main, 94
SATURN_DEFAULT_SPACE, 94
initial_residual
saturn::poisson::Result, 83
Int3
saturn, 9
Int4
saturn, 9
intersect
saturn::Box, 48
inviscidFaceFlux
saturn::compressible, 25
is_host_execution_space
saturn, 15

kappa
saturn::vof::Curvature2D, 55
saturn::vof::Curvature3D, 56
saturn::vof::LevelSetCurvature, 76
kAuthor
saturn, 15
kEpsilon
saturn::recon, 37

kInnerTile
saturn, 15
kProjectName
saturn, 15
kTenoCutoff
saturn::recon, 37
kTenoExponent
saturn::recon, 37
kVersion
saturn, 15
kWenoWeight0
saturn::recon, 37
kWenoWeight1
saturn::recon, 37
kWenoWeight2
saturn::recon, 37

laplacian
saturn::edac, 28
length
saturn::Box, 48
levels
saturn::poisson::GeometricMultigrid< ExecSpace >, 63
limitedUpwind
saturn::cavity, 17
Llf
saturn, 10
llf
saturn::riemann, 39
lo
saturn::Box, 49
saturn::UniformGrid, 85
loadPrimitive
saturn::compressible, 25
local
saturn::Decomposition, 57
saturn::UniformGrid, 85
local_hi
saturn::CheckpointHeader, 50
local_lo
saturn::CheckpointHeader, 50
localBox
saturn::UniformGrid, 85
ly
saturn::channel::Geometry, 68

machMinus1
saturn::riemann, 39
machMinus2
saturn::riemann, 39
machPlus1
saturn::riemann, 39
machPlus2
saturn::riemann, 39
magic
saturn::CheckpointHeader, 50
main
cavity_sim.cpp, 88
channel_sim.cpp, 89
compressible_mms.cpp, 90
compressible_sim.cpp, 91
edac_sim.cpp, 92
euler1d.cpp, 92
halo_bench.cpp, 93
incompressible_sim.cpp, 94
scalar_mms.cpp, 95
stencil_bench.cpp, 96
stream_bench.cpp, 97
makeMdrange3
saturn, 12
makeMdrange4
saturn, 12
mapX
saturn::cavity::Geometry, 66
mass
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 73
max_cycles
saturn::poisson::Options, 77
maxKinematicDiffusivity
saturn::compressible, 25
maxSignalSpeed
saturn::riemann, 40
maxWaveSpeed
saturn::compressible, 25
MDRange3
saturn, 9
MDRange4
saturn, 9
memory_space
saturn::compressible::CompressibleWorkspace< ExecSpace >, 53
saturn::FaceExchange< ExecSpace >, 59
saturn::PatchField< ExecSpace >, 78
saturn::poisson::GeometricMultigrid< ExecSpace >, 63
minmod
saturn::cavity, 17
minusIndex
saturn::poisson::detail, 31
mom_n
saturn::ConservativeState, 54
saturn::Flux, 60
mom_t1
saturn::ConservativeState, 54
saturn::Flux, 60
mom_t2
saturn::ConservativeState, 54
saturn::Flux, 60
momentumRhs
saturn::cavity, 18
saturn::channel, 21
saturn::incompressible, 30
mpiAll
saturn, 12
mpiMax

saturn, 12
mpiRank
saturn, 12
mpiSize
saturn, 12
mpiSum
saturn, 12
mu
saturn::compressible::GasProperties, 61

n
saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
n0
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 68
n1
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 68
n2
saturn::cavity::Geometry, 67
saturn::channel::Geometry, 69
Neumann
saturn::poisson, 31
ng
saturn::CheckpointHeader, 50
saturn::PatchField< ExecSpace >, 80
saturn::UniformGrid, 85
norm
saturn::poisson::GeometricMultigrid< ExecSpace >, 63
nu
saturn::channel::Geometry, 69
saturn::edac::Parameters, 78
nu_p
saturn::edac::Parameters, 78
numCells
saturn::Box, 48
saturn::UniformGrid, 85
nvar
saturn::CheckpointHeader, 50
saturn::PatchField< ExecSpace >, 80
nx
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 74
saturn::vof::Reconstruction, 82
saturn::vof::Reconstruction3D, 82
ny
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 74
saturn::vof::Reconstruction, 82
saturn::vof::Reconstruction3D, 83
nz
saturn::vof::Grid3D, 74
saturn::vof::Reconstruction3D, 83

operator!=
saturn::Box, 48
operator()
saturn::PatchField< ExecSpace >, 80
saturn::vof::Grid2D, 71
saturn::vof::Grid3D, 74
operator+
saturn::vof, 44
operator-
saturn::vof, 44
operator==
saturn::Box, 48
operator*
saturn::vof, 43

p
saturn::PrimitiveState, 81
pack
saturn::FaceExchange< ExecSpace >, 59
parseBoundaryType
saturn, 12
partner
saturn::Decomposition, 57
PatchField
saturn::PatchField< ExecSpace >, 79
Periodic
saturn, 10
saturn::poisson, 31
periodic
saturn::Decomposition, 58
physicalFlux
saturn::riemann, 40
plusIndex
saturn::poisson::detail, 32
polygonArea
saturn::vof, 44
Polyhedron
saturn::vof, 41
polyhedronVolume
saturn::vof, 44
post_smooth
saturn::poisson::Options, 77
prandtl
saturn::compressible::GasProperties, 61
pre_smooth
saturn::poisson::Options, 77
pressureMinus5
saturn::riemann, 40
pressurePlus5
saturn::riemann, 40
pressureRhs
saturn::edac, 28
project
saturn::cavity, 18
saturn::channel, 21
saturn::projection, 33
projectClustered
saturn::cavity, 18
projectMeanImpl
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
prolongAdd

saturn::poisson::GeometricMultigrid< ExecSpace >, 64

r
saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
rank
saturn::CheckpointHeader, 51
saturn::Decomposition, 58
rankSuffix
saturn, 13
readCheckpoint
saturn, 13
reconstruct
saturn::vof, 44
reconstructFace
saturn::compressible, 26
Reconstruction
saturn, 10
reconstructions
saturn::vof::Grid2D, 72
saturn::vof::Grid3D, 74
reconstructionStencilWidth
saturn, 13
reconstructLeft
saturn::recon, 35
reconstructRight
saturn::recon, 35
redistribute
saturn::vof, 44, 45
residualNorm
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
restrict
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
rho
saturn::ConservativeState, 55
saturn::Flux, 60
saturn::PrimitiveState, 81
RiemannSolver
saturn, 10
rowHeight
saturn::vof, 45

saturn, 7
advanceRk4, 10
advanceSsprk3, 10
applyBoundaryConditions, 11
AusmPlusUp, 10
BoundaryType, 9
Central2, 10
Central4, 10
Central6, 10
dispatchExecutionSpace, 11
ensureParent, 11
fieldAddScaled, 11
fieldCombine, 11
fieldCopy, 11
Hllc, 10
HllcLowMach, 10
Int3, 9
Int4, 9
is_host_execution_space, 15
kAuthor, 15
kInnerTile, 15
kProjectName, 15
kVersion, 15
Llf, 10
makeMdrange3, 12
makeMdrange4, 12
MDRange3, 9
MDRange4, 9
mpiAll, 12
mpiMax, 12
mpiRank, 12
mpiSize, 12
mpiSum, 12
parseBoundaryType, 12
Periodic, 10
rankSuffix, 13
readCheckpoint, 13
Reconstruction, 10
reconstructionStencilWidth, 13
RiemannSolver, 10
scalarAdvectionDiffusionRhs, 13
scalarAdvectionDiffusionRhsShell, 13
scalarRhsValue, 14
SlipWall, 10
Teno6, 10
Transmissive, 10
unavailableBackend, 14
Weno5Z, 10
writeCheckpoint, 14
writeVti, 14
writeVtiFields, 14
saturn::Box, 47
contains, 47
empty, 47
fromExtents, 48
grow, 48
hi, 49
intersect, 48
length, 48
lo, 49
numCells, 48
operator!=, 48
operator==, 48
shift, 48
shrink, 48
saturn::BoxExchange< ExecSpace >, 49
BoxExchange, 49
exchange, 49
saturn::cavity, 15
advanceSsprk3, 16
divergence, 17
filterVelocity, 17

limitedUpwind, 17
minmod, 17
momentumRhs, 18
project, 18
projectClustered, 18
weno5Derivative, 19
saturn::cavity::ClusteredPoisson2D, 51
apply, 52
ClusteredPoisson2D, 52
solve, 52
saturn::cavity::Geometry, 65
adv, 66
adv_blend, 66
beta, 66
dmapX, 66
hx, 67
hy, 67
hz, 67
mapX, 66
n0, 67
n1, 67
n2, 67
u_lid, 67
xc, 66
xf, 66
yc, 66
yf, 66
z_wall, 67
saturn::channel, 19
advanceSsprk3, 20
divergence, 20
filter4Channel, 20
filterDimension, 21
momentumRhs, 21
project, 21
saturn::channel::Geometry, 68
gx, 68
hx, 68
hy, 68
hz, 68
ly, 68
n0, 68
n1, 68
n2, 69
nu, 69
u_top, 69
saturn::CheckpointHeader, 50
global_hi, 50
global_lo, 50
local_hi, 50
local_lo, 50
magic, 50
ng, 50
nvar, 50
rank, 51
size, 51
step, 51
time, 51
version, 51
saturn::compressible, 22
assembleRhs, 23
central4Derivative, 23
compressibleRhs, 23
computeFaceFluxesX, 23
computeFaceFluxesY, 23
computeFaceFluxesZ, 24
computeGradients, 24
computeStress, 24
fieldDerivative, 24
fromNormal, 24
gradientLine, 25
inviscidFaceFlux, 25
loadPrimitive, 25
maxKinematicDiffusivity, 25
maxWaveSpeed, 25
reconstructFace, 26
soundSpeed, 26
temperature, 26
toNormal, 26
saturn::compressible::CompressibleWorkspace< ExecSpace >, 52
CompressibleWorkspace, 53
face_view, 53
fx, 53
fy, 53
fz, 53
grad, 54
memory_space, 53
stress, 54
saturn::compressible::GasProperties, 61
gamma, 61
gas_constant, 61
mu, 61
prandtl, 61
saturn::ConservativeState, 54
energy, 54
mom_n, 54
mom_t1, 54
mom_t2, 54
rho, 55
saturn::Decomposition, 56
boxForCoords, 57
coords, 57
create, 57
dims, 57
global, 57
local, 57
partner, 57
periodic, 58
rank, 58
size, 58
saturn::edac, 26
advanceSsprk3, 27
laplacian, 28
pressureRhs, 28
subtract, 28

velocityRhs, 28
saturn::edac::Parameters, 77
c, 78
nu, 78
nu_p, 78
saturn::FaceExchange< ExecSpace >, 58
begin, 59
buffer_type, 58
end, 59
exchange, 59
FaceExchange, 59
host_buffer_type, 58
memory_space, 59
pack, 59
unpack, 59
saturn::Flux, 60
energy, 60
mom_n, 60
mom_t1, 60
mom_t2, 60
rho, 60
saturn::GlobalExchange< ExecSpace >, 69
exchange, 70
GlobalExchange, 69
saturn::incompressible, 29
advanceSsprk3, 29
combine, 29
filter4, 30
filterDimension, 30
momentumRhs, 30
saturn::PatchField< ExecSpace >, 78
box, 79
data, 79
execution_space, 78
fillPeriodic, 79
memory_space, 78
ng, 80
nvar, 80
operator(), 80
PatchField, 79
view_type, 79
saturn::poisson, 30
Boundary, 31
Dirichlet, 31
Neumann, 31
Periodic, 31
saturn::poisson::detail, 31
extendedIndex, 31
minusIndex, 31
plusIndex, 32
saturn::poisson::GeometricMultigrid< ExecSpace >, 62
computeResidual, 63
GeometricMultigrid, 63
levels, 63
memory_space, 63
norm, 63
projectMeanImpl, 64
prolongAdd, 64
residualNorm, 64
restrict, 64
smooth, 64
solve, 64
vcycle, 64
view_type, 63
saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
b, 75
h, 75
n, 75
r, 75
x, 75
saturn::poisson::Options, 76
coarse_smooth, 77
max_cycles, 77
post_smooth, 77
pre_smooth, 77
tolerance, 77
verbose, 77
saturn::poisson::Result, 83
converged, 83
cycles, 83
final_residual, 83
initial_residual, 83
saturn::PrimitiveState, 81
p, 81
rho, 81
u, 81
v, 81
w, 81
saturn::projection, 32
divergence, 33
gradientX, 33
gradientY, 33
gradientZ, 33
project, 33
ScalarView, 32
saturn::recon, 34
central2, 34
central4, 34
central6, 35
kEpsilon, 37
kTenoCutoff, 37
kTenoExponent, 37
kWenoWeight0, 37
kWenoWeight1, 37
kWenoWeight2, 37
reconstructLeft, 35
reconstructRight, 35
stencil6Smoothness, 36
teno6, 36
tenoInverseSmoothness, 36
weno5Smoothness, 36
weno5z, 36
saturn::riemann, 38
ausmPlusUp, 38
averageFlux, 38

flux, 38
hllc, 38
hllcImpl, 39
hllcLowMach, 39
llf, 39
machMinus1, 39
machMinus2, 39
machPlus1, 39
machPlus2, 39
maxSignalSpeed, 40
physicalFlux, 40
pressureMinus5, 40
pressurePlus5, 40
toConservative, 40
toPrimitive, 40
valid, 40
saturn::UniformGrid, 84
fromCells, 84
fromCellsLocal, 84
global, 85
h, 85
hi, 85
lo, 85
local, 85
localBox, 85
ng, 85
numCells, 85
saturn::vof, 41
clipAxis, 42
clipHalfPlane, 42
columnHeight, 42
cross, 42
curvatureLevelset, 42
curvatureX, 43
curvatureY, 43
curvatureZ, 43
dot, 43
Face, 41
fluidPolygon, 43
fluidPolyhedron, 43
operator+, 44
operator-, 44
operator∗, 43
polygonArea, 44
Polyhedron, 41
polyhedronVolume, 44
reconstruct, 44
redistribute, 44, 45
rowHeight, 45
smoothBinomial, 45
unitCube, 45
unitSquare, 45
saturn::vof::Curvature2D, 55
kappa, 55
slope, 55
valid, 55
saturn::vof::Curvature3D, 55
hx, 56
hy, 56
kappa, 56
valid, 56
saturn::vof::Grid2D, 70
advance, 71
data, 71
Grid2D, 70
hx, 71
hy, 71
mass, 71
nx, 71
ny, 71
operator(), 71
reconstructions, 72
sweepX, 72
sweepY, 72
saturn::vof::Grid3D, 72
advance, 73
cellVolume, 73
data, 73
Grid3D, 73
hx, 73
hy, 73
hz, 73
mass, 73
nx, 74
ny, 74
nz, 74
operator(), 74
reconstructions, 74
sweepX, 74
sweepY, 74
sweepZ, 74
saturn::vof::LevelSetCurvature, 76
kappa, 76
valid, 76
saturn::vof::Point, 80
x, 80
y, 80
saturn::vof::Reconstruction, 82
alpha, 82
nx, 82
ny, 82
saturn::vof::Reconstruction3D, 82
alpha, 82
nx, 82
ny, 83
nz, 83
saturn::vof::Vec3, 86
x, 86
y, 86
z, 86
SATURN_DEFAULT_SPACE
cavity_sim.cpp, 88
channel_sim.cpp, 89
compressible_mms.cpp, 90
compressible_sim.cpp, 91
edac_sim.cpp, 92

halo_bench.cpp, 93
incompressible_sim.cpp, 94
scalar_mms.cpp, 95
stencil_bench.cpp, 96
stream_bench.cpp, 97
scalar_mms.cpp
main, 95
SATURN_DEFAULT_SPACE, 95
scalarAdvectionDiffusionRhs
saturn, 13
scalarAdvectionDiffusionRhsShell
saturn, 13
scalarRhsValue
saturn, 14
ScalarView
saturn::projection, 32
shift
saturn::Box, 48
shrink
saturn::Box, 48
size
saturn::CheckpointHeader, 51
saturn::Decomposition, 58
SlipWall
saturn, 10
slope
saturn::vof::Curvature2D, 55
smooth
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
smoothBinomial
saturn::vof, 45
solve
saturn::cavity::ClusteredPoisson2D, 52
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
soundSpeed
saturn::compressible, 26
src/core/execution_space.hpp, 97, 98
src/core/mpi.hpp, 99
src/core/parallel.hpp, 100
src/core/version.hpp, 101
src/fields/boundary_conditions.hpp, 101, 102
src/fields/patch_field.hpp, 103, 104
src/halo/box_exchange.hpp, 105
src/halo/face_exchange.hpp, 108
src/halo/global_exchange.hpp, 111
src/io/checkpoint.hpp, 113, 114
src/io/vtk_writer.hpp, 115, 116
src/mesh/box.hpp, 118, 119
src/mesh/decomposition.hpp, 120
src/mesh/uniform_grid.hpp, 122
src/numerics/compressible/compressible_ns.hpp, 123, 124
src/numerics/compressible/reconstruction.hpp, 131, 132
src/numerics/compressible/riemann.hpp, 135, 137
src/numerics/compressible/scalar_advection.hpp, 142
src/numerics/compressible/time_integration.hpp, 144
src/numerics/incompressible/cavity.hpp, 146, 148
src/numerics/incompressible/channel.hpp, 156, 157
src/numerics/incompressible/channel_poisson.hpp, 161
src/numerics/incompressible/edac.hpp, 163, 164
src/numerics/incompressible/fft_poisson.hpp, 166
src/numerics/incompressible/incompressible.hpp, 168, 169
src/numerics/incompressible/poisson.hpp, 172, 173
src/numerics/incompressible/projection.hpp, 179, 180
src/numerics/multiphase/dambreak2d.hpp, 181, 182
src/numerics/multiphase/twophase2d.hpp, 189
src/numerics/vof/vof2d.hpp, 195, 196
src/numerics/vof/vof3d.hpp, 199, 200
src/numerics/vof/vof_curvature2d.hpp, 204, 205
src/numerics/vof/vof_curvature3d.hpp, 206, 207
src/numerics/vof/vof_curvature_levelset2d.hpp, 208
src/numerics/vof/vof_redistribute.hpp, 209, 210
stencil6Smoothness
saturn::recon, 36
stencil_bench.cpp
main, 96
SATURN_DEFAULT_SPACE, 96
step
saturn::CheckpointHeader, 51
stream_bench.cpp
main, 97
SATURN_DEFAULT_SPACE, 97
stress
saturn::compressible::CompressibleWorkspace< ExecSpace >, 54
subtract
saturn::edac, 28
sweepX
saturn::vof::Grid2D, 72
saturn::vof::Grid3D, 74
sweepY
saturn::vof::Grid2D, 72
saturn::vof::Grid3D, 74
sweepZ
saturn::vof::Grid3D, 74

temperature
saturn::compressible, 26
Teno6
saturn, 10
teno6
saturn::recon, 36
tenoInverseSmoothness
saturn::recon, 36
time
saturn::CheckpointHeader, 51
toConservative
saturn::riemann, 40
tolerance
saturn::poisson::Options, 77
toNormal
saturn::compressible, 26
toPrimitive
saturn::riemann, 40

Transmissive
saturn, 10

u
saturn::PrimitiveState, 81
u_lid
saturn::cavity::Geometry, 67
u_top
saturn::channel::Geometry, 69
unavailableBackend
saturn, 14
unitCube
saturn::vof, 45
unitSquare
saturn::vof, 45
unpack
saturn::FaceExchange< ExecSpace >, 59

v
saturn::PrimitiveState, 81
valid
saturn::riemann, 40
saturn::vof::Curvature2D, 55
saturn::vof::Curvature3D, 56
saturn::vof::LevelSetCurvature, 76
vcycle
saturn::poisson::GeometricMultigrid< ExecSpace >, 64
velocityRhs
saturn::edac, 28
verbose
saturn::poisson::Options, 77
version
saturn::CheckpointHeader, 51
view_type
saturn::PatchField< ExecSpace >, 79
saturn::poisson::GeometricMultigrid< ExecSpace >, 63

w
saturn::PrimitiveState, 81
weno5Derivative
saturn::cavity, 19
weno5Smoothness
saturn::recon, 36
Weno5Z
saturn, 10
weno5z
saturn::recon, 36
writeCheckpoint
saturn, 14
writeVti
saturn, 14
writeVtiFields
saturn, 14

x
saturn::poisson::GeometricMultigrid< ExecSpace >::Level, 75
saturn::vof::Point, 80
saturn::vof::Vec3, 86
xc
saturn::cavity::Geometry, 66
xf
saturn::cavity::Geometry, 66

y
saturn::vof::Point, 80
saturn::vof::Vec3, 86
yc
saturn::cavity::Geometry, 66
yf
saturn::cavity::Geometry, 66

z
saturn::vof::Vec3, 86
z_wall
saturn::cavity::Geometry, 67